
Half Title

Solution Manual
Introduction to Bayesian Econometrics: A GUIDed
toolkit using R



Title Page

Solution Manual
Introduction to Bayesian Econometrics: A GUIDed
toolkit using R

by Andrés Ramírez-Hassan, PhD. Statistical Science.



*To my wife, Estephania, and to my parents, Nancy
and Orlando.*



Contents

Foreword	ix
Preface	xi
Symbols	xv
I Foundations: Theory, simulation methods and programming	1
1 Basic formal concepts	3
2 Conceptual differences of the Bayesian and Frequentist approaches	23
3 Cornerstone models: Conjugate families	29
4 Simulation methods	57
II Regression models: A GUIDed toolkit	79
5 Graphical user interface	81
5.1 Introduction	81
5.2 Univariate models	83
5.3 Multivariate models	86
5.4 Time series models	88
5.5 Longitudinal/panel models	91
5.6 Bayesian model averaging	92
5.7 Help	94
5.8 Warning	94
6 Univariate models	97
7 Multivariate models	119
8 Time series models	143
9 Longitudinal/Panel data models	169

10 Bayesian model average	197
III <i>Advanced</i> methods: A brief introduction	225
11 Semi-parametric and non-parametric models	227
12 Bayesian machine learning	255
13 Causal inference	281
14 Approximate Bayesian methods	309
Bibliography	331
A Appendix	335

Foreword



Preface

The main goal of this manual is to show the solution of the exercises of the book **Introduction to Bayesian Econometrics: A GUIDed toolkit using R**. Trying to solve the exercises is important for those who want to improve their understanding about the Bayesian inferential framework more approachable.

To instructors

This book is divided into three parts: foundations (chapters 1 to 4), regression analysis (chapters 5 to 10), and *Advanced* methods (chapters 11 to 14). Our graphical user interface (GUI) is designed for the second part. The source code can be found at <https://github.com/besmarter/BSTApp>. Instructors and students can access all the code, along with simulated and real datasets. There are four ways to install our GUI:

1. Install *shiny* package, and then, type
`shiny::runGitHub("besmarter/BSTApp", launch.browser=T)`
in the **R** console or any **R** code editor and execute it. We recommend to type directly, no copy and paste.
2. Visit <https://andres-ramirez-hassan.shinyapps.io/BSTApp/>. Please note: the free Posit Cloud tier sometimes runs out of memory, which can cause the app to stop. Sorry for the inconvenience.
3. Visit <https://fly-besmarter.fly.dev/>. As with Posit Cloud, occasional memory limits on the free tier may affect performance.
4. Use a **Docker** image by typing in the **Command Prompt**
 - (a) `docker pull aramir21/besmartergui:latest`
 - (b) `docker run -rm -p 3838:3838 aramir21/besmartergui`

Then users can access our GUI going to <http://localhost:3838/>. See Chapter 5 for details.

In addition, the `.tex` source files for the book's slides can be found in the GitHub repository:

<https://github.com/aramir21/PresentationsBookBayesianEconometrics>.

There is a free online version of the book at:

<https://andresramirezhasan-introduction-bayesian-econometrics-gui.share.connect.posit.cloud/>.

These allow users to copy and paste all the code and view every figure in full color.

Students should have a basic understanding of probability theory and statistics, as well as some background in econometrics and time series, particularly regression analysis. Familiarity with standard univariate and multivariate probability distributions is strongly recommended. See a nice summary of useful probability distributions in [16, p. 182-191]. Additionally, students who wish to master the material in this book should have programming skills in **R** software.¹

Instructors can send me an email to have access to this solution manual. They can use this book as a textbook for a course on introductory Bayesian Econometrics/Statistics, with a strong emphasis on implementation and applications. This book is intended to be complementary, rather than a substitute, for excellent resources on the topic, such as [11], [6], [44], [16], [14], [26], and [24].

Acknowledgments

I began developing our graphical user interface (GUI) in 2016, after being diagnosed with cervical dystonia. I worked on this side project during weekends, which I called “nerd weekends,” and it served as a form of release from my health condition. Once I began to recover, I invited Mateo Graciano, my former student, business partner, and friend, to join the project. He has been instrumental in developing our GUI, and I am enormously grateful to him.

I would also like to thank the members of the BEsmarter research group at Universidad EAFIT, as well as the NUMBATs members at Monash University, for their valuable feedback and recommendations to improve our GUI.

This book is an extension of the paper *A GUIDed tour of Bayesian regression* [42], which serves as a brief user guide for our GUI. I decided to write this book to explain the underlying theory and code in our GUI, and to use it as a textbook in my course on Bayesian econometrics/statistics. I am grateful to my students in this course; their insights and thoughtful questions have deepened my understanding of the material.

I would like to thank Chris Parmeter for his valuable suggestions on how to present our user guide; Professors Raúl Pericchi and Juan Carlos Correa for introducing me to Bayesian statistics; and Liana Jacobi, Tomasz Wozniak, and Chun Fung Kwok (Jackson) from the University of Melbourne, as well as David Frazier from Monash University, for their engaging discussions and fruitful collaborations in Bayesian econometrics and statistics. I am also deeply grateful to Professor Peter Diggle for his unwavering support of my career, and especially to Professor Gael Martin, who gave me the opportunity to work with her and has been a constant source of intellectual inspiration.

I also wish to acknowledge my colleagues and staff at Universidad EAFIT for their continuous support.

¹An excellent starting point for **R** programming is the *R Introduction Manual*: <https://cran.r-project.org/doc/manuals/r-release/R-intro.pdf>.

Finally, I acknowledge the use of ChatGPT, which assisted me in improving the grammar, clarity, and flow of the text, as well as in tidying up some of the code presented in this book. Nevertheless, all concepts, mathematical developments, and underlying logic are entirely my own, based on my understanding and readings of the literature. Any remaining errors are solely my responsibility, for which I apologize in advance. I sincerely thank the reader and hope that this book proves useful.

To my parents, Orlando and Nancy, who have always been there for me with their unconditional support. They have taught me that the primary aspect of human spiritual evolution is humility, a lesson I am still learning every day. To my wife, Estephania, for her unwavering love and support.



Symbols

Symbol Description

$\neg$	Negation symbol	$\mathbf{0}_l$	l -dimensional null vector
$\propto$	Proportional symbol	$\max_r$	Maximum over r
$\perp$	Independence symbol	$\min_r$	Minimum over r
$\mathcal{R}$	The Real set	∇	Gradiente operator
$\mathcal{R}^K$	Euclidean space of dimension K	$\stackrel{iid}{\sim}$	Independently and identically distributed
$\emptyset$	Empty set	$\stackrel{ind}{\sim}$	Independently and Not identically distributed
$\mathbb{1}$	Indicator function	$>$	Greater than
P	Probability measure	$<$	Less than
$:=$	Is defined as	$\approx$	Approximately equal to
argmax	Argument of the maximum	$\gtrsim$	Greater than or approximately equal to
argmin	Argument of the minimum	Δ	Difference operator
tr	Trace operator	$\subseteq$	Subset
vec	Vectorization operator	$\subset$	Proper subset
$\lim$	Limit	$\stackrel{d}{\rightarrow}$	Convergence in distribution (law)
$\otimes$	Kronecker product	$\stackrel{p}{\rightarrow}$	Convergence in probability
$\operatorname{diag}\{\cdot\}$	Diagonal matrix	$\stackrel{a.s.}{\rightarrow}$	Almost surely convergence
$\operatorname{dim}\{\cdot\}$	Dimension of an object		
$\mathbf{I}_l$	l -dimensional identity matrix		
$\mathbf{i}_N$	N -dimensional vector of ones		



Part I

Foundations: Theory, simulation methods and programming



1

Basic formal concepts

Solutions of Exercises

1. *The court case: the blue or green cap*

A cab was involved in a hit-and-run accident at night. There are two cab companies in the town: Blue and Green. The Blue company has 150 cabs, while the Green company has 850 cabs. A witness stated that a blue cab was involved in the accident. The court tested the reliability of the witness under similar circumstances and found that the witness correctly identified the color of the cab 80% of the time, but made an incorrect identification 20% of the time. *What is the probability that the cab involved in the accident was actually blue, given that the witness said it was blue?*

Answer

Set WB and WG equal to the events that the witness said the cab was blue and green, respectively. Set B and G equal to the events that the cabs are blue and green, respectively. We need to calculate $P(B|WB)$, then:

$$\begin{aligned} P(B|WB) &= \frac{P(B, WB)}{P(WB)} \\ &= \frac{P(WB|B) \times P(B)}{P(WB|B) \times P(B) + P(WB|\neg B) \times (1 - P(B))} \\ &= \frac{0.8 \times 0.15}{0.8 \times 0.15 + 0.20 \times 0.85} \\ &= 0.41 \end{aligned} \tag{1.1}$$

2. *The Monty Hall problem*

What is the probability of winning a car in the *Monty Hall problem* switching the decision if there are four doors, where there are three goats and one car? Solve this problem analytically and computationally. What if there are n doors, $n - 1$ goats and one car?

Answer

Let's name P_i the event *contestant picks door No. i* , H_i the event *host picks*

door No. i , and C_i the event *car is behind door No. i* . Let's assume that the contestant picked door number 1, and the host picked door number 3, then the contestant is interested in the probability of the event $P(C_i|H_3, P_1)$, $i = 2$ or 4. Then, $P(H_3|C_3, P_1) = 0$, $P(H_3|C_2, P_1) = P(H_3|C_4, P_1) = 1/2$ and $P(H_3|C_1, P_1) = 1/3$. Then,

$$\begin{aligned}
 P(C_i|H_3, P_1) &= \frac{P(C_i, H_3, P_1)}{P(H_3, P_1)} \\
 &= \frac{P(H_3|C_i, P_1)P(C_i|P_1)P(P_1)}{P(H_3|P_1) \times P(P_1)} \\
 &= \frac{P(H_3|C_i, P_1)P(C_i)}{P(H_3|P_1)} \\
 &= \frac{1/2 \times 1/4}{1/3} \\
 &= \frac{3}{8},
 \end{aligned} \tag{1.2}$$

where the third equation uses the fact that C_i and P_i are independent events, and $P(H_3|P_1) = 1/3$ due to this depending just on P_1 (not on C_i).

Therefore, changing the initial decision increases the probability of getting the car from $1/4$ to $3/8$!

Let's check the case with n doors, and assume that the contestant picks the door No. 1, the car is behind the door No. n , and the host, who knows what is behind each door, opens any of the remaining $n - 2$ doors, where there is a goat. The contestant is interested in the probability of the event:

$$\begin{aligned}
 P(C_n|(H_2 \cup \dots \cup H_{n-1}) \cap P_1) &= \frac{P((H_2 \cup H_3 \cup \dots \cup H_{n-1})|C_n \cap P_1)P(C_n|P_1)P(P_1)}{P((H_2 \cup H_3 \cup \dots \cup H_{n-1})|P_1)P(P_1)} \\
 &= \frac{[P(H_2|C_n \cap P_1) + \dots + P(H_{n-1}|C_n \cap P_1)]P(C_n)}{P(H_2|P_1) + P(H_3|P_1) + \dots + P(H_{n-1}|P_1)} \\
 &= \frac{1 \times (\frac{1}{n})}{\frac{1}{n-1} + \frac{1}{n-1} + \dots + \frac{1}{n-1}} \\
 &= \left(\frac{1}{n}\right) \left(\frac{n-1}{n-2}\right).
 \end{aligned} \tag{1.3}$$

In general, the probability of winning the car changing the pick is $\frac{1}{n} \frac{n-1}{n-2}$, while the probability of winning given no change is $\frac{1}{n}$. Given that $\frac{1}{n} \frac{n-1}{n-2} > \frac{1}{n}$ for all $n \geq 3$, where the difference between both probabilities is $\frac{1}{n(n-2)}$. We observe that as the number of doors increases, the difference between the two probabilities becomes zero.

Let's see a code for the general setting,

R code. The Monty Hall Problem

```

1 set.seed(0101) # Set simulation seed
2 S <- 100000 # Simulations
3 Game <- function(opt = 3){
4   # opt: number of options. opt > 2
5   opts <- 1:opt
6   car <- sample(opts, 1) # car location
7   guess1 <- sample(opts, 1) # Initial guess
8   if(opt == 3 && car != guess1) {
9     host <- opts[-c(car, guess1)]
10    } else {
11    host <- sample(opts[-c(car, guess1)], 1)
12  }
13  win1 <- guess1 == car # Win given no change
14  if(opt == 3) {
15    guess2 <- opts[-c(host, guess1)]
16  } else {
17    guess2 <- sample(opts[-c(host, guess1)], 1)
18  }
19  win2 <- guess2 == car # Win given change
20  return(c(win1, win2))
21 }
22 #Win probabilities
23 Prob <- rowMeans(replicate(S, Game(opt = 4)))
24 #Winning probabilities no changing door
25 Prob[1]
26 0.25151
27 #Winning probabilities changing door
28 Prob[2]
29 0.37267

```

3. Solve the health insurance example using a Gamma prior in the rate parametrization, that is, $\pi(\lambda) = \frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)} \lambda^{\alpha_0-1} \exp\{-\lambda\beta_0\}$.

Answer

First, we get the posterior distribution,

$$\pi(\lambda|\mathbf{y}) = \left(\frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)} \lambda^{\alpha_0-1} e^{-\lambda\beta_0} \right) \left(\prod_{i=1}^N \frac{\lambda^{y_i} e^{-\lambda}}{y_i!} \right) \quad (1.4)$$

$$\begin{aligned}
&= \left(\frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)} \lambda^{\alpha_0-1} e^{-\lambda\beta_0} \right) \left(\frac{\lambda^{\sum_{i=1}^N y_i} e^{-N\lambda}}{\prod_{i=1}^N y_i!} \right) \\
&= \left(\frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)} \frac{1}{\prod_{i=1}^N y_i!} \right) \lambda^{\sum_{i=1}^N y_i + \alpha_0 - 1} e^{-\lambda(\beta_0 + N)} \\
&\propto \lambda^{\sum_{i=1}^N y_i + \alpha_0 - 1} e^{-\lambda(\beta_0 + N)}.
\end{aligned} \tag{1.5}$$

The last expression is the kernel of a Gamma distribution with parameters $\alpha_n = \sum_{i=1}^N y_i + \alpha_0$ and $\beta_n = \beta_0 + N$.

Given that $\int_0^\infty \pi(\lambda|\mathbf{y}) d\lambda = 1$, then the constant of proportionality in the last expression is $\Gamma(\alpha_n) / \beta_n^{\alpha_n}$. Therefore the posterior density function $\pi(\lambda|\mathbf{y})$ is $G(\alpha_n, \beta_n)$.

The posterior mean is

$$\begin{aligned}
E[\lambda|\mathbf{y}] &= \frac{\alpha_n}{\beta_n} \\
&= \frac{\sum_{i=1}^N y_i + \alpha_0}{\beta_0 + N} \\
&= \left(\frac{N}{\beta_0 + N} \right) \bar{y} + \left(\frac{\beta_0}{\beta_0 + N} \right) \frac{\alpha_0}{\beta_0} \\
&= w\bar{y} + (1-w)E[\lambda],
\end{aligned} \tag{1.6}$$

where $w = \frac{N}{\beta_0 + N}$, $\bar{y}$ is the sample mean, and $E[\lambda] = \frac{\alpha_0}{\beta_0}$.

The posterior predictive distribution is given by

$$\begin{aligned}
\pi(Y_0|\mathbf{y}) &= \int_0^\infty \frac{\lambda^{y_0} e^{-\lambda}}{y_0!} \pi(\lambda|\mathbf{y}) d\lambda \\
&= \int_0^\infty \left(\frac{\lambda^{y_0} e^{-\lambda}}{y_0!} \right) \left(\frac{\beta_n^{\alpha_n}}{\Gamma(\alpha_n)} \lambda^{\alpha_n-1} e^{-\lambda\beta_n} \right) d\lambda \\
&= \frac{\beta_n^{\alpha_n}}{\Gamma(\alpha_n) y_0!} \int_0^\infty \lambda^{y_0 + \alpha_n - 1} e^{-\lambda(1+\beta_n)} d\lambda \\
&= \frac{\beta_n^{\alpha_n}}{\Gamma(\alpha_n) y_0!} \frac{\Gamma(y_0 + \alpha_n)}{(1 + \beta_n)^{y_0 + \alpha_n}} \\
&= \frac{\Gamma(y_0 + \alpha_n)}{\Gamma(\alpha_n) y_0!} \left(\frac{1}{1 + \beta_n} \right)^{y_0} \left(\frac{\beta_n}{1 + \beta_n} \right)^{\alpha_n}
\end{aligned} \tag{1.7}$$

$$\begin{aligned}
&= \frac{(y_0 + \alpha_n - 1)!}{(\alpha_n - 1)! y_0!} \left(\frac{1}{1 + \beta_n} \right)^{y_0} \left(\frac{\beta_n}{1 + \beta_n} \right)^{\alpha_n} \\
&= \binom{y_0 + \alpha_n - 1}{y_0} \left(\frac{1}{1 + \beta_n} \right)^{y_0} \left(\frac{\beta_n}{1 + \beta_n} \right)^{\alpha_n}.
\end{aligned}$$

Therefore $Y_0|y \sim NB(\alpha_n, p_n)$ where $p_n = \frac{1}{1+\beta_n}$.

To use empirical Bayes, we have the following setting

$$[\hat{\alpha}_0, \hat{\beta}_0] = \arg \max_{\alpha_0, \beta_0} \ln(p(\mathbf{y})),$$

where

$$\begin{aligned}
p(y) &= \int_0^\infty \left(\frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)} \lambda^{\alpha_0-1} e^{-\lambda \beta_0} \right) \left(\prod_{i=1}^N \frac{\lambda^{y_i} e^{-\lambda}}{y_i!} \right) d\lambda \quad (1.8) \\
&= \left(\frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0) \prod_{i=1}^N y_i!} \right) \int_0^\infty \lambda^{\sum_{i=1}^N y_i + \alpha_0 - 1} e^{-\lambda(\beta_0 + N)} d\lambda \\
&= \left(\frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0) \prod_{i=1}^N y_i!} \right) \left(\frac{\Gamma(\sum_{i=1}^N y_i + \alpha_0)}{(\beta_0 + N)^{\sum_{i=1}^N y_i + \alpha_0}} \right) \\
&= \frac{\Gamma(\sum_{i=1}^N y_i + \alpha_0)}{\Gamma(\alpha_0) \prod_{i=1}^N y_i!} \left(\frac{1}{\beta_0 + N} \right)^{\sum_{i=1}^N y_i} \left(\frac{\beta_0}{\beta_0 + N} \right)^{\alpha_0}.
\end{aligned}$$

*R code. Health insurance, predictive distribution
using vague hyperparameters*

```
1 set.seed(010101)
2 y <- c(0, 3, 2, 1, 0) # Data
3 N <- length(y)
4
5 # Predictive distribution
6 ProbBo <- function(y, a0, b0){
7   N <- length(y)
8   #sample size
9   aN <- a0 + sum(y)
10  # Posterior shape parameter
11  bN <- b0 + N
12  # Posterior scale parameter
13  p <- 1 / (bN + 1)
14  # Probability negative binomial density
15  Pr <- 1 - pbinom(0, size = aN, prob = (1 - p))
16  # Probability of visiting the Doctor
17  # Observe that in R there is a slightly
18  # different parametrization.
19  return(Pr)
20 }
21
22 # Using a vague prior:
23 a0 <- 0.001 # Prior shape parameter
24 b0 <- 0.001 # Prior scale parameter
25 PriMeanV <- a0 / b0 # Prior mean
26 PriVarV <- a0 / b0^2 # Prior variance
27 Pp <- ProbBo(y, a0 = 0.001, b0 = 0.001)
28 # This setting is vague prior information.
29 Pp
30 0.67
```


*R code. Health insurance, predictive distribution
using empirical Bayes*

```

1 # Using Empirical Bayes
2 LogMgLik <- function(theta, y){
3   N <- length(y)
4   #sample size
5   a0 <- theta[1]
6   # prior shape hyperparameter
7   b0 <- theta[2]
8   # prior scale hyperparameter
9   aN <- sum(y) + a0
10  # posterior shape parameter
11  if(a0 <= 0 || b0 <= 0){
12    #Avoiding negative values
13    lnp <- -Inf
14  }else{lnp <- lgamma(aN) - sum(y)*log(b0+N) + a0*log(b0/(b0
15    +N)) - lgamma(a0)}
16    # log marginal likelihood
17    return(-lnp)
18  }
19  theta0 <- c(0.01, 0.01)
20  # Initial values
21  control <- list(maxit = 1000)
22  # Number of iterations in optimization
23  EmpBay <- optim(theta0, LogMgLik, method = "BFGS", control =
24    control, hessian = TRUE, y = y)
25  # Optimization
26  EmpBay$convergence
27  # Checking convergence
28  EmpBay$value # Maximum
29  a0EB <- EmpBay$par[1]
30  # Prior shape using empirical Bayes
31  a0EB
32  128.383
33  b0EB <- EmpBay$par[2]
34  # Prior scale using empirical Bayes
35  b0EB
36  106.801
37  PriMeanEB <- a0EB / b0EB
38  # Prior mean
39  PriVarEB <- a0EB / b0EB^2
40  # Prior variance
41  PpEB <- ProbBo(y, a0 = a0EB, b0 = b0EB)
42  # This setting is using empirical Bayes.
43  PpEB
44  0.69

```

4. Suppose that you are analyzing to buy a car insurance next year. To make

a better decision you want to know *what is the probability that you have a car claim next year?* You have the records of your car claims in the last 15 years, $\mathbf{y} = \{0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0\}$.

Assume that this is a random sample from a data generating process (statistical model) that is Bernoulli, $Y_i \sim \text{Ber}(p)$, and your probabilistic prior beliefs about p are well described by a beta distribution with parameters α_0 and β_0 , $p \sim B(\alpha_0, \beta_0)$, then, you are interested in calculating the probability of a claim the next year $P(Y_0 = 1|\mathbf{y})$.

Solve this using an empirical Bayes approach and a non-informative approach where $\alpha_0 = \beta_0 = 1$ (uniform distribution).

Answer

The posterior distribution is given by

$$\begin{aligned} \pi(p|\mathbf{y}) &= \left[\frac{\Gamma(\alpha_0 + \beta_0)}{\Gamma(\alpha_0)\Gamma(\beta_0)} p^{\alpha_0-1} (1-p)^{\beta_0-1} \right] \left[\prod_{i=1}^N p^{y_i} (1-p)^{1-y_i} \right] \quad (1.9) \\ &= \frac{\Gamma(\alpha_0 + \beta_0)}{\Gamma(\alpha_0)\Gamma(\beta_0)} p^{\sum_{i=1}^N y_i + \alpha_0 - 1} (1-p)^{\beta_0 + N - \sum_{i=1}^N y_i - 1} \\ &\propto p^{\sum_{i=1}^N y_i + \alpha_0 - 1} (1-p)^{\beta_0 + N - \sum_{i=1}^N y_i - 1}. \end{aligned}$$

The last expression is the kernel of a Beta distribution with parameters $\alpha_n = \sum_{i=1}^N y_i + \alpha_0$ and $\beta_n = \beta_0 + N - \sum_{i=1}^N y_i$. Thus, the posterior mean is

$$\begin{aligned} E[p|\mathbf{y}] &= \frac{\alpha_n}{\alpha_n + \beta_n} \\ &= \frac{\sum_{i=1}^N y_i + \alpha_0}{\alpha_0 + \beta_0 + N} \quad (1.10) \\ &= \frac{N\bar{y}}{\alpha_0 + \beta_0 + N} + \frac{\alpha_0}{\alpha_0 + \beta_0 + N} \\ &= \frac{N}{\alpha_0 + \beta_0 + N} (\bar{y}) + \frac{\alpha_0 + \beta_0}{\alpha_0 + \beta_0 + N} \left(\frac{\alpha_0}{\alpha_0 + \beta_0} \right) \\ &= w(\bar{y}) + (1-w)E[p], \end{aligned}$$

where $w = \frac{N}{\alpha_0 + \beta_0 + N}$, $\bar{y}$ is the sample mean, and $E[p] = \frac{\alpha_0}{\alpha_0 + \beta_0}$ is the prior mean.

The posterior predictive distribution of claim the next year is given by

$$\begin{aligned}
\pi(Y_0 = 1|\mathbf{y}) &= \int_0^1 P(Y_0 = 1|\mathbf{y}, p) \pi(p|\mathbf{y}) dp \\
&= \int_0^1 p \times \pi(p|\mathbf{y}) dp \\
&= \mathbb{E}[p|\mathbf{y}] \\
&= \frac{\alpha_n}{\alpha_n + \beta_n}.
\end{aligned} \tag{1.11}$$

To use empirical Bayes, we have the following setting

$$[\hat{\alpha}_0 \ \hat{\beta}_0] = \arg \max_{\alpha_0, \beta_0} \ln(p(\mathbf{y})),$$

where

$$\begin{aligned}
p(\mathbf{y}) &= \int_0^1 \left[\frac{\Gamma(\alpha_0 + \beta_0)}{\Gamma(\alpha_0) \Gamma(\beta_0)} p^{\alpha_0-1} (1-p)^{\beta_0-1} \right] \left[\prod_{i=1}^N (1-p)^{1-y_i} \right] dp \tag{1.12} \\
&= \frac{\Gamma(\alpha_0 + \beta_0)}{\Gamma(\alpha_0) \Gamma(\beta_0)} \int_0^1 p^{\sum_{i=1}^N y_i + \alpha_0 - 1} (1-p)^{\beta_0 + N - \sum_{i=1}^N y_i - 1} dp \\
&= \frac{\Gamma(\alpha_0 + \beta_0)}{\Gamma(\alpha_0) \Gamma(\beta_0)} \frac{\Gamma\left(\sum_{i=1}^N y_i + \alpha_0\right) \Gamma\left(\beta_0 + N - \sum_{i=1}^N y_i\right)}{\Gamma(\alpha_0 + \beta_0 + N)}.
\end{aligned}$$

R code. Car claim, predictive distribution using vague hyperparameters

```

1 set.seed(010101)
2 y <- c(0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0)
3 # Data
4 N <- length(y)
5 #require(TailRank)
6 # Predictive distribution
7 ProbBo <- function(y, a0, b0){
8   N <- length(y)
9   #sample size
10  aN <- a0 + sum(y)
11  # Posterior shape parameter
12  bN <- b0 + N - sum(y)
13  # Posterior scale parameter
14  pr <- aN / (aN + bN)
15  # Probability of a claim the next year
16  return(pr)
17 }
18 # Using a vague prior:
19 a0 <- 1 # Prior shape parameter
20 b0 <- 1 # Prior scale parameter
21 PriMeanV <- a0 / (a0 + b0)
22 # Prior mean
23 PriVarV <- (a0*b0) / (((a0+b0)^2)*(a0+b0+1))
24 # Prior variance
25 Pp <- ProbBo(y, a0 = 1, b0 = 1)
26 # This setting is defining vague prior information.
27 # The probability of a claim
28 Pp
29 0.47

```

R code. Car claim, predictive distribution using empirical Bayes

```

1 # Using Empirical Bayes
2 LogMgLik <- function(theta, y){
3   N <- length(y)
4   #sample size
5   a0 <- theta[1]
6   # prior shape hyperparameter
7   b0 <- theta[2]
8   # prior scale hyperparameter
9   aN <- sum(y) + a0
10  # posterior shape parameter
11  if(a0 <= 0 || b0 <= 0){
12    #Avoiding negative values
13    lnp <- -Inf
14  }else{lnp <- lgamma(a0+b0) + lgamma(aN) + lgamma(b0+N-sum(
15    y)) -lgamma(a0) - lgamma(b0) - lgamma(a0+b0+N)}
16  # log marginal likelihood
17  return(-lnp)
18 }
19 theta0 <- c(0.1, 0.1)
20 # Initial values
21 control <- list(maxit = 1000)
22 # Number of iterations in optimization
23 EmpBay <- optim(theta0, LogMgLik, method = "BFGS", control =
24   control, hessian = TRUE, y = y)
25 # Optimization
26 EmpBay$convergence
27 # Checking convergence
28 EmpBay$value # Maximum
29 a0EB <- EmpBay$par[1]
30 # Prior shape using empirical Bayes
31 b0EB <- EmpBay$par[2]
32 # Prior scale using empirical Bayes
33 PriMeanEB <- a0EB / (a0EB + b0EB)
34 # Prior mean
35 PriVarEB <- (a0EB*b0EB) / (((a0EB+b0EB)^2)*(a0EB+b0EB+1))
36 # Prior variance
37 PpEB <- ProbBo(y, a0 = a0EB, b0 = b0EB)
38 # This setting is using empirical Bayes.
39 PpEB
40 0.47

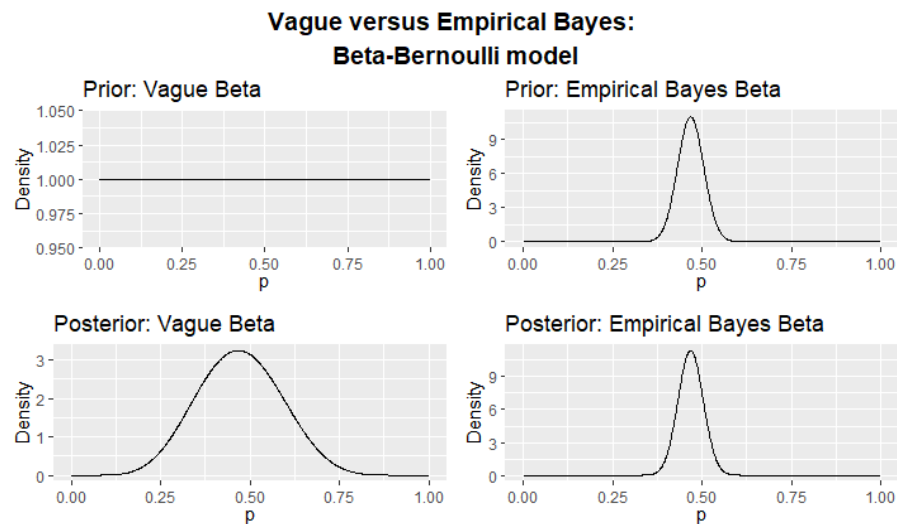
```

R code. Car claim, density plots

```

1 # Density figures
2 lambda <- seq(0.001, 1, 0.001)
3 # Values of lambda
4 VaguePrior <- dbeta(lambda, shape1 = a0, shape2 = b0)
5 EBPrior <- dbeta(lambda, shape1 = a0EB, shape2 = b0EB)
6 PosteriorV <- dbeta(lambda, shape1 = a0 + sum(y), shape2 =
  b0 + N - sum(y))
7 PosteriorEB <- dbeta(lambda, shape1 = a0EB + sum(y), shape2
  = b0EB + N - sum(y))
8 # Likelihood function
9 Likelihood <- function(theta, y){
10   LogL <- dbinom(y, 1, theta, log = TRUE)
11   # LogL <- dbern(y, theta)
12   Lik <- prod(exp(LogL))
13   return(Lik)
14 }
15 Liks <- sapply(lambda, function(par) {Likelihood(par, y = y)
  })
16 Sc <- max(PosteriorEB)/max(Liks)
17 #Scale for displaying in figure
18 LiksScale <- Liks * Sc
19 data <- data.frame(cbind(lambda, VaguePrior, EBPrior,
  PosteriorV, PosteriorEB, LiksScale))
20 #Data frame
21 require(ggplot2)
22 # Cool figures
23 require(latex2exp)
24 # LaTeX equations in figures
25 require(ggpubr)
26 # Multiple figures in one page
27 fig1 <- ggplot(data = data, aes(lambda, VaguePrior)) +
  geom_line() +
  xlab(TeX("$p$")) + ylab("Density") + ggtitle("Prior: Vague
  Beta")
28 fig2 <- ggplot(data = data, aes(lambda, EBPrior)) +
  geom_line() +
  xlab(TeX("$p$")) + ylab("Density") +
  ggtitle("Prior: Empirical Bayes Beta")
29 fig3 <- ggplot(data = data, aes(lambda, PosteriorV)) +
  geom_line() +
  xlab(TeX("$p$")) + ylab("Density") +
  ggtitle("Posterior: Vague Beta")
30 fig4 <- ggplot(data = data, aes(lambda, PosteriorEB)) +
  geom_line() +
  xlab(TeX("$p$")) + ylab("Density") +
  ggtitle("Posterior: Empirical Bayes Beta")
31 FIG <- ggarrange(fig1, fig2, fig3, fig4,
32 ncol = 2, nrow = 2)
33 annotate_figure(FIG,
34 top = text_grob("Vague versus Empirical Bayes: Beta-
  Bernoulli model", color = "black", face = "bold", size =
  14))

```

**FIGURE 1.1**

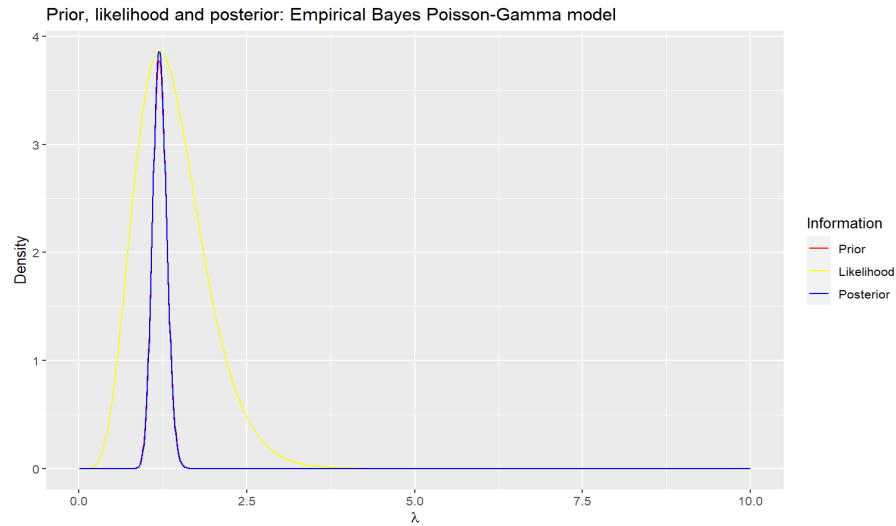
Vague versus Empirical Bayes: Bernoulli-Beta model.

*R code. Car claim, prior, likelihood and posterior
density plots*

```

1 # Prior, likelihood and posterior:
2 #Empirical Bayes Binomial-Beta model
3 dataNew <- data.frame(cbind(rep(lambda, 3),
4 c(EBPrior, PosteriorEB, LikScale),
5 rep(1:3, each = 1000)))
6 #Data frame
7
8 colnames(dataNew) <- c("Lambda", "Density", "Factor")
9 dataNew$Factor <- factor(dataNew$Factor, levels=c("1", "3",
10 "2"), labels=c("Prior", "Likelihood", "Posterior"))
11
12 ggplot(data = dataNew, aes_string(x = "Lambda",
13 y = "Density", group = "Factor")) +
14 geom_line(aes(color = Factor)) +
15 xlab(TeX("$\\lambda$")) + ylab("Density") +
16 ggtitle("Prior, likelihood and posterior: Empirical Bayes
17 Poisson-Gamma model") +
18 guides(color=guide_legend(title="Information")) +
19 scale_color_manual(values = c("red", "yellow", "blue"))

```

**FIGURE 1.2**

Prior, likelihood and posterior: Bernoulli-Beta model.

R code. Car claim, predictive probabilities plots

```

1 # Predictive distributions
2 require(TailRank)
3 PredDen <- function(y, y0, a0, b0){
4   N <- length(y)
5   aN <- a0 + sum(y) # Posterior shape parameter
6   bN <- b0 + N - sum(y) # Posterior scale parameter
7   Pr <- aN/(aN+bN)
8   Probs <- dbinom(y0, 1, prob = Pr)
9   return(Probs)
10 }
11 y0 <- 0:1
12 PredVague <- PredDen(y = y, y0 = y0, a0 = a0, b0 = b0)
13 PredEB <- PredDen(y = y, y0 = y0, a0 = a0EB, b0 = b0EB)
14 dataPred <- as.data.frame(cbind(y0, PredVague, PredEB))
15 colnames(dataPred) <- c("y0", "PredictiveVague",
16   "PredictiveEB")
17 ggplot(data = dataPred) +
18   geom_point(aes(y0, PredictiveVague, color = "red")) +
19   xlab(TeX("$y_0$")) + ylab("Density") +
20   ggtitle("Predictive density: Vague and Empirical Bayes
21     priors") + geom_point(aes(y0, PredictiveEB, color = "
22     yellow")) +
23   guides(color = guide_legend(title="Prior")) +
24   scale_color_manual(labels = c("Vague", "Empirical Bayes"),
25     values = c("red", "yellow")) +
26   scale_x_continuous(breaks=seq(0,1,by=1))

```

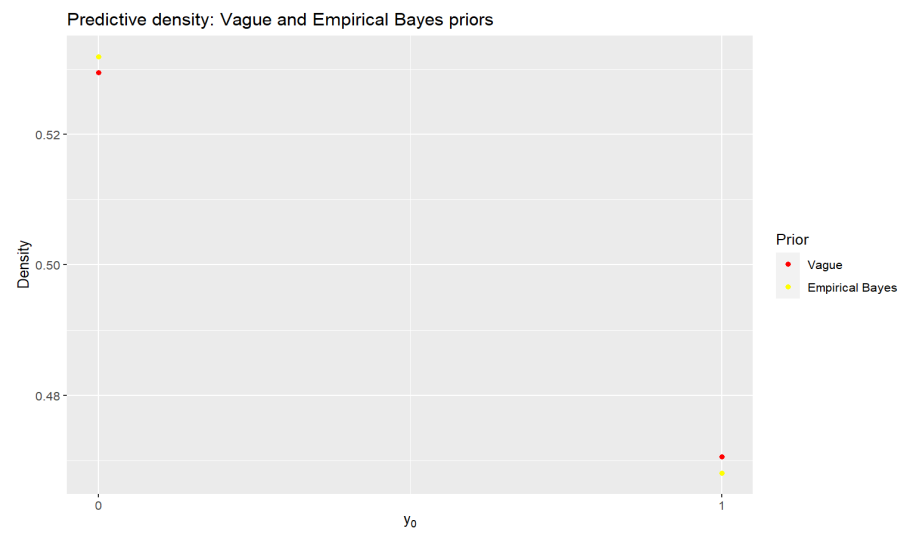



FIGURE 1.3
Predictive probabilities: Bernoulli-Beta model.

R code. Car claim, Bayesian model average

```

1 # Posterior odds: Vague vs Empirical Bayes
2 P012 <- exp(-LogMgLik(c(a0EB, b0EB), y = y))/exp(-LogMgLik(c
  (a0, b0), y = y))
3 PostProMEM <- P012/(1 + P012)
4 # Posterior model probability Empirical Bayes
5 PostProMEM
6 0.757
7 PostProbMV <- 1 - PostProMEM
8 # Posterior model probability vague prior
9 PostProbMV
10 0.242
11 # Bayesian model average (BMA)
12 PostMeanEB <- (a0EB + sum(y)) / (a0EB + b0EB + N)
13 # Posterior mean Empirical Bayes
14 PostMeanV <- (a0 + sum(y)) / (a0 + b0 + N)
15 # Posterior mean vague priors
16 BMAMean <- PostProMEM * PostMeanEB + PostProbMV * PostMeanV
17 # BMA posterior mean
18 PostVarEB <- (a0EB + sum(y))*(b0EB + N - sum(y)) / ((a0EB +
  b0EB + N)^2)*(a0EB + b0EB + N - 1)
19 # Posterior variance Empirical Bayes
20 PostVarV <- (a0 + sum(y))*(b0 + N - sum(y)) / ((a0 + b0 + N)
  ^2)*(a0 + b0 + N - 1)
21 # Posterior variance vague prior
22 BMAVar <- PostProMEM * PostVarEB + PostProbMV * PostVarV +
  PostProMEM * (PostMeanEB - BMAMean)^2 + PostProbMV * (
  PostMeanV - BMAMean)^2
23 # BMA posterior variance
24 # BMA: Predictive
25 BMAPred <- PostProMEM * PredEB + PostProbMV * PredVague
26 dataPredBMA <- as.data.frame(cbind(y0, BMAPred))
27 colnames(dataPredBMA) <- c("y0", "PredictiveBMA")
28 ggplot(data = dataPredBMA) +
29   geom_point(aes(y0, PredictiveBMA, color = "red")) +
30   xlab(TeX("$y_0$")) + ylab("Density") +
31   ggtitle("Predictive density: BMA") +
32   guides(color = guide_legend(title="BMA")) +
33   scale_color_manual(labels = c("Probability"), values = c("
  red")) + scale_x_continuous(breaks=seq(0,1,by=1))

```

R code. Car claim, Bayesian updating plots

```

1 # Bayesian updating
2 BayUp <- function(y, lambda, a0, b0){
3   N <- length(y)
4   aN <- a0 + sum(y)
5   # Posterior shape parameter
6   bN <- b0 + N - sum(y)
7   # Posterior scale parameter
8   p <- dbeta(lambda, shape1 = aN, shape2 = bN)
9   # Posterior density
10  return(list(Post = p, a0New = aN, b0New = bN))
11 }
12 PostUp <- NULL
13 for(i in 1:N){
14   if(i == 1){
15     PostUpi <- BayUp(y[i], lambda, a0 = 1, b0 = 1)}
16   else{
17     PostUpi <- BayUp(y[i], lambda,
18       a0 = PostUpi$a0New, b0 = PostUpi$b0New)
19   }
20   PostUp <- cbind(PostUp, PostUpi$Post)
21 }
22 DataUp <- data.frame(cbind(rep(lambda, 15), c(PostUp), rep
23   (1:15, each = 1000))) #Data frame
24 colnames(DataUp) <- c("Lambda", "Density", "Factor")
25 DataUp$Factor <- factor(DataUp$Factor, levels=c("1","2","3",
26   "4","5","6","7","8","9","10","11","12","13","14","15"),
27   labels=c("Iter_1","Iter_2","Iter_3","Iter_4","Iter_5",
28     "Iter_6","Iter_7","Iter_8","Iter_9","Iter_10","Iter_11",
29     "Iter_12","Iter_13","Iter_14","Iter_15"))
30 ggplot(data = DataUp, aes_string(x = "Lambda",
31   y = "Density", group = "Factor")) +
32   geom_line(aes(color = Factor)) +
33   xlab(TeX("$p$")) + ylab("Density") +
34   ggtitle("Bayesian updating:
35   Beta-Binomial model with vague prior") +
36   guides(color=guide_legend(title="Update"))

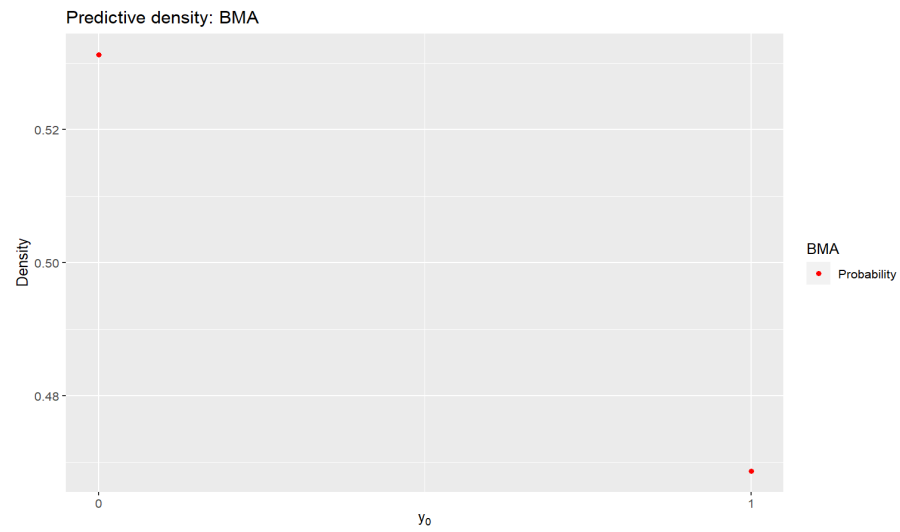
```

5. Show that given the loss function, $L(\theta, a) = |\theta - a|$, then the optimal decision rule minimizing the risk function, $a^*(\mathbf{y})$, is the median.

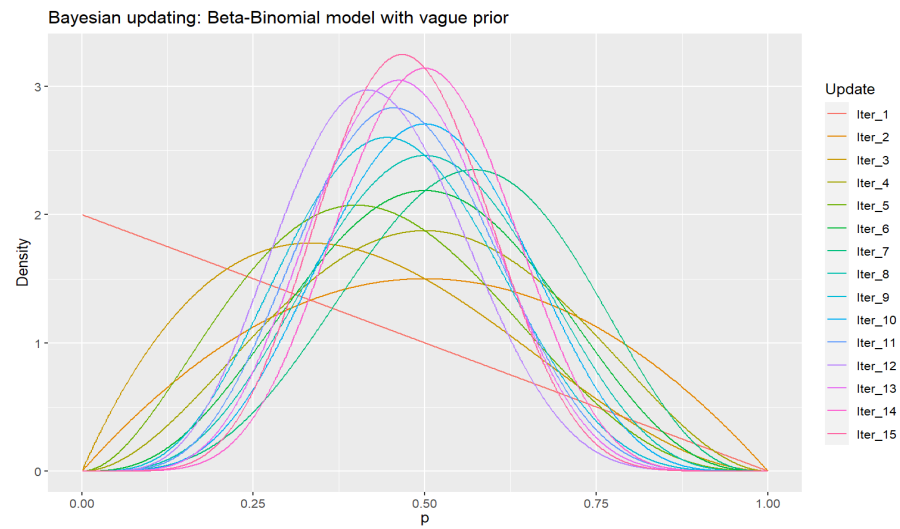
Answer

$\int_{\Theta} |\theta - a| \pi(\theta|\mathbf{y}) d\theta = \int_{-\infty}^a (a - \theta) \pi(\theta|\mathbf{y}) d\theta + \int_a^{\infty} (\theta - a) \pi(\theta|\mathbf{y}) d\theta$. Differentiating with respect to a , and equating to zero,

$$\int_{-\infty}^a \pi(\theta|\mathbf{y}) d\theta = \int_a^{\infty} \pi(\theta|\mathbf{y}) d\theta, \quad (1.13)$$

**FIGURE 1.4**

Predictive probabilities: Bernoulli-Beta Bayesian model average.

**FIGURE 1.5**

Predictive probabilities: Bernoulli-Beta Bayesian model updating.

then,

$$2 \int_{-\infty}^a \pi(\theta|\mathbf{y}) d\theta = \int_{-\infty}^{\infty} \pi(\theta|\mathbf{y}) d\theta = 1, \quad (1.14)$$

that is, $a^*(\mathbf{y})$ is the median.



2

Conceptual differences of the Bayesian and Frequentist approaches

Solutions of Exercises

1. Jeffreys-Lindley paradox

The **Jeffreys-Lindley paradox** [19, 28] is an apparent disagreement between the Bayesian and Frequentist frameworks to a hypothesis testing situation.

In particular, assume that in a city 49,581 boys and 48,870 girls have been born in 20 years. Assume that the male births is distributed Binomial with probability θ . We want to test the null hypothesis H_0 . $\theta = 0.5$ versus H_1 . $\theta \neq 0.5$.

- Show that the posterior model probability for the model under the null is approximately 0.95. Assume $\pi(H_0) = \pi(H_1) = 0.5$, and $\pi(\theta)$ equal to $U(0, 1)$ under H_1 .
- Show that the p -value for this hypothesis test is equal to 0.023 using the normal approximation, $Y \sim N(N \times \theta, N \times \theta \times (1 - \theta))$.

Answer

- The marginal likelihood under the null hypothesis is $p(y|H_0) = \binom{N}{y} \theta^y (1 - \theta)^{N-y} \approx 1.95 \times 10^{-4}$ given $\theta = 0.5$ under H_0 , $N = 49,581 + 48,870$ and $y = 49,581$. On the other hand, the marginal likelihood under the alternative hypothesis is

$$\begin{aligned}
p(y|H_1) &= \int_0^1 \binom{N}{y} \theta^y (1-\theta)^{N-y} d\theta \\
&= \binom{N}{y} B(y+1, N-k+1) \\
&= \frac{\Gamma(N+1)}{\Gamma(y+1)\Gamma(N-y+1)} \frac{\Gamma(y+1)\Gamma(N-y+1)}{\Gamma(N+2)} \\
&= \frac{N!}{(N+1)!} \\
&= \frac{1}{N+1} \\
&\approx 1.016 \times 10^{-5}.
\end{aligned}$$

Then, $PO_{01} = \frac{1.95 \times 10^{-4}}{1.016 \times 10^{-5}} = 19.19$, this implies that the posterior model probability under the null hypothesis is $\pi(H_0|y) = \frac{19.19}{1+19.19} = 0.95$.

- Under the null hypothesis,

$$\begin{aligned}
p &= 2 \int_{49,581}^{\infty} (2\pi\sigma^2)^{-1/2} \exp\left\{-\frac{1}{2\sigma^2}(y-\mu)^2\right\} dy \\
&= 0.0235,
\end{aligned}$$

where $\mu = N \times \theta = 49,225.5$, and $\sigma^2 = N \times \theta \times (1-\theta) = 24,612.75$ under the null hypothesis ($\theta = 0.5$).

Observe that the posterior model probability supports the null hypothesis, whereas the p-value implies rejection of the null hypothesis using a 5% significance level.

Observe that actually this is not a paradox, as we are answering two different questions. The Bayes factor is comparing two models ($\theta = 0.5$ versus $\theta \sim U(0,1)$), whereas the p-value is checking the compatibility between $\theta = 0.5$ and the sample information. Despite that $\theta = 0.5$ is not compatible with sample information, it is better than the models assuming $\theta \sim U(0,1)$ as most of these values of θ are far away from the sample mean. Thus, the model under the null is a bad description of the data, but it is better than the model under the alternative hypothesis.¹

¹Observe that there are at least another two issues in this example. First, the prior under the alternative is non-informative, this implies problems for Bayes factors, and second, the prior under the alternative is positive at $\theta = 0.5$, which is the null ([22] propose non-local prior densities in Bayesian hypothesis tests to tackle these issues).

2. We want to test $H_0: \mu = \mu_0$ vs $H_1: \mu \neq \mu_0$ given $y_i \stackrel{iid}{\sim} N(\mu, \sigma^2)$.

Assume $\pi(H_0) = \pi(H_1) = 0.5$, and $\pi(\mu, \sigma) \propto 1/\sigma$ under the alternative hypothesis.

Show that

$$p(\mathbf{y}|\mathcal{M}_1) = \frac{\pi^{-N/2}}{2} \Gamma(N/2) \left(\frac{1}{\alpha_n \hat{\sigma}^2} \right)^{N/2} \left(\frac{N}{\alpha_n \hat{\sigma}^2} \right)^{-1/2} \frac{\Gamma(1/2) \Gamma(\alpha_n/2)}{\Gamma((\alpha_n+1)/2)} \text{ and } p(\mathbf{y}|\mathcal{M}_0) = (2\pi)^{-N/2} \left[\frac{2}{\Gamma(N/2)} \left(\frac{N}{2} \frac{\sum_{i=1}^N (y_i - \mu_0)^2}{N} \right)^{N/2} \right]^{-1}. \text{ Then,}$$

$$\begin{aligned} PO_{01} &= \frac{p(\mathbf{y}|\mathcal{M}_0)}{p(\mathbf{y}|\mathcal{M}_1)} \\ &= \frac{\Gamma((\alpha_n + 1)/2)}{\Gamma(1/2) \Gamma(\alpha_n/2)} (\alpha_n \hat{\sigma}^2 / N)^{-1/2} \left[1 + \frac{(\mu_0 - \bar{y})^2}{\alpha_n \hat{\sigma}^2 / N} \right]^{-\left(\frac{\alpha_n + 1}{2}\right)}, \end{aligned}$$

where $\alpha_N = N - 1$ and $\hat{\sigma}^2 = \frac{\sum_{i=1}^N (y_i - \bar{y})^2}{N - 1}$.

Find the relationship between the posterior odds and the classical test statistic for the null hypothesis.

Answer

$$\begin{aligned} p(\mathbf{y}|\mathcal{M}_1) &= \int_{-\infty}^{\infty} \int_0^{\infty} (2\pi)^{-N/2} \sigma^{-N} \exp \left\{ -\frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \mu)^2 \right\} \frac{1}{\sigma} d\sigma d\mu \\ &= (2\pi)^{-N/2} \int_{-\infty}^{\infty} \int_0^{\infty} \sigma^{-(N+1)} \exp \left\{ -\frac{N}{2\sigma^2} \frac{\sum_{i=1}^N (y_i - \mu)^2}{N} \right\} d\sigma d\mu \\ &= (2\pi)^{-N/2} \frac{\Gamma(N/2)}{2} 2^{N/2} \int_{-\infty}^{\infty} \left[\sum_{i=1}^N (y_i - \mu)^2 \right]^{-N/2} d\mu \\ &= (2\pi)^{-N/2} \frac{\Gamma(N/2)}{2} 2^{N/2} \int_{-\infty}^{\infty} \left[\sum_{i=1}^N [(y_i - \bar{y}) - (\mu - \bar{y})]^2 \right]^{-N/2} d\mu \\ &= (2\pi)^{-N/2} \frac{\Gamma(N/2)}{2} 2^{N/2} \int_{-\infty}^{\infty} [\alpha_n \hat{\sigma}^2 + N(\mu - \bar{y})^2]^{-N/2} d\mu \\ &= (2\pi)^{-N/2} \frac{\Gamma(N/2)}{2} 2^{N/2} \left(\frac{\alpha_n \hat{\sigma}^2}{\alpha_n \hat{\sigma}^2} \right)^{-N/2} \int_{-\infty}^{\infty} [\alpha_n \hat{\sigma}^2 + N(\mu - \bar{y})^2]^{-N/2} d\mu \\ &= (2\pi)^{-N/2} \frac{\Gamma(N/2)}{2} 2^{N/2} (\alpha_n \hat{\sigma}^2)^{-N/2} \int_{-\infty}^{\infty} \left[1 + \frac{N(\mu - \bar{y})^2}{\alpha_n \hat{\sigma}^2} \right]^{-N/2} d\mu \\ &= \frac{\pi^{-N/2}}{2} \Gamma(N/2) \left(\frac{1}{\alpha_n \hat{\sigma}^2} \right)^{N/2} \left(\frac{N}{\alpha_n \hat{\sigma}^2} \right)^{-1/2} \frac{\Gamma(1/2) \Gamma(\alpha_n/2)}{\Gamma((\alpha_n + 1)/2)}. \end{aligned}$$

The third line takes into account that the integral in the second line is the kernel of an inverted-gamma distribution, and the last line takes into account that the integral in the previous line is the kernel of a student's t distribution [54].

$$\begin{aligned}
p(\mathbf{y}|\mathcal{M}_0) &= \int_0^\infty (2\pi)^{-N/2} \sigma^{-N} \exp \left\{ -\frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \mu_0)^2 \right\} \frac{1}{\sigma} d\sigma \\
&= (2\pi)^{-N/2} \int_0^\infty \sigma^{-(N+1)} \exp \left\{ -\frac{N}{2\sigma^2} \frac{\sum_{i=1}^N (y_i - \mu_0)^2}{N} \right\} d\sigma \\
&= (2\pi)^{-N/2} \left[\frac{2}{\Gamma(N/2)} \left(\frac{N}{2} \frac{\sum_{i=1}^N (y_i - \mu_0)^2}{N} \right)^{N/2} \right]^{-1}.
\end{aligned}$$

The third line takes into account that the integral in the second line is the kernel of an inverted-gamma distribution [54].

Given these results is easy to get PO_{01} .

In addition,

$$\begin{aligned}
PO_{01} &= \frac{\Gamma((\alpha_n + 1)/2)}{\Gamma(1/2)\Gamma(\alpha_n/2)} (\alpha_n \hat{\sigma}^2 / N)^{-1/2} \left[1 + \frac{(\mu_0 - \bar{y})^2}{\alpha_n \hat{\sigma}^2 / N} \right]^{-\left(\frac{\alpha_n + 1}{2}\right)} \\
&= \frac{\Gamma((\alpha_n + 1)/2)}{\Gamma(1/2)\Gamma(\alpha_n/2)} (\alpha_n \hat{\sigma}^2 / N)^{-1/2} \left[1 + \frac{1}{\alpha_n} \left(\frac{\mu_0 - \bar{y}}{\hat{\sigma} / \sqrt{N}} \right)^2 \right]^{-\left(\frac{\alpha_n + 1}{2}\right)} \\
&= \frac{\Gamma((\alpha_n + 1)/2)}{\Gamma(1/2)\Gamma(\alpha_n/2)} (\alpha_n \hat{\sigma}^2 / N)^{-1/2} \left[1 + \frac{1}{\alpha_n} t^2 \right]^{-\left(\frac{\alpha_n + 1}{2}\right)},
\end{aligned}$$

where $t = \frac{\bar{y} - \mu_0}{\hat{\sigma} / \sqrt{N}}$ is the classical statistical test. Then, as t increases then the PO_{01} decreases, both indicating support against the null hypothesis H_0 . $\mu = \mu_0$. However, there are other terms affecting the posterior odds, then, there is no necessary agreement between the classical test statistic and the posterior odds.

3. Math test continues

Using the setting of the **Example: Math test** in subsection 2.6.1 in the book, test H_0 . $\mu = \mu_0$ vs H_1 . $\mu \neq \mu_0$ where $\mu_0 = \{100, 100.5, 101, 101.5, 102\}$.

- What is the p -value for these hypothesis tests?
- Find the posterior model probability of the null model for each μ_0 .

Answer

R code. Example: Math test

```

1 N <- 50 # Sample size
2 y_bar <- 102 # Sample mean
3 s2 <- 10 # Sample variance
4 alpha <- N - 1
5 serror <- (s2/N)^0.5
6 y.H0 <- c(100, 100.5, 101, 101.5, 102)
7 test <- (y.H0 - y_bar)/serror
8 pval <- 2*pt(test, alpha)
9 pval
10 0.0000459 0.0015431 0.0299338 0.2690040 1
11 # p-values
12 P001 <- (gamma(N/2)*((N-1)*serror^2)^(-0.5)*(1+test^2/alpha)
13         ^(-N/2))/(gamma(1/2)*gamma((N-1)/2))
14 P001/(1+P001)
15 0.0001705 0.0050345 0.0725330 0.3210223 0.4702050
16 # Posterior model probability of the null hypothesis.

```



3

Cornerstone models: Conjugate families

Solutions of Exercises

1. Write in the canonical form the distribution of the Bernoulli example, and find the mean and variance of the sufficient statistic.

Answer

Given $p(\mathbf{y}|\theta) = (1-\theta)^N \exp \left\{ \sum_{i=1}^N y_i \log \left(\frac{\theta}{1-\theta} \right) \right\}$ where $\eta = \log \frac{\theta}{1-\theta}$ which implies $\theta = \frac{\exp(\eta)}{1+\exp(\eta)}$, then $p(\mathbf{y}|\theta) = \exp \left\{ \sum_{i=1}^N y_i \eta - N \log(1 + \exp(\eta)) \right\}$. Thus $B(\eta) = N \log(1 + \exp(\eta))$, $\nabla(B(\eta)) = N \frac{\exp(\eta)}{1+\exp(\eta)} = N\theta$ and $\nabla^2(B(\eta)) = N \left\{ \frac{\exp(\eta)(1+\exp(\eta))}{(1+\exp(\eta))^2} - \frac{\exp(\eta)\exp(\eta)}{(1+\exp(\eta))^2} \right\} = N\theta(1-\theta)$.

2. Given a random sample $\mathbf{y} = [y_1, y_2, \dots, y_N]^\top$ from N binomial experiments each having known size n_i and same unknown probability θ . Show that $p(\mathbf{y}|\theta)$ is in the exponential family, and find the posterior distribution, the marginal likelihood and the predictive distribution of the binomial-beta model assuming the number of trials is known.

Answer

The density function is

$$\begin{aligned} p(\mathbf{y}|\theta) &= \prod_{i=1}^N \binom{n_i}{y_i} \theta^{y_i} (1-\theta)^{n_i-y_i} \\ &= \prod_{i=1}^N \binom{n_i}{y_i} \theta^{\sum_{i=1}^N y_i} (1-\theta)^{\sum_{i=1}^N n_i - \sum_{i=1}^N y_i} \\ &= \prod_{i=1}^N \binom{n_i}{y_i} \exp \left\{ \sum_{i=1}^N y_i \log \left(\frac{\theta}{1-\theta} \right) + \sum_{i=1}^N n_i \log(1-\theta) \right\} \\ &= \prod_{i=1}^N \binom{n_i}{y_i} (1-\theta)^{\sum_{i=1}^N n_i} \exp \left\{ \sum_{i=1}^N y_i \log \left(\frac{\theta}{1-\theta} \right) \right\}, \end{aligned}$$

Observe that $\sum_{i=1}^N n_i$ is the total sample size of Bernoulli experiments.

Using Theorem 1 in Chapter 4, the prior distribution is

$$\begin{aligned}\pi(\theta) &\propto (1-\theta)^{B_0} \exp \left\{ a_0 \log \left(\frac{\theta}{1-\theta} \right) \right\} \\ &= \theta^{a_0} (1-\theta)^{B_0-a_0} \\ &= \theta^{\alpha_0-1} (1-\theta)^{\beta_0-1},\end{aligned}$$

where $\alpha_0 = a_0 + 1$ and $\beta_0 = B_0 - a_0 + 1$. This is the kernel of a beta distribution. Thus, the posterior distribution is

$$\begin{aligned}\pi(\theta|\mathbf{y}) &\propto \theta^{\alpha_0-1} (1-\theta)^{\beta_0-1} \times \theta^{\sum_{i=1}^N y_i} (1-\theta)^{\sum_{i=1}^N n_i - \sum_{i=1}^N y_i} \\ &= \theta^{\alpha_0 + \sum_{i=1}^N y_i - 1} (1-\theta)^{\beta_0 + \sum_{i=1}^N n_i - \sum_{i=1}^N y_i - 1} \\ &= \theta^{\alpha_n - 1} (1-\theta)^{\beta_n - 1},\end{aligned}$$

where $\alpha_n = \alpha_0 + \sum_{i=1}^N y_i$ and $\beta_n = \beta_0 + \sum_{i=1}^N n_i - \sum_{i=1}^N y_i$.

The marginal likelihood is

$$\begin{aligned}p(\mathbf{y}) &= \int_0^1 \frac{\theta^{\alpha_0-1} (1-\theta)^{\beta_0-1}}{B(\alpha_0, \beta_0)} \times \prod_{i=1}^N \binom{n_i}{y_i} \theta^{\sum_{i=1}^N y_i} (1-\theta)^{\sum_{i=1}^N n_i - \sum_{i=1}^N y_i} d\theta \\ &= \frac{\prod_{i=1}^N \binom{n_i}{y_i}}{B(\alpha_0, \beta_0)} \int_0^1 \theta^{\alpha_0 + \sum_{i=1}^N y_i - 1} (1-\theta)^{\beta_0 + \sum_{i=1}^N n_i - \sum_{i=1}^N y_i - 1} d\theta \\ &= \frac{\prod_{i=1}^N \binom{n_i}{y_i} B(\alpha_n, \beta_n)}{B(\alpha_0, \beta_0)}.\end{aligned}$$

The third line due to having the kernel of a Beta distribution.

Finally, the predictive distribution is

$$\begin{aligned}p(Y_0|\mathbf{y}) &= \int_0^1 \binom{n_{y_0}}{y_0} \theta^{y_0} (1-\theta)^{n_{y_0}-y_0} \frac{\theta^{\alpha_n-1} (1-\theta)^{\beta_n-1}}{B(\alpha_n, \beta_n)} d\theta \\ &= \frac{\binom{n_{y_0}}{y_0}}{B(\alpha_n, \beta_n)} \int_0^1 \theta^{\alpha_n + y_0 - 1} (1-\theta)^{\beta_n + n_{y_0} - y_0 - 1} d\theta \\ &= \binom{n_{y_0}}{y_0} \frac{B(\alpha_n + y_0, \beta_n + n_{y_0} - y_0)}{B(\alpha_n, \beta_n)},\end{aligned}$$

where n_{y_0} is the known size associated with y_0 , and the last line due to having the kernel of a beta distribution. The predictive is a *beta-binomial distribution*.

3. Given a random sample $\mathbf{y} = [y_1, y_2, \dots, y_N]^\top$ from a *exponential distribution*. Show that $p(\mathbf{y}|\lambda)$ is in the exponential family, and find the posterior distribution, marginal likelihood and predictive distribution of the exponential-gamma model.

Answer

We see that the exponential distribution belongs to the exponential family as $p(\mathbf{y}|\lambda) = \prod_{i=1}^N \lambda \exp(-\lambda y_i) = \lambda^N \exp(-\lambda \sum_{i=1}^N y_i)$.

Using the gamma distribution in the rate parametrization, we see that $\pi(\lambda|\mathbf{y}) \propto \lambda^{\alpha_0-1} \exp(-\lambda\beta_0) \times \lambda^N \exp(-\lambda \sum_{i=1}^N y_i) = \lambda^{\alpha_0+N-1} \exp(-\lambda(\beta_0 + \sum_{i=1}^N y_i))$. This is the kernel of a gamma distribution, that is, $\lambda|\mathbf{y} \sim G(\alpha_n, \beta_n)$ where $\alpha_n = \alpha_0 + N$ and $\beta_n = \beta_0 + \sum_{i=1}^N y_i$.

The marginal likelihood is

$$\begin{aligned} p(\mathbf{y}) &= \int_0^\infty \lambda^N \exp\left\{-\lambda \sum_{i=1}^N y_i\right\} \lambda^{\alpha_0-1} \exp\{-\beta_0\lambda\} \frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)} d\lambda \\ &= \frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)} \int_0^\infty \lambda^{\alpha_0+N-1} \exp\left\{-\lambda \left(\beta_0 + \sum_{i=1}^N y_i\right)\right\} d\lambda \\ &= \frac{\beta_0^{\alpha_0} \Gamma(\alpha_n)}{\Gamma(\alpha_0) \beta_n^{\alpha_n}}. \end{aligned}$$

Finally, the predictive distribution is

$$\begin{aligned} p(Y_0|\mathbf{y}) &= \int_0^\infty \lambda \exp\{-\lambda y_0\} \lambda^{\alpha_n-1} \exp\{-\beta_n\lambda\} \frac{\beta_n^{\alpha_n}}{\Gamma(\alpha_n)} d\lambda \\ &= \frac{\beta_n^{\alpha_n}}{\Gamma(\alpha_n)} \int_0^\infty \lambda^{\alpha_n+1-1} \exp\{-\lambda(\beta_n + y_0)\} d\lambda \\ &= \frac{\beta_n^{\alpha_n}}{\Gamma(\alpha_n)} \times \frac{\Gamma(\alpha_n + 1)}{(\beta_n + y_0)^{\alpha_n+1}} \\ &= \frac{\alpha_n \beta_n^{\alpha_n}}{(\beta_n + y_0)^{\alpha_n+1}}. \end{aligned}$$

This is a *Lomax distribution*.

4. Given $\mathbf{y} \sim N_N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, that is, a *multivariate normal distribution* show that $p(\mathbf{y}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is in the exponential family.

Answer

$$\begin{aligned}
p(\mathbf{y}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) &= (2\pi)^{-N/2} |\boldsymbol{\Sigma}|^{-1/2} \exp \left\{ -\frac{1}{2} (\mathbf{y} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{y} - \boldsymbol{\mu}) \right\} \\
&= (2\pi)^{-N/2} \exp \left\{ -\frac{1}{2} (\mathbf{y}^\top \boldsymbol{\Sigma}^{-1} \mathbf{y} - 2\mathbf{y}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu} + \boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu} + \log(|\boldsymbol{\Sigma}|)) \right\} \\
&= (2\pi)^{-N/2} \exp \left\{ -\frac{1}{2} (\text{tr} \{ \mathbf{y}^\top \boldsymbol{\Sigma}^{-1} \mathbf{y} \} - 2\mathbf{y}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu} + \boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu} + \log(|\boldsymbol{\Sigma}|)) \right\} \\
&= (2\pi)^{-N/2} \exp \left\{ -\frac{1}{2} \left(\text{vec}(\mathbf{y}\mathbf{y}^\top)^\top \text{vec}(\boldsymbol{\Sigma}^{-1}) - 2\mathbf{y}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu} + \boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu} + \log(|\boldsymbol{\Sigma}|) \right) \right\},
\end{aligned}$$

where tr and vec are the trace and vectorization operators, respectively.

Then, $h(\mathbf{y}) = (2\pi)^{-N/2}$, $\eta(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = [\boldsymbol{\Sigma}^{-1} \boldsymbol{\mu} \text{vec}(\boldsymbol{\Sigma}^{-1})]$, $T(\mathbf{y}) = [\mathbf{y} \text{ } -\frac{1}{2} \text{vec}(\mathbf{y}\mathbf{y}^\top)]$ and $C(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \exp \left\{ -\frac{1}{2N} (\boldsymbol{\mu}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu} + \log(|\boldsymbol{\Sigma}|)) \right\}$.

5. Find the marginal likelihood in the normal/inverse-Wishart model.

Answer

$$\begin{aligned}
p(\mathbf{Y}) &= \int_{\mathcal{R}^p} \int_{\mathcal{S}} (2\pi)^{-pN/2} |\boldsymbol{\Sigma}|^{-N/2} \exp \left\{ -\frac{1}{2} \text{tr}[(\mathbf{S} + N(\boldsymbol{\mu} - \hat{\boldsymbol{\mu}})(\boldsymbol{\mu} - \hat{\boldsymbol{\mu}})^\top) \boldsymbol{\Sigma}^{-1}] \right\} \\
&\quad \times (2\pi)^{-p/2} \beta_0^{p/2} |\boldsymbol{\Sigma}|^{-1/2} \exp \left\{ -\frac{\beta_0}{2} \text{tr}[(\boldsymbol{\mu} - \boldsymbol{\mu}_0)(\boldsymbol{\mu} - \boldsymbol{\mu}_0)^\top \boldsymbol{\Sigma}^{-1}] \right\} \\
&\quad \times |\boldsymbol{\Sigma}|^{-(\alpha_0 + p + 1)/2} \frac{2^{-\alpha_0 p/2} |\boldsymbol{\Psi}_0|^{\alpha_0/2}}{\Gamma_p(\alpha_0/2)} \exp \left\{ -\frac{1}{2} \text{tr}(\boldsymbol{\Psi}_0 \boldsymbol{\Sigma}^{-1}) \right\} d\boldsymbol{\Sigma} d\boldsymbol{\mu} \\
&= \frac{(2\pi)^{-\frac{1}{2}(pN + p)} |\boldsymbol{\Psi}_0|^{\alpha_0/2} \beta_0^{p/2} 2^{-\alpha_0 p/2}}{\Gamma_p(\alpha_0/2)} \int_{\mathcal{R}^p} \int_{\mathcal{S}} |\boldsymbol{\Sigma}|^{-\frac{1}{2}(N + 1 + \alpha_0 + p + 1)} \\
&\quad \times \exp \left\{ -\frac{1}{2} \text{tr}[(\mathbf{S} + N(\boldsymbol{\mu} - \hat{\boldsymbol{\mu}})(\boldsymbol{\mu} - \hat{\boldsymbol{\mu}})^\top + \beta_0(\boldsymbol{\mu} - \boldsymbol{\mu}_0)(\boldsymbol{\mu} - \boldsymbol{\mu}_0)^\top + \boldsymbol{\Psi}_0) \boldsymbol{\Sigma}^{-1}] \right\} d\boldsymbol{\Sigma} d\boldsymbol{\mu}.
\end{aligned}$$

We have in the integral the kernel of an Inverse-Wishart distribution, then

$$\begin{aligned}
p(\mathbf{Y}) &= \frac{\Gamma_p\left(\frac{N+1+\alpha_0}{2}\right) |\Psi_0|^{\alpha_0/2} \beta_0^{p/2}}{\Gamma_p(\alpha_0/2) \pi^{p(N+1)/2}} \\
&\quad \times \int_{\mathcal{R}^p} |\mathbf{S} + \Psi_0 + (N + \beta_0)(\boldsymbol{\mu} - \boldsymbol{\mu}_n)(\boldsymbol{\mu} - \boldsymbol{\mu}_n)^\top \\
&\quad + N\beta_0/(N + \beta_0)(\hat{\boldsymbol{\mu}} - \boldsymbol{\mu}_0)(\hat{\boldsymbol{\mu}} - \boldsymbol{\mu}_0)^\top|^{-(\alpha_n+1)/2} d\boldsymbol{\mu} \\
&= \frac{\Gamma_p\left(\frac{N+1+\alpha_0}{2}\right) |\Psi_0|^{\alpha_0/2} \beta_0^{p/2}}{\Gamma_p(\alpha_0/2) \pi^{p(N+1)/2}} \\
&\quad \times \int_{\mathcal{R}^p} (|\Psi_n| |1 + \beta_n(\boldsymbol{\mu} - \boldsymbol{\mu}_n) \Psi_n^{-1}(\boldsymbol{\mu} - \boldsymbol{\mu}_n)^\top|)^{-\frac{1}{2}(\alpha_n+1)} d\boldsymbol{\mu} \\
&= \frac{\Gamma_p\left(\frac{\alpha_n+1}{2}\right) |\Psi_0|^{\alpha_0/2} \beta_0^{p/2}}{\Gamma_p(\alpha_0/2) \pi^{p(N+1)/2}} |\Psi_n|^{-\frac{1}{2}(\alpha_n+1)} \\
&\quad \times \int_{\mathcal{R}^p} [1 + \beta_n(\boldsymbol{\mu} - \boldsymbol{\mu}_n)^\top \Psi_n^{-1}(\boldsymbol{\mu} - \boldsymbol{\mu}_n)]^{-\frac{1}{2}(\alpha_n+1)} d\boldsymbol{\mu}.
\end{aligned}$$

The last equality uses the definition of Ψ_n , β_n and α_n , and the Sylvester's determinant theorem. Observe that we have the kernel of a multivariate t distribution. Then,

$$\begin{aligned}
p(\mathbf{Y}) &= \frac{\Gamma_p\left(\frac{\alpha_n+1}{2}\right) |\Psi_0|^{\alpha_0/2} \beta_0^{p/2}}{\Gamma_p(\alpha_0/2) \pi^{p(N+1)/2}} |\Psi_n|^{-\frac{1}{2}(\alpha_n+1)} \\
&\quad \times \int_{\mathcal{R}^p} \left[1 + \frac{1}{\alpha_n + 1 - p} (\boldsymbol{\mu} - \boldsymbol{\mu}_n)^\top \left(\frac{\Psi_n}{\beta_n(\alpha_n + 1 - p)}\right)^{-1} (\boldsymbol{\mu} - \boldsymbol{\mu}_n)\right]^{-\frac{1}{2}(\alpha_n+1-p+p)} d\boldsymbol{\mu} \\
&= \frac{\Gamma_p\left(\frac{\alpha_n+1}{2}\right) \Gamma_p\left(\frac{\alpha_n+1-p}{2}\right) |\Psi_0|^{\alpha_0/2} \beta_0^{p/2} (\alpha_n + 1 - p)^{p/2} \pi^{p/2} |\Psi_n|^{-\frac{1}{2}(\alpha_n+1)}}{\Gamma_p(\alpha_0/2) \pi^{p(N+1)/2} \Gamma_p\left(\frac{\alpha_n+1-p+p}{2}\right) \left|\frac{\Psi_n}{\beta_n(\alpha_n+1-p)}\right|^{-1/2}} \\
&= \frac{\Gamma_p\left(\frac{v_n}{2}\right) |\Psi_0|^{\alpha_0/2}}{\Gamma_p\left(\frac{\alpha_0}{2}\right) |\Psi_n|^{\alpha_n/2}} \left(\frac{\beta_0}{\beta_n}\right)^{p/2} (\pi)^{-Np/2},
\end{aligned}$$

where $v_n = \alpha_n + 1 - p$.

6. Find the posterior predictive distribution in the normal/inverse-Wishart model, and show that $\mathbf{Y}_0|\mathbf{Y} \sim T_{N_0,M}(\alpha_n - M + 1, \mathbf{X}_0\mathbf{B}_n, \mathbf{I}_{N_0} + \mathbf{X}_0\mathbf{V}_n\mathbf{X}_0^\top, \Psi_n)$ in the multivariate regression linear model.

Answer

$$\begin{aligned}
p(\mathbf{Y}_0|\mathbf{Y}) &\propto \int_{\mathcal{R}^p} \int_S |\Sigma|^{-1/2} \exp \left\{ -\frac{1}{2} \text{tr}[(\mathbf{y}_0 - \boldsymbol{\mu})(\mathbf{y}_0 - \boldsymbol{\mu})^\top \Sigma^{-1}] \right\} \\
&\quad \times |\Sigma|^{-1/2} \exp \left\{ -\frac{\beta_n}{2} \text{tr}[(\boldsymbol{\mu} - \boldsymbol{\mu}_n)(\boldsymbol{\mu} - \boldsymbol{\mu}_n)^\top \Sigma^{-1}] \right\} \\
&\quad \times |\Sigma|^{-(\alpha_n + p + 1)/2} \exp \left\{ -\frac{1}{2} \text{tr}(\Psi_n \Sigma^{-1}) \right\} d\Sigma d\boldsymbol{\mu} \\
&\propto \int_{\mathcal{R}^p} |(\mathbf{y}_0 - \boldsymbol{\mu})(\mathbf{y}_0 - \boldsymbol{\mu})^\top + \beta_n(\boldsymbol{\mu} - \boldsymbol{\mu}_n)(\boldsymbol{\mu} - \boldsymbol{\mu}_n)^\top + \Psi_n|^{-(\alpha_n + 2)/2} d\boldsymbol{\mu}.
\end{aligned}$$

The last equality uses that there is the kernel of an Inverse Wishart distribution.

Taking into account that

$$\begin{aligned}
(\mathbf{y}_0 - \boldsymbol{\mu})(\mathbf{y}_0 - \boldsymbol{\mu})^\top + \beta_n(\boldsymbol{\mu} - \boldsymbol{\mu}_n)(\boldsymbol{\mu} - \boldsymbol{\mu}_n)^\top &= (1 + \beta_n) \left(\boldsymbol{\mu} - \frac{(\mathbf{y}_0 + \beta_n \boldsymbol{\mu}_n)}{1 + \beta_n} \right) \left(\boldsymbol{\mu} - \frac{(\mathbf{y}_0 + \beta_n \boldsymbol{\mu}_n)}{1 + \beta_n} \right)^\top \\
&\quad + \frac{\beta_n}{1 + \beta_n} (\mathbf{y}_0 - \boldsymbol{\mu}_n)(\mathbf{y}_0 - \boldsymbol{\mu}_n)^\top.
\end{aligned}$$

Then,

$$\begin{aligned}
p(\mathbf{Y}_0|\mathbf{Y}) &\propto \int_{\mathcal{R}^p} |(\mathbf{y}_0 - \boldsymbol{\mu})(\mathbf{y}_0 - \boldsymbol{\mu})^\top + \beta_n(\boldsymbol{\mu} - \boldsymbol{\mu}_n)(\boldsymbol{\mu} - \boldsymbol{\mu}_n)^\top + \Psi_n|^{-(\alpha_n + 2)/2} d\boldsymbol{\mu} \\
&= \int_{\mathcal{R}^p} \left| (1 + \beta_n) \left(\boldsymbol{\mu} - \frac{(\mathbf{y}_0 + \beta_n \boldsymbol{\mu}_n)}{1 + \beta_n} \right) \left(\boldsymbol{\mu} - \frac{(\mathbf{y}_0 + \beta_n \boldsymbol{\mu}_n)}{1 + \beta_n} \right)^\top \right. \\
&\quad \left. + \frac{\beta_n}{1 + \beta_n} (\mathbf{y}_0 - \boldsymbol{\mu}_n)(\mathbf{y}_0 - \boldsymbol{\mu}_n)^\top + \Psi_n \right|^{-(\alpha_n + 2)/2} d\boldsymbol{\mu} \\
&= \int_{\mathcal{R}^p} \left| \underbrace{\Psi_n + \frac{\beta_n}{1 + \beta_n} (\mathbf{y}_0 - \boldsymbol{\mu}_n)(\mathbf{y}_0 - \boldsymbol{\mu}_n)^\top}_{\Lambda_n} \right| \\
&\quad \left| 1 + (1 + \beta_n) \left(\boldsymbol{\mu} - \frac{(\mathbf{y}_0 + \beta_n \boldsymbol{\mu}_n)}{1 + \beta_n} \right)^\top \frac{1}{\alpha_n + 2 - p} \left(\frac{\Lambda_n}{\alpha_n + 2 - p} \right)^{-1} \left(\boldsymbol{\mu} - \frac{(\mathbf{y}_0 + \beta_n \boldsymbol{\mu}_n)}{1 + \beta_n} \right) \right|^{-(\alpha_n + 2 - p + p)/2} d\boldsymbol{\mu} \\
&\propto \left| \Psi_n + \frac{\beta_n}{1 + \beta_n} (\mathbf{y}_0 - \boldsymbol{\mu}_n)(\mathbf{y}_0 - \boldsymbol{\mu}_n)^\top \right|^{-(\alpha_n + 2)/2} \\
&\quad \times \left| \Psi_n + \frac{\beta_n}{1 + \beta_n} (\mathbf{y}_0 - \boldsymbol{\mu}_n)(\mathbf{y}_0 - \boldsymbol{\mu}_n)^\top \right|^{1/2} \\
&= \left| \Psi_n + \frac{\beta_n}{1 + \beta_n} (\mathbf{y}_0 - \boldsymbol{\mu}_n)(\mathbf{y}_0 - \boldsymbol{\mu}_n)^\top \right|^{-(\alpha_n + 1)/2} \\
&\propto \left[1 + (\mathbf{y}_0 - \boldsymbol{\mu}_n)^\top \frac{1}{\alpha_n + 1 - p} \left(\frac{\Psi_n(1 + \beta_n)}{(\alpha_n + 1 - p)\beta_n} \right)^{-1} (\mathbf{y}_0 - \boldsymbol{\mu}_n) \right]^{-(\alpha_n + 1 - p + p)}.
\end{aligned}$$

The second equality uses the Sylvester's determinant theorem, and that there is the kernel of a multivariate t distribution.

Then, we have that the predictive distribution is a multivariate t distribution centered at $\boldsymbol{\mu}_n$, $\alpha_n + 1 - p$ degrees of freedom, and scale matrix $\frac{\boldsymbol{\Psi}_n(1+\beta_n)}{(\alpha_n+1-p)\beta_n}$.

To show the second statement, let's start by the definition of the predictive density to show that $\mathbf{Y}_0|\mathbf{Y} \sim T_{N_0,M}(\alpha_n - M + 1, \mathbf{X}_0\mathbf{B}_n, \mathbf{I}_{N_0} + \mathbf{X}_0\mathbf{V}_n\mathbf{X}_0^\top, \boldsymbol{\Psi}_n)$.

$$\begin{aligned} \pi(\mathbf{Y}_0|\mathbf{Y}) &\propto \int_{\mathcal{S}} \int_{\mathcal{B}} \left\{ |\boldsymbol{\Sigma}|^{-N_0/2} \exp \left\{ -\frac{1}{2} \text{tr}[(\mathbf{Y}_0 - \mathbf{X}_0\mathbf{B})^\top (\mathbf{Y}_0 - \mathbf{X}_0\mathbf{B}) \boldsymbol{\Sigma}^{-1}] \right\} \right. \\ &\quad \times |\boldsymbol{\Sigma}|^{-K/2} \exp \left\{ -\frac{1}{2} \text{tr}[(\mathbf{B} - \mathbf{B}_n)^\top \mathbf{V}_n^{-1} (\mathbf{B} - \mathbf{B}_n) \boldsymbol{\Sigma}^{-1}] \right\} \\ &\quad \times |\boldsymbol{\Sigma}|^{-(\alpha_n+M+1)/2} \exp \left\{ -\frac{1}{2} \text{tr}[\boldsymbol{\Psi}_n \boldsymbol{\Sigma}^{-1}] \right\} \Big\} d\mathbf{B} d\boldsymbol{\Sigma} \\ &= \int_{\mathcal{S}} \int_{\mathcal{B}} \left\{ |\boldsymbol{\Sigma}|^{-(N_0+K+\alpha_n+M+1)/2} \exp \left\{ -\frac{1}{2} \text{tr} [((\mathbf{Y}_0 - \mathbf{X}_0\mathbf{B})^\top (\mathbf{Y}_0 - \mathbf{X}_0\mathbf{B}) \right. \right. \\ &\quad \left. \left. + (\mathbf{B} - \mathbf{B}_n)^\top \mathbf{V}_n^{-1} (\mathbf{B} - \mathbf{B}_n) + \boldsymbol{\Psi}_n) \boldsymbol{\Sigma}^{-1}] \right\} \right\} d\mathbf{B} d\boldsymbol{\Sigma}. \end{aligned}$$

Setting $\mathbf{M} = (\mathbf{X}_0^\top \mathbf{X}_0 + \mathbf{V}_n^{-1})$, and $\mathbf{B}_* = \mathbf{M}^{-1}(\mathbf{V}_n \mathbf{B}_n + \mathbf{X}_0^\top \mathbf{Y}_0)$, we have that $(\mathbf{B} - \mathbf{B}_*)^\top \mathbf{M} (\mathbf{B} - \mathbf{B}_*) + \mathbf{B}_n^\top \mathbf{V}_n^{-1} \mathbf{B}_n + \mathbf{Y}_0^\top \mathbf{Y}_0 - \mathbf{B}_*^\top \mathbf{M} \mathbf{B}_* = (\mathbf{Y}_0 - \mathbf{X}_0\mathbf{B})^\top (\mathbf{Y}_0 - \mathbf{X}_0\mathbf{B}) + (\mathbf{B} - \mathbf{B}_n)^\top \mathbf{V}_n^{-1} (\mathbf{B} - \mathbf{B}_n)$. Then,

$$\begin{aligned} \pi(\mathbf{Y}_0|\mathbf{Y}) &\propto \int_{\mathcal{S}} |\boldsymbol{\Sigma}|^{-(N_0+K+\alpha_n+M+1)/2} \\ &\quad \times \exp \left\{ -\frac{1}{2} \text{tr}[(\boldsymbol{\Psi}_n + \mathbf{B}_n^\top \mathbf{V}_n^{-1} \mathbf{B}_n + \mathbf{Y}_0^\top \mathbf{Y}_0 - \mathbf{B}_*^\top \mathbf{M} \mathbf{B}_*) \boldsymbol{\Sigma}^{-1}] \right\} \\ &\quad \times \int_{\mathcal{B}} \exp \left\{ -\frac{1}{2} \text{tr}[(\mathbf{B} - \mathbf{B}_*)^\top \mathbf{M} (\mathbf{B} - \mathbf{B}_*) \boldsymbol{\Sigma}^{-1}] \right\} d\mathbf{B} d\boldsymbol{\Sigma}. \end{aligned}$$

The latter is the kernel of a matrix normal distribution, thus

$$\begin{aligned} \pi(\mathbf{Y}_0|\mathbf{Y}) &\propto \int_{\mathcal{S}} |\boldsymbol{\Sigma}|^{-(N_0+\alpha_n+M+1)/2} \\ &\quad \times \exp \left\{ -\frac{1}{2} \text{tr}[(\boldsymbol{\Psi}_n + \mathbf{B}_n^\top \mathbf{V}_n^{-1} \mathbf{B}_n + \mathbf{Y}_0^\top \mathbf{Y}_0 - \mathbf{B}_*^\top \mathbf{M} \mathbf{B}_*) \boldsymbol{\Sigma}^{-1}] \right\} d\boldsymbol{\Sigma} \end{aligned}$$

This is the kernel of an inverse-Wishart distribution, then

$$\pi(\mathbf{Y}_0|\mathbf{Y}) \propto |\boldsymbol{\Psi}_n + \mathbf{B}_n^\top \mathbf{V}_n^{-1} \mathbf{B}_n + \mathbf{Y}_0^\top \mathbf{Y}_0 - \mathbf{B}_*^\top \mathbf{M} \mathbf{B}_*|^{-(N_0+\alpha_n)/2}.$$

Setting $\mathbf{C}^{-1} = \mathbf{I}_{N_0} + \mathbf{X}_0 \mathbf{V}_n \mathbf{X}_0^\top$ such that $\mathbf{C} = \mathbf{I}_{N_0} - \mathbf{X}_0 (\mathbf{X}_0^\top \mathbf{X}_0 + \mathbf{V}_n^{-1})^{-1} \mathbf{X}_0^\top$ (see footnote 4 in Chapter 4), then $\mathbf{B}_n^\top \mathbf{V}_n^{-1} \mathbf{B}_n + \mathbf{Y}_0^\top \mathbf{Y}_0 - \mathbf{B}_*^\top \mathbf{M} \mathbf{B}_* = (\mathbf{Y}_0 - \mathbf{X}_0 \mathbf{B}_n)^\top \mathbf{C} (\mathbf{Y}_0 - \mathbf{X}_0 \mathbf{B}_n)$. This is done following exactly same procedure as deducing the predictive distribution in the linear regression model in the book. Thus,

$$\begin{aligned} \pi(\mathbf{Y}_0 | \mathbf{Y}) &\propto |\boldsymbol{\Psi}_n + (\mathbf{Y}_0 - \mathbf{X}_0 \mathbf{B}_n)^\top \mathbf{C} (\mathbf{Y}_0 - \mathbf{X}_0 \mathbf{B}_n)|^{-(N_0 + \alpha_n)/2} \\ &\propto |\mathbf{I}_{N_0} + \mathbf{C} (\mathbf{Y}_0 - \mathbf{X}_0 \mathbf{B}_n) \boldsymbol{\Psi}^{-1} (\mathbf{Y}_0 - \mathbf{X}_0 \mathbf{B}_n)^\top|^{-(\alpha_n + 1 - M + N_0 + M - 1)/2}. \end{aligned}$$

The second proportionality follows from the Sylvester's theorem. Observe that this is the kernel of a matrix t distribution with $\alpha_n + 1 - M$ degrees of freedom, location $\mathbf{X}_0 \mathbf{B}_n$ and scale matrices $\boldsymbol{\Psi}_n$ and $\mathbf{C}^{-1} = \mathbf{I}_{N_0} + \mathbf{X}_0 \mathbf{V}_n \mathbf{X}_0^\top$.

7. Show that $\delta_n = \delta_0 + (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^\top (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) + (\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}_0)^\top ((\mathbf{X}^\top \mathbf{X})^{-1} + \mathbf{B}_0)^{-1} (\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}_0)$ in the linear regression model, and that $\boldsymbol{\Psi}_n = \boldsymbol{\Psi}_0 + \mathbf{S} + (\hat{\mathbf{B}} - \mathbf{B}_0)^\top \mathbf{V}_n (\hat{\mathbf{B}} - \mathbf{B}_0)$ in the linear multivariate regression model.

Answer

Taking into account that

$$\begin{aligned} \delta^* &= \delta_0 + \mathbf{y}^\top \mathbf{y} + \boldsymbol{\beta}_0^\top \mathbf{B}_0^{-1} \boldsymbol{\beta}_0 - \boldsymbol{\beta}_n^\top \mathbf{B}_n^{-1} \boldsymbol{\beta}_n \\ &= \delta_0 + \mathbf{y}^\top \mathbf{y} + \boldsymbol{\beta}_0^\top \mathbf{B}_0^{-1} \boldsymbol{\beta}_0 - (\mathbf{B}_0^{-1} \boldsymbol{\beta}_0 + \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}})^\top \mathbf{B}_n (\mathbf{B}_0^{-1} \boldsymbol{\beta}_0 + \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}}) \\ &= \delta_0 + \mathbf{y}^\top \mathbf{y} - \hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{X} \mathbf{B}_n \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}} - 2\hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{X} \mathbf{B}_n \mathbf{B}_0^{-1} \boldsymbol{\beta}_0 + \boldsymbol{\beta}_0^\top (\mathbf{B}_0^{-1} - \mathbf{B}_0^{-1} \mathbf{B}_n \mathbf{B}_0^{-1}) \boldsymbol{\beta}_0 \\ &\quad - \hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}} + \hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}} \\ &= \delta_0 + \mathbf{y}^\top \mathbf{y} - \hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}} + \hat{\boldsymbol{\beta}}^\top (\mathbf{X}^\top \mathbf{X} - \mathbf{X}^\top \mathbf{X} \mathbf{B}_n \mathbf{X}^\top \mathbf{X}) \hat{\boldsymbol{\beta}} \\ &\quad - 2\hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{X} \mathbf{B}_n \mathbf{B}_0^{-1} \boldsymbol{\beta}_0 + \boldsymbol{\beta}_0^\top (\mathbf{B}_0^{-1} - \mathbf{B}_0^{-1} \mathbf{B}_n \mathbf{B}_0^{-1}) \boldsymbol{\beta}_0. \end{aligned}$$

Observe that

$$\begin{aligned} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^\top (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) &= \mathbf{y}^\top \mathbf{y} - 2\hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{y} + \hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}} \\ &= \mathbf{y}^\top \mathbf{y} - 2\hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top (\mathbf{X}\hat{\boldsymbol{\beta}} + \hat{\boldsymbol{\mu}}) + \hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}} \\ &= \mathbf{y}^\top \mathbf{y} - \hat{\boldsymbol{\beta}}^\top \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}}, \end{aligned}$$

where $\mathbf{y} = \mathbf{X}\hat{\boldsymbol{\beta}} + \hat{\boldsymbol{\mu}}$, and $\mathbf{X}^\top \hat{\boldsymbol{\mu}} = 0$.

The following matrix identities are useful [48]:

$$(\mathbf{D} + \mathbf{E})^{-1} = \mathbf{D}^{-1} - \mathbf{D}^{-1} (\mathbf{D}^{-1} + \mathbf{E}^{-1})^{-1} \mathbf{D}^{-1},$$

and

$$(\mathbf{D} + \mathbf{E})^{-1} = \mathbf{D}^{-1} (\mathbf{E}^{-1} + \mathbf{D}^{-1}) \mathbf{E}^{-1}.$$

Using these identities,

$$\begin{aligned} [(\mathbf{X}^\top \mathbf{X})^{-1} + \mathbf{B}_0]^{-1} &= \mathbf{X}^\top \mathbf{X} - \mathbf{X}^\top \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \mathbf{B}_0^{-1})^{-1} \mathbf{X}^\top \mathbf{X} \\ &= \mathbf{B}_0^{-1} - \mathbf{B}_0^{-1}(\mathbf{X}^\top \mathbf{X} + \mathbf{B}_0^{-1})^{-1} \mathbf{B}_0^{-1} \\ &= \mathbf{X}^\top \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \mathbf{B}_0^{-1})^{-1} \mathbf{B}_0^{-1}. \end{aligned}$$

Then,

$$\begin{aligned} \delta^* &= \delta_0 + (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^\top (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) + \hat{\boldsymbol{\beta}}^\top [(\mathbf{X}^\top \mathbf{X})^{-1} + \mathbf{B}_0]^{-1} \hat{\boldsymbol{\beta}} \\ &\quad - 2\hat{\boldsymbol{\beta}}^\top [(\mathbf{X}^\top \mathbf{X})^{-1} + \mathbf{B}_0]^{-1} \boldsymbol{\beta}_0 + \boldsymbol{\beta}_0^\top [(\mathbf{X}^\top \mathbf{X})^{-1} + \mathbf{B}_0]^{-1} \boldsymbol{\beta}_0 \\ &= \delta_0 + (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^\top (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) \\ &\quad + (\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}_0)^\top [(\mathbf{X}^\top \mathbf{X})^{-1} + \mathbf{B}_0]^{-1} (\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}_0). \end{aligned}$$

In a similar way for the second part,

$$\begin{aligned} (\mathbf{V}_0 + (\mathbf{X}^\top \mathbf{X})^{-1})^{-1} &= \mathbf{V}_0^{-1} - \mathbf{V}_0^{-1}(\mathbf{V}_0^{-1} + \mathbf{X}^\top \mathbf{X})^{-1} \mathbf{V}_0^{-1} \\ &= \mathbf{X}^\top \mathbf{X} - \mathbf{X}^\top \mathbf{X}(\mathbf{V}_0^{-1} + \mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{X} \\ &= \mathbf{X}^\top \mathbf{X}((\mathbf{X}^\top \mathbf{X})^{-1} + \mathbf{V}_0)^{-1} \mathbf{V}_0^{-1}, \end{aligned}$$

we use these results and some algebra to show that $\mathbf{B}_0^\top \mathbf{V}_0^{-1} \mathbf{B}_0 + \hat{\mathbf{B}}^\top \mathbf{X}^\top \mathbf{X} \hat{\mathbf{B}} - \mathbf{B}_n^\top \mathbf{V}_n^{-1} \mathbf{B}_n = (\hat{\mathbf{B}} - \mathbf{B}_0)^\top \mathbf{V}_n (\hat{\mathbf{B}} - \mathbf{B}_0)$ taking into account that $\mathbf{V}_n = (\mathbf{V}_0^{-1} + \mathbf{X}^\top \mathbf{X})^{-1}$ and $\hat{\mathbf{B}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}$.

8. Show that in the linear regression model $\boldsymbol{\beta}_n^\top (\mathbf{B}_n^{-1} - \mathbf{B}_n^{-1} \mathbf{M}^{-1} \mathbf{B}_n^{-1}) \boldsymbol{\beta}_n = \boldsymbol{\beta}_{**}^\top \mathbf{C} \boldsymbol{\beta}_{**}$ and $\boldsymbol{\beta}_{**} = \mathbf{X}_0 \boldsymbol{\beta}_n$.

Answer

Taking into account that $(\mathbf{A} + \mathbf{B})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}(\mathbf{A}^{-1} + \mathbf{B}^{-1})^{-1} \mathbf{A}^{-1}$ [48], then we observe that $(\mathbf{B}_n^{-1} - \mathbf{B}_n^{-1} \mathbf{M}^{-1} \mathbf{B}_n^{-1}) = (\mathbf{B}_n + (\mathbf{X}_0^\top \mathbf{X}_0)^{-1})^{-1}$, where $(\mathbf{B}_n + (\mathbf{X}_0^\top \mathbf{X}_0)^{-1})^{-1} = \mathbf{X}_0^\top \mathbf{X}_0 - \mathbf{X}_0^\top \mathbf{X}_0 (\mathbf{B}_n^{-1} + \mathbf{X}_0^\top \mathbf{X}_0)^{-1} \mathbf{X}_0^\top \mathbf{X}_0 = \mathbf{X}_0^\top \mathbf{X}_0 - \mathbf{X}_0^\top \mathbf{X}_0 \mathbf{M}^{-1} \mathbf{X}_0^\top \mathbf{X}_0$, thus

$$\begin{aligned} \boldsymbol{\beta}_n^\top (\mathbf{B}_n^{-1} - \mathbf{B}_n^{-1} \mathbf{M}^{-1} \mathbf{B}_n^{-1}) \boldsymbol{\beta}_n &= \boldsymbol{\beta}_n^\top (\mathbf{X}_0^\top \mathbf{X}_0 - \mathbf{X}_0^\top \mathbf{X}_0 \mathbf{M}^{-1} \mathbf{X}_0^\top \mathbf{X}_0) \boldsymbol{\beta}_n \\ &= \boldsymbol{\beta}_n^\top \mathbf{X}_0^\top (\mathbf{I}_{N_0} - \mathbf{X}_0 \mathbf{M}^{-1} \mathbf{X}_0^\top) \mathbf{X}_0 \boldsymbol{\beta}_n \\ &= \boldsymbol{\beta}_n^\top \mathbf{X}_0^\top \mathbf{C} \mathbf{X}_0 \boldsymbol{\beta}_n \\ &= \boldsymbol{\beta}_{**}^\top \mathbf{C} \boldsymbol{\beta}_{**}. \end{aligned}$$

Let's show that $\boldsymbol{\beta}_{**} = \mathbf{X}_0 \boldsymbol{\beta}_n$,

$$\begin{aligned}
\beta_{**} &= \mathbf{C}^{-1} \mathbf{X}_0 \mathbf{M}^{-1} \mathbf{B}_n^{-1} \beta_n \\
&= (\mathbf{I}_{N_0} + \mathbf{X}_0 \mathbf{B}_n \mathbf{X}_0^\top) \mathbf{X}_0 \mathbf{M}^{-1} \mathbf{B}_n^{-1} \beta_n \\
&= (\mathbf{I}_{N_0} + \mathbf{X}_0 \mathbf{B}_n \mathbf{X}_0^\top) \mathbf{X}_0 (\mathbf{B}_n - \mathbf{B}_n ((\mathbf{X}_0^\top \mathbf{X}_0)^{-1} + \mathbf{B}_n)^{-1} \mathbf{B}_n) \mathbf{B}_n^{-1} \beta_n \\
&= (\mathbf{I}_{N_0} + \mathbf{X}_0 \mathbf{B}_n \mathbf{X}_0^\top) (\mathbf{X}_0 \beta_n - \mathbf{X}_0 \mathbf{B}_n ((\mathbf{X}_0^\top \mathbf{X}_0)^{-1} + \mathbf{B}_n)^{-1} \beta_n) \\
&= \mathbf{X}_0 \beta_n - \mathbf{X}_0 \mathbf{B}_n ((\mathbf{X}_0^\top \mathbf{X}_0)^{-1} + \mathbf{B}_n)^{-1} \beta_n + \mathbf{X}_0 \mathbf{B}_n \mathbf{X}_0^\top \mathbf{X}_0 \beta_n \\
&\quad - \mathbf{X}_0 \mathbf{B}_n \mathbf{X}_0^\top \mathbf{X}_0 \mathbf{B}_n ((\mathbf{X}_0^\top \mathbf{X}_0)^{-1} + \mathbf{B}_n)^{-1} \beta_n \\
&= \mathbf{X}_0 \beta_n - \mathbf{X}_0 \mathbf{B}_n [((\mathbf{X}_0^\top \mathbf{X}_0)^{-1} + \mathbf{B}_n)^{-1} - \mathbf{X}_0^\top \mathbf{X}_0 + \mathbf{X}_0^\top \mathbf{X}_0 \mathbf{B}_n ((\mathbf{X}_0^\top \mathbf{X}_0)^{-1} + \mathbf{B}_n)^{-1}] \beta_n.
\end{aligned}$$

Using that $(\mathbf{A} + \mathbf{B})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1} \mathbf{B} (\mathbf{A} + \mathbf{B})^{-1}$, we observe that the expression in brackets is equal to $\mathbf{0}$, then we have the result.

9. Show that $(\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}})^\top (\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}}) = \mathbf{S} + (\mathbf{B} - \hat{\mathbf{B}})^\top \mathbf{X}^\top \mathbf{X} (\mathbf{B} - \hat{\mathbf{B}})$ where $\mathbf{S} = (\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}})^\top (\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}})$, $\hat{\mathbf{B}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}$ in the multivariate regression model.

Answer

$$\begin{aligned}
(\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}})^\top (\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}}) &= (\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}} + \mathbf{X}\hat{\mathbf{B}} - \mathbf{X}\mathbf{B})^\top (\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}} + \mathbf{X}\hat{\mathbf{B}} - \mathbf{X}\mathbf{B}) \\
&= (\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}})^\top (\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}}) + 2(\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}})^\top (\mathbf{X}\hat{\mathbf{B}} - \mathbf{X}\mathbf{B}) \\
&\quad + (\mathbf{X}\mathbf{B} - \mathbf{X}\hat{\mathbf{B}})^\top (\mathbf{X}\mathbf{B} - \mathbf{X}\hat{\mathbf{B}}) \\
&= \mathbf{S} + (\mathbf{B} - \hat{\mathbf{B}})^\top \mathbf{X}^\top \mathbf{X} (\mathbf{B} - \hat{\mathbf{B}}),
\end{aligned}$$

given that $(\mathbf{Y} - \mathbf{X}\hat{\mathbf{B}})^\top (\mathbf{X}\hat{\mathbf{B}} - \mathbf{X}\mathbf{B}) = \hat{\mathbf{U}}^\top \mathbf{X} (\hat{\mathbf{B}} - \mathbf{B})$, using that $\hat{\mathbf{B}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}$ which implies $\mathbf{X}^\top \mathbf{X}\hat{\mathbf{B}} = \mathbf{X}^\top \mathbf{Y} = \mathbf{X}^\top \mathbf{X}\hat{\mathbf{B}} + \mathbf{X}^\top \hat{\mathbf{U}}$, then $\mathbf{X}^\top \hat{\mathbf{U}} = \mathbf{0}$.

10. What is the probability that the Sun will rise tomorrow?

This is the most famous Richard Price's example developed in the Appendix of the Bayes' theorem paper [4]. Here, we implicitly use *Laplace's Rule of Succession* to solve this question. In particular, if we were a priori uncertain about the probability the Sun will rise on a specified day, we can assume a prior uniform distribution over $(0,1)$, that is, a beta $(1,1)$ distribution. Then, what is the probability that the Sun will rise tomorrow?

Answer

This exercise is an application of the Bernoulli-beta model. Thus, the likelihood is given by a binomial distribution where the probability of success is θ , $p(\mathbf{y}|\theta) \propto \theta^{\sum_{i=1}^N y_i} (1-\theta)^{N-\sum_{i=1}^N y_i}$. In addition, the prior distribution is beta, that is, $\pi(\theta) \propto \theta^{\alpha_0-1} (1-\theta)^{\beta_0-1}$, where $\alpha_0 = \beta_0 = 1$. Then, the predictive distribution that the sun will rise tomorrow is $p(Y_0 = 1|\mathbf{y}) = \frac{1+S}{2+N}$, where $S = \sum_{i=1}^N y_i$ is the number of successes (the Sun rise). $\frac{1+S}{2+N}$ is known

as the *Laplace's Rule of Succession* that was introduced by Laplace in the 18th century in the course of treating the sunrise problem.

11. Using information from Public Policy Polling in September 27th-28th for the 2016 presidential five-way race in USA, there are 411, 373 and 149 sampled people supporting Hillary Clinton, Donald Trump and other, respectively.
 - Find the posterior probability of the percentage difference of people supporting Hillary versus Trump according to this data using a non-informative prior, that is, $\alpha_0 = [1 \ 1 \ 1]$ in the multinomial-Dirichlet model. What is the probability of having more supporters of Hillary vs Trump?
 - What is the probability that sampling one hundred independent individuals 44, 40 and 16 support Hillary, Trump and other, respectively?

Answer

R code. Multinomial-Dirichlet model: Polling 2016 USA presidential race

```

1 set.seed(010101)
2 # Multinomial-Dirichlet example:
3 # Polling 2016 USA presidential race
4 y <- c(411, 373, 149)
5 # Clinton, Trump, Other
6 # Public Policy Polling September 27-28,
7 # 2016 five-way race
8 alpha0 <- rep(1, 3)
9 # Hyperparameters: non-informative distribution
10 alphan <- alpha0 + y
11 S <- 100000
12 # Sample draws of posterior
13 thetas <- MCMCpack::rdirichlet(S, alphan)
14 colnames(thetas) <- c("Clinton", "Trump", "Other")
15 head(thetas)
16      Clinton      Trump      Other
17 [1,] 0.4211346 0.4188607 0.1600046
18 [2,] 0.4244207 0.4224523 0.1531270
19 [3,] 0.4349268 0.3843953 0.1806779
20 [4,] 0.4533499 0.4005530 0.1460972
21 [5,] 0.4381799 0.3968502 0.1649699
22 [6,] 0.4436852 0.3971321 0.1591827
23 dif <- thetas[,1] - thetas[,2]
24 # Difference of shares Hillary vs Trump
25 data <- data.frame(dif)
26 names(data) <- c("Difference")
27 library(ggplot2)
28 p <- ggplot(data) +
29   geom_histogram(aes(x = Difference), binwidth = 0.01) +
30   geom_vline(xintercept=0.0, lwd=1, colour="red") +
31   ggtitle("Percentage difference Clinton vs Trump 2016
32     presidential race") + xlab("Percentage Difference") +
33     ylab("")
34 difmcmc <- coda::mcmc(dif)
35 # Declaring a MCMC object
36 summary(difmcmc)
37
38 Iterations = 1:1e+05
39 Thinning interval = 1
40 Number of chains = 1
41 Sample size per chain = 1e+05
42
43 1. Empirical mean and standard deviation for each
44 variable, plus standard error of the mean:
45
46      Mean          SD      Naive SE Time-series SE
47 4.062e-02  2.996e-02  9.474e-05  9.474e-05
48
49 2. Quantiles for each variable:
50
51      2.5%      25%      50%      75%      97.5%
52 -0.01817  0.02033  0.04058  0.06089  0.09923
53
54 CW <- mean(difmcmc>0)
55 CW
56 0.91339

```

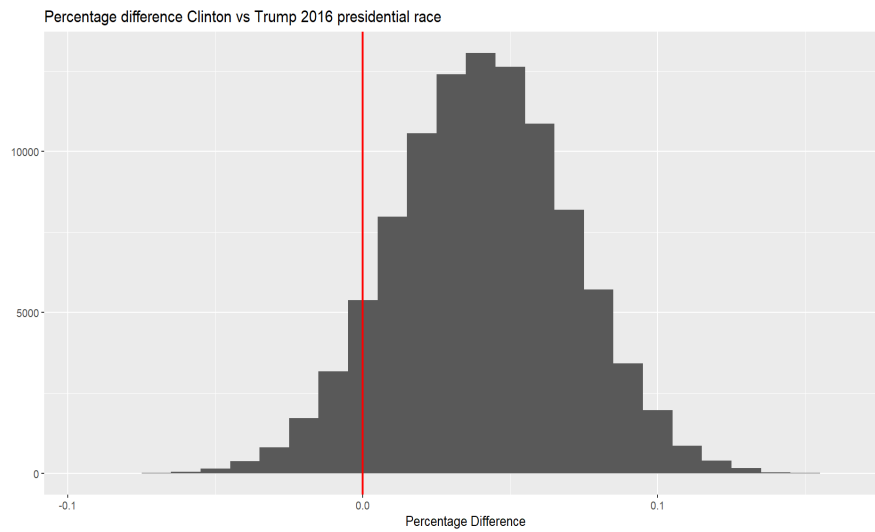



FIGURE 3.1

Percentage difference: Hillary Clinton vs Donald Trump, five-way race.

There is a 95% probability that the percentage difference between Hillary and Trump according to this poll is (-1.8%, 9.9%). The probability of Hillary having more supporters is 91.3%

***R code. Multinomial-Dirichlet model: Polling 2016
USA presidential race***

```

1 # Predictive distribution by simulation
2 y0 <- c(44, 40, 16)
3 Pred <- apply(thetas, 1, function(p) {rmultinom(1, size =
4   sum(y0), prob = p)})
5 sum(sapply(1:S, function(s) {sum(Pred[,s] == y0) == 3}))/S
6 # Predictive distribution by analytical expression
7 PredY0 <- function(y0){
8   n <- sum(y0)
9   Res1 <- sum(sapply(1:length(y), function(l){lgamma(alphan[
10     l]+y0[l]) - lgamma(alphan[l])-lfactorial(y0[l]))})
11   Res <- lfactorial(n)+lgamma(sum(alphan))-lgamma(sum(alphan
12     )+n) + Res1
13   return(exp(Res))
14 }
15 PredY0(y0)
16 0.00850

```

The probability that from one hundred random selected people 44 support Hillary, 40 support Trump and 16 support other candidate is 0.85%.

12. Math test example continues

You have a random sample of math scores of size $N = 50$ from a normal distribution, $Y_i \sim N(\mu, \sigma^2)$. The sample mean and variance are equal to 102 and 10, respectively. Using the normal-normal/inverse-gamma model where $\mu_0 = 100$, $\beta_0 = 1$, $\alpha_0 = \delta_0 = 0.001$

- Get 95% confidence and credible intervals for μ .
- What is the posterior probability that $\mu > 103$?

Answer

R code. Math test example continues

```

1 set.seed(010101)
2 N <- 50
3 # Sample size
4 muhat <- 102
5 # Sample mean
6 sig2hat <- 10
7 # Sample variance
8 # Hyperparameters
9 mu0 <- 100
10 beta0 <- 1
11 delta0 <- 0.001
12 alpha0 <- 0.001
13 S <- 100000
14 # Posterior draws
15 alphan <- alpha0 + N
16 deltan <- sig2hat*(N - 1) + delta0 + beta0*N/(beta0 + N)*(
    muhat - mu0)^2
17 sig2Post <- invgamma::rinvgamma(S, shape = alphan, rate =
    deltan)
18 summary(sig2Post)
19 betan <- beta0 + N
20 mun <- (beta0*mu0 + N*muhat)/betan
21 muPost_t <- ((deltan/(alphan*betan))^0.5)*rt(S, alphan) +
    mun
22 c1 <- rgb(173,216,230,max = 255, alpha = 50, names = "lt.
    blue")
23 c2 <- rgb(255,192,203, max = 255, alpha = 50, names = "lt.
    pink")
24 hist(muPost_t, main = "Histogram: Posterior mean", xlab = "
    Posterior mean", col = c1)
25 muPost_tq <- quantile(muPost_t, c(0.025, 0.5, 0.975))
26 muPost_tq
27 2.5%      50%      97.5%
28 101.0837 101.9608 102.8435
29 PmuPost_tcutoff <- mean(muPost_t > cutoff)
30 PmuPost_tcutoff
31 0.01087

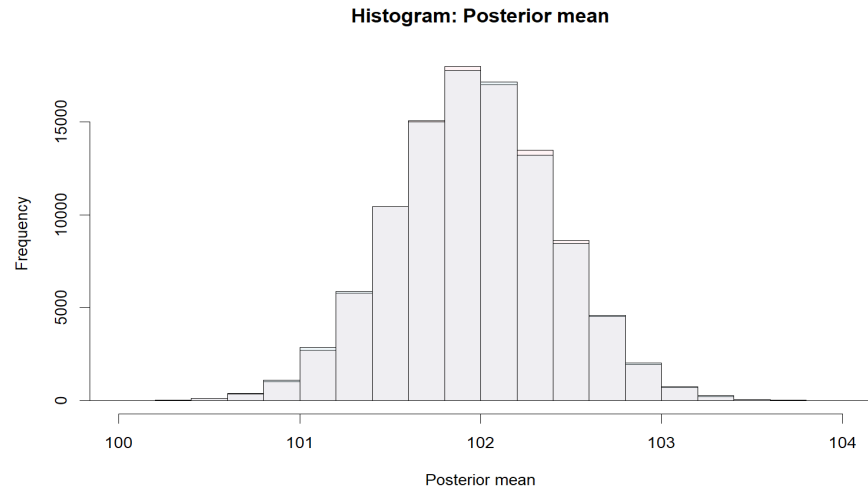
```

Figure 3.2 shows the posterior distribution of the mean.

We have that the 95% credible interval is (101.08, 102.84), and the probability of having a value greater than 103 is 1.09%.

13. Demand of electricity example continues

Set c_0 such that maximizes the marginal likelihood in the specifications with and without electricity price in the example of demand of electricity (empirical Bayes). Then, calculate the Bayes factor, and conclude if there is

**FIGURE 3.2**

Histogram of the posterior distribution of the mean

evidence supporting the inclusion of the price of electricity in the demand equation.

Answer

R code. Demand of electricity

```

1 rm(list = ls())
2 set.seed(010101)
3 # Electricity demand
4 DataUt <- read.csv("https://raw.githubusercontent.com/
    besmarter/BSTApp/refs/heads/master/DataApp/Utilities.csv",
    sep = ",", header = TRUE, quote = "")
5 DataUtEst <- DataUt %>%
6 filter(Electricity != 0)
7 attach(DataUtEst)
8 # Dependent variable: Monthly consumption (kWh) in log
9 Y <- log(Electricity)
10 N <- length(Y)
11 # Regressors quantity including intercept
12 X <- cbind(LnPriceElect, IndSocio1, IndSocio2, Altitude,
    Nrooms, HouseholdMem, Children, Lnincome, 1)
13 # Regressor without price
14 Xnew <- cbind(IndSocio1, IndSocio2, Altitude, Nrooms,
    HouseholdMem, Children, Lnincome, 1)
15 # Log marginal function (multiply by -1 due to minimization)
16 LogMarLikLM <- function(X, c0){
17   k <- dim(X)[2]
18   N <- dim(X)[1]
19   # Hyperparameters
20   B0 <- c0*diag(k)
21   b0 <- rep(0, k)
22   # Posterior parameters
23   bhat <- solve(t(X)%*%X)%*%t(X)%*%Y
24   # Force this matrix to be symmetric
25   Bn <- as.matrix(Matrix::forceSymmetric(solve(solve(B0) + t
    (X)%*%X)))
26   bn <- Bn%*(solve(B0)%*%b0 + t(X)%*%X)%*%bhat)
27   dn <- as.numeric(d0 + t(Y)%*%Y+t(b0)%*%solve(B0)%*%b0-t(bn
    )%*%solve(Bn)%*%bn)
28   an <- a0 + N
29   # Log marginal likelihood
30   logpy <- (N/2)*log(1/pi)+(a0/2)*log(d0)-(an/2)*log(dn) +
    0.5*log(det(Bn)/det(B0)) + lgamma(an/2)-lgamma(a0/2)
31   return(-logpy)
32 }
33 # Hyperparameters
34 d0 <- 0.001
35 a0 <- 0.001
36 # Empirical Bayes: Obtain c0 maximizing the log marginal
    likelihood
37 c0 <- 1000
38 EB <- optim(c0, fn = LogMarLikLM, method = "Brent", lower =
    0.0001, upper = 10^6, X = X)
39 EBnew <- optim(c0, fn = LogMarLikLM, method = "Brent", lower
    = 0.0001, upper = 10^6, X = Xnew)
40 # Change of order to take into account the -1 in the
    LogMarLikLM function
41 BFEM <- exp(EBnew$value - EB$value)
42 BFEM
43 71897025

```

The Bayes factor based on the empirical Bayes of the model with electricity price versus the model without electricity price is equal to 71897938, this gives very strong evidence to include the price in the specification.

14. Utility demand

Use the file *Utilities.csv* to estimate a multivariate linear regression model where $\mathbf{Y}_i = [\log(\text{electricity}_i) \log(\text{water}_i) \log(\text{gas}_i)]$ as function of $\log(\text{electricity price}_i)$, $\log(\text{water price}_i)$, $\log(\text{gas price}_i)$, IndSocio1_i , IndSocio2_i , Altitude_i , Nrooms_i , HouseholdMem_i , Children_i , and $\log(\text{Income}_i)$, where electricity, water and gas are monthly consumption of electricity (kWh), water (m^3) and gas (m^3), and other definitions are given in the Example of Section 4.3. Omit households that do not consume any of the utilities in this exercise.

Set a non-informative prior framework, $\mathbf{B}_0 = [0]_{11 \times 3}$, $\mathbf{V}_0 = 1000\mathbf{I}_{11}$, $\Psi_0 = 1000\mathbf{I}_3$ and $\alpha_0 = 3$, where we have $K = 11$ (regressors plus intercept) and $M = 3$ (equations) in this exercise.

- (a) Find the posterior mean estimates and the highest posterior density intervals at 95% of $\mathbf{B}$ and Σ .
- (b) Find the Bayes factor comparing the baseline model in this exercise with the same specification but using the income in dollars. Now, calculate the Bayes factor using the income in thousand dollars. Is there any difference?
- (c) Find the predictive distribution for the monthly demand of electricity, water and gas in the baseline specification of a household located in the lowest socioeconomic condition in a municipality located below 1000 meters above the sea level, 2 rooms, 3 members with children, a monthly income equal to USD 500, an electricity price equal to USD/kWh 0.15, a water price equal to USD/ M^3 0.70, and a gas price equal to USD/ M^3 0.75.

Answer

We find that the Bayes factor of the baseline model ($\log(\text{Income})$) versus the two alternative models using income in dollars and thousand dollars are 108925764 and 0.1089261. The former gives strong evidence in favor of the baseline model, whereas the latter gives positive evidence for the model using the income in thousand dollars. This result despite that the location coefficients are the same in the two alternative specifications, except for the change in scale of the coefficients associated with income. This example shows that Bayes factors are sensitive to units of measure, and consequently, it is relevant to think carefully about the priors when performing hypothesis testing using a Bayesian framework. Observe that a nice feature in Bayesian inference is that we followed the same conceptual framework (Bayes factor) in the previous exercise and this exercise. In one

hand, the previous exercise is an example of nested models, that is, one model is a restricted version of a more general model. On the other hand, this exercise is an example of non-nested models. This is not the case in the Frequentist approach. The statistical framework is not the same when testing nested and non-nested models.

*R code. Utilities demand: Multivariate regression,
posterior inference*

```

1 rm(list = ls())
2 set.seed(010101)
3 library(dplyr)
4 # Electricity demand
5 DataUt <- read.csv("https://raw.githubusercontent.com/
    besmarter/BSTApp/refs/heads/master/DataApp/Utilities.csv
    ", sep = ",", header = TRUE, quote = "")
6 DataUtEst <- DataUt %>% filter(Electricity != 0 & Water !=0
    & Gas != 0)
7 attach(DataUtEst)
8 Y <- cbind(log(Electricity), log(Water), log(Gas))
9 X <- cbind(LnPriceElect, LnPriceWater, LnPriceGas, IndSocio1
    , IndSocio2, Altitude, Nrooms, HouseholdMem, Children,
    Lnincome, 1)
10 M <- dim(Y)[2]
11 K <- dim(X)[2]
12 N <- dim(Y)[1]
13 # Hyperparameters
14 B0 <- matrix(0, K, M)
15 c0 <- 1000
16 V0 <- c0*diag(K)
17 Psi0 <- c0*diag(M)
18 a0 <- M
19 # Posterior parameters
20 Bhat <- solve(t(X)%*%X)%*%t(X)%*%Y
21 S <- t(Y - X%*%Bhat)%*%(Y - X%*%Bhat)
22 Vn <- solve(solve(V0) + t(X)%*%X)
23 Bn <- Vn%*(solve(V0)%*%B0 + t(X)%*%X%*%Bhat)
24 Psin <- Psi0 + S + t(B0)%*%solve(V0)%*%B0 + t(Bhat)%*%t(X)%*
    %X%*%Bhat - t(Bn)%*%solve(Vn)%*%Bn
25 an <- a0 + N
26 #Posterior draws
27 s <- 10000 #Number of posterior draws
28 SIGs <- replicate(s, LaplacesDemon::rinvwishart(an, Psin))
29 Bs <- sapply(1:s, function(s) {MixMatrix::rmatrixt(n = 1,
    mean=Bn, U = Vn,V = Psin, df = an + 1 - M)})
30 summary(coda::mcmc(t(Bs)))
31 SIGMs <- t(sapply(1:s, function(l) {gdata::lowerTriangle(
    SIGs[,l], diag=TRUE, byrow=FALSE)}))
32 summary(coda::mcmc(SIGMs))
33 hdiBs <- HDInterval::hdi(t(Bs), credMass = 0.95) # Highest
    posterior density credible interval
34 hdiBs
35 hdiSIG <- HDInterval::hdi(SIGMs, credMass = 0.95) # Highest
    posterior density credible interval
36 hdiSIG
37

```


*R code. Utilities demand: Multivariate regression,
Bayes factors*

```

1 # Log marginal function (multiply by -1 due to minimization)
2 LogMarLikLM <- function(X, c0){
3   c10 <- c0[1]; c20 <- c0[2]
4   k <- dim(X)[2]
5   N <- dim(X)[1]
6   # Hyperparameters
7   V0 <- c10*diag(k)
8   Psi0 <- c20*diag(M)
9   # Posterior parameters
10  Bhat <- solve(t(X)%*%X)%*%t(X)%*%Y
11  S <- t(Y - X%*%Bhat)%*%(Y - X%*%Bhat)
12  Vn <- solve(solve(V0) + t(X)%*%X)
13  Bn <- Vn%*(solve(V0)%*%B0 + t(X)%*%X)%*%Bhat)
14  Psin <- Psi0 + S + t(B0)%*%solve(V0)%*%B0 + t(Bhat)%*%t(X)
    %*%X)%*%Bhat - t(Bn)%*%solve(Vn)%*%Bn
15  # Log marginal likelihood
16  logpy <- (N*M/2)*log(1/pi)+(a0/2)*log(det(Psi0)) - (an/2)*
    log(det(Psin)) + (M/2)*(log(det(Vn)) - log(det(V0))) +
    lgamma(an/2)-lgamma(a0/2)
17  return(-logpy)
18 }
19 c0 <- rep(1000, 2)
20 LogML <- LogMarLikLM(X=X, c0 = c0)
21 # Using income in dollars as regressor
22 Xnew <- cbind(LnPriceElect, LnPriceWater, LnPriceGas,
    IndSocio1, IndSocio2, Altitude, Nrooms, HouseholdMem,
    Children, exp(Lnincome), 1)
23 LogMLnew <- LogMarLikLM(X=Xnew, c0 = c0)
24 # Bayes factor
25 BF12 <- exp(LogMLnew - LogML)
26 BF12
27 # Using income in thousand dollars as regressor
28 XnewT <- cbind(LnPriceElect, LnPriceWater, LnPriceGas,
    IndSocio1, IndSocio2, Altitude, Nrooms, HouseholdMem,
    Children, exp(Lnincome)/1000, 1)
29 LogMLnewT <- LogMarLikLM(X=XnewT, c0 = c0)
30 # Bayes factor
31 BF13 <- exp(LogMLnewT - LogML)
32 BF13
33

```

R code. Utilities demand: Multivariate regression, predictive distribution

```

1 # Predictive distribution
2 Xpred <- c(log(0.15), log(0.70), log(0.75), 1, 0, 0, 2, 3,
3           1, log(500), 1)
4 Mean <- Xpred%%Bn
5 Hn <- 1+t(Xpred)%%Vn%%Xpred
6 UtilDemand <- exp(replicate(s, MixMatrix::rmatrixt(n = 1,
7             mean=Mean, U = Hn, V = Psin, df = an + 1 - M)))
8 ElePred <- UtilDemand[1,1,]
9 WatPred <- UtilDemand[1,2,]
10 GasPred <- UtilDemand[1,3,]
11 data <- data.frame(cbind(ElePred, WatPred, GasPred)) #Data
12           frame
13 annotations1 <- data.frame(
14   x = round(quantile(data$ElePred, c(0.025, 0.5, 0.975)),1),
15   y = c(600, 1000, 600),
16   label = c("2.5%", "50%", "97.5%:")
17 )
18 annotations2 <- data.frame(
19   x = round(quantile(data$WatPred, c(0.025, 0.5, 0.975)),1),
20   y = c(600, 1000, 600),
21   label = c("2.5%", "50%", "97.5%:")
22 )
23 annotations3 <- data.frame(
24   x = round(quantile(data$GasPred, c(0.025, 0.5, 0.975)),1),
25   y = c(600, 1000, 600),
26   label = c("2.5%", "50%", "97.5%:")
27 )
28 require(ggplot2) # Cool figures
29 require(ggpubr) # Multiple figures in one page
30 require(latex2exp) # LaTeX equations in figures
31 fig1 <- ggplot(data = data, aes(ElePred)) + geom_histogram(
32   bins = 40, color = "#000000", fill = "#0099F8") + xlab(
33     "kWh") + ylab("Frequency") + ggtitle("Electricity") +
34     xlim(0, 1050) + geom_text(data = annotations1, aes(x = x,
35       y = y, label = paste(label, x)), size = 3, fontface = "
36     bold")
37 fig2 <- ggplot(data = data, aes(WatPred)) + geom_histogram(
38   bins = 40, color = "#000000", fill = "#0099F8") + xlab(
39     TeX("$M^3$")) + ylab("Frequency") + ggtitle("Water") +
40     xlim(0, 100) + geom_text(data = annotations2, aes(x = x,
41       y = y, label = paste(label, x)), size = 3, fontface = "
42     bold")
43 fig3 <- ggplot(data = data, aes(GasPred)) + geom_histogram(
44   bins = 40, color = "#000000", fill = "#0099F8") + xlab(
45     TeX("$M^3$")) + ylab("Frequency") + ggtitle("Gas") +
46     xlim(0, 80) + geom_text(data = annotations3, aes(x = x,
47       y = y, label = paste(label, x)), size = 3, fontface = "
48     bold")

```

Figures 3.3, 3.4 and 3.5 show the marginal predictive distributions of electricity, water and gas for the reference household. The median predictive values are kWh 168.8, M³ 12.3 and M³ 10.1, respectively. In addition, the 95% credible intervals are (27.7, 1028.9), (1.5, 98.7) and (1.5, 67.5) for electricity, water and gas.

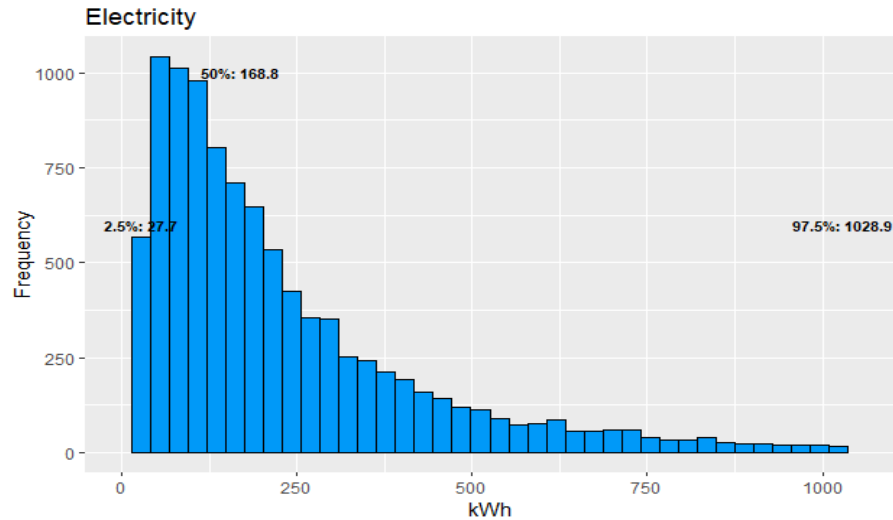


FIGURE 3.3

Histogram using the posterior predictive distribution of electricity demand

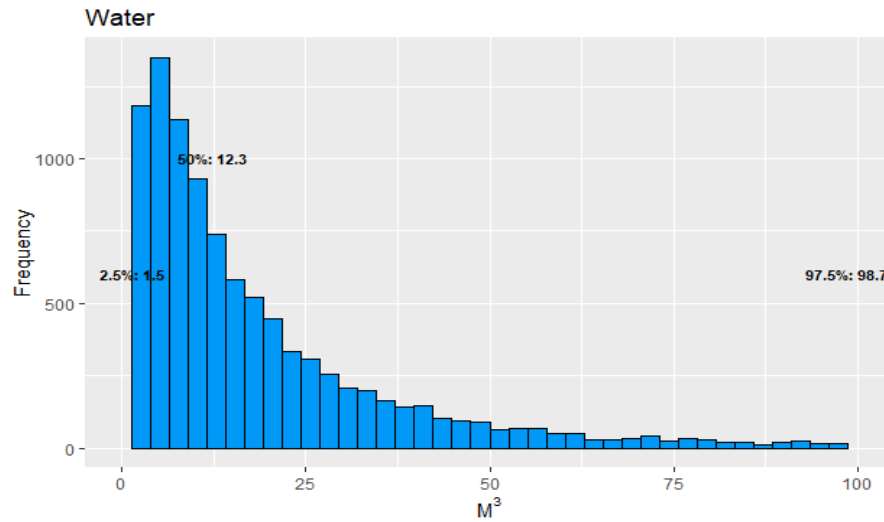
15. Ph.D. students sleeping hours [1, Chap. 2]

We are interested in learning about the proportion of Ph.D. students who sleep at least 6 hours per day. We have a sample of 52 students, where 15 report sleeping at least 6 hours, and the remaining 37 report not sleeping at least 6 hours. The prior distribution is a Beta distribution, with hyperparameters calibrated so that the prior probabilities of the proportion of students who sleep less than 6 hours being less than 0.4 and 0.75 are 0.6 and 0.95, respectively. Estimate the 95% posterior credible interval for the proportion of Ph.D. students. Then, assume there is a group of experts whose beliefs about the proportion of Ph.D. students sleeping at least 6 hours are represented in the following table:

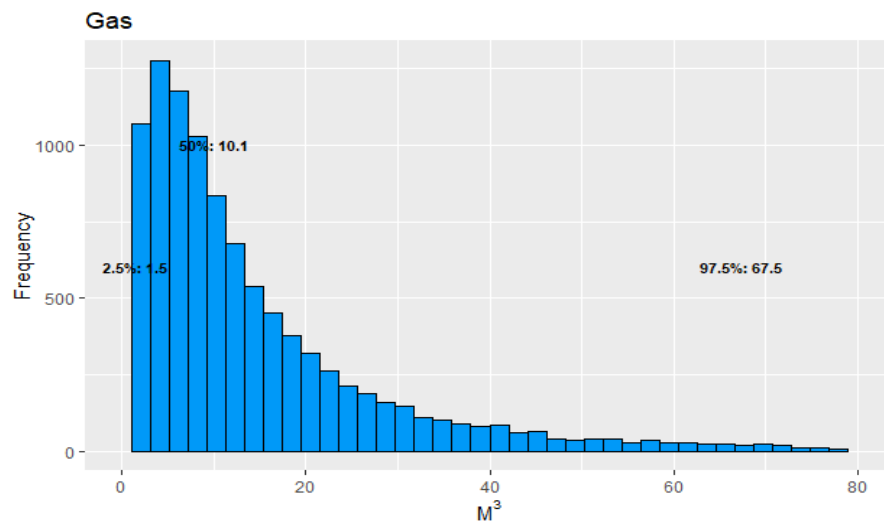
TABLE 3.1

Probability distribution: Ph.D students who sleep at least 6 hours per day.

h	0.05	0.10	0.15	0.20	0.25	0.30	0.35	0.40	0.45	0.50	0.55
$P(p = h)$	0.05	0.07	0.10	0.12	0.15	0.17	0.15	0.11	0.06	0.01	0.01

**FIGURE 3.4**

Histogram using the posterior predictive distribution of water demand

**FIGURE 3.5**

Histogram using the posterior predictive distribution of gas demand

Use Table 3.1 as prior information, and find the posterior distribution of the proportion of students who sleep at least 6 hours.

Answer

We know from the Bernoulli-beta model that the posterior distribution of the proportion of Ph.D. students that sleep at least 6 hours is beta, $\theta|\mathbf{y} \sim B(\alpha_n, \beta_n)$, $\alpha_n = \alpha_0 + 15$ and $\beta_n = \beta_0 + 37$, where $\alpha_0 = 1.44$ and $\beta_0 = 2.57$.

The following code shows how to get the posterior distribution in this exercise.

R code. Ph.D. students sleeping hours: Posterior distribution

```

1 rm(list = ls()); set.seed(010101)
2 # Function to calibrate the hyperparameters
3 elec <- function(theta, perc, q){
4   E1 <- perc[1] - pbeta(q[1], theta[1], theta[2])
5   E2 <- perc[2] - pbeta(q[2], theta[1], theta[2])
6   loss <- E1^2 + E2^2
7   return(loss)
8 }
9 q1 <- 0.4; q2 <- 0.75; q <- c(q1, q2) # Quantiles
10 p1e <- 0.6; p2e <- 0.95; perc <- c(p1e, p2e) # Cumulate
    probability from experts
11 theta0 <- c(1, 1) # Parameters
12 elec(theta0, perc, q)
13 Res <- optim(theta0, elec, perc = perc, q = q)
14 p <- seq(0, 1, length = 500)
15 a <- Res$par[1]; b <- Res$par[2]
16 pbeta(q1, a, b) # P(p <= q1) = p1e
17 pbeta(q2, a, b) # P(p <= q2) = p2e
18 s <- 15 # Success; f <- 37 # Failures
19 prior <- dbeta(p, a, b)
20 likelihood <- dbeta(p, s + 1, f + 1) # Seeing the binomial
    distribution as a likelihood is a beta distribution!!!
21 post <- dbeta(p, a+s, b+f)
22 plot(p, post, type="l", ylab="Density", main = "Proportion of
    students who sleep at least 6 hours per day", lty=2, lwd
    =3, col=gray.colors(1, start=0.1))
23 lines(p, likelihood, lty=1, lwd=3, col=gray.colors(1, start=0.4))
24 lines(p, prior, lty=3, lwd=3, col=gray.colors(1, start=0.6))
25 legend(.7, 4, c("Prior", "Likelihood", "Posterior"), lty=c
    (3, 1, 2), lwd=c(3, 3, 3), col=c(col=gray.colors(1, start=0.6),
    col=gray.colors(1, start=0.4), col=gray.colors(1, start
    =0.1)))
26 S <- 10000; postdraws1 <- rbeta(S, a+s, b+f)
27 mean(postdraws1); quantile(postdraws1, c(0.025, 0.975))
28 # Experts
29 p <- c(seq(0.05, 0.55, by = 0.05)) # Probabilities
30 prior <- c(0.05, 0.07, 0.1, 0.12, 0.15, 0.17, 0.15, 0.11,
    0.06, 0.01, 0.01) # Expert assessment
31 llikelihood <- s * log(p) + f * log(1 - p) # Log likelihood
32 likelihood2 <- exp(llikelihood - max(llikelihood)) # Scale
    the likelihood for plotting
33 product <- likelihood2 * prior # Proportional to posterior
34 posterior <- product/sum(product) # Posterior
35 # Figure
36 plot(p, prior, type = "l", ylab="Probability", lwd=2, lty=2,
    ylim=c(0, 0.99), xlim = c(0, 0.6), col=gray.colors(1,
    start=0.1), main = "Posterior distribution: Proportion
    sleep hours")
37 lines(p, posterior, type = "l", lwd=2, col=gray.colors(1, start
    =0.6))
38 lines(p, likelihood2, type = "l", lwd=4, col=gray.colors(1, start
    =0.3))
39 legend(0.45, 0.85, c('Prior', 'Likelihood', 'Posterior'), lty=c
    (2, 1, 1), col=c(gray.colors(1, start=0.1), gray.colors(1,
    start=0.3), gray.colors(1, start=0.6)), lwd=2)
40 postdraws2 <- sample(p, S, replace = TRUE, prob = posterior)
41 mean(postdraws2); quantile(postdraws2, c(0.025, 0.975))

```

Figures 3.6 and 3.7 show the prior, likelihood, and posterior distributions of the proportion of Ph.D. students who sleep at least 6 hours per day based on the binomial-beta and binomial-discrete models. The mean under both models is 29%, and the 95% credible intervals are (18%,42%) and (20%,40%), respectively.

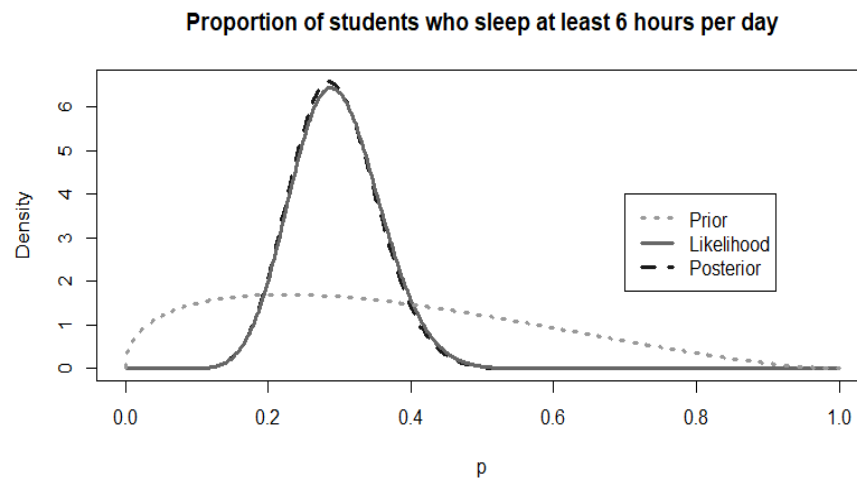
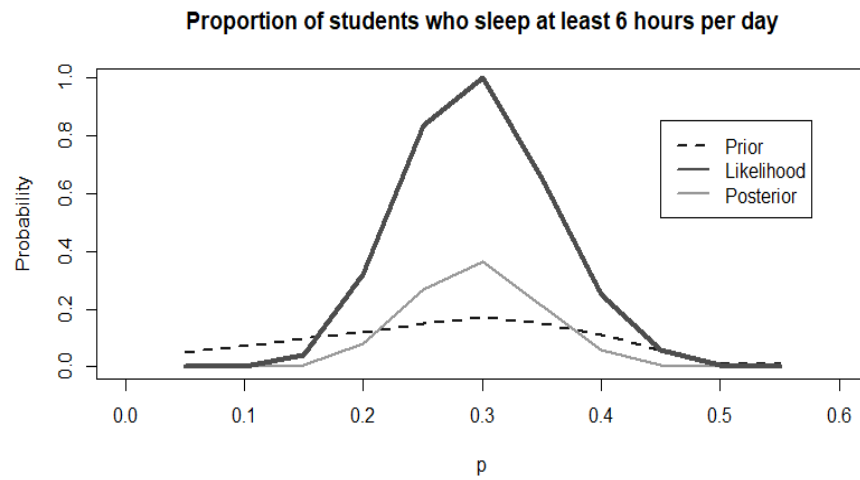


FIGURE 3.6

Posterior distribution binomial-beta model: Proportion Ph.D. students that sleep at least 6 hours per day

**FIGURE 3.7**

Posterior distribution binomial-discrete model: Proportion Ph.D. students that sleep at least 6 hours per day

4

Simulation methods

Solutions of Exercises

1. Example: The normal model with independent priors

Let's recap the math test exercise in Chapter 3, this time assuming independent priors. Specifically, let $Y_i \sim N(\mu, \sigma^2)$, where $\mu \sim N(\mu_0, \sigma_0^2)$ and $\sigma^2 \sim IG(\alpha_0/2, \delta_0/2)$. The sample size is 50, and the mean and standard deviation of the math scores are 102 and 10, respectively. We set $\mu_0 = 100$, $\sigma_0^2 = 100$, and $\alpha_0 = \delta_0 = 0.001$.

- Find the posterior distribution of μ and σ^2 .
- Program a Gibbs sampler algorithm and plot the histogram of the posterior draws of μ .

Answer

The posterior distribution is

$$\begin{aligned}\pi(\mu, \sigma^2 | \mathbf{y}) &\propto (\sigma^2)^{-N/2} \exp \left\{ -\frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \mu)^2 \right\} \\ &\times \exp \left\{ -\frac{1}{2\sigma_0^2} (\mu - \mu_0)^2 \right\} \times \left(\frac{1}{\sigma^2} \right)^{\alpha_0/2+1} \exp \left\{ -\frac{\delta_0}{2\sigma^2} \right\}.\end{aligned}$$

Thus, the conditional posterior distribution of μ is

$$\begin{aligned}\pi(\mu, \sigma^2 | \mathbf{y}) &\propto \exp \left\{ -\frac{1}{2} \left[\frac{1}{\sigma^2} \sum_{i=1}^N (y_i - \mu)^2 + \frac{1}{\sigma_0^2} (\mu - \mu_0)^2 \right] \right\} \\ &\propto \exp \left\{ -\frac{1}{2} \left[\mu^2 \left(\frac{1}{\sigma^2/N} + \frac{1}{\sigma_0^2} \right) - 2\mu \left(\frac{\bar{y}}{\sigma^2/N} + \frac{\mu_0}{\sigma_0^2} \right) \right] \right\}.\end{aligned}$$

We set $\mu_n = \sigma_n^2 \left(\frac{\bar{y}}{\sigma^2/N} + \frac{\mu_0}{\sigma_0^2} \right)$ and $\sigma_n^2 = \left(\frac{1}{\sigma^2/N} + \frac{1}{\sigma_0^2} \right)^{-1}$. Thus,

$$\begin{aligned} \pi(\mu, \sigma^2 | \mathbf{y}) &\propto \exp \left\{ -\frac{1}{2\sigma_n^2} [\mu^2 - 2\mu\mu_n + \mu_n^2 - \mu_n^2] \right\} \\ &\propto \exp \left\{ -\frac{1}{2\sigma_n^2} (\mu - \mu_n)^2 \right\}. \end{aligned}$$

This is the kernel of a normal distribution, that is, $\mu | \sigma^2, \mathbf{y} \sim N(\mu_n, \sigma_n^2)$.

The conditional posterior distribution of σ^2 is given by

$$\begin{aligned} \pi(\sigma^2 | \mu, \mathbf{y}) &\propto (\sigma^2)^{-N/2} \exp \left\{ -\frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \mu)^2 \right\} \\ &\quad \times \left(\frac{1}{\sigma^2} \right)^{\alpha_0/2+1} \exp \left\{ -\frac{\delta_0}{2\sigma^2} \right\} \\ &\propto (\sigma^2)^{-N/2-\alpha_0/2-1} \exp \left\{ -\frac{1}{2\sigma^2} \left[\sum_{i=1}^N (y_i - \mu)^2 + \delta_0 \right] \right\}. \end{aligned}$$

Thus, $\sigma^2 | \mu, \mathbf{y} \sim IG(\alpha_n/2, \delta_n/2)$, where $\alpha_n = N + \alpha_0$ and $\delta_n = \sum_{i=1}^N (y_i - \mu)^2 + \delta_0 = N\hat{\sigma}^2 + N(\bar{y} - \mu)^2 + \delta_0$ given that $\sum_{i=1}^N (y_i - \bar{y}) = 0$, where $\bar{y}$ and $\hat{\sigma}$ are the mean and standard deviation estimates.

As we have the conditional posterior distributions, we can use the Gibbs sampling algorithm to perform inference in this model. The following code shows how to do it.

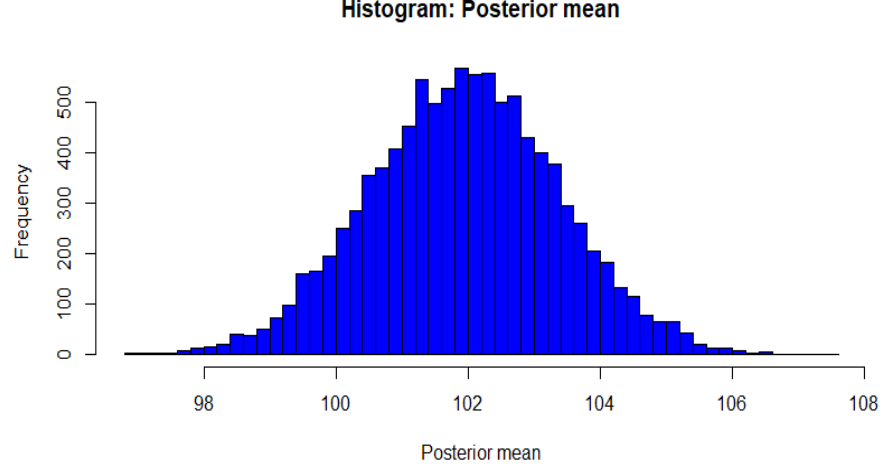
R code. Gibbs sampler: The math example

```

1 rm(list = ls())
2 set.seed(010101)
3 N <- 50
4 # Sample size
5 muhat <- 102
6 # Sample mean
7 sig2hat <- 100
8 # Sample variance
9
10 # Hyperparameters
11 mu0 <- 100
12 sig20 <- 100
13 delta0 <- 0.001
14 alpha0 <- 0.001
15
16 MCMC <- 10000; burnin <- 1000; S <- MCMC + burnin
17 keep <- (burnin+1):S
18 # Posterior draws
19 alphan <- alpha0 + N
20 sig2Post <- rep(NA, S)
21 muPost <- rep(NA, S)
22 sig2 <- sig20
23 for(s in 1:S){
24   sig2n <- (1/(sig2/N)+1/sig20)^(-1)
25   mun <- sig2n*(muhat/(sig2/N)+mu0/sig20)
26   mu <- rnorm(1, mun, sig2n^0.5)
27   muPost[s] <- mu
28   deltan <- N*(sig2hat + (muhat - mu)^2)
29   sig2 <- invgamma::rinvgamma(1, shape = alphan, rate =
30     deltan)
31   sig2Post[s] <- sig2
32 }
33 sig2s <- coda::mcmc(sig2Post[keep])
34 mus <- coda::mcmc(muPost[keep])
35 summary(sig2s)
36 summary(mus)
37 hist(mus, main = "Histogram: Posterior mean", xlab = "
38   Posterior mean", col = "blue", breaks = 50)
39 muPost_tq <- quantile(mus, c(0.025, 0.5, 0.975))
40 muPost_tq
41 cutoff <- 103
42 PmuPost_t cutoff <- mean(mus > cutoff)
43 PmuPost_t cutoff

```

Figure 4.1 shows the histogram of the draws of the posterior mean of the math test results. The posterior mean and median are 101.94 and 101.95,

**FIGURE 4.1**

Histogram of posterior draws of mean: Math test

and the 95% credible interval is (99.14, 104.79).

2. Show that the Gibbs sampler is a particular case of the Metropolis-Hastings where the acceptance probability is equal to 1.

Answer

Without loss of generality let's assume that there are two blocks, $\boldsymbol{\theta} = [\boldsymbol{\theta}_1 \ \boldsymbol{\theta}_2]^\top$ such that the Metropolis-Hastings (M-H) algorithm generates a candidate sample $(\boldsymbol{\theta}_1^c)$ using a proposal distribution $q(\boldsymbol{\theta}_1^c | \boldsymbol{\theta}^{(s-1)})$. The candidate is accepted with probability:

$$\alpha(\boldsymbol{\theta}^{(s-1)}, \boldsymbol{\theta}^c) = \min \left\{ 1, \frac{q(\boldsymbol{\theta}_1^{(s-1)} | \boldsymbol{\theta}^c) \pi(\boldsymbol{\theta}^c | \mathbf{y})}{q(\boldsymbol{\theta}_1^c | \boldsymbol{\theta}^{(s-1)}) \pi(\boldsymbol{\theta}^{(s-1)} | \mathbf{y})} \right\}.$$

For Gibbs sampling, the candidate $\boldsymbol{\theta}_1^c$ is drawn directly from the *full conditional distribution*, so $q(\boldsymbol{\theta}_1^c | \boldsymbol{\theta}^{(s-1)}) = \pi(\boldsymbol{\theta}_1^c | \boldsymbol{\theta}_2^{(s-1)}, \mathbf{y})$. Since Gibbs sampling uses the full conditional distributions as the proposal, the key terms simplify. In particular, $q(\boldsymbol{\theta}_1^c | \boldsymbol{\theta}^{(s-1)}) = \pi(\boldsymbol{\theta}_1^c | \boldsymbol{\theta}_2^c, \mathbf{y})$, and similarly $q(\boldsymbol{\theta}_1^{(s-1)} | \boldsymbol{\theta}^c) = \pi(\boldsymbol{\theta}_1^{(s-1)} | \boldsymbol{\theta}_2^{(s-1)}, \mathbf{y})$. Thus, the acceptance probability is

given by

$$\begin{aligned}\alpha(\boldsymbol{\theta}^{(s-1)}, \boldsymbol{\theta}^c) &= \min \left\{ 1, \frac{\pi(\boldsymbol{\theta}_1^{(s-1)} | \boldsymbol{\theta}_2^{(s-1)}, \mathbf{y}) \pi(\boldsymbol{\theta}^c | \mathbf{y})}{\pi(\boldsymbol{\theta}_1^c | \boldsymbol{\theta}_2^c, \mathbf{y}) \pi(\boldsymbol{\theta}^{(s-1)} | \mathbf{y})} \right\} \\ &= \min \left\{ 1, \frac{\pi(\boldsymbol{\theta}_1^{(s-1)} | \boldsymbol{\theta}_2^{(s-1)}, \mathbf{y}) \pi(\boldsymbol{\theta}_1^c | \boldsymbol{\theta}_2^c, \mathbf{y}) \pi(\boldsymbol{\theta}_2^c | \mathbf{y})}{\pi(\boldsymbol{\theta}_1^c | \boldsymbol{\theta}_2^c, \mathbf{y}) \pi(\boldsymbol{\theta}_1^{(s-1)} | \boldsymbol{\theta}_2^{(s-1)}, \mathbf{y}) \pi(\boldsymbol{\theta}_2^{(s-1)} | \mathbf{y})} \right\} \\ &= 1,\end{aligned}$$

due to $\boldsymbol{\theta}_2^{(s-1)} = \boldsymbol{\theta}_2^c$. Thus, the Gibbs sampling algorithm is implicitly a M-H algorithm where the acceptance probability is 1.

3. Implement a Metropolis-Hastings (M-H) to sample from the Cauchy distribution, $C(0, 1)$, using as proposals a standard normal distribution and a Student's t distribution with 5 degrees of freedom.

Answer

The following code shows to program the M-H to get draws from the Cauchy distribution, $C(0, 1)$, using as proposal a standard normal distribution and a Student's t distribution with 5 degree of freedom.

R code. Metropolis-Hastings algorithm: Cauchy distribution

```

1 rm(list = ls()); set.seed(010101)
2 S <- 100000; df <- 5
3 theta1 <- runif(S); accept1 <- rep(0, S)
4 theta2 <- runif(S); accept2 <- rep(0, S)
5 for (s in 2:S){
6   thetac1 <- rnorm(1) # Candidate normal standard
7   thetac2 <- rt(1, df = df) # Candidate Students' t
8   #Acceptance rate
9   a1 <- (dcauchy(thetac1)*dnorm(theta1[s-1]))/(dcauchy(
10     theta1[s-1]*dnorm(thetac1))
11   a2 <- (dcauchy(thetac2)*dt(theta2[s-1], df = df))/(dcauchy(
12     theta2[s-1]*dt(thetac2, df = df))
13   U1 <- runif(1)
14   if(U1 <= a1){
15     theta1[s] <- thetac1; accept1[s] <- 1
16   }else{
17     theta1[s] <- theta1[s-1]; accept1[s] <- 0
18   }
19   if(U1 <= a2){
20     theta2[s] <- thetac2; accept2[s] <- 1
21   }else{
22     theta2[s] <- theta2[s-1]; accept2[s] <- 0
23   }
24 }
25 mean(accept1); mean(accept2)
26 plot(coda::mcmc(theta1)); plot(coda::mcmc(theta2))
27 h <- hist(theta1, breaks=50, col="blue", xlab="x", main="
28   Cauchy draws from a Metropolis-Hastings algorithm:
29   Normal standard proposal")
30 pfit <- seq(min(theta1),max(theta1),length=50)
31 yfit<-dcauchy(pfit)
32 yfit <- yfit*diff(h$mids[1:2])*length(theta1)
33 lines(pfit, yfit, col="red", lwd=2)
34 h <- hist(theta2, breaks=50, col="blue", xlab="x", main="
35   Cauchy draws from a Metropolis-Hastings algorithm:
36   Student's t proposal")
37 pfit <- seq(min(theta2),max(theta2),length=50)
38 yfit<-dcauchy(pfit)
39 yfit <- yfit*diff(h$mids[1:2])*length(theta2)
40 lines(pfit, yfit, col="red", lwd=2)

```

Figures 4.2 and 4.3 show the histograms of the posterior draws using the normal and Student's t distributions, along with the density of the Cauchy distribution. The spike in the posterior draws from the standard normal proposal is due to the algorithm being stuck for many iterations at a par-

ticular value, as the normal distribution has lighter tails compared to the Cauchy distribution. The convergence using the normal proposal is slower than when using the Student's t proposal, owing to the lighter tails of the former.

Cauchy draws from a Metropolis-Hastings algorithm: Normal standard proposal

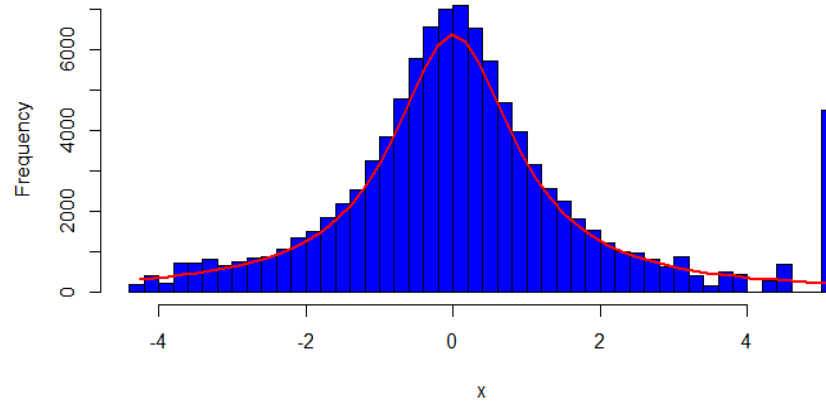


FIGURE 4.2

Histogram of posterior draws of beta distribution and the density of the beta distribution.

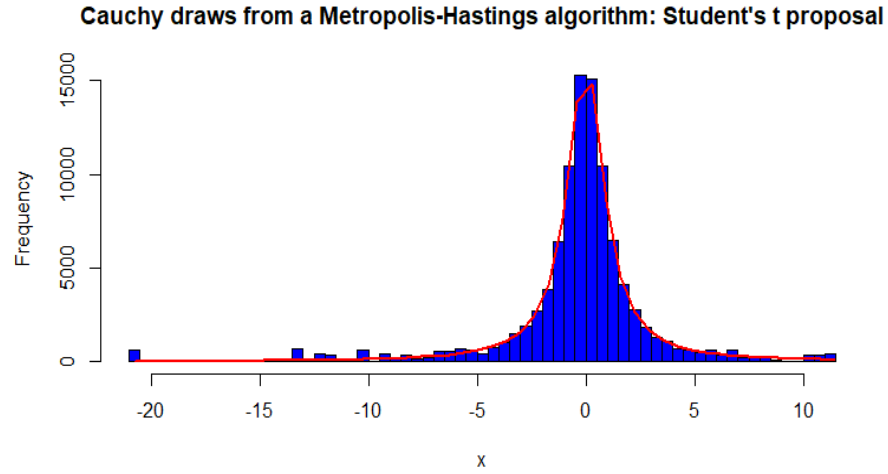
4. This exercise was proposed by Professor Hedibert Freitas Lopes, who cites [50] as a useful reference for an introduction to Hamiltonian Monte Carlo in **R** and the *hmclearn* package. The task is to obtain posterior draws using the Metropolis-Hastings and Hamiltonian Monte Carlo algorithms for the posterior distribution given by

$$\pi(\theta_1, \theta_2 | \mathbf{y}) \propto \exp \left\{ -\frac{1}{2} (\theta_1^2 \theta_2^2 + \theta_1^2 + \theta_2^2 - 8\theta_1 - 8\theta_2) \right\}.$$

Answer

The following code demonstrates the implementation of the M-H and HMC algorithms for this exercise. For the M-H algorithm, the number of MCMC iterations, burn-in period, and thinning parameters are set to 20,000, 1,000, and 20, respectively. The variance of the proposal distribution is tuned to achieve an acceptance rate close to 25%.

For the HMC algorithm, the number of iterations and burn-in period are set to 5,000 and 500, respectively. The step size and number of leapfrog iterations are set to 0.001 and 100, and the mass matrix is defined as $\mathbf{M} = \mathbf{I}_2$. These parameters yield an acceptance rate near 65%.

**FIGURE 4.3**

Histogram of posterior draws of beta distribution and the density of the beta distribution.

Figures 4.4 and 4.6 present the contour plots and posterior draws from the M-H and HMC algorithms for the joint posterior distribution. It can be observed that this distribution is bimodal, and both algorithms successfully capture this feature.

*R code. Metropolis-Hastings and Hamiltonian
Monte Carlo algorithms*

```

1 rm(list = ls()); set.seed(010101)
2 # Posterior distribution
3 PostDist <- function(theta){
4   theta1 <- theta[1]; theta2 <- theta[2]
5   post <- exp(-0.5*(theta1^2*theta2^2+theta1^2+theta2^2-8*
6     theta1-8*theta2))
7   return(post)
8 }
9 # Metropolis-Hastings
10 MH <- function(theta, c2){
11   thetac <- MASS::mvrnorm(1, mu = theta, Sigma = c2*diag(2))
12   a <- PostDist(thetac)/PostDist(theta)
13   U <- runif(1)
14   if(U <= a){
15     theta <- thetac
16     accept <- 1
17   }else{
18     theta <- theta
19     accept <- 0
20   }
21   return(list(theta = theta, accept = accept))
22 }
23 # Posterior draws M-H
24 S <- 20000; burnin <- 1000; thin <- 20; tot <- S + burnin
25 K <- 2; c2 <- 1.5
26 thetaPostMH <- matrix(NA, tot, K)
27 AcceptMH <- rep(NA, tot)
28 thetaMH <- c(0, 0)
29 for(s in 1:tot){
30   ResMH <- MH(thetaMH, c2 = c2)
31   thetaMH <- ResMH$theta
32   thetaPostMH[s,] <- thetaMH
33   AcceptMH[s] <- ResMH$accept
34 }
35 keep <- seq((burnin), tot, thin)
36 mean(AcceptMH[keep])
37 thetaPostMHMCMC <- coda::mcmc(thetaPostMH[keep,])
38 plot(thetaPostMHMCMC)
39 coda::autocorr.plot(thetaPostMHMCMC)
40 # Contour plot
41 ngrid <- 400
42 theta1 <- seq(-1, 6, length = ngrid)
43 theta2 <- seq(-1, 6, length = ngrid)
44 f <- matrix(0, ngrid, ngrid)
45 for (i in 1:ngrid){
46   for (j in 1:ngrid){
47     f[i,j] = PostDist(c(theta1[i],theta2[j]))
48   }
49 }
50 plot(thetaPostMH[keep,], xlim=range(theta1), ylim=range(
51   theta2), pch=16, col=grey(0.8), xlab=expression(theta
52   [1]), ylab=expression(theta[2]))
53 contour(theta1,theta2,f, drawlabels=FALSE, add=TRUE, col = "
54   blue", lwd = 1.2)
55 title("Random walk Metropolis-Hastings")

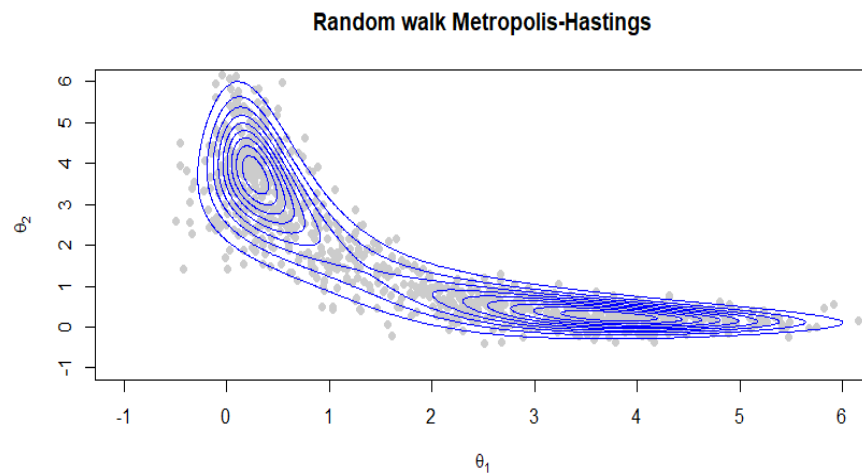
```

R code. Metropolis-Hastings and Hamiltonian Monte Carlo algorithms

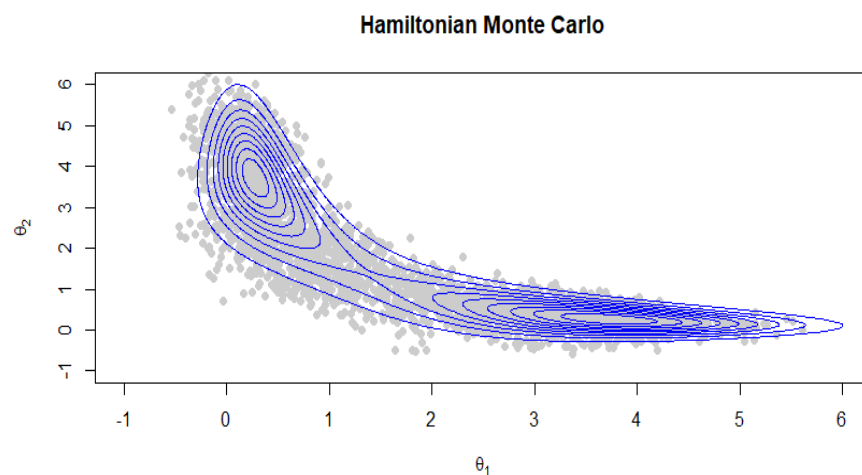
```

1 HMC <- function(theta, epsilon, L, M){
2   Minv <- solve(M); thetat <- theta
3   K <- length(thetat)
4   mom <- t(mvtnorm::rmvnorm(1, rep(0, K), M))
5   logPost_Mom_t <- log(PostDist(thetat)) + mvtnorm::dmvnorm
6     (t(mom), rep(0, K), M, log = TRUE)
7   theta1 <- theta[1]; theta2 <- theta[2]
8   for(l in 1:L){
9     if(l == 1 | l == L){
10      mom <- mom + 0.5*epsilon*(-0.5*c(2*theta1*theta2^2+2*
11        theta1-8, 2*theta2*theta1^2+2*theta2-8))
12      theta <- theta + epsilon*Minv%*%mom
13    }else{
14      mom <- mom + epsilon*(-0.5*c(2*theta1*theta2^2+2*
15        theta1-8, 2*theta2*theta1^2+2*theta2-8))
16      theta <- theta + epsilon*Minv%*%mom
17    }
18  }
19  logPost_Mom_star <- log(PostDist(theta)) + mvtnorm::
20    dmvnorm(t(mom), rep(0, K), M, log = TRUE)
21  alpha <- min(1, exp(logPost_Mom_star-logPost_Mom_t))
22  u <- runif(1)
23  if(u <= alpha){
24    thetaNew <- c(theta)
25  }else{
26    thetaNew <- thetat
27  }
28  rest <- list(theta = thetaNew, Prob = alpha)
29  return(rest)
30 }
31 # Posterior draws HMC
32 S <- 5000; burnin <- 500; tot <- S + burnin
33 epsilon <- 0.0025; L <- 100; M <- diag(2)
34 thetaPostHMC <- matrix(NA, tot, K)
35 ProbAcceptHMC <- rep(NA, tot)
36 thetaHMC <- c(0, 0)
37 for(s in 1:tot){
38   ResHMC <- HMC(theta = thetaHMC, epsilon = epsilon, L = L,
39     M = M)
40   thetaHMC <- ResHMC$theta
41   thetaPostHMC[s,] <- thetaHMC
42   ProbAcceptHMC[s] <- ResHMC$Prob
43 }
44 keep <- burnin:S
45 summary(ProbAcceptHMC[keep])
46 thetaPostHMC[keep,] <- coda::mcmc(thetaPostHMC[keep,])
47 plot(thetaPostHMC[keep,]); coda::autocorr.plot(thetaPostHMC[keep,])
48 plot(thetaPostHMC[keep,], xlim=range(theta1), ylim=range(
49   theta2), pch=16, col=grey(0.8),
50   xlab=expression(theta[1]), ylab=expression(theta[2]))
51 contour(theta1,theta2,f, drawlabels=FALSE, add=TRUE, col = "
52   blue", lwd = 1.2)
53 title("Hamiltonian Monte Carlo")

```

**FIGURE 4.4**

Contour plot: Metropolis-Hastings posterior draws.

**FIGURE 4.5**

Contour plot: Hamiltonian Monte Carlo posterior drwas.

5. Ph.D. students sleeping hours continues

- (a) Use importance sampling based on a $U(0, 1)$ proposal to obtain draws of $\theta|\mathbf{y} \sim B(16.55, 39.57)$ in the Ph.D. students' sleeping hours example

in Chapter 3. Note that, based on Exercise 15 in Chapter 3, $\alpha_0 = 1.44$ and $\beta_0 = 2.57$.

- (b) Compute the marginal likelihood in this context (Bernoulli-Beta model) and compare it to the result obtained using the Gelfand-Dey method.

Answer

The posterior distribution of the Bernoulli-beta model is

$$p(\theta|\mathbf{y}) = \frac{\prod_{i=1}^N \theta^{y_i} (1-\theta)^{1-y_i} \theta^{\alpha_0-1} (1-\theta)^{\beta_0-1} \Gamma(\alpha_0 + \beta_0)}{\Gamma(\alpha_0)\Gamma(\beta_0)p(\mathbf{y})},$$

where

$$\begin{aligned} p(\mathbf{y}) &= \int_{\Theta} \frac{\prod_{i=1}^N \theta^{y_i} (1-\theta)^{1-y_i} \theta^{\alpha_0-1} (1-\theta)^{\beta_0-1} \Gamma(\alpha_0 + \beta_0)}{\Gamma(\alpha_0)\Gamma(\beta_0)} d\theta \\ &= \int_{\Theta} \frac{\theta^{\sum_{i=1}^N y_i + \alpha_0 - 1} (1-\theta)^{N - \sum_{i=1}^N y_i + \beta_0 - 1} \Gamma(\alpha_0 + \beta_0)}{\Gamma(\alpha_0)\Gamma(\beta_0)} d\theta \\ &= \frac{B(\alpha_n, \beta_n)}{B(\alpha_0, \beta_0)} \int_{\Theta} \frac{\theta^{\sum_{i=1}^N y_i + \alpha_0 - 1} (1-\theta)^{N - \sum_{i=1}^N y_i + \beta_0 - 1}}{B(\alpha_n, \beta_n)} d\theta \\ &= \frac{B(\alpha_n, \beta_n)}{B(\alpha_0, \beta_0)}. \end{aligned}$$

The integral is equal to 1 due to being over the support of $B(\alpha_n, \beta_n)$ distribution.

R code. Importance sampling: Bernoulli-Beta model

```

1 rm(list = ls()); set.seed(010101); S <- 50000
2 # Importance sampling from standard normal proposal
3 an <- 16.55; bn <- 39.57 # Posterior parameters
4 theta <- runif(S) # Proposal
5 ws <- dbeta(theta, an, bn) # Weights
6 wstars <- ws/sum(ws) # Standardized weights
7 L <- 20000 # Size of posterior sample
8 thetaBeta <- sample(theta, L, replace = TRUE, prob = wstars)
9 # Posterior draws
10 # Figure
11 h <- hist(thetaBeta, breaks=50, col="blue", xlab="x", main="
12   Beta draws from importance sampling: Uniform (0,1)
13   proposal")
14 pfit <- seq(min(thetaBeta), max(thetaBeta), length=50)
15 yfit <- dbeta(pfit, an, bn)
16 yfit <- yfit*diff(h$mids[1:2])*length(thetaBeta)
17 lines(pfit, yfit, col="red", lwd=2)
18 a0 <- 1.44; b0 <- 2.57 # Hyperparameters
19 s <- 15; n <- 52 # Data
20 LogMarLik <- lbeta(an, bn)-lbeta(a0, b0) # Marginal
21   likelihood
22 LogMarLik
23 # Gelfand-Day method
24 LikPrior <- function(theta){
25   Liki <- theta^s*(1-theta)^(n-s)
26   Priori <- dbeta(theta, a0, b0)
27   LikPriori <- 1 / Liki * Priori
28   return(LikPriori)
29 }
30 LogMarLikGD <- log(1/mean(sapply(thetaBeta, LikPrior)))
31 LogMarLikGD

```

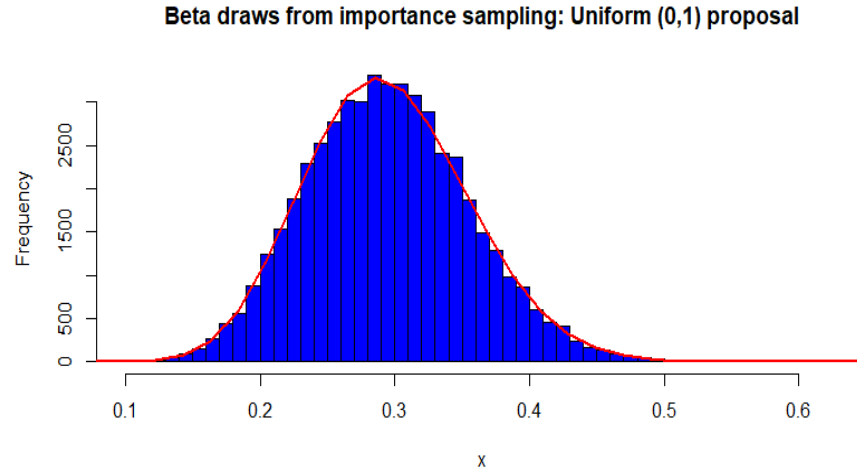
Figure 4.6 shows the histogram of the posterior draws obtained using the importance sampling algorithm with a uniform proposal. The algorithm performs well. Additionally, the analytic solution for the log marginal likelihood is -32.75, while the Gelfand-Dey method yields -32.76.

6. Example 4.1 in [15] is

$$\theta_t = 0.5\theta_{t-1} + 25\frac{\theta_{t-1}}{1 + \theta_{t-1}^2} + 8\cos(1.2t) + w_t$$

$$y_t = \frac{\theta_t^2}{20} + \mu_t,$$

where $\theta_0 \sim N(0, \sqrt{10})$, $w_t \sim N(0, \sqrt{10})$ and $\mu_t \sim N(0, \sqrt{1})$.

**FIGURE 4.6**

Importance sampling: Beta distribution.

- Perform sequential importance sampling in this example
- Perform particle (Bootstrap) filtering in this example
- Estimate the marginal likelihood in this example

Answer

The following code shows to perform sequential importance sampling and particle (Bootstrap) filtering in the Example 1 in [15]. Take into account that we use same strategy as in the dynamic linear model example in the book to calculate the mean and standard deviation of the posterior distribution.

*R code. Sequential importance sampling and
particle (Bootstrap) filtering: Gordon (1993)
Example 1.*

```

1 rm(list = ls()); set.seed(010101)
2 S <- 50000 # Number of particles
3 sigma_w <- 10^0.5 # State noise
4 sigma_mu <- 1 # Observation noise
5 T <- 200 # Sample size
6 # Simulate true states and observations
7 theta_true <- numeric(T)
8 y_obs <- numeric(T)
9 theta_true[1] <- rnorm(1, mean = 0, sd = sigma_w) # Initial
  state
10 y_obs[1] <- rnorm(1, mean = theta_true[1]^2/20, sd = sigma_
  mu)
11 for (t in 2:T) {
12   meant <- 0.5*theta_true[t-1] + 25*(theta_true[t-1]/(1+
  theta_true[t-1]^2)) + 8*cos(1.2*t)
13   theta_true[t] <- rnorm(1, mean = meant, sd = sigma_w)
14   y_obs[t] <- rnorm(1, mean = theta_true[t]^2/20, sd = sigma
  _mu)
15 }
16 plot(theta_true, type = "l"); plot(y_obs, type = "l")
17 # Sequential Importance Sampling (SIS)
18 particles <- matrix(0, nrow = S, ncol = T)
19 weights <- matrix(0, nrow = S, ncol = T)
20 weightsSt <- matrix(0, nrow = S, ncol = T)
21 # Initialization
22 particles[, 1] <- rnorm(S, mean = 0, sd = sigma_w)
23 weights[, 1] <- dnorm(y_obs[1], mean = particles[, 1]^2/20,
  sd = sigma_mu)
24 weightsSt[, 1] <- weights[, 1] / sum(weights[, 1])
25 # Sequential updating
26 for (t in 2:T) {
27   particles[, t] <- rnorm(S, mean = 0.5*particles[, t-1] +
  25*(particles[, t-1]/(1+particles[, t-1]^2)) + 8*cos(1.2
  *t), sd = sigma_w) # Compute weights
28   weights[, t] <- weightsSt[, t-1] * dnorm(y_obs[t], mean =
  particles[, t]^2/20, sd = sigma_mu)
29   weightsSt[, t] <- weights[, t] / sum(weights[, t])
30 }
31 FilterDist <- colSums(particles * weightsSt)
32 SDFilterDist <- (colSums(particles^2 * weightsSt) -
  FilterDist^2)^0.5
33 library(dplyr); library(ggplot2); library(latex2exp)
34 ggplot2::theme_set(theme_bw())
35 df <- tibble(t = 1:T, mean = FilterDist, lower = FilterDist
  - 2*SDFilterDist, upper = FilterDist + 2*SDFilterDist,
  theta_true = theta_true)
36 plot_filtering_estimates <- function(df) {
37   p <- ggplot(data = df, aes(x = t)) + geom_ribbon(aes(ymin
  = lower, ymax = upper), alpha = 1, fill = "lightblue") +
  geom_line(aes(y = theta_true), colour = "black", alpha
  = 1, linewidth = 0.5) + geom_line(aes(y = mean), colour
  = "blue", linewidth = 0.5) + ylab(TeX("$\\theta_{t}$"))
  + xlab("Time")
38   print(p)
39 }
40 plot_filtering_estimates(df)

```

*R code. Sequential importance sampling and
particle (Bootstrap) filtering: Gordon (1993)
Example 1.*

```

1 # Particle filtering
2 particles <- matrix(0, nrow = S, ncol = T)
3 particlesT <- matrix(0, nrow = S, ncol = T)
4 weights <- matrix(0, nrow = S, ncol = T)
5 weightsSt <- matrix(0, nrow = S, ncol = T)
6 weightsSTT <- matrix(1/S, nrow = S, ncol = T)
7 logalphas <- matrix(0, nrow = S, ncol = T)
8 # Initialization
9 particles[, 1] <- rnorm(S, mean = 0, sd = sigma_w) # Sample
    initial particles
10 weights[, 1] <- dnorm(y_obs[1], mean = particles[, 1]^2/20,
    sd = sigma_mu) # Importance weights
11 weightsSt[, 1] <- weights[, 1] / sum(weights[, 1]) #
    Normalize weights
12 ind <- sample(1:S, size = S, replace = TRUE, prob =
    weightsSt[, 1]) # Resample
13 particles[, 1] <- particles[ind, 1] # Resampled particles
14 particlesT[, 1] <- particles[, 1] # Resampled particles
15 # Sequential updating
16 pb <- txtProgressBar(min = 0, max = T, style = 3)
17 for (t in 2:T) {
18   # Propagate particles
19   particles[, t] <- rnorm(S, mean = 0.5*particles[, t-1] +
    25*(particles[, t-1]/(1+particles[, t-1]^2)) + 8*cos(1.2
    *t), sd = sigma_w)
20   # Compute weights
21   logalphas[, t] <- dnorm(y_obs[t], mean = particles[, t]^2/
    20, sd = sigma_mu, log = TRUE)
22   weights[, t] <- exp(logalphas[, t])
23   weightsSt[, t] <- weights[, t] / sum(weights[, t])
24   if(t < T){
25     ind <- sample(1:S, size = S, replace = TRUE, prob =
    weightsSt[, t])
26     particles[, 1:t] <- particles[ind, 1:t]
27   }else{
28     ind <- sample(1:S, size = S, replace = TRUE, prob =
    weightsSt[, t])
29     particlesT[, 1:t] <- particles[ind, 1:t]
30   }
31   setTxtProgressBar(pb, t)
32 }
33 close(pb)
34 FilterDist <- colSums(particles * weightsSt)
35 SDFilterDist <- (colSums(particles^2 * weightsSt) -
    FilterDist^2)^0.5
36 FilterDistT <- colSums(particlesT * weightsSTT)
37 SDFilterDistT <- (colSums(particlesT^2 * weightsSTT) -
    FilterDistT^2)^0.5

```


***R code. Sequential importance sampling and
particle (Bootstrap) filtering: Gordon (1993)
Example 1.***

```

1 library(dplyr); library(ggplot2); library(latex2exp)
2 ggplot2::theme_set(theme_bw())
3 df <- tibble(t = 1:T, mean = FilterDist, lower = FilterDist
4             - 2*SDFilterDist, upper = FilterDist+ 2*SDFilterDist,
5             meanT = FilterDistT, lowerT = FilterDistT - 2*
6             SDFilterDistT,
7             upperT = FilterDistT + 2*SDFilterDistT, x_true = theta_true)
8 plot_filtering_estimates <- function(df) {
9   p <- ggplot(data = df, aes(x = t)) + geom_ribbon(aes(ymin
10              = lower, ymax = upper), alpha = 1, fill = "lightblue") +
11     geom_line(aes(y = x_true), colour = "black", alpha = 1,
12              linewidth = 0.5) + geom_line(aes(y = mean), colour = "
13              blue", linewidth = 0.5) +
14     geom_line(aes(y = meanT), colour = "purple", linewidth =
15              0.5) + ylab(TeX("$\\theta_{t}$")) + xlab("Time")
16   print(p)
17 }
18 plot_filtering_estimates(df)
19 MargLik <- colMeans(weights)
20 plot(MargLik, type = "l", xlab = "Time", ylab = "Marginal
21      likelihood", main = "Marginal likelihood")

```

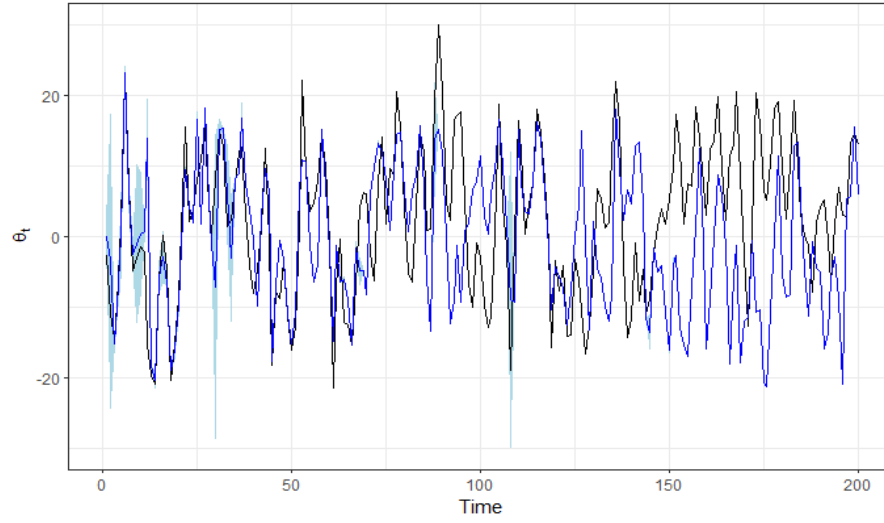
Figure 4.7 shows the true trajectory of the state vector (black line), the posterior mean (blue line), and the $\pm 2\hat{\sigma}_\theta$ posterior estimate (light blue shaded area) from the sequential importance sampling algorithm. We observe that there is no light blue shaded area after time period 40, and the algorithm exhibits poor performance after $t = 75$. This is due to weight degeneracy.

Figure 4.8 shows the performance of particle filtering (see Figure 3 in [15]). There is the true state vector (black line), the means based on $\{\theta_{1:t}^{(s)}, w_t^{*(s)}\}$ (blue line) and $\{\theta_{1:t}^{r(s)}, 1/S\}$ (purple line), and the area defined by $\pm 2\hat{\sigma}_\theta$ based on the former (light blue shaded area). Note that the particle filtering algorithm has better performance than the SIS algorithm. This is due to restarting the system every stage by resampling, which in turn mitigates the weight degeneracy issue.

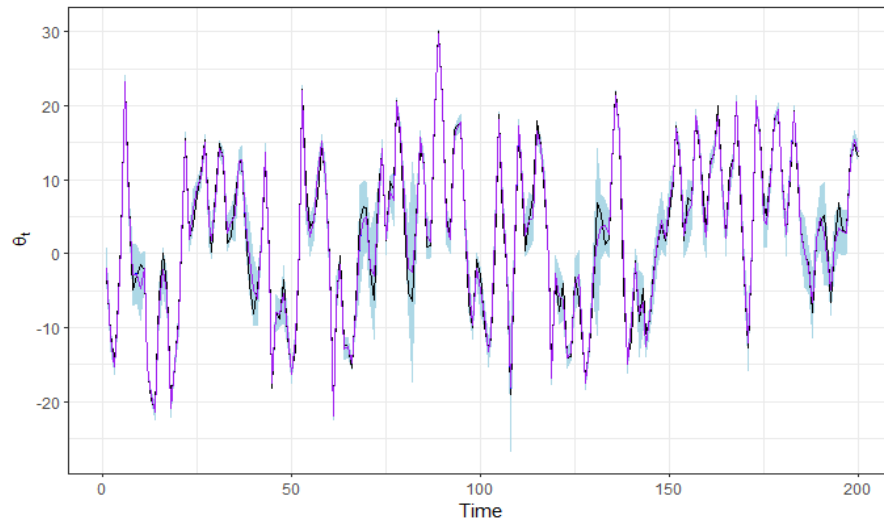
Finally, Figure 4.9 shows the posterior estimate of marginal likelihood based on the particle filtering algorithm.

7. Ph.D. students sleeping hours continues

- Perform the diagnostics of Section 4.4 in this example.

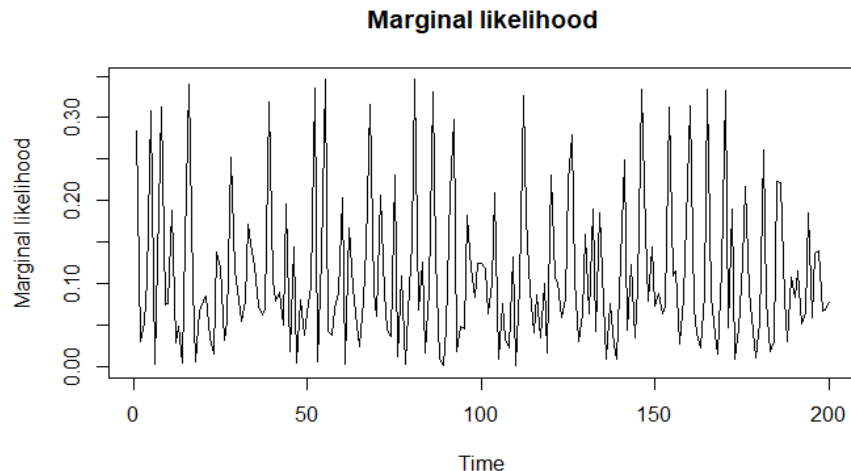
**FIGURE 4.7**

Sequential importance sampling: Example 1, [15].

**FIGURE 4.8**

Particle (Bootstrap) filtering: Example 1, [15].

- Check if there are errors in the posterior simulator of the Metropolis-Hastings algorithm in this example using the Geweke approach, with the first moments of p and p_2 as test functions. Remember from Ex-

**FIGURE 4.9**

Marginal likelihood: Example 1, [15].

ercise 15 in Chapter 3 that the sample size is 52, and $\alpha_0 = 1.22$ and $\beta_0 = 2.57$.

- Run the Geweke test using $\alpha_0 = 2.57$ and $\beta_0 = 1.22$, and check the results.

Answer

We initially run the Metropolis-Hastings (M-H) algorithm for 100,000 iterations and obtain an effective sample size of 13,606. The naive and time-series standard errors are 0.00019 and 0.00052, respectively, indicating correlation—a feature also observed in the autocorrelation plot (not shown). However, the trace plot appears satisfactory. The Geweke test statistic is 1.97, marginally rejecting the null hypothesis that the means of the first 10% and the last 50% of the posterior draws are equal. Additionally, the dependence factor is 6.1, which exceeds the rule-of-thumb threshold of 5. The posterior draws pass both stages of Heidelberger and Welch’s convergence diagnostic.

Given the autocorrelation issues, we set a burn-in period of 20,000 iterations and apply a thinning parameter of 10. Figures 4.10 and 4.11 display the trace and autocorrelation plots of the posterior draws. We observe that the draws appear stationary around their mean, and the autocorrelation decreases rapidly to zero.

The naive and time-series standard errors are 0.00068 and 0.00072, respectively. The similarity between these values indicates a low level of auto-

correlation, consistent with the results in Figure 4.11. The effective sample size of the posterior draws is 7,066, while the total number of posterior draws is 8,000.

The Geweke test statistic is 0.23, providing no statistical evidence to reject the null hypothesis of equal means in the two segments of the posterior draws. The Raftery and Lewis test yields a dependence factor close to 1, indicating minimal dependence. The Heidelberger and Welch test does not reject the null hypothesis of stationarity for the posterior draws and confirms that the mean is both accurate and stable.

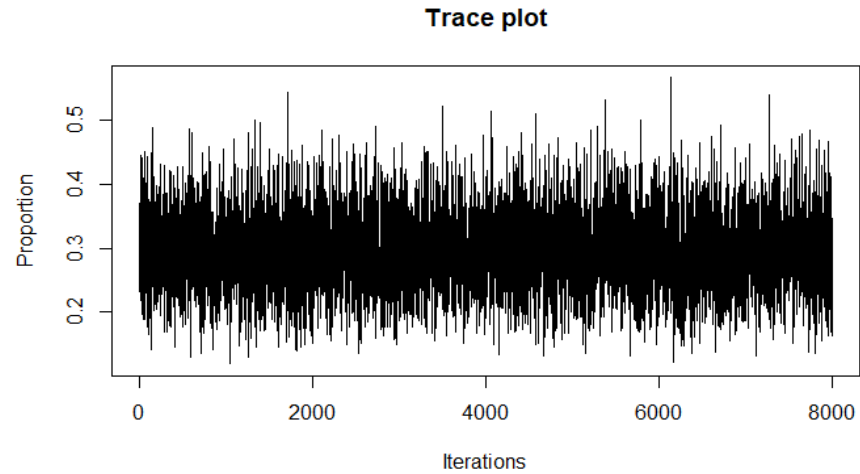
Then, we implement the proposal by [13] to assess the reliability of the posterior simulator. We use as test functions the first moments of p and p^2 . We fail to reject the null hypothesis of equal means across the two test functions, indicating that the posterior simulator is functioning correctly. To evaluate the effectiveness of the test, we run the *marginal-conditional simulator* with prior parameters $\alpha_0 = 1.22$ and $\beta_0 = 2.57$. In contrast, for the *successive-conditional simulator*, we use prior parameters $\alpha_0 = 2.57$ and $\beta_0 = 1.22$. In this case, we reject the null hypothesis in the two test functions, suggesting that the test performs well in this example.

*R code. Metropolis-Hastings algorithm: Ph.D.
students sleeping hours.*

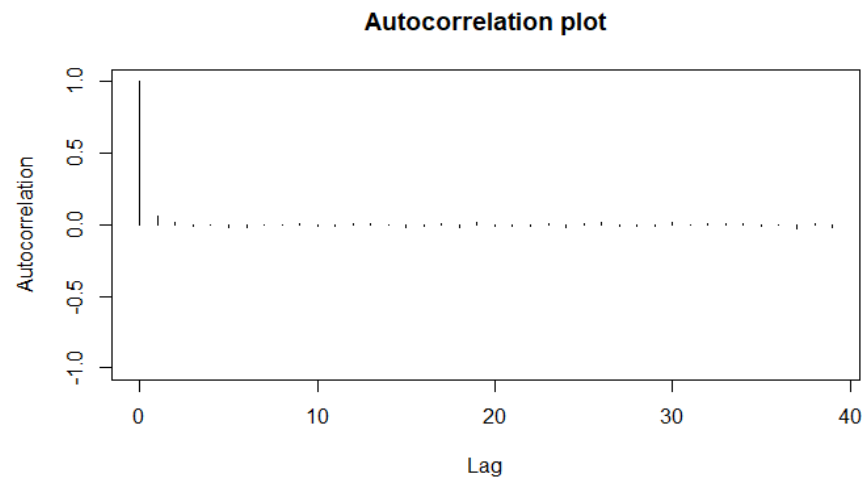
```

1 rm(list = ls()); set.seed(010101); an <- 16.55; bn <- 39.57
2 S <- 100000; p <- runif(S); accept <- rep(0, S)
3 for (s in 2:S){
4   pc <- runif(1)
5   a <- dbeta(pc, an, bn)/dbeta(p[s-1], an, bn)
6   U <- runif(1)
7   if(U <= a){ p[s] <- pc; accept[s] <- 1
8   }else{p[s] <- p[s-1]; accept[s] <- 0}
9 }
10 library(coda); library(latex2exp)
11 thetaPost <- mcmc(p)
12 plot(thetaPost, density = FALSE, main = "Trace plot", ylab =
   "Proportion")
13 autocorr.plot(thetaPost, main = "Autocorrelation plot")
14 summary(thetaPost); effectiveSize(thetaPost)
15 geweke.diag(thetaPost, frac1 = 0.1, frac2 = 0.5)
16 raftery.diag(thetaPost, q = 0.025, r = 0.005, s = 0.95)
17 heidel.diag(thetaPost, eps = 0.1, pvalue = 0.05)
18 burnin <- 20000; thin <- 10; keep <- seq(burnin, S, thin)
19 thetaPostNew <- mcmc(p[keep])
20 plot(thetaPostNew, density = FALSE, main = "Trace plot",
   ylab = "Proportion")
21 autocorr.plot(thetaPostNew, main = "Autocorrelation plot")
22 summary(thetaPostNew); effectiveSize(thetaPostNew)
23 geweke.diag(thetaPostNew, frac1 = 0.1, frac2 = 0.5)
24 raftery.diag(thetaPostNew, q = 0.025, r = 0.005, s = 0.95)
25 heidel.diag(thetaPostNew, eps = 0.1, pvalue = 0.05)
26 # Marginal-conditional simulator
27 a0 <- 1.22; b0 <- 2.57; ThetaPrior <- rbeta(S,a0,b0)
28 yPrior <- rbinom(S, 1, prob = ThetaPrior)
29 parsmcmc1 <- coda::mcmc(cbind(ThetaPrior, ThetaPrior^2))
30 Summ1 <- summary(parsmcmc1)
31 # Successive-conditional simulator
32 N <- 52
33 SucConSim <- function(a0, b0, par){
34   y <- rbinom(N, 1, prob = par)
35   an <- a0 + sum(y); bn <- b0 + N - sum(y); pc <- runif(1)
36   a <- dbeta(pc, an, bn)/dbeta(par, an, bn); U <- runif(1)
37   if(U <= a){par <- pc}else{par <- par}
38   return(par)
39 }
40 a0 <- 1.22; b0 <- 2.57 # a0 <- 2.57; b0 <- 1.22
41 par2 <- rbeta(1, a0, b0); pars2 <- rep(NA, S); pars2[1] <-
   par2
42 for(s in 2:S){
43   Res <- SucConSim(a0 = a0, b0 = b0, par = pars2[s-1])
44   pars2[s] <- Res
45 }
46 parsmcmc2 <- coda::mcmc(cbind(pars2, pars2^2))
47 Summ2 <- summary(parsmcmc2)
48 TestGeweke <- function(j){
49   Test <- (Summ1[["statistics"]][j,1] - Summ2[["statistics"]
   ][j,1])/(Summ1[["statistics"]][j,4]+Summ2[["statistics"]
   ][j,4])^0.5
50   Reject <- abs(Test) > qnorm(0.975); return(list(Test =
   Test, Reject = Reject))
51 }
52 TestGeweke(1); TestGeweke(2)

```

**FIGURE 4.10**

Trace plot of posterior draws: Proportion of students that sleep at least 6 hours.

**FIGURE 4.11**

Autocorrelation plot of posterior draws: Proportion of students that sleep at least 6 hours.

Part II

Regression models: A GUIded toolkit



5

Graphical user interface

This chapter introduces our graphical user interface (GUI) for conducting Bayesian regression analysis in a user-friendly environment that requires no programming skills, using a simple drag-and-drop design. The GUI is implemented as an interactive web application built with *shiny* [7] and integrates packages such as *MCMCpack* [29] and *bayesm* [43] within the **R** software environment [36]. It is primarily designed for teaching and applied purposes at the introductory level. In the subsequent chapters of the second part of this book, we present several applications that illustrate the potential of our GUI for applied researchers and practitioners.

5.1 Introduction

Our GUI enables users to perform inference using Bayesian regression analysis without requiring programming skills. The lack of such skills is often a significant impediment to the widespread adoption of the Bayesian framework [52, 23].

Several other graphical user interfaces are available for Bayesian regression analysis. *ShinyStan* [49] is a highly flexible and open-source program; however, it requires users to have some programming skills, as it is based on the *Stan* software for Bayesian data analysis [5]. *BugsXLA* [52] is also open source but less flexible, though it does not require any programming experience. *Bayesian Regression: Nonparametric and Parametric Models* [23] is a user-friendly and flexible GUI based on the *MATLAB Compiler* for 64-bit Windows systems. It primarily focuses on Bayesian nonparametric regression and is designed for users already familiar with basic parametric models, such as those implemented in our GUI. Additionally, there are tools such as the *MATLAB Toolkit*, *Stata*, and *BayES*, but these are not open source.

We developed our GUI as an interactive web application using *shiny* [7] and various libraries in **R** [35]. The specific libraries and commands used in our GUI are listed in Table A.1. It includes ten univariate models, four multivariate models, four time-series models, three hierarchical longitudinal models, and seven Bayesian model averaging frameworks. In addition, it provides basic

summaries and diagnostics of posterior chains, along with visualizations such as trace plots, autocorrelation plots, and density plots.

In terms of flexibility and functionality, our GUI lies between *ShinyStan* and *BugsXLA*: it does not require programming skills but is not as advanced as the software described in [23]. However, unlike the latter, our GUI runs on any operating system. We call our GUI **BEsmarter**,¹ and it is freely available at <https://github.com/besmarter/BSTApp>, where users can access all source code and datasets.

Simulated and applied datasets are stored in the **DataSim** and **DataApp** folders of our **GitHub** repository (see Tables A.2 and A.3 for details). The **DataSim** folder contains files used to simulate various processes, providing access to population parameters. These files serve as valuable pedagogical tools for illustrating the statistical properties of the inferential frameworks available in our GUI. The **DataApp** folder includes the datasets used throughout this book, which users can adopt as templates when structuring their own data.

There are four ways to install our GUI. The simplest method, requiring only the installation of **R** and, optionally, an **R** code editor, is to install *shiny*, and then type:

R code. How to display our graphical user interface

```
1 shiny::runGitHub("besmarter/BSTApp", launch.browser = T)
```

in the **R** package console or any **R** code editor, we strongly recommend typing this command manually rather than copying and pasting it, as quotation marks can sometimes cause issues.

The second option is to visit

<https://andres-ramirez-hassan.shinyapps.io/BSTApp/>

Please note: the free Posit Cloud tier sometimes runs out of memory, which can cause the app to stop. Sorry for the inconvenience.

The third option is to visit <https://fly-besmarter.fly.dev/>. As with Posit Cloud, occasional memory limits on the free tier may affect performance.

The fourth approach, and our recommendation, is using a **Docker** image by running:

1. `docker pull aramir21/besmartergui:latest`
2. `docker run -rm -p 3838:3838 aramir21/besmartergui`

¹Bayesian Econometrics: Simulations, Models, and Applications to Research, Teaching, and Encoding with Responsibility.

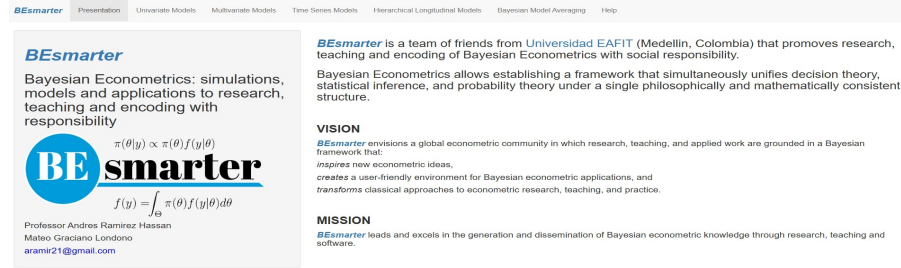


FIGURE 5.1
Display of graphical user interface.

in your **Command Prompt**. This command creates an isolated environment for our GUI, ensuring consistent performance across different systems. Note that **Docker** must be installed to deploy our GUI using this method. Before running these commands, make sure Docker Desktop is open and running. Users can then access the app by navigating to `127.0.0.1:3838` or `http://localhost:3838/`.

After using any of the four methods to run our GUI, users will see a new window displaying a presentation of our research team (see Figure 5.1). Additionally, the top panel in Figure 5.1 shows the categories of models that can be estimated in our GUI.

5.2 Univariate models

After deploying our GUI (see Figure 5.1), the user should select *Univariate Models* from the top panel. Then, Figure 5.2 is displayed, showing a radio button on the left-hand side that lists the specific models within this category. In particular, users can see that the normal model is selected from the univariate models class.

The right-hand panel then displays a widget for uploading the input dataset, which must be a *csv* file with headers in the first row. Users must also select the type of separator used in the input file —comma, semicolon, or tab- (see the **DataSim** and **DataApp** folders for input file templates). Once the dataset is uploaded, users can preview the data. Range sliders allow users to set the number of iterations for the Markov Chain Monte Carlo algorithm, specify the burn-in period, and adjust the thinning parameter (see the following chapters in this part of the book for technical details).

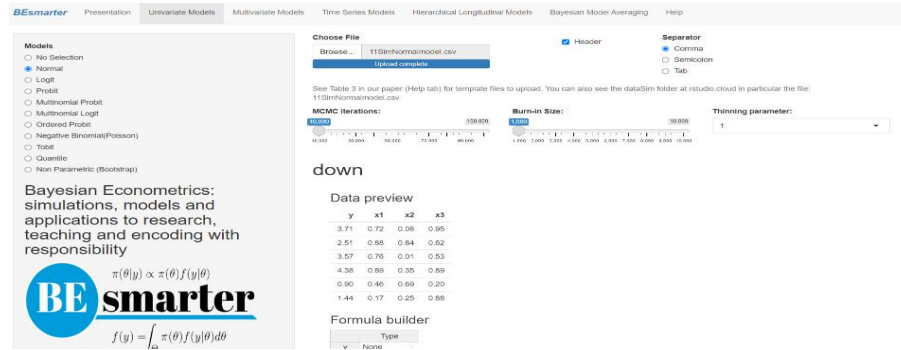


FIGURE 5.2
Univariate models: Specification.

Next, users must specify the model equation. This can be done using the formula builder, where they select the dependent and independent variables and then click on the *Build Formula* tab. The equation is displayed in the *Main Equation* window, formatted according to **R** syntax, for example, $y \sim x1 + x2 + x3$. Users can modify this expression as needed to include higher-order terms, interaction effects, or other transformations. These modifications must follow standard formula syntax.²

By default, univariate models include an intercept, except for ordered probit models, where the specification must explicitly exclude it due to *identification* constraints (see details below).³ Thus, users should specify this explicitly as follows: $y \sim x1 + x2 + x3 - 1$.

Finally, users must define the prior hyperparameters. For example, in the normal-inverse-gamma model, these include the mean vector, covariance matrix, shape parameter, and scale parameter (see Figure 5.3). However, our GUI uses *noninformative* hyperparameters by default across all modeling frameworks, so this step is optional.

After completing the specification process, users should click the *Go!* button to initiate estimation (see Figure 5.3). Once the process is complete, the GUI displays summary statistics and convergence diagnostics. In addition, widgets allow users to download the posterior chains (as *csv* files) and graphs (as *pdf* or *eps* files). Note that in the results—summary tables, posterior chains, and graphs—the coefficients are ordered such that location parameters appear first, followed by scale parameters.

For multinomial models (probit and logit), the dataset must be structured as follows: the first column should contain the dependent variable, followed by alternative-specific regressors (e.g., alternatives' price), and then

²See <https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/formula>

³An *identification* issue arises when multiple sets of model parameters yield the same likelihood function value.

Build formula

Main Equation
 $y = x1 + x2 + x3$

Introduce Formula. Example: $y = x1 + x2$ where y is the dependent variable, and $x1$ and $x2$ are independent variables. Model specification includes the constant term by default.

Introduce prior mean vector location parameters

Prior mean

	cte	x1	x2	x3
cte	0			
x1		0		
x2			0	
x3				0

Introduce prior covariances location parameters by row. It has to be symmetric.

	cte	x1	x2	x3
cte	1	0	0	0
x1		1	0	0
x2			1	0
x3				1

Prior Shape Parameter:
 Variance Parameter: 0.001

Prior Scale Parameter:
 Variance Parameter: 0.001

Introduce Prior shape Parameter. Example: 0.001

Introduce prior scale parameter. Example: 0.001

Click the button (Get) after importing the dataset and selecting the model to update the value displayed in the main panel.

Get

Warning: Be patient this may take several minutes!!!

[Download Posterior Chains](#) | [Download Posterior Graphs](#)

Working on it. Running MCMC sampling

FIGURE 5.3
 Univariate models: Priors.

non-alternative-specific regressors (e.g., income). The formula builder allows users to specify the dependent variable as well as both types of independent variables (see technical details in the next chapter). Users must also define the base category, the number of alternatives (which is also required for ordered probit models), the number of alternative-specific regressors, and the number of non-alternative-specific regressors (see Figure 5.4).

For multinomial logit models, users can additionally specify a tuning parameter corresponding to the degrees of freedom of the Metropolis–Hastings algorithm (see technical details in the next chapter). This tuning option is available in our GUI given that estimation relies on the Metropolis–Hastings algorithm.

In the results of these models, the coefficients are ordered as follows:

1. Intercepts (cte_l in the summary display, where l represents the alternative).
2. Non-alternative-specific regressors (NAS_{jl} in the summary display, where l represents the alternative and j the non-alternative regressor).
3. Alternative-specific regressors (AS_j in the summary display, where j represents the alternative-specific regressor).

Note that the non-alternative-specific regressors associated with the base category are set to zero and do not appear in the results. Additionally, due to identification constraints in multinomial and multivariate probit models, some coefficients in the main diagonal of the covariance matrix remain constant.

For the negative binomial model, users must specify a dispersion parameter (see the next chapter for details). Similarly, for Tobit and quantile models, users need to define the censorship points and quantiles, respectively.

The Bayesian bootstrap method only requires uploading a dataset, specifying the number of MCMC iterations, burn-in size, and defining the equation

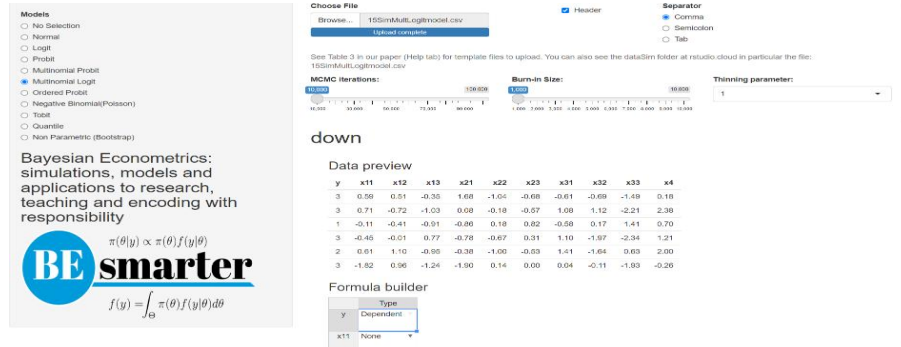


FIGURE 5.4
Univariate models: Multinomial.

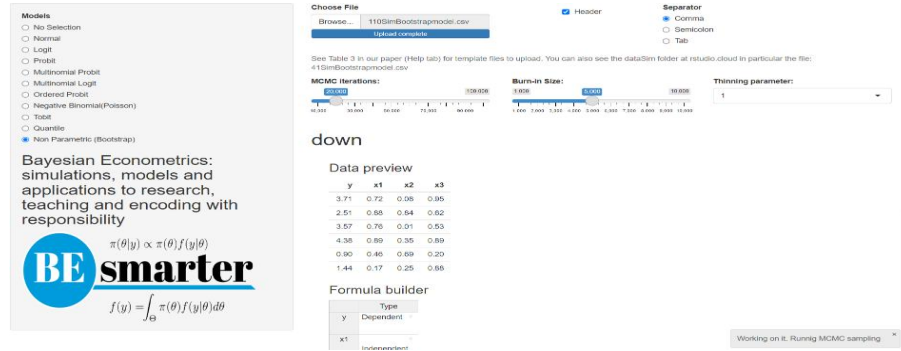


FIGURE 5.5
Univariate models: Bootstrap.

(see Figure 5.5). The input file should follow the same structure as the one used for the univariate normal model.

5.3 Multivariate models

After deploying our GUI (see Figure 5.1), the user should select *Multivariate Models* from the top panel. Figure 5.6 will then appear, displaying a radio button on the left-hand side that lists the specific models within this category.

Figure 5.6 illustrates the setup for multivariate regression models. The input file should first include the dependent variables, followed by the regressors. If each equation includes an intercept, a column of ones must be added after

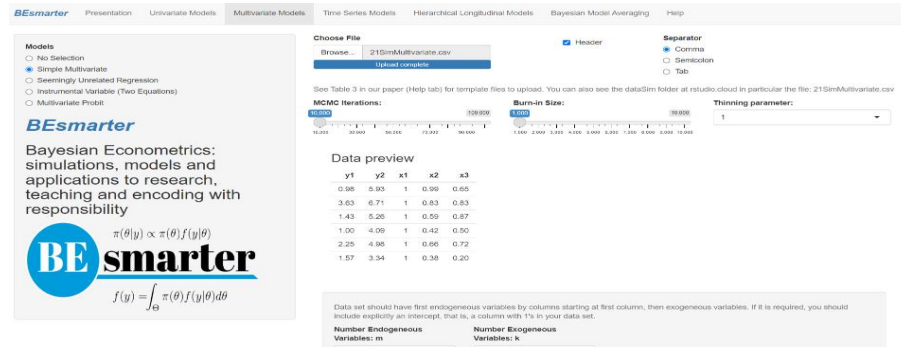


FIGURE 5.6
Multivariate models: Simple multivariate.

the dependent variables in the input file. Once the file is uploaded, users can preview the data.

The user must then specify the number of dependent variables and regressors, indicate whether an intercept should be included, and define the hyperparameter values (see Figure 5.6).

In seemingly unrelated regressions, the input file should first include the dependent variables, followed by the regressors for each equation, including the intercept (a column of ones) if applicable. Users must specify the number of dependent variables (equations), the total number of regressors (i.e., the sum of all regressors across equations), and the number of regressors per equation (including the intercept if relevant). Users may also define the values of the hyperparameters if prior information is available.

For both simple multivariate and seemingly unrelated regression models, the results first display the posterior estimates of the location parameters by equation, followed by the posterior covariance matrix.

In the instrumental variable setting, users must specify both the main equation and the instrumental equation. Intercepts are included by default. The first variable on the right-hand side of the main equation must correspond to the endogenous regressor. In the instrumental equation, this endogenous variable serves as the dependent variable and is modeled as a function of the instruments. Users may also specify hyperparameter values if prior information is available. The input file should contain the dependent variable, the endogenous regressor, the instruments, and the exogenous regressors. The results first report the posterior estimates of the endogenous regressor, followed by the location parameters of the auxiliary (instrumental) regression, the location parameters of the exogenous regressors, and finally, the posterior covariance matrix.

The multivariate probit model requires the input dataset to be ordered by unit. For example, if there are three choices, each unit should appear three

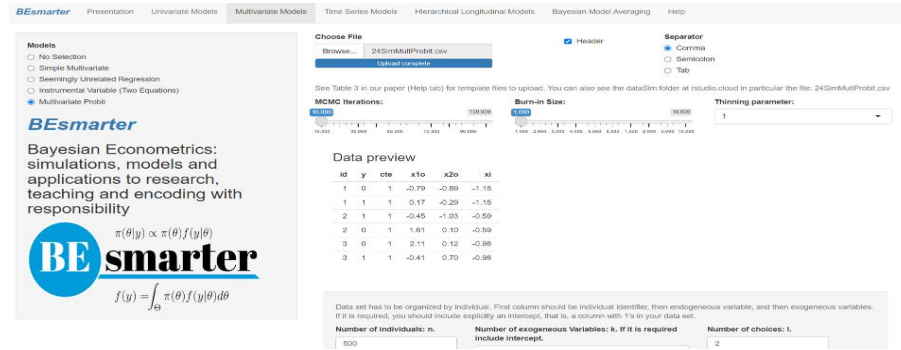


FIGURE 5.7
Multivariate models: Multivariate probit.

times in the dataset. The first column must contain a unique identifier for each unit, using ordered integers. The next column should contain the dependent variable, represented as a single vector of 0s and 1s, followed by the regressors, which must include a column of ones for the intercepts. Users must specify the number of units, the number of regressors, and the number of choices (see Figure 5.7). The results first display the posterior estimates of the location parameters by equation, followed by the posterior covariance matrix.

5.4 Time series models

After our GUI is deployed (see Figure 5.1), the user should select *Time Series Models* from the top panel. Then, Figure 5.8 will be displayed, and the user will see the radio button on the left-hand side, which shows the specific models within this general class.

Users can perform inference using dynamic linear models (DLM), autoregressive moving average (ARMA) models, stochastic volatility models (SVM), and vector autoregressive (VAR) models. Users should upload a dataset, which must be a *csv* file with headers in the first row. The files for DLMs and SVMs have the same structure: the first column contains the dependent variable, followed by the independent variables. For ARMA models, there is only one column with the modeled variable, while VAR models have each modeled variable in a separate column. Note that this version of the GUI does not allow for exogenous variables in VAR models. Users should specify the separator used in the input file: comma, semicolon, or tab. A dataset preview is displayed once the file is uploaded. Dataset templates can be found in the folders **DataSim**

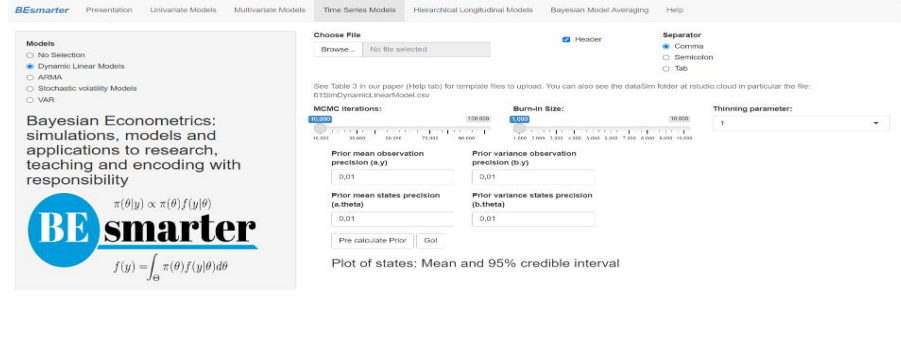


FIGURE 5.8
Time series models.

(see Table A.2 for details) and **DataApp** (see Table A.3 for details) in our *GitHub* repository.

Next, users should set the MCMC and burn-in iterations using the range sliders and the thinning parameter using the input box.

To estimate DLMs, users should set the hyperparameters for the precision of the observation equation and the state equations (means and variances) if prior information is available. Otherwise, users can click the *Pre Calculate Prior* button, where these hyperparameters are estimated based on a recursive model estimation using ordinary least squares (OLS). The sample size is progressively increased, and the location parameters are saved. The GUI then computes the covariance matrix of this sequence and uses it to set the prior mean for the precision of the state vector, which is equal to the inverse of the maximum element on the main diagonal of the covariance matrix (*a.theta*). The prior variance is set to ten times this value (*b.theta*). For the observation equation, the prior mean of the precision is set to the inverse of the OLS variance estimate (*a.y*), and the prior variance is set to ten times this value (*b.y*). This is a rudimentary approach to setting these hyperparameters, and users are encouraged to use a more thoughtful process.

Next, users should click the *Go!* button to start estimating the model. This may take a few minutes, as DLMs are complex to estimate. Users should be patient. Once the estimation is complete, the GUI will display graphs of the states (mean and 95% credible intervals), summary statistics of the posterior chains for the observation and state variances, and convergence diagnostics. Users can download the mean and the lower and upper limits of the 95% credible intervals of the states, as well as the posterior chains for the variances.

For ARMA models, users need to set the frequency (annual -1-, quarterly -4-, and monthly -12-), as well as the AR and MA orders (see Figure 5.9). Then, users should set the location and scale hyperparameters for the intercept, autoregressive (AR), moving average (MA), and standard deviation terms. Note that there is only one set of hyperparameters for the AR and MA

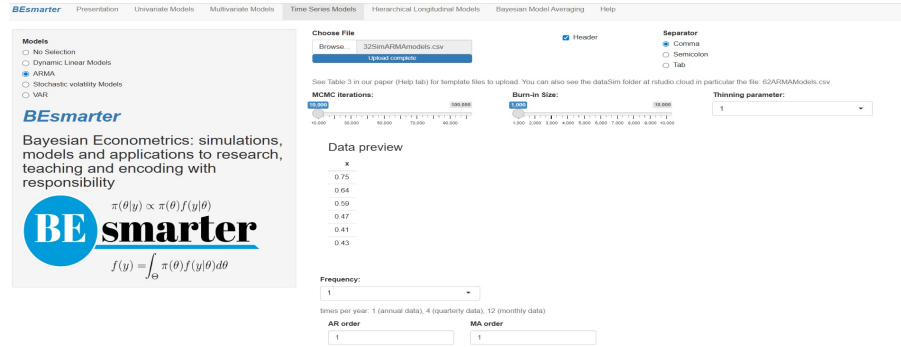


FIGURE 5.9
Time series models: ARMA specification.

coefficients. This step is optional, as the GUI uses non-informative priors by default.

Then, users should click the *Go!* button, and the GUI will start estimating the model. The GUI will display the summary statistics of the posterior draws and the convergence diagnostics. The order is AR coefficients (if any), MA coefficients (if any), intercept, and standard deviation. Users can download the posterior chains and figures (density, autocorrelation, and trace plots).

Estimation of the SVMs requires setting the coefficients of the mean and standard deviation of the Gaussian prior for the regression parameters, the mean and standard deviation for the Gaussian prior distribution of the level of the log-volatility, shape parameters for the Beta prior distribution of the transformed persistence parameter, and a positive real number representing the scaling of the transformed volatility of log-volatility. However, this step is not necessary, as by default our GUI uses the default values in the *stochvol* package.

Then, click the *Go!* button, wait for the estimation to be completed, and the GUI will display the stochastic volatility plot (mean and 95% credible interval). Users can also view the summary and diagnostics of the posterior chains (see Figure 5.10). In addition, users can download the mean and the lower and upper limits of the 95% credible intervals of the stochastic volatility, as well as the posterior chains of the variances.

To estimate VAR models, users should set the number of lags, the impulse response and forecast periods, the three coefficients of the Minnesota prior, and the type of impulse response (*forecast error impulse response* -feir- or *orthogonalized impulse response* -other-, both cumulative or non-cumulative). See Chapter 8 for details, Section ??.

Click the *Go!* button, and after a few minutes, users will be able to see the plots of the impulse responses and forecasts (means and 95% credible intervals). Click the *Download Results* button, and a zip file with .csv files

**FIGURE 5.10**

Time series models: Stochastic volatility results.

containing the impulse responses and forecasts, along with their plots, will be downloaded.

5.5 Longitudinal/panel models

After our GUI is deployed (see Figure 5.1), the user should select *Hierarchical Longitudinal Models* in the top panel. Then, Figure 5.11 will be displayed, and the user can see the radio button on the left-hand side that shows the specific models inside this generic class.

The hierarchical longitudinal models tab allows for estimating models that account for within-subject correlation when the dependent variable is continuous (Normal), binary (Logit), or a count (Poisson).

The input files for hierarchical longitudinal models should first include the dependent variable, followed by the regressors and a cross-sectional identifier ($i = 1, 2, \dots, N$). It is not a requirement to have a balanced dataset: T_i can be different for each i (see Chapter 9 for technical details). Users can see templates of datasets in the folders **DataSim** (see Table A.2 for details) and **DataApp** (see Table A.3 for details) in our *GitHub* repository. When the dataset is uploaded, users will have a preview of it.

Users should also specify the fixed part equation and the random part equation, both in **R** format. If only random intercepts are required, do not enter anything in the latter part (see Figure 5.11). Users should also type the name of the cross-sectional identifier variable (*species* in Figure 5.11). The results displayed and the posterior graphs are associated with the fixed effects and covariance matrix. However, users can download the posterior chains of all posterior estimates: fixed and random effects, and the covariance matrix.

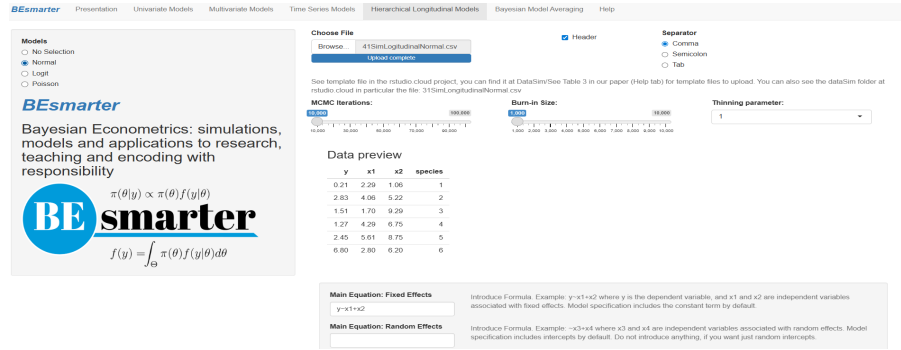


FIGURE 5.11
Hierarchical longitudinal models: Specification.

5.6 Bayesian model averaging

After our GUI is deployed (see Figure 5.1), the user should select *Bayesian Model Averaging* in the top panel. Then, Figure 5.12 will be displayed, and the user can see the radio button on the left-hand side that shows the specific models inside this generic class.

Bayesian model averaging (BMA) based on a Gaussian distribution can be carried out using the Bayesian information criterion (BIC) approximation, Markov chain Monte Carlo model composition (MC3), instrumental variables, and dynamic BMA (see Figure 5.12). The first two approaches require an input dataset where the first column is the dependent variable, followed by the potentially important regressors.

Users should set the bandwidth model selection parameter (O_R) and the number of iterations for BIC and MC3, respectively (see Chapter 10 for technical details). The results include the posterior inclusion probability ($p \neq 0$), expected value (EV), and standard deviation (SD) of the coefficients associated with each regressor. The BIC framework also displays the most relevant models with their posterior model probabilities (PMP). Users can download two *csv* files: *Best models* and *Descriptive statistics coefficients*. The former is a 0-1 matrix such that the columns are the regressors and the rows are the models; a 1 indicates the presence of a specific regressor in a specific model, and 0 indicates its absence. Note that the last column of this file is the posterior model probability for each model (row). The latter file shows the posterior inclusion probabilities, expected values, and standard deviations associated with each regressor, taking into account the BMA procedure based on the best models.

Bayesian model averaging with endogeneity issues requires two input files. The first file should have the dependent variable in the first column, followed

BEsmarter Bayesian Model Averaging

Bayesian Econometrics: simulations, models and applications to research, teaching and encoding with responsibility

$\pi(\theta|y) \propto \pi(\theta)f(y|\theta)$

$f(y) = \int_{\Theta} \pi(\theta)f(y|\theta)d\theta$

Which type do you want to perform?

☒ BIC

☐ MC3

☐ Instrumental variable

☐ Dynamic BMA

Choose File

No file selected

Header

Separator

☒ Comma

☐ Semicolon

☐ Tab

See Table 3 in our paper (help tab) for template files to upload. You can also see the dataSim folder at rstudio cloud in particular the file: 513SimNormalBMA.csv

OR: Number between 5 and 50.

50

Using BIC approximation: be patient! This can take time.

Data preview

Show 10 entries

V1

No results yet

Upload data and click the go button

FIGURE 5.12

Bayesian model averaging: Specification and results.

BEsmarter Bayesian Model Averaging

Bayesian Econometrics: simulations, models and applications to research, teaching and encoding with responsibility

$\pi(\theta|y) \propto \pi(\theta)f(y|\theta)$

$f(y) = \int_{\Theta} \pi(\theta)f(y|\theta)d\theta$

Which type do you want to perform?

☐ BIC

☒ MC3

☒ Instrumental variable

☐ Dynamic BMA

Choose File

See Table 3 in our paper (help tab) for template files to upload. You can also see the dataSim folder at rstudio cloud in particular the file: 513SimNormalBMA.csv

OR: Number between 5 and 50.

50

Using BIC approximation: be patient! This can take time.

Data preview

Show 10 entries

V1

No results yet

Upload data and click the go button

Instrumental variable setting

$$Y = X\beta_{end} + W\beta_{ex} + \epsilon_Y$$

$$X = Z\gamma + W\beta_{ex} + \epsilon_X$$

Find file should include a matrix [Y X W], where Y are main interest variables, X are endogenous regressors, and W are exogenous regressors. Second file includes only the instruments (Z).

Run in Sample

Working on it. Running MCMC sampling.

FIGURE 5.13

Bayesian model averaging: Instrumental variable specification.

by the regressors with endogeneity issues, and then the exogenous regressors. The user should include a column of 1's if an intercept is required. The second input file contains all the instruments. Users should also specify the number of regressors with endogeneity issues (see Figure 5.13).

The results include the posterior inclusion probabilities and expected values for each regressor. The user can find the results of the main equation, and then of the auxiliary equations. Users can download *csv* files of BMA results for both the second stage (main equation) and the first stage (auxiliary equations). In addition, users can download the posterior chains of the location parameters of the main equation, β_l , $l = 1, 2, \dots, \dim\{\beta\}$, the location parameters of the auxiliary equations, $\gamma_{j,i}$, $j = 1, 2, \dots, \dim\{\beta_s\}$ where $\dim\{\beta_s\}$ is the number of regressors with endogeneity issues, $i = 1, 2, \dots, \dim\{\gamma\}$, where $\dim\{\gamma\}$ is the number of regressors in the auxiliary regressors (exogenous regressors + instruments), and the elements of the covariance matrix $\sigma_{j,k}$ (see Chapter 10 for technical details).

Dynamic BMA also requires two files. The first is the dataset with the dependent variable and potential regressors, and the second file describes the competing models. There is one column for each regressor and one row for each competing model; 0 indicates that the regressor is not in the model, and 1 indicates that it is in the model. Users can see templates of this file in the folders **DataSim** (see Table A.2 for details) and **DataApp** (see Table A.3 for details) of our *GitHub* repository.

Then, the users should set the *forgetting parameters* of the covariance and transition matrices and click the *Go!* button. A plot of the PMPs of the competing models is displayed, and users can click the *Download the results for DBMA*. Two files are downloaded: the first contains the dynamic Bayesian model averaging filtering recursions for each state, and the second contains the PMP of each model and the dynamic Bayesian model averaging prediction.

Bayesian model averaging based on BIC approximation for non-linear models (Logit, Gamma, and Poisson) requires an input dataset where the first column is the dependent variable, and the other columns are the potentially relevant regressors. Users should specify the bandwidth model selection parameters, also referred to as Occam's window parameters (O_R and O_L). Our GUI displays the posterior inclusion probabilities ($p^i = 0$), the expected value of the posterior coefficients (EV), and the standard deviation (SD). In addition, users can view the results associated with the models with the highest posterior model probabilities and download *csv* files with the results of specifications of the best models and descriptive statistics of the posterior coefficients from the BMA procedure. These files are similar to the results of the BIC approximation for the Gaussian model.

5.7 Help

The last tab in our GUI is *Help*. There, users can find links to both the online *HTML* version of the book,⁴ and the *PDF* version.⁵ Users are also welcome to contact me at *aramir21@gmail.com* with any questions, comments, or suggestions.

5.8 Warning

Users should also note that sometimes our GUI shuts down. In our experience, this is due to computational issues arising from the implicit commands we call

⁴<https://bookdown.org/aramir21/IntroductionBayesianEconometricsGuidedTour/>

⁵https://drive.google.com/file/d/1_IBKe7vS5a2XLnvg74T_UNESRoKNDUUt/edit

when estimating certain models. These issues may include computationally singular systems, missing values where TRUE/FALSE are needed, L-BFGS-B requiring finite values for “fn”, NA/NaN/Inf values, or errors in `backsolve`. These issues can sometimes be resolved by adjusting the dataset, such as avoiding high levels of multicollinearity.

It should also be noted that when warning messages are displayed in our GUI, there is a high likelihood of convergence issues with the posterior chains. Therefore, the results may not be trustworthy. Users can identify these problems by checking the console in their *RStudio* sessions, where the specific folder/file where the issue occurred will be specified. In any case, we would appreciate your feedback to improve and enhance our GUI.

We should also mention that there are many ways to improve the codes presented in this book. For instance, the *MCMCpack* and *bayesm* packages perform most of the matrix operations in C++ using the *Rcpp* package. This substantially speeds up the algorithms compared to the codes presented in the next chapters when we program from scratch the samplers. We could further improve the computational time of our codes using parallel computing and the *Rcpp* package, but this requires more advanced skills that are not covered in this book.



6

Univariate models

Solutions of Exercises

1. Derive the posterior conditional distributions of the Gaussian linear model assuming independent priors $\pi(\boldsymbol{\beta}, \sigma^2) = \pi(\boldsymbol{\beta}) \times \pi(\sigma^2)$, where $\boldsymbol{\beta} \sim N(\boldsymbol{\beta}_0, \mathbf{B}_0)$ and $\sigma^2 \sim IG(\alpha_0/2, \delta_0/2)$.

Answer

The joint posterior distribution of the parameters is

$$\begin{aligned}
 \pi(\boldsymbol{\beta}, \sigma^2 | \mathbf{y}, \mathbf{X}) &\propto p(\mathbf{y} | \boldsymbol{\beta}, \sigma^2, \mathbf{X}) \pi(\boldsymbol{\beta}) \pi(\sigma^2) \\
 &\propto (\sigma^2)^{-\frac{N}{2}} \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \right\} \\
 &\quad \times \exp \left\{ -\frac{1}{2} (\boldsymbol{\beta} - \boldsymbol{\beta}_0)^\top \mathbf{B}_0^{-1} (\boldsymbol{\beta} - \boldsymbol{\beta}_0) \right\} \\
 &\quad \times \frac{(\delta_0/2)^{(\alpha_0/2)}}{\Gamma(\alpha_0/2)} \frac{1}{(\sigma^2)^{(\alpha_0/2+1)}} \exp \left\{ -\frac{\delta_0}{2\sigma^2} \right\} \\
 &\propto \exp \left\{ -\frac{1}{2} (\boldsymbol{\beta} - \boldsymbol{\beta}_0)^\top \mathbf{B}_0^{-1} (\boldsymbol{\beta} - \boldsymbol{\beta}_0) \right\} \\
 &\quad \times \frac{1}{(\sigma^2)^{(\alpha_0+N)/2+1}} \exp \left\{ -\frac{\delta_0 + (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})}{2\sigma^2} \right\} \\
 &= \exp \left\{ -\frac{1}{2} (\boldsymbol{\beta} - \boldsymbol{\beta}_0)^\top \mathbf{B}_0^{-1} (\boldsymbol{\beta} - \boldsymbol{\beta}_0) \right\} \times \underbrace{\frac{1}{(\sigma^2)^{(\frac{\alpha_n}{2}+1)}} \exp \left\{ -\frac{\delta_n}{2\sigma^2} \right\}}_1.
 \end{aligned}$$

Observe that (1) is the kernel of an inverse-gamma density function. Thus, $\sigma^2 | \boldsymbol{\beta}, \mathbf{y}, \mathbf{X} \sim IG(\alpha_n/2, \delta_n/2)$, where $\alpha_n = \alpha_0 + N$ and $\delta_n = \delta_0 + (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$.

Let's see the posterior distribution of β ,

$$\begin{aligned}
\pi(\beta, \sigma^2 | \mathbf{y}, \mathbf{X}) &\propto p(\mathbf{y} | \beta, \sigma^2, \mathbf{X}) \pi(\beta) \pi(\sigma^2) \\
&\propto (\sigma^2)^{-\frac{N}{2}} \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\beta)^\top (\mathbf{y} - \mathbf{X}\beta) \right\} \\
&\times \exp \left\{ -\frac{1}{2} (\beta - \beta_0)^\top \mathbf{B}_0^{-1} (\beta - \beta_0) \right\} \\
&\times \frac{(\delta_0/2)^{(\alpha_0/2)}}{\Gamma(\alpha_0/2)} \frac{1}{(\sigma^2)^{(\alpha_0/2+1)}} \exp \left\{ -\frac{\delta_0}{2\sigma^2} \right\} \\
&= (\sigma^2)^{-\frac{N}{2}} \exp \left\{ -\frac{\sigma^{-2}}{2} [\mathbf{y}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\beta - \beta^\top \mathbf{X}^\top \mathbf{y} + \beta^\top \mathbf{X}^\top \mathbf{X}\beta] \right\} \\
&\times \exp \left\{ -\frac{1}{2} [\beta^\top \mathbf{B}_0^{-1} \beta - \beta^\top \mathbf{B}_0^{-1} \beta_0 - \beta_0^\top \mathbf{B}_0^{-1} \beta + \beta_0^\top \mathbf{B}_0^{-1} \beta_0] \right\} \\
&\times \frac{(\delta_0/2)^{(\alpha_0/2)}}{\Gamma(\alpha_0/2)} \frac{1}{(\sigma^2)^{(\alpha_0/2+1)}} \exp \left\{ -\frac{\delta_0}{2\sigma^2} \right\} \\
&\propto \exp \left\{ -\frac{1}{2} [\beta^\top (\mathbf{B}_0^{-1} + \sigma^{-2} \mathbf{X}^\top \mathbf{X}) \beta - 2\beta^\top (\mathbf{B}_0^{-1} \beta_0 + \sigma^{-2} \mathbf{X}^\top \mathbf{X} \hat{\beta})] \right\} \\
&\times \frac{1}{(\sigma^2)^{(\alpha_0+N)/2+1}} \exp \left\{ -\frac{\delta_0 + \mathbf{y}^\top \mathbf{y}}{2\sigma^2} \right\},
\end{aligned}$$

where $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

Adding and subtracting $\beta_n^\top \mathbf{B}_n^{-1} \beta_n$ where

$$\begin{aligned}
\mathbf{B}_n &= (\mathbf{B}_0^{-1} + \sigma^{-2} \mathbf{X}^\top \mathbf{X})^{-1} \\
\beta_n &= \mathbf{B}_n (\mathbf{B}_0^{-1} \beta_0 + \sigma^{-2} \mathbf{X}^\top \mathbf{X} \hat{\beta}) = \mathbf{B}_n (\mathbf{B}_0^{-1} \beta_0 + \sigma^{-2} \mathbf{X}^\top \mathbf{y}),
\end{aligned}$$

and completing the square

$$\begin{aligned}
\pi(\beta, \sigma^2 | \mathbf{y}, \mathbf{X}) &\propto \exp \left\{ -\frac{1}{2} [\beta^\top (\mathbf{B}_0^{-1} + \sigma^{-2} \mathbf{X}^\top \mathbf{X}) \beta - 2\beta^\top \mathbf{B}_n^{-1} \mathbf{B}_n (\mathbf{B}_0^{-1} \beta_0 + \sigma^{-2} \mathbf{X}^\top \mathbf{X} \hat{\beta}) \right. \\
&\quad \left. + \beta_n^\top \mathbf{B}_n^{-1} \beta_n - \beta_n^\top \mathbf{B}_n^{-1} \beta_n] \right\} \times \frac{1}{(\sigma^2)^{(\alpha_0+N)/2+1}} \exp \left\{ -\frac{\delta_0 + \mathbf{y}^\top \mathbf{y}}{2\sigma^2} \right\} \\
&= \exp \left\{ -\frac{1}{2} [\beta^\top \mathbf{B}_n^{-1} \beta - 2\beta^\top \mathbf{B}_n^{-1} \beta_n + \beta_n^\top \mathbf{B}_n^{-1} \beta_n] \right\} \\
&\times \frac{1}{(\sigma^2)^{(\alpha_0+N)/2+1}} \exp \left\{ -\frac{\delta_0 + \mathbf{y}^\top \mathbf{y} - \sigma^2 \beta_n^\top \mathbf{B}_n^{-1} \beta_n}{2\sigma^2} \right\} \\
&= \exp \left\{ -\frac{1}{2} (\beta - \beta_n)^\top \mathbf{B}_n^{-1} (\beta - \beta_n) \right\} \\
&\quad \underbrace{\hspace{10em}}_1 \\
&\times (\sigma^2)^{-(\frac{\alpha_n}{2}+1)} \exp \left\{ -\frac{\delta^*}{2\sigma^2} \right\},
\end{aligned}$$

where $\delta^* = \delta_0 + \mathbf{y}^\top \mathbf{y} + \sigma^2 \boldsymbol{\beta}_n^\top \mathbf{B}_n^{-1} \boldsymbol{\beta}_n$ does not depend on $\boldsymbol{\beta}$.

We can see that (1) is the kernel of a multivariate normal distribution with mean equal to $\boldsymbol{\beta}_n$ and covariance matrix $\mathbf{B}_n$, that is, $\boldsymbol{\beta} | \sigma^2, \mathbf{y}, \mathbf{X} \sim N(\boldsymbol{\beta}_n, \mathbf{B}_n)$.

We see that the posterior distributions are from the same family as the prior distributions.

2. Consider the model $y_i \sim N(\mathbf{x}_i^\top \boldsymbol{\beta}, \sigma^2 / \tau_i)$ (Gaussian linear model with heteroskedasticity) and independent priors $\pi(\boldsymbol{\beta}, \sigma^2, \boldsymbol{\tau}) = \pi(\boldsymbol{\beta}) \times \pi(\sigma^2) \times \prod_{i=1}^N \pi(\tau_i)$, where $\boldsymbol{\beta} \sim N(\boldsymbol{\beta}_0, \mathbf{B}_0)$, $\sigma^2 \sim IG(\alpha_0/2, \delta_0/2)$, and $\tau_i \sim G(v/2, v/2)$.

Show that $\boldsymbol{\beta} | \sigma^2, \boldsymbol{\tau}, \mathbf{y}, \mathbf{X} \sim N(\boldsymbol{\beta}_n, \mathbf{B}_n)$, $\sigma^2 | \boldsymbol{\beta}, \boldsymbol{\tau}, \mathbf{y}, \mathbf{X} \sim IG(\alpha_n/2, \delta_n/2)$, and $\tau_i | \boldsymbol{\beta}, \sigma^2, \mathbf{y}, \mathbf{X} \sim G(v_{1n}/2, v_{2in}/2)$, where $\boldsymbol{\tau} = [\tau_1 \dots \tau_N]^\top$, $\mathbf{B}_n = (\mathbf{B}_0^{-1} + \sigma^{-2} \mathbf{X}^\top \Psi \mathbf{X})^{-1}$, $\boldsymbol{\beta}_n = \mathbf{B}_n(\mathbf{B}_0^{-1} \boldsymbol{\beta}_0 + \sigma^{-2} \mathbf{X}^\top \Psi \mathbf{y})$, $\alpha_n = \alpha_0 + N$, $\delta_n = \delta_0 + (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top \Psi (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$, $v_{1n} = v + 1$, $v_{2in} = v + \sigma^{-2} (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2$, and $\Psi = \text{diag}(\tau_i)$.

Answer

The joint posterior distribution of the parameters is

$$\begin{aligned} \pi(\boldsymbol{\beta}, \sigma^2, \boldsymbol{\tau} | \mathbf{y}, \mathbf{X}) &\propto p(\mathbf{y} | \boldsymbol{\beta}, \sigma^2, \boldsymbol{\tau}, \mathbf{X}) \pi(\boldsymbol{\beta}) \pi(\sigma^2) \prod_{i=1}^N \pi(\tau_i) \\ &\propto \left(\prod_{i=1}^N \tau_i^{1/2} \right) (\sigma^2)^{-\frac{N}{2}} \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top \Psi (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \right\} \\ &\quad \times \exp \left\{ -\frac{1}{2} (\boldsymbol{\beta} - \boldsymbol{\beta}_0)^\top \mathbf{B}_0^{-1} (\boldsymbol{\beta} - \boldsymbol{\beta}_0) \right\} \\ &\quad \times \frac{1}{(\sigma^2)^{(\alpha_0/2+1)}} \exp \left\{ -\frac{\delta_0}{2\sigma^2} \right\} \prod_{i=1}^N \tau_i^{v/2-1} \exp \{ -v\tau_i/2 \}. \end{aligned}$$

Thus, the conditional posterior distribution of $\sigma^2 | \boldsymbol{\beta}, \boldsymbol{\tau}, \mathbf{y}, \mathbf{X}$ is given by

$$\pi(\sigma^2 | \boldsymbol{\beta}, \boldsymbol{\tau}, \mathbf{y}, \mathbf{X}) \propto (\sigma^2)^{-\frac{N+\alpha_0}{2}} \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top \Psi (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + \delta_0 \right\}.$$

This is the kernel of an inverse-gamma distribution with shape parameter $\alpha_n/2$ and rate parameter $\delta_n/2$.

The conditional posterior distribution of $\tau_i | \boldsymbol{\beta}, \sigma^2, \mathbf{y}, \mathbf{X}$ is

$$\pi(\tau_i | \boldsymbol{\beta}, \sigma^2, \mathbf{y}, \mathbf{X}) \propto \tau_i^{(v+1)/2-1} \exp \left\{ -\frac{\tau_i}{2} [\sigma^{-2} (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + v] \right\}.$$

This is the kernel of a gamma distribution with parameter $(v + 1)/2$ and $(\sigma^{-2}(y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + v)/2$.

The conditional posterior distribution of $\boldsymbol{\beta} | \sigma^2, \boldsymbol{\tau}, \mathbf{y}, \mathbf{X}$ is given by

$$\pi(\boldsymbol{\beta} | \sigma^2, \boldsymbol{\tau}, \mathbf{y}, \mathbf{X}) \propto \exp \left\{ -\frac{1}{2} [\sigma^{-2}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top \Psi(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + (\boldsymbol{\beta} - \boldsymbol{\beta}_0)^\top \mathbf{B}_0^{-1}(\boldsymbol{\beta} - \boldsymbol{\beta}_0)] \right\}.$$

Following same steps as in the previous exercise we get $\boldsymbol{\beta} | \sigma^2, \boldsymbol{\tau}, \mathbf{y}, \mathbf{X} \sim N(\boldsymbol{\beta}_n, \mathbf{B}_n)$.

3. The market value of soccer players in Europe continues.

Use the setting of the previous exercise to perform inference using a Gibbs sampling algorithm for the market value of soccer players in Europe, setting $v = 5$ and using the same hyperparameters as in the homoscedastic case. Is there any meaningful difference in the coefficient associated with the national team compared to the homoscedastic case?

Answer

R. code. The value of soccer players, programming our Gibbs sampler (heteroskedastic case)

```

1 rm(list = ls())
2 set.seed(010101)
3 ##### Linear regression: Value of
  soccer players #####
4 Data <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/1
  ValueFootballPlayers.csv", sep = ",", header = TRUE,
  quote = "")
5 attach(Data)
6 y <- log(Value)
7 # Value: Market value in Euros (2017) of soccer players
8 # Regressors quantity including intercept
9 X <- cbind(1, Perf, Age, Age2, NatTeam, Goals, Exp, Exp2)
10 # Perf: Performance. Perf2: Performance squared. Age: Age;
  Age2: Age squared.
11 # NatTeam: Indicator of national team. Goals: Scored goals.
  Goals2: Scored goals squared
12 # Exp: Years of experience. Exp2: Years of experience
  squared. Assists: Number of assists
13 k <- dim(X)[2]
14 N <- dim(X)[1]
15 # Hyperparameters
16 d0 <- 0.001
17 a0 <- 0.001
18 b0 <- rep(0, k)
19 c0 <- 1000
20 B0 <- c0*diag(k)
21 B0i <- solve(B0)
22 v <- 5
23 # MCMC parameters
24 mcmc <- 5000
25 burnin <- 5000
26 tot <- mcmc + burnin
27 thin <- 1
28 # Posterior distributions programming the Gibbs sampling
29 # Auxiliary parameters
30 an <- a0 + N
31 v1n <- v + 1
32 # Gibbs sampling functions
33 PostSig2 <- function(Beta, tau){
34   dn <- d0 + t(y - X%*Beta)%*diag(tau)%*(y - X%*Beta)
35   sig2 <- invgamma::rinvgamma(1, shape = an/2, rate = dn/2)
36   return(sig2)
37 }
38 PostBeta <- function(sig2, tau){
39   Bn <- solve(B0i + sig2^(-1)*t(X)%*diag(tau)%*X)
40   bn <- Bn%*(B0i%*b0 + sig2^(-1)*t(X)%*diag(tau)%*y)
41   Beta <- MASS::mvrnorm(1, bn, Bn)
42   return(Beta)
43 }
44 PostTau <- function(sig2, Beta, i){
45   v2n <- v + sig2^(-1)*(y[i]-X[i,]%*Beta)^2
46   tau_i <- rgamma(1, v1n/2, v2n/2)
47   return(tau_i)
48 }

```

R. code. The value of soccer players, programming our Gibbs sampler (heteroskedastic case)

```

1 PostBetas <- matrix(0, mcmc+burnin, k)
2 PostSigma2 <- rep(0, mcmc+burnin)
3 Beta <- rep(0, k)
4 tau <- rep(1, N)
5 # create progress bar in case that you want to see
  iterations progress
6 pb <- txtProgressBar(min = 0, max = tot, style = 3)
7 for(s in 1:tot){
8   sig2 <- PostSig2(Beta = Beta, tau = tau)
9   PostSigma2[s] <- sig2
10  Beta <- PostBeta(sig2 = sig2, tau = tau)
11  PostBetas[s,] <- Beta
12  tau <- sapply(1:N, function(i){PostTau(sig2 = sig2, Beta =
    Beta, i)})
13  setTxtProgressBar(pb, s)
14 }
15 close(pb)
16 keep <- seq((burnin+1), tot, thin)
17 PosteriorBetas <- PostBetas[keep,]
18 colnames(PosteriorBetas) <- c("Intercept", "Perf", "Age", "
  Age2", "NatTeam", "Goals", "Exp", "Exp2")
19 summary(coda::mcmc(PosteriorBetas))
20 PosteriorSigma2 <- PostSigma2[keep]
21 summary(coda::mcmc(PosteriorSigma2))
22 summary(coda::mcmc(exp(PosteriorBetas[,5])-1))
23 Iterations = 1:5000
24 Thinning interval = 1
25 Number of chains = 1
26 Sample size per chain = 5000
27 1. Empirical mean and standard deviation for each variable,
28 plus standard error of the mean:
29 Mean          SD          Naive SE Time-series SE
30 1.256091      0.254974      0.003606      0.004579
31 2. Quantiles for each variable:
32 2.5%      25%      50%      75%      97.5%
33 0.8078 1.0746 1.2367 1.4189 1.7959

```

We see in this application that the value of a top soccer player in Europe increases 124% ($\exp(0.80) - 1$) on average when he has played in the national team, the credible interval at 95% is (81%, 180%). These values are not very different from the application assuming homoscedasticity in the book.

4. Example: Determinants of hospitalization continues.

Program a Gibbs sampling algorithm for the application on the determinants of hospitalization

Answer

R. code. Determinants of hospitalization, programming our Gibbs sampler

```

1 set.seed(010101)
2 Data <- read.csv("https://raw.githubusercontent.com/
   besmarter/BSTApp/refs/heads/master/DataApp/2HealthMed.
   csv", sep = ",", header = TRUE, quote = "")
3 attach(Data)
4 str(Data)
5 y <- Hosp # Dependent variables
6 X <- cbind(1, SHI, Female, Age, Age2, Est2, Est3, Fair, Good
   , Excellent) # Regressors
7 K <- dim(X)[2]
8 N <- dim(X)[1]
9 # Hyperparameters
10 b0 <- rep(0, K) # Prio mean
11 B0 <- diag(K) # Prior covariance
12 B0i <- solve(B0)
13 mcmc <- 1000; burnin <- 500; thin <- 2; tot <- mcmc + burnin
   ; keep <- seq(burnin, tot, thin)
14 # Posterior distributions programming the Gibbs sampling
15 # Auxiliary parameters
16 XtX <- t(X)%*%X
17 # Gibbs sampling functions
18 PostBeta <- function(Yl){
19   Bn <- solve(B0i + XtX)
20   bn <- Bn%*%(B0i%*%b0 + t(X)%*%Yl)
21   Beta <- MASS::mvrnorm(1, bn, Bn)
22   return(Beta)
23 }
24 PostYl <- function(Beta, i){
25   Ylmean <- X[i,]%*%Beta
26   if(y[i] == 1){
27     Yli <- truncnorm::rtruncnorm(1, a = 0, b = Inf, mean =
   Ylmean, sd = 1)
28   }else{
29     Yli <- truncnorm::rtruncnorm(1, a = -Inf, b = 0, mean =
   Ylmean, sd = 1)
30   }
31   return(Yli)
32 }
33 PostBetas <- matrix(0, mcmc+burnin, K)
34 Beta <- rep(0, K)
35 # create progress bar in case that you want to see
   iterations progress
36 pb <- txtProgressBar(min = 0, max = tot, style = 3)
37 for(s in 1:tot){
38   Yl <- sapply(1:N, function(i){PostYl(Beta = Beta, i)})
39   Beta <- PostBeta(Yl = Yl)
40   PostBetas[s,] <- Beta
41   setTxtProgressBar(pb, s)
42 }
43 close(pb)
44 keep <- seq((burnin+1), tot, thin)
45 PosteriorBetas <- PostBetas[keep,]
46 colnames(PosteriorBetas) <- c("Intercept", "SHI", "Female",
   "Age", "Age2", "Est2", "Est3", "Fair", "Good", "
   Excellent")
47 summary(coda::mcmc(PosteriorBetas))

```


5. **Choice of fishing mode continues.**

- Run Algorithm A9 to report the results of the Geweke [12], Raftery [38], and Heidelberg [17] tests using our GUI.
- Use the command *rmnpGibbs* to replicate the fishing mode choice example.

Answer

R. code. Fishing choice mode, results our GUI

```

1 GewekeTestLocationCoef
2 Fraction in 1st window = 0.1
3 Fraction in 2nd window = 0.5
4 cte_1 cte_2 cte_3 NAS_1_1 NAS_1_2 NAS_1_3 AS_1 AS_2
5 -1.821 -0.714 0.792 2.275 -3.944 -2.071 1.627 -2.729
6
7 RafteryTestLocationCoef
8 Quantile (q) = 0.5
9 Accuracy (r) = +/- 0.025
10 Probability (s) = 0.95
11 Burn-in Total Lower bound Dependence
12 (M) (N) (Nmin) factor (I)
13 cte_1 780 365690 1537 238.0
14 cte_2 360 193950 1537 126.0
15 cte_3 660 340120 1537 221.0
16 NAS_1_1 120 70320 1537 45.8
17 NAS_1_2 475 243960 1537 159.0
18 NAS_1_3 440 248930 1537 162.0
19 AS_1 3010 1438135 1537 936.0
20 AS_2 550 297770 1537 194.0
21
22 HeidelTestLocationCoef
23 Stationarity start p-value
24 test iteration
25 cte_1 passed 6001 6.54e-01
26 cte_2 failed NA 3.72e-02
27 cte_3 failed NA 4.99e-02
28 NAS_1_1 failed NA 4.77e-07
29 NAS_1_2 failed NA 1.82e-05
30 NAS_1_3 failed NA 1.19e-04
31 AS_1 passed 2001 3.71e-01
32 AS_2 passed 8001 4.48e-01
33 Halfwidth Mean Halfwidth
34 test
35 cte_1 passed -0.34236 0.017025
36 cte_2 <NA> NA NA
37 cte_3 <NA> NA NA
38 NAS_1_1 <NA> NA NA
39 NAS_1_2 <NA> NA NA
40 NAS_1_3 <NA> NA NA
41 AS_1 passed -0.00708 0.000306
42 AS_2 passed 0.27982 0.009994

```

R. code. Fishing choice mode, library bayesm

```

1 remove(list = ls()); set.seed(12345)
2 Data <- read.csv("https://raw.githubusercontent.com/
   besmarter/BSTApp/refs/heads/master/DataApp/3Fishing.csv"
   , sep = ",", header = TRUE, quote = "")
3 attach(Data); str(Data)
4 p <- 4; na <- 2; nd <- 1; N <- dim(Data)[1]
5 Xa <- Data[,2:9]
6 Xd <- matrix(income, N, 1)
7 X <- bayesm::createX(p = p, na = na, nd = nd, Xa = Xa, Xd =
   Xd, INT = TRUE, base = p, DIFF = TRUE)
8 df <- list(y = mode, X = X, p = 4)
9 # Hyperparameters
10 k <- dim(X)[2]; b0 <- rep(0, k); c0 <- 1000
11 B0 <- c0*diag(k); B0i <- solve(B0)
12 a0 <- p - 1 + 3; Psi0 <- a0*diag(p-1)
13 Prior <- list(betabar = b0, A = B0i, nu = a0, V = Psi0)
14 # MCMC parameters
15 mcmc <- 100000; thin <- 5
16 Mcmc <- list(R = mcmc, keep = thin)
17 Results <- bayesm::rmnpGibbs(Data = df, Prior = Prior, Mcmc
   = Mcmc)
18 betatilde <- Results$betadraw / sqrt(Results$sigmadraw[,1])
19 attributes(betatilde)$class <- "bayesm.mat"
20 summary(coda::mcmc(betatilde))
21 Quantiles for each variable:
22      2.5%      25%      50%      75%      97.5%
23 var1 -6.371e-01 -4.670e-01 -3.834e-01 -3.028e-01 -1.422e-01
24 var2 -2.149e-01 -6.681e-02 -7.870e-03  4.414e-02  1.493e-01
25 var3 -8.873e-01 -6.071e-01 -4.982e-01 -4.040e-01 -2.446e-01
26 var4  1.859e-05  4.577e-05  5.952e-05  7.326e-05  9.861e-05
27 var5 -2.589e-05 -3.050e-06  8.212e-06  2.089e-05  5.330e-05
28 var6  4.549e-05  7.451e-05  9.144e-05  1.098e-04  1.555e-04
29 var7 -9.425e-03 -7.612e-03 -6.586e-03 -5.573e-03 -3.136e-03
30 var8  1.214e-01  2.068e-01  2.592e-01  3.154e-01  4.477e-01
31 sigmadraw <- Results$sigmadraw / Results$sigmadraw[,1]
32 attributes(sigmadraw)$class = "bayesm.var"
33 summary(coda::mcmc(sigmadraw))

```

6. Simulation exercise: Multinomial logit model continues.

Perform inference in the simulated multinomial logit model using the command *rmnlIndepMetrop* from the *bayesm* package in **R**, and compare the results with those obtained from our GUI.

Answer

R. code. Simulation of the multinomial logit model

```

1  remove(list = ls())
2  set.seed(12345)
3  # Simulation of data
4  N<-1000 # Sample Size
5  B<-c(0.5,0.8,-3)
6  B1<-c(-2.5,-3.5,0)
7  B2<-c(1,1,0)
8  # Alternative specific attributes of choice 1, for instance,
   price, quality and duration of choice 1
9  X1<-matrix(cbind(rnorm(N,0,1),rnorm(N,0,1),rnorm(N,0,1)),N,
   length(B))
10 # Alternative specific attributes of choice 2, for instance,
   price, quality and duration of choice 2
11 X2<-matrix(cbind(rnorm(N,0,1),rnorm(N,0,1),rnorm(N,0,1)),N,
   length(B))
12 # Alternative specific attributes of choice 3, for instance,
   price, quality and duration of choice 3
13 X3<-matrix(cbind(rnorm(N,0,1),rnorm(N,0,1),rnorm(N,0,1)),N,
   length(B))
14 X4<-matrix(rnorm(N,1,1),N,1)
15 V1<-B2[1]+X1%*%B+B1[1]*X4
16 V2<-B2[2]+X2%*%B+B1[2]*X4
17 V3<-B2[3]+X3%*%B+B1[3]*X4
18 suma<-exp(V1)+exp(V2)+exp(V3)
19 p1<-exp(V1)/suma
20 p2<-exp(V2)/suma
21 p3<-exp(V3)/suma
22 p<-cbind(p1,p2,p3)
23 y<- apply(p,1, function(x) sample(1:3, 1, prob = x, replace =
   TRUE))
24 table(y)
25 L <- length(table(y))
26 dat <-data.frame(mode,X1[,1],X2[,1],X3[,1],X1[,2],X2[,2],X3
   [,2],X1[,3],X2[,3],X3[,3],X4)
27 colnames(dat) <- c("mode","V1.1","V1.2","V1.3","V2.1","V2.2"
   ,"V2.3","V3.1","V3.2","V3.3","V4")
28 attach(dat)
29 LongData <- mlogit::mlogit.data(dat, shape = "wide", varying
   =2:10, choice = "mode")
30 Xa <- cbind(LongData$V1, LongData$V2, LongData$V3)
31 Xa <- cbind(X1[,1],X2[,1],X3[,1],X1[,2],X2[,2],X3[,2],X1
   [,3],X2[,3],X3[,3])
32 na <- 3
33 Xd <- X4
34 X <- bayesm::createX(p = L, na = na, nd = 1, Xa = Xa, Xd =
   Xd, base = L)
35 DataMlogit <- list(y=y, X = X, p = L)
36 # MCMC parameters
37 mcmc <- 11000+1
38 thin <- 5
39 df <- 6
40 mcmcpair <- list(R = mcmc, keep = 5, nu = df)
41 PostBeta <- bayesm::rmnlIndepMetrop(Data = DataMlogit, Mcmc
   = mcmcpair)
42 summary(PostBeta[["betadraw"]])

```

7. Simulation of the ordered probit model.

Simulate an ordered probit model where the first regressor follows $N(6, 5)$ and the second follows $G(1, 1)$. The location vector is $\beta = [0.5 \ -0.25 \ 0.5]^\top$, and the cutoffs are $\alpha = [0 \ 1 \ 2.5]^\top$. Program a Metropolis-within-Gibbs sampling algorithm from scratch to perform inference in this simulation.

Answer

R. code. Simulation of the ordered probit model

```

1 rm(list = ls()); set.seed(010101); N <- 1000
2 x1 <- rnorm(N, 6, 5); x2 <- rgamma(N, shape = 1, scale = 1)
3 X <- cbind(1, x1, x2)
4 beta <- c(0.5, -0.25, 0.5); cutoffs <- c(0, 1, 2.5)
5 e <- rnorm(N,0,1)
6 y_latent <- X%*%beta + e; y <- rep(0,N)
7 for (i in 1:N) {
8   if (y_latent[i] < cutoffs[1]){
9     y[i] <- 0}else{
10    if (y_latent[i] >= cutoffs[1] & y_latent[i] < cutoffs
11      [2]) {
12      y[i] <- 1
13    }else{
14      if (y_latent[i] >= cutoffs[2] & y_latent[i] < cutoffs
15        [3]) {
16        y[i] <- 2
17      }else{y[i] <- 3
18      }
19    }
20  }
21 }
22 # Likelihood function
23 LogLikOP <- function(param){
24   beta_g <- param[1:ncol(X)]
25   delta <- param[(ncol(X)+1):(ncol(X) + dplyr::n_distinct(y)
26     - 1)]
27   Xbeta <- X%*%beta_g
28   logLik <- 0
29   for (i in 1:length(y)){
30     if (y[i]==0){logLiki <- log(pnorm(-Xbeta[i]))
31     }else if (y[i]==1){
32       logLiki <- log(pnorm(exp(delta[1]) - Xbeta[i]) - pnorm
33         (-Xbeta[i]))
34     }else if (y[i]==2){
35       logLiki <- log(pnorm(exp(delta[2]) + exp(delta[1]) -
36         Xbeta[i]) - pnorm(exp(delta[1]) - Xbeta[i]))
37     }else {logLiki <- log(1 - pnorm(exp(delta[2]) + exp(
38       delta[1]) - Xbeta[i]))
39     }
40     logLik <- logLik + logLiki
41   }
42   return(-logLik)
43 }
44 # ML Estimation
45 param0 <- rep(0, ncol(X) + n_distinct(y)-2)
46 mle <- optim(param0, LogLikOP, hessian = T, method = "BFGS")
47 mle$par
48 exp(mle$par[length(beta)+1])
49 exp(mle$par[length(beta)+1])+exp(mle$par[length(beta)+2])
50 CovarML <- solve(mle$hessian)

```

R. code. Simulation of the ordered probit model

```

1 # M-H within Gibbs
2 mhop <- function(param0, G){
3   betasamples <- matrix(c(0), nrow = G, ncol = ncol(X))
4   betasamples[1,] <- param0[1:ncol(X)]
5   tau <- matrix(c(0), nrow = G, ncol = dplyr::n_distinct(y)
6     - 2)
7   tau[1,] <- param0[(ncol(X)+1):(ncol(X) + dplyr::n_distinct(
8     y) - 2)]
9   yl <- rep(0,length(y)); ar <- rep(0,G); B1 <- solve(t(X)%*
10     %X+solve(B0))
11   pb <- txtProgressBar(min = 0, max = G, style = 3)
12   for(g in 2:G){
13     bg <- betasamples[g-1,]; tg <- tau[g-1,]
14     #Random walk M-H for delta
15     delta_prime <- tg + mvtnorm::rmvnorm(1, mean = rep(0,2),
16       sigma = VarProp)
17     alpha <- min(1,(mvtnorm::dmvnorm(delta_prime, mean = d0,
18       sigma = D0)*exp(-LogLikOP(c(bg, delta_prime)) +
19       LogLikOP(c(bg, tg)))/mvtnorm::dmvnorm(tg, mean = d0,
20       sigma = D0))
21     if(is.nan(alpha) | is.na(alpha)) {
22       alpha <- 0
23     }
24     #Acceptance step
25     u <- runif(1, min = 0, max = 1)
26     if(u<=alpha){tau[g,] <- delta_prime; ar[g] <- 1
27     }else{tau[g,] <- tg
28     }
29     #Generation of latent variables
30     for (i in 1:length(y)){
31       if (y[i]==0) {
32         yl[i] <- EnvStats::rnormTrunc(1, mean = X[i,]%*%bg,
33         sd = 1, max = 0)
34       }else if(y[i]==1){
35         yl[i] <- EnvStats::rnormTrunc(1, mean = X[i,]%*%bg,
36         sd = 1, min = 0, max = exp(tau[g,1]))
37       }else if(y[i]==2){
38         yl[i] <- EnvStats::rnormTrunc(1, mean = X[i,]%*%bg,
39         sd = 1, min = exp(tau[g,1]), max = exp(tau[g,2])+exp(tau
40         [g,1]))
41       }else{
42         yl[i] <- EnvStats::rnormTrunc(1, mean = X[i,]%*%bg,
43         sd = 1, min = exp(tau[g,2])+exp(tau[g,1]))
44       }
45     }
46     #Gibbs sampling for beta
47     if(sum(is.nan(yl))>0 | sum(is.na(yl))>0 | sum(yl)==Inf){
48       betasamples[g,] <- betasamples[g-1,]
49     }else{
50       b1 <- B1%*(t(X)%*%yl + solve(B0)%*%b0)
51       betasamples[g,] <- mvrnorm(1, mu = b1, Sigma = B1)
52     }
53     setTxtProgressBar(pb, g)
54   }
55   close(pb)
56   return(cbind(betasamples, tau, ar))
57 }

```

R. code. Simulation of the ordered probit model

```

1 #Hyperparameters
2 d0 <- rep(0,2)
3 D0 <- diag(2)*10000
4 b0 <- rep(0,ncol(X))
5 B0 <- diag(ncol(X))*10000
6 #Estimation
7 param0 <- rep(0, ncol(X) + dplyr::n_distinct(y)-1)
8 G <- 1000
9 tun <- 1
10 VarProp <- tun*solve(solve(CovarML[4:5, 4:5]) + solve(D0))
11 param_sample <- mhop(param0, G)
12 #Burn in
13 B <- round(0.2*G)
14 param_sample <- param_sample[(B+1):G,]
15 mcmc0 <- coda::mcmc(param_sample[, 1:(ncol(X) + dplyr::n_
    distinct(y) - 2)])
16 summary(mcmc0)
17 Iterations = 1:800
18 Thinning interval = 1
19 Number of chains = 1
20 Sample size per chain = 800
21 1. Empirical mean and standard deviation for each variable,
22 plus standard error of the mean:
23 Mean      SD Naive SE Time-series SE
24 0.49120 0.08465 0.0029929      0.007140
25 -0.24919 0.01222 0.0004319      0.001269
26 0.49440 0.03942 0.0013937      0.002739
27 0.06716 0.06419 0.0022695      0.008479
28 0.41926 0.07414 0.0026212      0.009479
29 2. Quantiles for each variable:
30 2.5%      25%      50%      75%      97.5%
31 0.31947 0.43558 0.49710 0.5479 0.6496
32 -0.27180 -0.25740 -0.24948 -0.2408 -0.2238
33 0.42229 0.46706 0.49253 0.5189 0.5762
34 -0.06338 0.02328 0.06819 0.1104 0.1932
35 0.25857 0.37195 0.41742 0.4672 0.5558
36 summary(coda::mcmc(cbind(exp(param_sample[, 4]),exp(param_
    sample[, 4])+exp(param_sample[, 5]))))
37 Iterations = 1:800
38 Thinning interval = 1
39 Number of chains = 1
40 Sample size per chain = 800
41 1. Empirical mean and standard deviation for each variable,
42 plus standard error of the mean:
43 Mean      SD Naive SE Time-series SE
44 [1,] 1.072 0.06854 0.002423      0.008999
45 [2,] 2.597 0.13593 0.004806      0.019700
46 2. Quantiles for each variable:
47 2.5%      25%      50%      75% 97.5%
48 var1 0.9386 1.024 1.071 1.117 1.213
49 var2 2.3224 2.500 2.603 2.686 2.877

```


All posterior mean estimates are close to the population parameters, and the 95% credible intervals encompass the population parameters. We use the definition of γ in the last line of the code.

8. Simulation of the negative binomial model continues.

Perform inference in the simulated negative binomial model using the *bayesm* package in R.

Answer

R. code. Simulation of the negative binomial model

```

1 rm(list = ls())
2 set.seed(010101)
3 N <- 2000 # Sample size
4 x1 <- runif(N); x2 <- rnorm(N)
5 X <- cbind(1, x1, x2)
6 k <- dim(X)[2]
7 B <- rep(1, k)
8 alpha <- 1.2
9 gamma <- exp(alpha)
10 lambda <- exp(X%*%B)
11 y <- rbinom(N, mu = lambda, size = gamma)
12 table(y)
13 # MCMC parameters
14 mcmc <- 10000
15 burnin <- 1000
16 thin <- 5
17 iter <- mcmc + burnin
18 keep <- seq(burnin, iter, thin)
19 sbeta <- 2.93/sqrt(k); salpha <- 2.93
20 # Hyperparameters: Priors
21 B0 <- 1000*diag(k); b0 <- rep(0, k)
22 alpha0 <- 0.5; delta0 <- 0.1
23 DataNB <- list(y = y, X = X)
24 mcmcNB <- list(R = mcmc, keep = thin, s_beta = sbeta, s_
    alpha = salpha)
25 PriorNB <- list(betabar = b0, A = solve(B0), a = alpha0, b =
    delta0)
26 ResultBayesm <- bayesm::rnegbinRw(Data = DataNB, Mcmc =
    mcmcNB, Prior = PriorNB)
27 summary(ResultBayesm$alphadraw)
28 summary(ResultBayesm$betadraw)

```

9. The market value of soccer players in Europe continues.

Perform the application on soccer players' market value with left censoring

at one million euros in our GUI using Algorithm ?? and the hyperparameters of the example.

Answer

These are the results of running the application in our GUI. These are very similar compared with the results in the book.

R. code. The value of soccer players with left censoring in our GUI

```

1 Summary
2 Iterations = 10001:60000
3 Thinning interval = 1
4 Number of chains = 1
5 Sample size per chain = 50000
6 1. Empirical mean and standard deviation for each variable,
7 plus standard error of the mean:
8
9      Mean      SD Naive SE Time-series SE
10 (Intercept)  1.014765  2.626395  1.175e-02   1.659e-02
11 Perf         0.033935  0.004529  2.025e-05   2.245e-05
12 Age          1.027285  0.212413  9.499e-04   1.326e-03
13 Age2        -0.021914  0.003989  1.784e-05   2.528e-05
14 NatTeam      0.847412  0.124884  5.585e-04   6.423e-04
15 Goals        0.010092  0.001644  7.351e-06   7.712e-06
16 Exp          0.175324  0.069653  3.115e-04   3.727e-04
17 Exp2        -0.005696  0.002950  1.319e-05   1.527e-05
18 sigma2       0.983337  0.096194  4.302e-04   6.724e-04
19
20 2. Quantiles for each variable:
21      2.5%      25%      50%      75%      97.5%
22 (Intercept) -4.212563 -0.732856  1.038281  2.796603  6.051e
23      +00
24 Perf         0.025121  0.030873  0.033896  0.036988  4.286e
25      -02
26 Age          0.618743  0.883011  1.026760  1.168860  1.450e
27      +00
28 Age2        -0.029854 -0.024580 -0.021886 -0.019203 -1.424e
29      -02
30 NatTeam      0.605030  0.763154  0.846671  0.930593  1.095e
31      +00
32 Goals        0.006893  0.008987  0.010097  0.011201  1.330e
33      -02
34 Exp          0.038522  0.128032  0.175261  0.222110  3.118e
35      -01
36 Exp2        -0.011484 -0.007671 -0.005685 -0.003706  5.502e
37      -05
38 sigma2       0.813118  0.915664  0.977298  1.043661  1.189e
39      +00

```

10. The market value of soccer players in Europe continues.

Program from scratch the Gibbs sampling algorithm for the example of soccer players' market value at the 0.75 quantile.

Answer

R. code. The value of soccer players, quantile regression

```

1 rm(list = ls()); set.seed(010101)
2 Data <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/1
  ValueFootballPlayers.csv", sep = ",", header = TRUE,
  quote = "")
3 attach(Data)
4 y <- log(Value); X <- cbind(1, Perf, Age, Age2, NatTeam,
  Goals, Exp, Exp2)
5 RegLS <- lm(y ~ X -1)
6 k <- dim(X)[2]; N <- dim(X)[1]
7 # Hyperparameters
8 b0 <- rep(0, k); c0 <- 1000
9 B0 <- c0*diag(k); B0i <- solve(B0)
10 # MCMC parameters
11 mcmc <- 5000; burnin <- 1000
12 tot <- mcmc + burnin; thin <- 1
13 # Quantile
14 tau <- 0.5; theta <- (1-2*tau)/(tau*(1-tau))
15 psi2 <- 2/(tau*(1-tau)); an2 <- 2+theta^2/psi2
16 # Gibbs sampler
17 PostBeta <- function(e){
18   Bn <- solve(B0i + psi2^(-1)*t(X)%*%diag(1/e)%*%X)
19   bn <- Bn%*%(B0i%*%b0 + psi2^(-1)*t(X)%*%diag(1/e)%*%(y-
    theta*e))
20   Beta <- MASS::mvrnorm(1, bn, Bn)
21   return(Beta)
22 }
23 PostE <- function(Beta, i){
24   dn2 <- (y[i]-X[i,]%*%Beta)^2/psi2
25   ei <- GIGrvg::rgig(1, chi = dn2, psi = an2, lambda = 1/2)
26   return(ei)
27 }
28 PostBetas <- matrix(0, mcmc+burnin, k)
29 Beta <- RegLS$coefficients
30 pb <- txtProgressBar(min = 0, max = tot, style = 3)
31 for(s in 1:tot){
32   e <- sapply(1:N, function(i){PostE(Beta = Beta, i)})
33   Beta <- PostBeta(e = e)
34   PostBetas[s,] <- Beta
35   setTxtProgressBar(pb, s)
36 }
37 close(pb)
38 keep <- seq((burnin+1), tot, thin)
39 PosteriorBetas <- PostBetas[keep,]
40 colnames(PosteriorBetas) <- c("Intercept", "Perf", "Age", "
  Age2", "NatTeam", "Goals", "Exp", "Exp2")
41 summary(coda::mcmc(PosteriorBetas))

```

11. Use the *bayesboot* package to perform inference in the simulation exercise of Section ?? and compare the results with those obtained from our GUI, setting $S = 10,000$.

Answer

R. code. The value of soccer players, quantile regression

```

1 ##### Bayesian bootstrap: Simulation
  #####
2 rm(list = ls())
3 set.seed(010101)
4 N <- 1000 # Sample size
5 x1 <- runif(N); x2 <- rnorm(N)
6 X <- cbind(x1, x2)
7 k <- dim(X)[2]
8 B <- rep(1, k+1)
9 sig2 <- 1
10 u <- rnorm(N, 0, sig2)
11 y <- cbind(1, X)%*%B + u
12 data <- as.data.frame(cbind(y, X))
13 names(data) <- c("y", "x1", "x2")
14 Reg <- function(d){
15   Reg <- lm(y ~ x1 + x2, data = d)
16   Bhat <- Reg$coef
17   return(Bhat)
18 }
19 Reg(data)
20 S <- 10000
21 BB <- bayesboot::bayesboot(data = data, statistic = Reg, R =
  S)
22 plot(BB)

```



7

Multivariate models

Solutions of Exercises

1. Show that $\mathbb{E}[u_1 \text{PAER}] = \frac{\alpha_1}{1-\beta_1\alpha_1}\sigma_1^2$, assuming that $\mathbb{E}[u_1 u_2] = 0$ and $\text{Var}(u_1) = \sigma_1^2$, in the example on the effect of institutions on per capita GDP.

Answer

The point of departure is the following *simultaneous structural* economic model:

$$\log(\text{pcGDP95}_i) = \beta_1 + \beta_2 \text{PAER}_i + \beta_3 \text{Africa} + \beta_4 \text{Asia} + \beta_5 \text{Other} + u_{1i}, \quad (7.1)$$

$$\text{PAER}_i = \alpha_1 + \alpha_2 \log(\text{pcGDP95}_i) + \alpha_3 \log(\text{Mort}_i) + u_{2i}, \quad (7.2)$$

where *pcGDP95*, *PAER* and *Mort* are the per capita gross domestic product (GDP) in 1995, the average index of protection against expropriation between 1985 and 1995, and the settler mortality rate during the time of colonization. *Africa*, *Asia* and *Other* are dummies for continents, with *America* as the baseline group.

Replacing Equation 7.2 into Equation 7.1, and solving for $\log(\text{pcGDP95})$,

$$\log(\text{pcGDP95}_i) = \pi_1 + \pi_2 \log(\text{Mort}_i) + \pi_3 \text{Africa} + \pi_4 \text{Asia} + \pi_5 \text{Other} + e_{1i}, \quad (7.3)$$

where $\pi_1 = \frac{\beta_1 + \beta_2 \alpha_1}{1 - \beta_2 \alpha_2}$, $\pi_2 = \frac{\beta_2 \alpha_3}{1 - \beta_2 \alpha_2}$, $\pi_3 = \frac{\beta_3}{1 - \beta_2 \alpha_2}$, $\pi_4 = \frac{\beta_4}{1 - \beta_2 \alpha_2}$, $\pi_5 = \frac{\beta_5}{1 - \beta_2 \alpha_2}$, and $e_{1i} = \frac{\beta_2 u_{1i} + u_{2i}}{1 - \beta_2 \alpha_2}$.

Then, replacing Equation 7.3 into Equation 7.2, and solving for *PAER*,

$$\text{PAER}_i = \gamma_1 + \gamma_2 \log(\text{Mort}_i) + \gamma_3 \text{Africa} + \gamma_4 \text{Asia} + \gamma_5 \text{Other} + e_{2i}, \quad (7.4)$$

where $\gamma_1 = \frac{\alpha_1 + \alpha_2 \beta_1}{1 - \beta_2 \alpha_2}$, $\gamma_2 = \frac{\alpha_3}{1 - \beta_2 \alpha_2}$, $\gamma_3 = \frac{\alpha_2 \beta_3}{1 - \beta_2 \alpha_2}$, $\gamma_4 = \frac{\alpha_2 \beta_4}{1 - \beta_2 \alpha_2}$, $\gamma_5 = \frac{\alpha_2 \beta_5}{1 - \beta_2 \alpha_2}$, and $e_{2i} = \frac{\alpha_2 u_{1i} + u_{2i}}{1 - \beta_2 \alpha_2}$.

Then, $\mathbb{E}[u_1 \text{PAER}] = \mathbb{E}[u_1 (\gamma_1 + \gamma_2 \log(\text{Mort}_i) + \gamma_3 \text{Africa} + \gamma_4 \text{Asia} + \gamma_5 \text{Other} + e_{2i})] = \mathbb{E}\left[u_1 \left(\frac{\alpha_2 u_1 + u_2}{1 - \beta_2 \alpha_2}\right)\right] = \frac{\alpha_2}{1 - \beta_2 \alpha_2} \sigma_1^2$ given the assumptions.

2. Show that $\beta_1 = \pi_1/\gamma_1$ in the example on the effect of institutions on per capita GDP.

Answer

Given that $\pi_2 = \frac{\beta_2 \alpha_3}{1 - \beta_2 \alpha_2}$ and $\gamma_2 = \frac{\alpha_3}{1 - \beta_2 \alpha_2}$ from the previous exercise, consequently, $\beta_2 = \pi_2/\gamma_2$.

3. **The effect of institutions on per capita GDP continues I**

Use the *rmultireg* command from the *bayesm* package to perform inference in the example of the effect of institutions on per capita GDP.

Answer

R code. The effect of institutions on per capita GDP

```

1 rm(list = ls())
2 set.seed(010101)
3 DataInst <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/4Institutions
  .csv", sep = ",", header = TRUE, quote = "")
4 attach(DataInst)
5 Y <- cbind(logpcGDP95, PAER)
6 X <- cbind(1, logMort, Africa, Asia, Other)
7 M <- dim(Y)[2]
8 K <- dim(X)[2]
9 # Hyperparameters
10 B0 <- matrix(0, K, M)
11 c0 <- 100
12 V0 <- c0*diag(K)
13 Psi0 <- 5*diag(M)
14 a0 <- 5
15 S <- 10000 #Number of posterior draws
16 betadraw = matrix(double(S*K*M), ncol=K*M)
17 Sigmadraw = matrix(double(S*M*M), ncol=M*M)
18 pb <- txtProgressBar(min = 0, max = S, style = 3)
19 for (s in 1:S) {
20   Results <- bayesm::rmultireg(Y, X, Bbar = B0, A = solve(V0
    ), nu = a0, V = Psi0)
21   betadraw[s,] <- Results$B
22   Sigmadraw[s,] <- Results$Sigma
23   setTxtProgressBar(pb, s)
24 }
25 close(pb)
26 summary(coda::mcmc(betadraw))
27 summary(coda::mcmc(Sigmadraw))

```

4. Demand and supply simulation

Consider the structural demand–supply model:

$$\begin{aligned} q_i^d &= \beta_1 + \beta_2 p_i + \beta_3 y_i + \beta_4 pc_i + \beta_5 ps_i + u_{i1}, \\ q_i^s &= \alpha_1 + \alpha_2 p_i + \alpha_3 er_i + u_{i2}, \end{aligned}$$

where q^d and q^s represent demand and supply, respectively; p , y , pc , ps , and er denote price, income, complementary price, substitute price, and exchange rate. The variables pc and ps refer to the prices of complementary and substitute goods for q . Assume that $\beta = [5 \ -0.5 \ 0.8 \ -0.4 \ 0.7]^\top$, $\alpha = [-2 \ 0.5 \ -0.4]^\top$, $u_1 \sim N(0, 0.5^2)$, and $u_2 \sim N(0, 0.5^2)$. Additionally, assume that $y \sim N(10, 1)$, $pc \sim N(5, 1)$, $ps \sim N(5, 1)$, and $er \sim N(15, 1)$.

- Derive the *reduced-form* model under the equilibrium condition $q^d = q^s$, which defines the observable quantity q .
- Simulate p and q from the *reduced-form* equations.
- Perform inference for the *reduced-form* model using the *rmultireg* command from the *bayesm* package.
- Use the posterior draws of the *reduced-form* parameters to perform inference for the *structural* parameters. Discuss any potential issues.
Hint: Are all structural parameters exactly identified?

Answer

We should equal demand and supply, and solve for price,

$$p = \pi_1 + \pi_2 er + \pi_3 y + \pi_4 pc + \pi_5 ps + v_1,$$

where $\pi_1 = \frac{\alpha_1 - \beta_1}{\beta_2 - \alpha_2}$, $\pi_2 = \frac{\alpha_3}{\beta_2 - \alpha_2}$, $\pi_3 = \frac{-\beta_3}{\beta_2 - \alpha_2}$, $\pi_4 = \frac{-\beta_4}{\beta_2 - \alpha_2}$, $\pi_5 = \frac{-\beta_5}{\beta_2 - \alpha_2}$, and $v_1 = \frac{u_2 - u_1}{\beta_2 - \alpha_2}$ given $\beta_2 \neq \alpha_2$, that is, the equations should be independent. This condition is given by economic theory due to $\beta_2 < 0$ and $\alpha_2 > 0$, the effect of price on demand and supply should be negative and positive, respectively.

The equation of price into the demand equation gives

$$q = \tau_1 + \tau_2 er + \tau_3 y + \tau_4 pc + \tau_5 ps + v_2,$$

where $\tau_1 = \beta_1 + \beta_2 \pi_1$, $\tau_2 = \beta_2 \pi_2$, $\tau_3 = \beta_2 \pi_3 + \beta_3$, $\tau_4 = \beta_2 \pi_4 + \beta_4$, $\tau_5 = \beta_2 \pi_5 + \beta_5$, and $v_2 = \beta_2 v_1 + u_1$. We can use the equations para π_k and τ_k , $k = \{1, 2, 3, 4, 5\}$ to simulate the *reduced-form* equations.

Observe that estimating the *reduced-form* equations, we can get the *structural* parameters for the demand equation, $\beta_2 = \tau_2 / \pi_2$ ($\pi_2 \neq 0$), $\beta_3 = \tau_3 - \beta_2 \pi_3$, $\beta_4 = \tau_4 - \beta_2 \pi_4$ and $\beta_5 = \tau_5 - \beta_2 \pi_5$, whereas the *structural* parameters of the supply equation cannot be recovered just in one way, $\alpha_2 = \beta_2 + \beta_3 / \pi_3 = \beta_2 + \beta_4 / \pi_4 = \beta_2 + \beta_5 / \pi_5$. This in turn implies different

values for $\alpha_3 = \pi_2(\beta_2 - \alpha_2)$. This is because the demand equation is *exactly identified*, whereas the supply equation is *over identified*.

In this exercise, $K = 5$, $M = 2$, $K_1 = 4$, $K_2 = 2$, $M_1 = 2$ and $M_2 = 2$. This means that $K - K_1 = 1 = M - 1$ and $K - K_2 = 3 > M - 1 = 1$, that is, the order condition says that both equations (demand and supply) satisfy the necessary condition of identification, the demand would be *exactly identified*, and the supply equation would be *over identified*.

Regarding the rank condition (necessary and sufficient), let's see the identification matrix:

TABLE 7.1

Identification matrix.

q	p	constant	er	y	pc	ps
1	$-\beta_2$	$-\beta_1$	0	$-\beta_3$	$-\beta_4$	$-\beta_5$
1	$-\alpha_2$	$-\alpha_1$	$-\alpha_3$	0	0	0

The demand equation excludes the exchange rate (*exclusion restriction*), and given $\alpha_3 \neq 0$, that is, the exchange rate is relevant in the supply equation, then the rank condition is satisfied in the demand equation. The supply equation excludes the income, the complementary price and substitute price (*exclusion restrictions*), then as far as $\beta_k \neq 0$ for any $k = \{3, 4, 5\}$, the rank condition is satisfied in the supply equation.

The following code shows how to do the simulation, and perform inference in this exercise. We can see that all 95% credible intervals encompass the population parameters, and the posterior means are very close to them.

R code. Demand and supply simulation

```

1 rm(list = ls()); set.seed(12345)
2 B0 <- 5; B1 <- -0.5; B2 <- 0.8; B3 <- -0.4; B4 <- 0.7; SD <-
  0.5
3 A0 <- -2; A1 <- 0.5; A2 <- -0.4; SS <- 0.5
4 P0 <- (A0-B0)/(B1-A1); P2 <- -B2/(B1-A1); P3 <- -B3/(B1-A1);
  P1 <- A2/(B1-A1); P4 <- -B4/(B1-A1)
5 T0 <- B0+B1*P0; T2 <- B2+B1*P2; T3 <- B3+B1*P3; T1 <- B1*P1;
  T4 <- B4+B1*P4;
6 n <- 5000
7 ED <- rnorm(n, 0, SD); ES <- rnorm(n, 0, SS)
8 VP <- (ES-ED)/(B1-A1); UQ <- B1*VP+ED
9 y <- rnorm(n, 10, 1); pc <- rnorm(n, 5, 1); er <- rnorm(n,
  15, 1); ps <- rnorm(n, 5, 1);
10 p <- P0+P1*er+P2*y+P3*pc+P4*ps+VP
11 q <- T0+T1*er+T2*y+T3*pc+T4*ps+UQ
12 #Inference
13 Y <- cbind(p, q); X <- cbind(1, er, y, pc, ps)
14 M <- dim(Y)[2]; K <- dim(X)[2]
15 # Hyperparameters
16 B0 <- matrix(0, K, M); c0 <- 100; V0 <- c0*diag(K)
17 Psi0 <- 5*diag(M); a0 <- 5; S <- 10000 #Posterior draws
18 betadraw = matrix(double(S*K*M), ncol=K*M)
19 Sigmadraw = matrix(double(S*M*M), ncol=M*M)
20 pb <- txtProgressBar(min = 0, max = S, style = 3)
21 for (s in 1:S) {
22   Results <- bayesm::rmultireg(Y, X, Bbar = B0, A = solve(V0
  ), nu = a0, V = Psi0)
23   betadraw[s,] <- Results$B
24   Sigmadraw[s,] <- Results$Sigma
25   setTxtProgressBar(pb, s)
26 }
27 close(pb)
28 summary(coda::mcmc(betadraw))
29 summary(coda::mcmc(Sigmadraw))
30 beta2 <- betadraw[,7]/betadraw[,2] # Effect of price on
  demand
31 summary(coda::mcmc(beta2))
32 beta3 <- betadraw[,8] - beta2*betadraw[,3] # Effect of
  income on demand
33 summary(coda::mcmc(beta3))
34 beta4 <- betadraw[,9] - beta2*betadraw[,4] # Effect of
  complementary price on demand
35 summary(coda::mcmc(beta4))
36 beta5 <- betadraw[,10] - beta2*betadraw[,5] # Effect of
  substitute price on demand
37 summary(coda::mcmc(beta5))

```

5. Utility demand continues

- Run the **Utility demand** application using our GUI and the dataset *Utilities.csv*. *Hint:* This file must be modified to match the structure required by our GUI (see the dataset *5Institutions.csv* in the *DataApp* folder of our GitHub repository—<https://github.com/besmarter/BSTApp>—as a template).
- Program the Gibbs sampling algorithm for this application from scratch.

Answer

R code. Utility demand in Colombia, Gibbs sampler

```

1 rm(list = ls()); set.seed(010101); library(dplyr)
2 DataUt <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/Utilities.csv",
  sep = ",", header = TRUE, quote = "")
3 DataUtEst <- DataUt %>% filter(Electricity != 0 & Water != 0
  & Gas != 0)
4 attach(DataUtEst)
5 y1 <- log(Electricity); y2 <- log(Water); y3 <- log(Gas)
6 X1 <- cbind(1, LnPriceElect, LnPriceWater, LnPriceGas,
  IndSocio1, IndSocio2, Altitude, Nrooms, HouseholdMem,
  Lnincome)
7 X2 <- cbind(1, LnPriceElect, LnPriceWater, LnPriceGas,
  IndSocio1, IndSocio2, Nrooms, HouseholdMem)
8 X3 <- cbind(1, LnPriceElect, LnPriceWater, LnPriceGas,
  IndSocio1, IndSocio2, Altitude, Nrooms, HouseholdMem)
9 y <- c(y1, y2, y3)
10 X <- as.matrix(Matrix::bdiag(X1, X2, X3))
11 M <- 3; K1 <- dim(X1)[2]; K2 <- dim(X2)[2]; K3 <- dim(X3)[2]
12 K <- K1 + K2 + K3; N <- length(y1)
13 # Hyperparameters
14 b0 <- rep(0, K); c0 <- 100; B0 <- c0*diag(K)
15 B0i <- solve(B0); Psi0 <- 5*diag(M); Psi0i <- solve(Psi0)
16 a0 <- M; IN <- diag(N); an <- a0 + N
17 #Posterior draws
18 S <- 6000; burnin <- 1000; thin <- 1
19 tot <- S+burnin
20 # Gibbs functions
21 PostBeta <- function(Sigma){
22   Aux <- solve(Sigma)%x%IN
23   Bn <- solve(B0i + t(X)%*%Aux)%*X
24   bn <- Bn%*(B0i%*%b0 + t(X)%*%Aux)%*y
25   Beta <- MASS::mvrnorm(1, bn, Bn)
26   return(Beta)
27 }
28 PostSigma <- function(Beta){
29   B1 <- Beta[1:K1]; B2 <- Beta[(K1+1):(K1+K2)]; B3 <- Beta[(
  K1+K2+1):(K1+K2+K3)]
30   U1 <- y1 - X1%*%B1; U2 <- y2 - X2%*%B2; U3 <- y3 - X3%*%B3
31   U <- cbind(U1, U2, U3)
32   Psin <- solve(Psi0i + t(U)%*%U)
33   Sigmai <- rWishart::rWishart(1, df = an, Sigma = Psin)
34   Sigma <- solve(Sigmai[, ,1])
35   return(Sigma)
36 }
37 PostBetas <- matrix(0, tot, K)
38 PostSigmas <- matrix(0, tot, M*(M+1)/2); Beta <- rep(1, K)
39 pb <- txtProgressBar(min = 0, max = tot, style = 3)
40 for(s in 1:tot){
41   Sigma <- PostSigma(Beta = Beta); PostSigmas[s,] <-
  matrixcalc::vech(Sigma)
42   Beta <- PostBeta(Sigma = Sigma); PostBetas[s,] <- Beta
43   setTxtProgressBar(pb, s)
44 }
45 close(pb); keep <- seq((burnin+1), tot, thin)
46 Bs <- PostBetas[keep,]; summary(coda::mcmc(Bs))
47 Sigmas <- PostSigmas[keep,]; summary(coda::mcmc(Sigmas))

```

R code. Utility demand in Colombia, results

```

1 keep <- seq((burnin+1), tot, thin)
2 Bs <- PostBetas[keep,]; summary(coda::mcmc(Bs))
3 Sigmas <- PostSigmas[keep,]; summary(coda::mcmc(Sigmas))
4 Quantiles for each variable:
5
6      2.5%      25%      50%      75%      97.5%
7 Const      0.43304  1.04508  1.337751  1.64728  2.25228
8 LnPriceElect -2.40343 -2.05533 -1.882159 -1.70463 -1.35979
9 LnPriceWater -0.44027 -0.38524 -0.356049 -0.32676 -0.27021
10 LnPriceGas -0.21665 -0.13807 -0.098877 -0.05712  0.01673
11 IndSocio1 -0.87408 -0.78454 -0.736561 -0.68898 -0.59137
12 IndSocio2 -0.24504 -0.18344 -0.150827 -0.11845 -0.06029
13 Altitude -0.27153 -0.23866 -0.221539 -0.20468 -0.17325
14 Nrooms 0.04598 0.06191 0.069971 0.07837 0.09379
15 HouseholdMem 0.06610 0.07999 0.087320 0.09406 0.10708
16 Lnincome 0.03897 0.05516 0.063130 0.07152 0.08751
17 Const 0.88247 1.73546 2.162290 2.60793 3.48450
18 LnPriceElect -0.83128 -0.31258 -0.051838 0.20446 0.72087
19 LnPriceWater -0.49760 -0.40946 -0.363138 -0.31943 -0.23498
20 LnPriceGas 0.05880 0.16932 0.228734 0.28677 0.39226
21 IndSocio1 -0.63928 -0.50417 -0.427978 -0.35529 -0.21024
22 IndSocio2 -0.50363 -0.40976 -0.359576 -0.30929 -0.21097
23 Nrooms 0.05733 0.08126 0.093291 0.10550 0.12935
24 HouseholdMem 0.09984 0.12098 0.132126 0.14265 0.16290
25 Const -2.26403 -1.57842 -1.201124 -0.84494 -0.15985
26 LnPriceElect -2.41128 -2.00330 -1.786150 -1.57292 -1.17185
27 LnPriceWater -0.10791 -0.03978 -0.003399 0.03329 0.10082
28 LnPriceGas -0.76450 -0.67575 -0.627062 -0.57949 -0.48807
29 IndSocio1 -0.91346 -0.80094 -0.743421 -0.68334 -0.57228
30 IndSocio2 -0.31613 -0.24086 -0.202828 -0.16367 -0.08824
31 Altitude 0.24986 0.28987 0.310784 0.33183 0.37126
32 Nrooms 0.06071 0.07899 0.089606 0.09949 0.11842
33 HouseholdMem 0.14485 0.16148 0.169509 0.17797 0.19467
34 Sigmas <- PostSigmas[keep,]
35 summary(coda::mcmc(Sigmas))
36      2.5%      25%      50%      75%      97.5%
37 var1 0.19664 0.20567 0.21072 0.21593 0.22636
38 var2 0.08201 0.09289 0.09865 0.10447 0.11623
39 var3 0.05222 0.06015 0.06452 0.06903 0.07799
40 var4 0.47390 0.49575 0.50789 0.52046 0.54446
41 var5 0.07355 0.08672 0.09381 0.10061 0.11427
42 var6 0.29259 0.30618 0.31384 0.32140 0.33700

```

6. Simulation exercise of instrumental variables continues I

- Use the setting of the simulation exercise with instrumental variables to analyze the impact of a weak instrument. For example, set $\gamma_2 =$

0.2 and compare the posterior mean estimates of the ordinary and instrumental variable models.

- Perform a simulation to analyze how the degree of exogeneity of the instrument affects the performance of the posterior mean in the instrumental variable model.

Answer

*R code. Simulation exercise, sampling properties
of instrumental variable*

```

1 rm(list = ls())
2 set.seed(010101)
3 N <- 100
4 k <- 2
5 B <- rep(1, k)
6 G <- c(1, 0.2)
7 s12 <- 0.8
8 SIGMA <- matrix(c(1, s12, s12, 1), 2, 2)
9 z <- rnorm(N); # w <- rnorm(N)
10 Z <- cbind(1, z); w <- matrix(1,N,1)
11 S <- 100
12 U <- replicate(S, MASS::mvrnorm(n = N, mu = rep(0, 2), SIGMA
   ))
13 x <- G[1] + G[2]*z + U[,2,]
14 y <- B[1] + B[2]*x + U[,1,]
15 VarX <- G[2]^2+1 # Population variance of x
16 EU1U2 <- s12 # Covariance U1
17 BiasPopB2 <- EU1U2/VarX
18 # Hyperparameters
19 d0 <- 0.001/2
20 a0 <- 0.001/2
21 b0 <- rep(0, k)
22 c0 <- 1000
23 B0 <- c0*diag(k)
24 B0i <- solve(B0)
25 g0 <- rep(0, 2)
26 G0 <- 1000*diag(2)
27 G0i <- solve(G0)
28 nu <- 3
29 Psi0 <- nu*diag(2)
30 # MCMC parameters
31 mcmc <- 5000
32 burnin <- 1000
33 tot <- mcmc + burnin
34 thin <- 1
35 # Gibbs sampling
36 Gibbs <- function(x, y){
37   Data <- list(y = y, x = x, w = w, z = Z)
38   Mcmc <- list(R = mcmc, keep = thin, nprint = 0)
39   Prior <- list(md = g0, Ad = G0i, mbg = b0, Abg = B0i, nu =
     nu, V = Psi0)
40   RestIV <- bayesm::rivGibbs(Data = Data, Mcmc = Mcmc, Prior
     = Prior)
41   PostBIV <- mean(RestIV[["betadraw"]])
42   ResLM <- MCMCpack::MCMCregress(y ~ x + w - 1, b0 = b0, B0
     = B0i, c0 = a0, d0 = d0)
43   PostB <- mean(ResLM[,1])
44   Res <- c(PostB, PostBIV)
45   return(Res)
46 }
47 PosteriorMeans <- sapply(1:S, function(s) {Gibbs(x = x[,s],
   y = y[,s])})
48 rowMeans(PosteriorMeans)
49 1.781513 1.470648

```

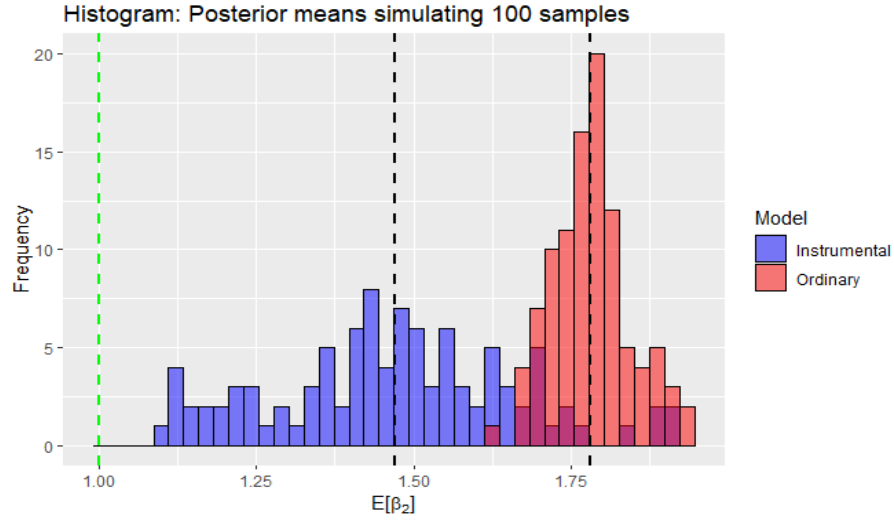



FIGURE 7.1

Histogram of posterior means to assess the effects of *weakness* of the instrument: Ordinary and instrumental models.

Figure 7.1 shows that having a *weak instrument* generates poor performance of the posterior means of the variable instrumental model. Observe that the mean of the posterior means is less precise and biased. This is due to an identification problem in the likelihood function of y and x_s given z (see [45, Chap. 7]).

*R code. Simulation exercise, sampling properties
of instrumental variable*

```

1 # Endogenous instruments
2 rm(list = ls())
3 set.seed(010101)
4 N <- 100
5 k <- 2
6 B <- rep(1, k)
7 G <- c(1, 1)
8 delta <- 0.2 # Effect of instrument on y
9 s12 <- 0.8
10 SIGMA <- matrix(c(1, s12, s12, 1), 2, 2)
11 z <- rnorm(N); # w <- rnorm(N)
12 Z <- cbind(1, z); w <- matrix(1,N,1)
13 S <- 100
14 U <- replicate(S, MASS::mvrnorm(n = N, mu = rep(0, 2), SIGMA
  ))
15 x <- G[1] + G[2]*z + U[,2,]
16 y <- B[1] + B[2]*x + delta*z + U[,1,]
17 VarX <- G[2]^2+1 # Population variance of x
18 EU1U2 <- s12 # Covariance U1
19 BiasPopB2 <- EU1U2/VarX
20 # Hyperparameters
21 d0 <- 0.001/2
22 a0 <- 0.001/2
23 b0 <- rep(0, k)
24 c0 <- 1000
25 B0 <- c0*diag(k)
26 B0i <- solve(B0)
27 g0 <- rep(0, 2)
28 G0 <- 1000*diag(2)
29 G0i <- solve(G0)
30 nu <- 3
31 Psi0 <- nu*diag(2)
32 # MCMC parameters
33 mcmc <- 5000
34 burnin <- 1000
35 tot <- mcmc + burnin
36 thin <- 1
37 # Gibbs sampling
38 Gibbs <- function(x, y){
39   Data <- list(y = y, x = x, w = w, z = Z)
40   Mcmc <- list(R = mcmc, keep = thin, nprint = 0)
41   Prior <- list(md = g0, Ad = G0i, mbg = b0, Abg = B0i, nu =
     nu, V = Psi0)
42   RestIV <- bayesm::rivGibbs(Data = Data, Mcmc = Mcmc, Prior
     = Prior)
43   PostBIV <- mean(RestIV[["betadraw"]])
44   ResLM <- MCMCpack::MCMCregress(y ~ x + w - 1, b0 = b0, B0
     = B0i, c0 = a0, d0 = d0)
45   PostB <- mean(ResLM[,1])
46   Res <- c(PostB, PostBIV)
47   return(Res)
48 }
49 PosteriorMeans <- sapply(1:S, function(s) {Gibbs(x = x[,s],
     y = y[,s])})
50 rowMeans(PosteriorMeans)
51 1.508009 1.217419

```

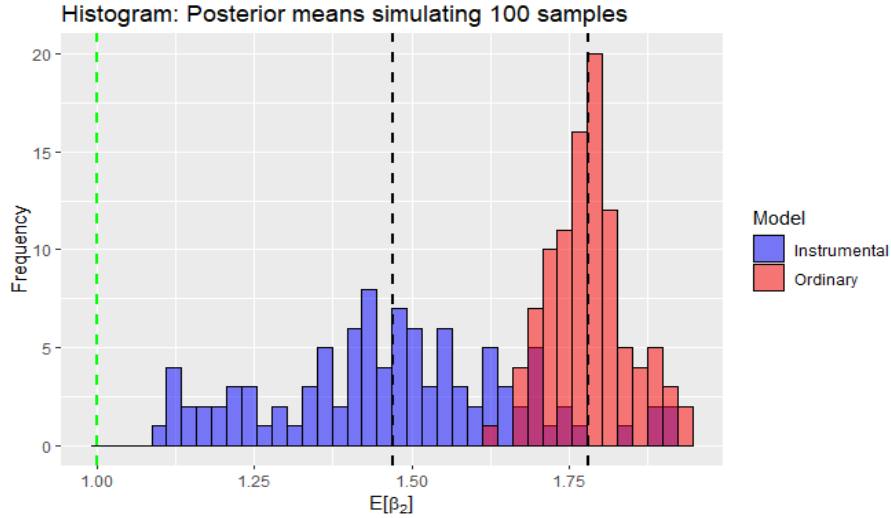


FIGURE 7.2

Histogram of posterior means to assess the effects of *endogeneity* of the instrument: Ordinary and instrumental models.

A simple way to assess the effects of *endogeneity* of the instrument in the instrumental variable model is to introduce the instrument into the *structural equation*, but assume that $\mathbb{E}[yz|x_s] = 0$. In particular, we generate $y_i = \beta_1 + \beta_2 x_{si} + \beta_3 z_i + \mu_i$, but estimate $y_i = \beta_1 + \beta_2 x_{si} + \mu_i$. Figure 7.2 shows that assuming that an instrument is exogenous, when actually it is not, generates bias in the posterior mean of the instrumental variable model. Observe that *exogeneity* of the instrument is equivalent to the dogmatic prior belief that the instrument does not affect the structural equation. Thus, [8] propose to assess the plausibility of the *exclusion restrictions* ($\beta_3 = 0$ and $\gamma_2 \neq 0$) defining *plausible exogeneity* as having prior information that the effect of the instrument in the *structural equation* is near zero, but perhaps not exactly zero.

7. Simulation exercise of instrumental variables continues II

Program the Gibbs sampling algorithm for the instrumental variable model from scratch.

Answer

The following code shows how to implement the Gibbs sampling algorithm from scratch. We observe that all posterior means are close to the population values, and 95% credible intervals encompass them.

*R code. Simulation exercise, Gibbs sampling
instrumental model*

```

1 rm(list = ls()); set.seed(010101)
2 N <- 100; k <- 2; kz <- 2
3 B <- rep(1, k); G <- rep(1, kz)
4 s12 <- 0.8; SIGMA <- matrix(c(1, s12, s12, 1), 2, 2)
5 z <- rnorm(N); Z <- cbind(1, z); w <- matrix(1, N, 1)
6 S <- 100; U <- MASS::mvrnorm(n = N, mu = rep(0, 2), SIGMA)
7 x <- G[1] + G[2]*z + U[,2]; y <- B[1] + B[2]*x + U[,1]
8 X <- cbind(w, x)
9 # Hyperparameters
10 b0 <- rep(0, k); c0 <- 1000; B0 <- c0*diag(k)
11 B0i <- solve(B0); g0 <- rep(0, kz)
12 G0 <- 1000*diag(kz); G0i <- solve(G0)
13 nu <- 3; Psi0 <- nu*diag(2); Psi0i <- solve(Psi0)
14 # MCMC parameters
15 mcmc <- 5000; burnin <- 1000; tot <- mcmc + burnin
16 thin <- 1
17 # Auxiliary elements
18 XtX <- t(X)%*%X; ZtZ <- t(Z)%*%Z; nun <- nu + N
19 # Gibbs sampling
20 PostBeta <- function(Sigma, Gamma){
21   w1 <- Sigma[1,1] - Sigma[1,2]^2/Sigma[2,2]
22   Bn <- solve(w1^(-1)*XtX + B0i)
23   yaux <- y - (Sigma[1,2]/Sigma[2,2])*(x - Z%*%Gamma)
24   bn <- Bn%*%(B0i%*%b0 + w1^(-1)*t(X)%*%yaux)
25   Beta <- MASS::mvrnorm(1, bn, Bn)
26   return(Beta)
27 }
28 PostGamma <- function(Sigma, Beta){
29   w2 <- Sigma[2,2] - Sigma[1,2]^2/Sigma[1,1]
30   Gn <- solve(w2^(-1)*ZtZ + G0i)
31   xaux <- x - (Sigma[1,2]/Sigma[1,1])*(y - X%*%Beta)
32   gn <- Gn%*(G0i%*%g0 + w2^(-1)*t(Z)%*%xaux)
33   Gamma <- MASS::mvrnorm(1, gn, Gn)
34   return(Gamma)
35 }
36 PostSigma <- function(Beta, Gamma){
37   Uy <- y - X%*%Beta; Ux <- x - Z%*%Gamma
38   U <- cbind(Uy, Ux)
39   Psin <- solve(Psi0i + t(U)%*%U)
40   Sigmai <- rWishart::rWishart(1, df = nun, Sigma = Psin)
41   Sigma <- solve(Sigmai[, , 1])
42   return(Sigma)
43 }

```

*R code. Simulation exercise, Gibss sampling
instrumental model*

```

1 PostBetas <- matrix(0, tot, k); PostGammas <- matrix(0, tot,
  kz)
2 PostSigmas <- matrix(0, tot, 2*(2+1)/2); Beta <- rep(0, k);
  Gamma <- rep(0, kz)
3 pb <- txtProgressBar(min = 0, max = tot, style = 3)
4 for(s in 1:tot){
5   Sigma <- PostSigma(Beta = Beta, Gamma = Gamma)
6   Beta <- PostBeta(Sigma = Sigma, Gamma = Gamma)
7   Gamma <- PostGamma(Sigma = Sigma, Beta = Beta)
8   PostBetas[s,] <- Beta; PostGammas[s,] <- Gamma;
    PostSigmas[s,] <- matrixcalc::vech(Sigma)
9   setTxtProgressBar(pb, s)
10 }
11 close(pb)
12 keep <- seq((burnin+1), tot, thin)
13 Bs <- PostBetas[keep,]
14 Gs <- PostGammas[keep,]
15 Sigmas <- PostSigmas[keep,]
16 summary(coda::mcmc(Bs))
17 1. Empirical mean and standard deviation for each variable,
18 plus standard error of the mean:
19 Mean      SD Naive SE Time-series SE
20 [1,] 1.1519 0.1683 0.00238      0.006321
21 [2,] 0.9896 0.1146 0.00162      0.005177
22 summary(coda::mcmc(Gs))
23 1. Empirical mean and standard deviation for each variable,
24 plus standard error of the mean:
25 Mean      SD Naive SE Time-series SE
26 [1,] 1.133 0.1068 0.001511      0.003096
27 [2,] 1.030 0.1066 0.001507      0.004193
28 summary(coda::mcmc(Sigmas))
29 1. Empirical mean and standard deviation for each variable,
30 plus standard error of the mean:
31 Mean      SD Naive SE Time-series SE
32 [1,] 1.3883 0.3149 0.004454      0.012707
33 [2,] 0.9992 0.2133 0.003017      0.007166
34 [3,] 1.1637 0.1685 0.002382      0.002481

```

8. The effect of institutions on per capita GDP continues II

Estimate the structural Equation 7.1 using the instrumental variable model, where $\log(Mort)$ serves as the instrument for PAER. Compare the estimated effect of property rights on per capita GDP with that obtained in the example on the effect of institutions on per capita GDP. Use the file *6Institutions.csv* to conduct this exercise in our GUI, setting $B_0 = 100I_5$, $\beta_0 = 0_5$, $\gamma_0 = 0_2$, $G_0 = 100I_2$, $\alpha_0 = 3$, and $\Psi_0 = 3I_2$. Set the MCMC pa-

rameters as follows: 50,000 iterations, 1,000 burn-in, and a thinning interval of 5.

Answer

The following code shows how to do this using the command *rivGibbs* from the *bayesm* package. We see that the posterior mean of the effect of property rights on per capita GDP is 0.72, and the 95% credible interval is (0.47, 1.00). The posterior mean estimate of this effect was 0.98 in the example, and the 95% credible interval was (0.56, 2.87) (see Section 8.1 in the book). We can observe that the indirect approach, where the *reduced-form* parameters are estimated, and then, plug into the equations of the *structural parameters* produce less efficient posterior estimations.

*R code. The effects of institutions on GDP,
variable instrumental model*

```

1 rm(list = ls()); set.seed(010101)
2 DataInst <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/6Institutions
  .csv", sep = ",", header = TRUE, quote = "")
3 attach(DataInst)
4 y <- logpcGDP95; x <- PAER
5 w <- cbind(1, Africa, Asia, Other); Z <- cbind(1, logMort)
6 # Hyperparameters
7 k <- 5; kz <- 2; b0 <- rep(0, k); c0 <- 100
8 B0 <- c0*diag(k); B0i <- solve(B0); g0 <- rep(0, kz)
9 G0 <- 100*diag(kz); G0i <- solve(G0); nu <- 5
10 Psi0 <- nu*diag(2)
11 # MCMC parameters
12 mcmc <- 50000; burnin <- 10000; tot <- mcmc + burnin; thin
  <- 5
13 # Gibbs sampling
14 Data <- list(y = y, x = x, w = w, z = Z)
15 Mcmc <- list(R = mcmc, keep = thin, nprint = 100)
16 Prior <- list(md = g0, Ad = G0i, mbg = b0, Abg = B0i, nu =
  nu, V = Psi0)
17 RestIV <- bayesm::rivGibbs(Data = Data, Mcmc = Mcmc, Prior =
  Prior)
18 summary(RestIV[["deltadraw"]])
19 summary(coda::mcmc(RestIV[["betadraw"]]))
20 Iterations = 1:10000
21 Thinning interval = 1
22 Number of chains = 1
23 Sample size per chain = 10000
24 1. Empirical mean and standard deviation for each variable,
25 plus standard error of the mean:
26 Mean          SD          Naive SE Time-series SE
27 0.759497      0.140874      0.001409      0.002477
28 2. Quantiles for each variable:
29 2.5%    25%    50%    75%   97.5%
30 0.5110 0.6654 0.7502 0.8424 1.0707
31 summary(RestIV[["gammadraw"]])
32 summary(RestIV[["sigmadraw"]])

```

9. Multivariate probit with different regressors

Simulate the model

$$y_{i1}^* = 0.5 - 1.2x_{i11} + 0.7x_{i12} + 0.8x_{i13} + \mu_{i1}, \quad y_{i2}^* = 1.5 - 0.8x_{i21} + 0.5x_{i22} + \mu_{i2},$$

with

$$\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix},$$

where all regressors follow a standard normal distribution and $N = 5000$. Use $\beta_0 = \mathbf{0}$, $\mathbf{B}_0 = 1000\mathbf{I}$, $\alpha_0 = 4$, and $\Psi_0 = 4\mathbf{I}_2$. Set the number of iterations to 2,000 and the thinning parameter to 5.

- Perform inference under the setting of Section 7.4, assuming that x_{i3} may affect y_{i2} .
- Program a Gibbs sampling algorithm that accounts for different regressors in each binary decision, that is, where x_{i3} does not affect y_{i2} .

Answer

The following code shows how to perform inference assuming same number of regressors in both binary results. We can check that all 95% credible intervals encompass the population values, and the posterior mean estimates are very close to the population parameters.

R code. Multivariate probit model: Same number of regressors in each binary response

```

1 remove(list = ls())
2 n <- 5000 #Individuals
3 p <- 2 #Number dependent variables (For instance to buy or
4       #not to buy different products)
5 xo <- 2 #Number of choice dependent variables (For instance
6       #price of products)
7 xi <- 1 #Number of regressors that dependt on individuals (
8       #For instance income)
9 B1 <- c(0.5, -1.2, 0.7, 0.8)
10 B2 <- c(1.5, -0.8, 0.5, 0) # The last regressor is not relevant
11      #for the second product
12 Cor <- matrix(c(1, 0.5, 0.5, 1), p, p)
13 yX <- NULL; y1 <- NULL
14 for (i in 1:n){
15   ei <- MASS::mvrnorm(1, c(0,0), Cor); xs <- rnorm(xi)
16   x1i <- c(1, rnorm(xo), xs); y11i <- sum(x1i*B1)
17   y1i <- y11i + ei[1] > 0; x2i <- c(1, rnorm(xo), xs)
18   y12i <- sum(x2i*B2); y2i <- y12i + ei[2] > 0
19   yXi <- rbind(c(y1i, x1i), c(y2i, x2i))
20   yXi <- cbind(i, yXi)
21   colnames(yXi) <- c("id", "y", "cte", "x1o", "x2o", "xi")
22   yX <- rbind(yX, yXi)
23   y1 <- c(y1, c(y11i + ei[1], y12i + ei[2]))
24 }
25 nd <- 4 # Number of regressors
26 Xd <- as.matrix(yX[,3:6])
27 XcreateMP <- function(p, nxs, nind, Data){
28   pandterm = function(message) {
29     stop(message, call. = FALSE)
30   }
31   if (missing(nxs))
32     pandterm("requires number of regressors: include intercept
33             #if required")
34   if (missing(nind))
35     pandterm("requires number of units (individuals)")
36   if (missing(Data))
37     pandterm("requires dataset")
38   if (nrow(Data) != nind*2)
39     pandterm("check dataset! number of units times number
40             #alternatives should be equal to dataset rows")
41   XXDat <- array(0, c(p, 1+nxs, nind))
42   XX <- array(0, c(p, nxs*p, nind))
43   YY <- array(0, c(p, 1, nind))
44   is <- seq(p, nind*p, p)
45   cis <- seq(nxs, nxs*p+1, nxs)
46   for(i in is){
47     j <- which(i==is)
48     XXDat[,j] <- as.matrix(Data[c((i-(p-1)):i), -1])
49     YY[,j] <- XXDat[,1,j]
50     for(l in 1:p){
51       XX[l, (cis[l]-(nxs-1)):cis[l], j] <- XXDat[l, -1, j]
52     }
53   }
54   return(list(y=YY, X=XX))
55 }
56 Dat <- XcreateMP(p = p, nxs = nd, nind = n, Data = yX)

```

R code. Multivariate probit model: Same number of regressors in each binary response

```

1 y<-NULL
2 X<-NULL
3 for(i in 1:dim(Dat$y)[3]){
4   y<-c(y,Dat$y[,i])
5   X<-rbind(X,Dat$X[,i])
6 }
7 DataMP = list(p=p, y=y, X=X)
8 # Hyperparameters
9 k <- dim(X)[2]
10 b0 <- rep(0, k)
11 c0 <- 1000
12 B0 <- c0*diag(k)
13 B0i <- solve(B0)
14 a0 <- p - 1 + 3
15 Psi0 <- a0*diag(p)
16 Prior <- list(betabar = b0, A = B0i, nu = a0, V = Psi0)
17 # MCMC parameters
18 mcmc <- 2000
19 burnin <- 1000
20 thin <- 5
21 Mcmc <- list(R = mcmc, keep = thin)
22 Results <- bayesm::rmvpGibbs(Data = DataMP, Mcmc = Mcmc,
   Prior = Prior)
23 betatildeEq1 <- Results$betadraw[,1:4] / sqrt(Results$
   sigmadraw[,1])
24 summary(coda::mcmc(betatildeEq1))
25 betatildeEq2 <- Results$betadraw[,5:8] / sqrt(Results$
   sigmadraw[,4])
26 summary(coda::mcmc(betatildeEq2))
27 sigmadraw12 <- Results$sigmadraw[,2] / (Results$sigmadraw
   [,1]*Results$sigmadraw[,4])^0.5
28 summary(coda::mcmc(sigmadraw12))

```

The following code shows the implementation of a Gibbs sampling algorithm using the *bayesm* package, particularly, the *rmvpGibbs* command.

R code. Multivariate probit model: Different number of regressors in each binary response, the rmvpGibbs command

```

1 # Omitting last regressor for y2
2 Xnew <- X[,-8]
3 DataMP = list(p=p, y=y, X=Xnew)
4 # Hyperparameters
5 k <- dim(Xnew)[2]
6 b0 <- rep(0, k)
7 c0 <- 1000
8 B0 <- c0*diag(k)
9 B0i <- solve(B0)
10 Prior <- list(betabar = b0, A = B0i, nu = a0, V = Psi0)
11 Results <- bayesm::rmvpGibbs(Data = DataMP, Mcmc = Mcmc,
    Prior = Prior)
12 betatildeEq1 <- Results$betadraw[,1:4] / sqrt(Results$
    sigmadraw[,1])
13 summary(coda::mcmc(betatildeEq1))
14 betatildeEq2 <- Results$betadraw[,5:7] / sqrt(Results$
    sigmadraw[,4])
15 summary(coda::mcmc(betatildeEq2))
16 sigmadraw12 <- Results$sigmadraw[,2] / (Results$sigmadraw
    [,1]*Results$sigmadraw[,4])^0.5
17 summary(coda::mcmc(sigmadraw12))

```

The following code shows how to implement a Gibbs sampling algorithm from scratch.

R code. Multivariate probit model: Different number of regressors in each binary response, Gibbs from scratch

```

1 PostBetaNew <- function(Sigma, Yl){
2   Sigmai <- solve(Sigma); C <- chol(Sigmai)
3   Xstar <- matrix(0, n*p, k); Ystar <- matrix(0, n*p, 1)
4   for(i in seq(1, n*p, 2)){
5     Xstari <- C%%Xnew[i:(i+1),]
6     Xstar[i:(i+1),] <- Xstari
7     Ystari <- C%%Yl[i:(i+1)]
8     Ystar[i:(i+1)] <- Ystari
9   }
10  Bn <- solve(B0i + t(Xstar)%%Xstar)
11  bn <- Bn%*(B0i%%b0 + t(Xstar)%%Ystar)
12  Beta <- MASS::mvrnorm(1, bn, Bn)
13  return(Beta)
14 }
15 PostSigmaNew <- function(Beta, Yl){
16  U <- matrix(0, p, p)
17  for(i in seq(1, n*p, 2)){
18    Ui <- (Yl[i:(i+1)] - Xnew[i:(i+1),]%%Beta)%%t(Yl[i:(i+1)] - Xnew[i:(i+1),]%%Beta)
19    U <- U + Ui
20  }
21  Psin <- solve(Psi0i + U)
22  Sigmai <- rWishart::rWishart(1, df = an, Sigma = Psin)
23  Sigma <- solve(Sigmai[, ,1])
24 }
25 PostYlNew <- function(Beta, Sigma, Yl){
26  w1 <- Sigma[1, ]
27  w2 <- Sigma[2, ]
28  f1 <- w1[-1]*Sigma[2, 2]^(-1)
29  f2 <- w2[-2]*Sigma[1, 1]^(-1)
30  t11 <- Sigma[1, 1] - w1[-1]*Sigma[2, 2]^(-1)*w1[-1]
31  t22 <- Sigma[2, 2] - w2[-2]*Sigma[1, 1]^(-1)*w2[-2]
32  Ylnew <- Yl
33  for(i in seq(1, n*p, 2)){
34    Ylmean1 <- Xnew[i,]%%Beta + f1*(Yl[i+1] - Xnew[i+1,]%%Beta)
35    if(y[i] == 1){
36      Ylnew[i] <- truncnorm::rtruncnorm(1, a = 0, b = Inf,
37        mean = Ylmean1, sd = t11^0.5)
38    }else{
39      Ylnew[i] <- truncnorm::rtruncnorm(1, a = -Inf, b = 0,
40        mean = Ylmean1, sd = t11^0.5)
41    }
42    Ylmean2 <- Xnew[i+1,]%%Beta + f2*(Ylnew[i] - Xnew[i,]%%Beta)
43    if(y[i+1] == 1){
44      Ylnew[i+1] <- truncnorm::rtruncnorm(1, a = 0, b = Inf,
45        mean = Ylmean2, sd = t22^0.5)
46    }else{
47      Ylnew[i+1] <- truncnorm::rtruncnorm(1, a = -Inf, b = 0,
48        mean = Ylmean2, sd = t22^0.5)
49    }
50  }
51  return(Ylnew)
52 }

```

R code. Multivariate probit model: Different number of regressors in each binary response, Gibbs from scratch

```

1 an <- a0 + n
2 Psi0i <- solve(Psi0)
3 K1 <- 4; K2 <- 3; K <- K1 + K2
4 tot <- mcmc + burnin
5 PostBetas <- matrix(0, tot, K)
6 PostSigmas <- matrix(0, tot, p*(p+1)/2)
7 Beta <- rep(1, K)
8 Sigma <- diag(p)
9 pb <- txtProgressBar(min = 0, max = tot, style = 3)
10 for(s in 1:tot){
11   Yl <- PostYlNew(Beta = Beta, Sigma = Sigma, Yl = yl)
12   Sigma <- PostSigmaNew(Beta = Beta, Yl = Yl)
13   Beta <- PostBetaNew(Sigma = Sigma, Yl = Yl)
14   PostBetas[s,] <- Beta
15   PostSigmas[s,] <- matrixcalc::vech(Sigma)
16   setTxtProgressBar(pb, s)
17 }
18 close(pb)
19 keep <- seq((burnin+1), tot, thin)
20 Bs <- PostBetas[keep,]
21 betatilde1 <- Bs[,1:K1] / sqrt(PostSigmas[keep,1])
22 summary(coda::mcmc(betatilde1))
23 betatilde2 <- Bs[, (K1+1):(K1+K2)] / sqrt(PostSigmas[keep,3])
24 summary(coda::mcmc(betatilde2))
25 sigmadraw12 <- PostSigmas[keep,2] / (PostSigmas[keep,1] *
   PostSigmas[keep,3])^0.5
26 summary(coda::mcmc(sigmadraw12))

```



8

Time series models

Solutions of Exercises

1. Simulate the *dynamic linear model* assuming $X_t \sim N(1, 0.1\sigma^2)$, $w_t \sim N(0, 0.5\sigma^2)$, $\mu_t \sim N(0, \sigma^2)$, $\beta_0 = 1$, $B_0 = 0.5\sigma^2$, $\sigma^2 = 0.25$, and $G_t = 1$, $t = 1, \dots, 100$. Then, perform the filtering recursion fixing $\Sigma = 25 \times 0.25$, $\Omega_1 = 0.5\Sigma$ (high signal-to-noise ratio) and $\Omega_2 = 0.1\Sigma$ (low signal-to-noise ratio). Plot and compare the results.

Answer

The following code shows how to perform this simulation and perform the filtering recursion using the *dlm* package in **R**.

R code. Simulation: Scenarios signal-to-noise ratio

```

1 rm(list = ls()); set.seed(010101)
2 T <- 100; sig2 <- 0.5^2; r <- 0.5; sigW2 <- sig2*r
3 x <- rnorm(T, mean = 1, sd = 0.1*sig2^0.5)
4 e <- rnorm(T, mean = 0, sd = sig2^0.5)
5 w <- rnorm(T, mean = 0, sd = sigW2^0.5)
6 K <- 1
7 Bt <- matrix(NA, T, K); Bt[1] <- 1
8 yt <- rep(NA, T)
9 yt[1] <- x[1]*Bt[1] + e[1]
10 for(t in 1:T){
11   if(t == 1){
12     Bt[t,] <- w[t]
13   }else{
14     Bt[t,] <- Bt[t-1,] + w[t]
15   }
16   yt[t] <- x[t]*Bt[t,] + e[t]
17 }
18 # State space model
19 ModelReg <- function(par){
20   Mod <- dlm::dlmModReg(x, dV = exp(par[1]), dW = exp(par
21     [2]), m0 = rep(1, K),
22   C0 = sigW2*diag(K), addInt = FALSE)
23   return(Mod)
24 }
25 sig2New <- sig2*25
26 RegFilter1 <- dlm::dlmFilter(yt, ModelReg(c(sig2New, sig2New
27   *0.5)))
28 ytfil1 <- RegFilter1[["m"]][-1]
29 RegFilter2 <- dlm::dlmFilter(yt, ModelReg(c(sig2New, sig2New
30   *0.1)))
31 ytfil2 <- RegFilter2[["m"]][-1]
32 Time <- 1:T
33 df1 <- as.data.frame(cbind(Time,yt, ytfil1, ytfil2))
34 library(ggplot2)
35 ggplot(df1, aes(x=Time)) +
36   geom_line(aes(y=yt), colour="black", linewidth=1, linetype
37     =1) +
38   geom_line(aes(y=ytfil1), colour="green", linewidth=1, alpha
39     =0.9, linetype=2) +
40   geom_line(aes(y=ytfil2), color="blue", linewidth=1, alpha
41     =0.9, linetype=3)

```

Figure 8.1 shows the results of the filtering recursion. We see in this figure that the filtered recursion associated with the high signal-to-noise ratio (green dashed line) follows closer the data than the low signal-to-noise ratio scenario (blue dotted line). We see that the latter is smoother as it

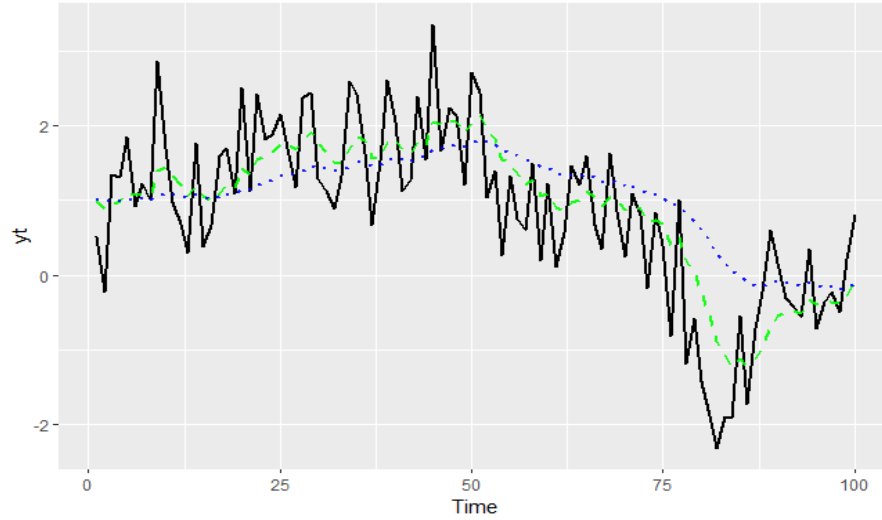


FIGURE 8.1

Signal-to-noise ratio: Dynamic linear model.

gives more weight to the prior mean, which takes into account historical information.

2. Simulate the *dynamic linear model* $y_t = \beta_t x_t + \mu_t$, $\beta_t = \beta_{t-1} + w_t$, where $x_t \sim N(1, 0.1\sigma^2)$, $w_t \sim N(0, 0.5\sigma^2)$, $\mu_t \sim N(0, \sigma^2)$, $\beta_0 = 0$, $B_0 = 0.5\sigma^2$, and $\sigma^2 = 1$, $t = 1, \dots, 100$. Perform the filtering and smoothing recursions from scratch.

Answer

The following code shows how to perform this simulation and perform the filtering recursion from scratch in **R**.

R code. Simulation: Filtering and smoothing recursions from scratch

```

1 rm(list = ls()); set.seed(010101)
2 T <- 100; sig2 <- 1; r <- 0.5; sigW2 <- sig2*r
3 xt <- rnorm(T, mean = 1, sd = 0.1*sig2^0.5)
4 e <- rnorm(T, mean = 0, sd = sig2^0.5)
5 w <- rnorm(T, mean = 0, sd = sigW2^0.5)
6 K <- 1; Bt <- matrix(NA, T, K); Bt[1] <- 0
7 yt <- rep(NA, T)
8 for(t in 1:T){
9   if(t == 1){
10     Bt[t,] <- w[t]
11   }else{
12     Bt[t,] <- Bt[t-1,] + w[t]
13   }
14   yt[t] <- xt[t]%*%Bt[t,] + e[t]
15 }
16 # Filtering
17 KFR <- function(y, x, b1, B1, Omega, sig2){
18   e <- as.numeric(y - t(x)%*%b1) # Error
19   R <- B1 + Omega
20   q <- as.numeric(t(x)%*%R%*%x + sig2)
21   K <- (R%*%x)/q
22   bt <- b1 + K*e
23   Bt <- R - R%*%x%*%t(x)%*%R/q
24   Result <- list(bt = bt, Bt = Bt)
25   return(Result)
26 }
27 KFresb <- list(); KFresB <- list()
28 b0 <- 0; B0 <- sigW2
29 for(t in 1:T){
30   KFrest <- KFR(y = yt[t], x = xt[t], b1 = b0, B1 = B0,
31     Omega = sigW2, sig2 = sig2)
32   b0 <- KFrest[["bt"]]; B0 <- KFrest[["Bt"]]
33   KFresb[[t]] <- b0; KFresB[[t]] <- B0
34 }
35 ModelReg <- function(par){
36   Mod <- dlm::dlmModReg(xt, dV = exp(par[1]), dW = exp(par
37     [2]), m0 = rep(0, K),
38   C0 = sigW2*diag(K), addInt = FALSE)
39   return(Mod)
40 }
41 RegFilter1 <- dlm::dlmFilter(yt, ModelReg(c(sig2, sigW2)))
42 plot(Bt, type = "l")
43 lines(unlist(KFresb), type = "l", col = "red")
44 lines(RegFilter1[["m"]][-1], type = "l", col = "blue")

```

R code. Simulation: Filtering and smoothing recursions from scratch

```

1 # Smoothing
2 BSR <- function(b, B, slead, Slead, Omega){
3   b = bfilter[t]; B = Bfilter[t]; slead = slead; Slead =
4     slead; Omega = sigW2
5   R1 <- B + Omega
6   s <- b + B%*%solve(R1)%*%(slead - b)
7   S <- B - B%*%solve(R1)%*%(R1 - Slead)%*%solve(R1)%*%B
8   Result <- list(st = s, St = S)
9   return(Result)
10 }
11 bfilter <- unlist(KFresb); Bfilter <- unlist(KFresB)
12 slead <- bfilter[T]; Slead <- Bfilter[T]
13 smooth <- rep(slead, T); Smooth <- rep(Slead, T);
14 for(t in (T-1):1){
15   BSRrest <- BSR(b = bfilter[t], B = Bfilter[t], slead =
16     slead, Slead = Slead, Omega = sigW2)
17   slead <- BSRrest[["st"]]; Slead <- BSRrest[["St"]]
18   smooth[t] <- slead
19 }
20 RegSmoth <- dlm::dlmSmooth(yt, ModelReg(c(sig2, sigW2)))
21 VarSmooth <- dlm::dlmSvd2var(u = RegSmoth[["U.S"]], RegSmoth
22   [["D.S"]])
23 SDVarSmoothB2 <- sapply(2:(T+1), function(t){VarSmooth[[t]][
24   K,K]^0.5})
25 plot(Bt, type = "l")
26 lines(smooth, type = "l", col = "orange")
27 lines(RegSmoth[["s"]][-1], type = "l", col = "green")

```

3. Simulate the process $y_t = \alpha z_t + \beta_t x_t + \mathbf{h}^\top \boldsymbol{\epsilon}_t$, $\beta_t = \beta_{t-1} + \mathbf{H}^\top \boldsymbol{\epsilon}_t$, where $\mathbf{h}^\top = [1 \ 0]$, $\mathbf{H}^\top = [0 \ 1/\tau]$, $\mathbf{v}_t \sim N(\mathbf{0}_2, \sigma^2 \mathbf{I}_2)$, $x_t \sim N(1, 2\sigma^2)$, $z_t \sim N(0, 2\sigma^2)$, $\alpha = 2$, $\tau^2 = 5$ and $\sigma^2 = 0.1$, $t = 1, \dots, 200$. Assume $\pi(\beta_0, \alpha, \sigma^2, \tau) = \pi(\beta_0)\pi(\alpha)\pi(\sigma^2)\pi(\tau^2)$ where $\sigma^2 \sim IG(\alpha_0/2, \delta_0/2)$, $\tau^2 \sim G(v_0/2, v_0/2)$, $\alpha \sim N(a_0, A_0)$ and $\beta_0 \sim N(b_0, B_0)$ such that $\alpha_0 = \delta_0 = 1$, $v_0 = 5$, $a_0 = 0$, $A_0 = 1$, $\beta_0 = 0$, $B_0 = \sigma^2/\tau^2$. Program the MCMC algorithm including the *simulation smoother*.

Answer

*R code. Simulation: MCMC algorithm with
simulation smoother*

```

1 rm(list = ls()); set.seed(010101)
2 T <- 200; sig2 <- 0.1; tau2 <- 5; tau <- tau2^0.5; sigW2 <-
  sig2/tau2
3 xt <- rnorm(T, mean = 1, sd = 2*sig2^0.5)
4 zt <- rnorm(T, mean = 1, sd = 2*sig2^0.5)
5 alpha <- 2
6 e <- MASS::mvrnorm (T, mu = c(0, 0), Sigma = sig2*diag(2))
7 K <- 1
8 h1 <- matrix(c(1, 0), K + 1, 1); h2 <- matrix(c(0, 1/tau), K
  + 1, 1); t(h1)%*%h2
9 e1 <- e%*%h1; e2 <- e%*%h2; var(e1); var(e2)
10 Bt <- matrix(NA, T, K); Bt[1] <- 0
11 yt <- rep(NA, T)
12 for(t in 1:T){
13   if(t == 1){
14     Bt[t,] <- e2[t]
15   }else{
16     Bt[t,] <- Bt[t-1,] + e2[t]
17   }
18   yt[t] <- zt[t]*alpha + xt[t]*Bt[t] + e1[t]
19 }
20 # Recursion functions
21 # Filter
22 KFRDSnew <- function(y, z, x, b, B, alpha, tau2){
23   e <- as.numeric(y - z*alpha - t(x)%*%b)
24   q <- as.numeric(t(x)%*%B%*x + t(h1)%*%h1)
25   K <- (B%*x)/q
26   bt <- b + K*e
27   h2 <- matrix(c(0, 1/tau2^0.5))
28   Bt <- B - B%*x%*t(K) + t(h2)%*%h2
29   Result <- list(bt = bt, Bt = Bt, et = e, Kt = K, qt = q)
30   return(Result)
31 }
32 # Smooth
33 BSRDSnew <- function(r, M, Q, e, K, x, sig2, tau2){
34   h2 <- matrix(c(0, 1/tau2^0.5))
35   Lambda <- t(h2)%*%h2
36   C <- Lambda - Lambda%*%M%*t(Lambda)
37   xi <- rnorm(1, 0, sd = (sig2*C)^0.5)
38   L <- 1 - K*x
39   V <- Lambda%*%M%*L
40   rtl <- x*e/Q + t(L)*r - t(V)%*%solve(C)%*%xi
41   Mtl <- x%*%t(x)/Q + t(L)%*%M%*L + t(V)%*%solve(C)%*%V
42   eta <- Lambda%*%r + xi
43   Result <- list(rtl = rtl, Mtl = Mtl, etat = eta)
44   return(Result)
45 }
46 # Gibbs functions
47 PostSig2 <- function(bbt, alpha, tau2){
48   an <- T*(K + 1) + alpha0
49   term1 <- diff(c(0, bbt))
50   term2 <- c(yt - alpha*zt - xt*bbt)
51   dn <- delta0 + sum(term1^2)*tau2 + sum(term2^2)
52   sig2 <- invgamma::rinvgamma(1, shape = an/2, rate = dn/2)
53   return(sig2)
54 }

```

*R code. Simulation: MCMC algorithm with
simulation smoother*

```

1 PosAlpha <- function(bbt, sig2){
2   An <- solve(A0i + sig2^(-1)*t(zt)%*%zt)
3   term <- yt-xt*bbt
4   an <- An%*%(A0i%*%a0 + sig2^(-1)*t(zt)%*%term)
5   Alpha <- MASS::mvrnorm(1, an, An)
6   return(Alpha)
7 }
8 PostTau2 <- function(bbt, sig2){
9   v1n <- v0 + T
10  term1 <- diff(c(0,bbt))
11  v2n <- v0 + sig2^(-1)*sum(term1^2)
12  tau2 <- rgamma(1, v1n/2, v2n/2)
13  return(tau2)
14 }
15 # Hyperparameter
16 alpha0 <- 1; delta0 <- 1; v0 <- 5
17 a0 <- 0; A0 <- 1; A0i <- 1/A0
18 tau2post <- 2; sig2post <- 0.2; alphapost <- 0
19 # Inference
20 S <- 2000; burnin <- 500; tot <- S + burnin; thin <- 5
21 # Posterior draws
22 sig2s <- rep(NA, tot); tau2s <- rep(NA, tot)
23 alphas <- rep(NA, tot); betas <- matrix(NA, tot, T)
24 pb <- txtProgressBar(min = 0, max = tot, style = 3)

```

*R code. Simulation: MCMC algorithm with
simulation smoother*

```

1 for (s in 1:tot){
2   b0 <- 0; B0 <- sigW2
3   KFDSresb <- list(); KFDSresB <- list()
4   KFDSresE <- list(); KFDSresK <- list(); KFDSresQ <- list()
5   for(t in 1:T){
6     KFDSrest <- KFRDSnew(y = yt[t], z = zt[t], x = xt[t], b
7       = b0, B = B0, alpha = alphapost, tau2 = tau2post)
8     b0 <- KFDSrest[["bt"]]; B0 <- KFDSrest[["Bt"]]
9     KFDSresb[[t]] <- b0; KFDSresB[[t]] <- B0
10    KFDSresE[[t]] <- KFDSrest[["et"]]; KFDSresK[[t]] <-
11      KFDSrest[["Kt"]]; KFDSresQ[[t]] <- KFDSrest[["qt"]]
12  }
13  et <- unlist(KFDSresE); Qt <- unlist(KFDSresQ); Kt <-
14    unlist(KFDSresK)
15  rT <- 0; MT <- 0
16  BSRDSreseta <- rep(0, T); BSRDSresetaMean <- rep(0, T)
17  for(t in (T-1):1){
18    BSRDSrest <- BSRDSnew(r = rT, M = MT, Q = Qt[t+1], e =
19      et[t+1], K = Kt[t+1], x = xt[t+1], sig2 = sig2post, tau2
20      = tau2post)
21    rT <- BSRDSrest[["rt1"]]; MT <- BSRDSrest[["Mt1"]]
22    BSRDSreseta[t+1] <- BSRDSrest[["etat"]]
23  }
24  Bt1DSpost <- rep(0,T)
25  for(t in 1:T){
26    if(t == 1){
27      Bt1DSpost[t] <- BSRDSreseta[t]
28    }else{
29      Bt1DSpost[t] <- Bt1DSpost[t-1] + BSRDSreseta[t]
30    }
31  }
32  sig2post <- PostSig2(bbt = Bt1DSpost, alpha = alphapost,
33    tau2 = tau2post)
34  alphapost <- PosAlpha(bbt = Bt1DSpost, sig2 = sig2post)
35  tau2post <- PostTau2(bbt = Bt1DSpost, sig2 = sig2post)
36  sig2s[s] <- sig2post
37  tau2s[s] <- tau2post
38  alphas[s] <- alphapost
39  betas[s,] <- Bt1DSpost
40  setTxtProgressBar(pb, s)
41 }
42 close(pb); keep <- seq((burnin+1), tot, thin)
43 betasF <- betas[keep,]; sig2sF <- coda::mcmc(sig2s[keep])
44 tau2sF <- coda::mcmc(tau2s[keep]); alphasF <- coda::mcmc(
45   alphas[keep])
46 summary(sig2sF); plot(sig2sF)
47 summary(tau2sF); plot(tau2sF)
48 summary(alphasF); plot(alphasF)
49 library(fanplot); df <- as.data.frame(betasF)
50 plot(NULL, main="Percentiles", xlim = c(1, T+1), ylim = c
51   (-4, 1), xlab = "Time", ylab = TeX("$\\beta_{t1}$"))
52 fan(data = df); lines(colMeans(betasF), col = "black", lw =
53   2)
54 lines(Bt, col = "blue")

```

Figure 8.2 shows the path of the state vector (blue line), and the results from the *simulation smoother* using posterior draws, the black line is the mean, and the red-yellow shadows are different deciles.

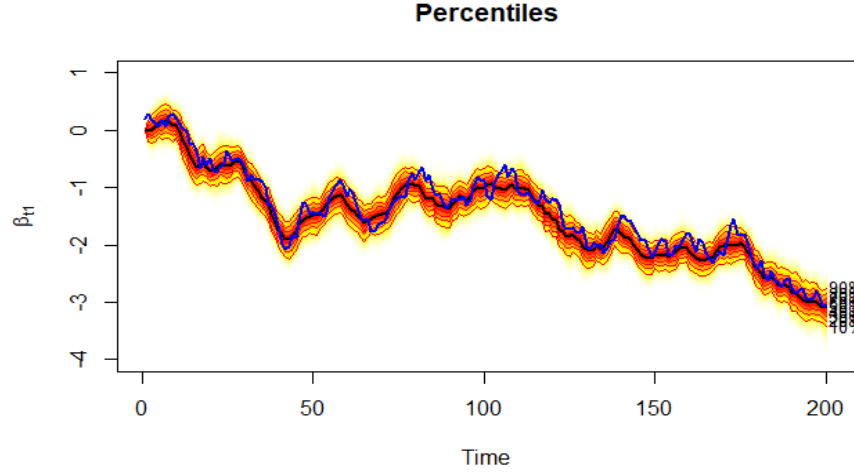


FIGURE 8.2

State vector: Dynamic linear model.

4. Show that the posterior distribution of $\phi|\beta, \sigma^2, \mathbf{y}, \mathbf{X}$ in the model $Y_t = \mathbf{x}_t^\top \beta + \mu_t$ where $\phi(L)\mu_t = \epsilon_t$ and $\epsilon_t \stackrel{iid}{\sim} N(0, \sigma^2)$ is $N(\phi_n, \Phi_n) \mathbb{1}[\phi \in S_\phi]$, where $\Phi_n = (\Phi_0^{-1} + \sigma^{-2} \mathbf{U}^\top \mathbf{U})$, $\phi_n = \Phi_n(\Phi_0^{-1} \phi_0 + \sigma^{-2} \mathbf{U}^\top \boldsymbol{\mu})$, and S_ϕ is the stationary region of ϕ .

Answer

The conditional posterior distribution can be got as follows:

$$\begin{aligned}
 \pi(\phi|\beta, \sigma^2, \mathbf{y}) &\propto \exp \left\{ -\frac{1}{2\sigma^2} (\boldsymbol{\mu} - \mathbf{U}\phi)^\top (\boldsymbol{\mu} - \mathbf{U}\phi) \right\} \\
 &\quad \times \exp \left\{ -\frac{1}{2} (\phi - \phi_0)^\top \Phi_0^{-1} (\phi - \phi_0) \right\} \\
 &\propto \exp \left\{ -\frac{1}{2} [\phi^\top (\sigma^{-2} \mathbf{U}^\top \mathbf{U} + \Phi_0^{-1}) \phi - 2\phi^\top (\sigma^{-2} \mathbf{U}^\top \boldsymbol{\mu} + \Phi_0^{-1} \phi_0) \right. \\
 &\quad \left. + \phi_n^\top \Phi_n^{-1} \phi_n - \phi_n^\top \Phi_n^{-1} \phi_n] \right\} \\
 &\propto \exp \left\{ -\frac{1}{2} [(\phi - \phi_n)^\top \Phi_n^{-1} (\phi - \phi_n)] \right\},
 \end{aligned}$$

where $\Phi_n = (\sigma^{-2} \mathbf{U}^\top \mathbf{U} + \Phi_0^{-1})^{-1}$ and $\phi_n = \Phi_n(\sigma^{-2} \mathbf{U}^\top \boldsymbol{\mu} + \Phi_0^{-1} \phi_0)$. The

last expression is the kernel of a multivariate normal distribution with mean ϕ_n and variance matrix Φ_n .

5. Show that in the $AR(2)$ stationary process, $Y_t = \mu + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \epsilon_t$, where $\epsilon_t \sim N(0, \sigma^2)$, $\mathbb{E}[Y_t] = \frac{\mu}{1-\phi_1-\phi_2}$, and $Var[Y_t] = \frac{\sigma^2(1-\phi_2)}{1-\phi_2-\phi_1^2\phi_2-\phi_2^2+\phi_2^3}$.

Answer

A necessary condition to achieve second-order stationarity in the $AR(2)$ process is that roots of the polynomial $1 - \phi_1 L - \phi_2 L^2 = 0$ lie outside the unit circle. This is achieved if $|\phi_2| < 1$, $\phi_1 + \phi_2 < 1$ and $\phi_2 - \phi_1 < 1$.

Let's calculate the expected value:

$$\begin{aligned}\mathbb{E}[Y_t] &= \bar{\mu} \\ &= \mathbb{E}[\mu + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \epsilon_t] \\ &= \mu + \phi_1 \mathbb{E}[Y_{t-1}] + \phi_2 \mathbb{E}[Y_{t-2}] + \mathbb{E}[\epsilon_t] \\ &= \mu + \phi_1 \bar{\mu} + \phi_2 \bar{\mu},\end{aligned}$$

then, $\bar{\mu}(1 - \phi_1 - \phi_2) = \mu$, then, $\mathbb{E}[Y_t] = \bar{\mu} = \frac{\mu}{1-\phi_1-\phi_2}$.

See that $Y_t = \mu + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \epsilon_t$ implies that $Y_t = \bar{\mu}(1 - \phi_1 - \phi_2) + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \epsilon_t$, which means that $Y_t - \bar{\mu} = \phi_1(Y_{t-1} - \bar{\mu}) + \phi_2(Y_{t-2} - \bar{\mu}) + \epsilon_t$, and setting $z_t = Y_t - \bar{\mu}$, we have $z_t = \phi_1 z_{t-1} + \phi_2 z_{t-2} + \epsilon_t$ where $\mathbb{E}[z_t] = 0$. Then, we can calculate the variance of Y_t using z_t , given that both have same variance.

Let's define $\gamma_s = Cov(z_t, z_{t-s}) = \mathbb{E}[z_t z_{t-s}]$, then the Yule-Walker equations are:

$$\begin{aligned}\gamma_0 &= \phi_1 \gamma_1 + \phi_2 \gamma_2 + \sigma^2 \\ \gamma_1 &= \phi_1 \gamma_0 + \phi_2 \gamma_1 \\ \gamma_2 &= \phi_1 \gamma_1 + \phi_2 \gamma_0.\end{aligned}$$

Solving this system of equations for γ_0 , γ_1 and γ_2 , we find that $\gamma_0 = \frac{\sigma^2(1-\phi_2)}{1-\phi_2-\phi_1^2\phi_2-\phi_2^2+\phi_2^3}$.

6. Program a Hamiltonian Monte Carlo (HMC) taking into account the stationary restrictions on ϕ_1 and ϕ_2 , and ϵ_0 such that the acceptance rate is near 65%.

Answer

We know from the previous exercise that $|\phi_2| < 1$, $\phi_1 + \phi_2 < 1$ and $\phi_2 - \phi_1 < 1$ satisfied the stationary condition of the process. Thus, we should impose this restrictions in the HMC. The following code does this.

*R code. AR(2) model: Hamiltonian Monte Carlo
with stationary restrictions*

```

1 # Simulation AR(2)
2 rm(list = ls()); set.seed(010101); T <- 1000; K <- 4
3 mu <- 0.5; phi1 <- 0.5; phi2 <- 0.3; sig <- 0.5
4 Ey <- mu/(1-phi1-phi2); Sigy <- sig*((1-phi2)/(1-phi2-phi1
   ^2-phi2*phi1^2-phi2^2+phi2^3))^0.5
5 y <- rnorm(T, mean = Ey, sd = Sigy); e <- rnorm(T, mean = 0,
   sd = sig)
6 for(t in 3:T){
7   y[t] <- mu + phi1*y[t-1] + phi2*y[t-2] + e[t]
8 }
9 # Hyperparameters
10 d0 <- 0.01; a0 <- 0.01; mu0 <- 0; MU0 <- 1
11 phi0 <- c(0, 0); Phi0 <- diag(2)
12 # Log posterior multiply by -1 to use optim
13 LogPost <- function(theta, y){
14   mu <- theta[1]; phi1 <- theta[2]; phi2 <- theta[3]
15   tau <- theta[4]; sig2 <- exp(tau); logLik <- NULL
16   for(t in 3:T){
17     logLikt <- dnorm(y[t], mean = mu + phi1*y[t-1] + phi2*y[
18       t-2], sd = sig2^0.5, log = TRUE)
19     logLik <- c(logLik, logLikt)
20   }
21   logLik <- sum(logLik)
22   logPrior <- dnorm(mu, mean = mu0, sd = MU0^0.5, log = TRUE
23     ) + dnorm(phi1, mean = phi0[1], sd = Phi0[1,1]^0.5, log
24     = TRUE) + dnorm(phi2, mean = phi0[2], sd = Phi0
25     [2,2]^0.5, log = TRUE) + invgamma::dinvgamma(sig2, shape
26     = a0/2, rate = d0/2, log = TRUE)
27   logPosterior <- logLik + logPrior + tau
28   return(-logPosterior) # Multiply by -1 to minimize using
29     optim
30 }
31 theta0 <- c(mean(y), 0, 0, var(y))
32 Opt <- optim(theta0, LogPost, y = y, hessian = TRUE)
33 theta0 <- Opt$par; VarPost <- solve(Opt$hessian)
34 # Gradient log posterior
35 GradientTheta <- function(theta, y){
36   mu <- theta[1]; phi1 <- theta[2]; phi2 <- theta[3]
37   tau <- theta[4]; sig2 <- exp(tau); SumLik <- matrix(0, 3,
38     1)
39   SumLik2 <- NULL
40   for(t in 3:T){
41     xt <- matrix(c(1, y[t-1], y[t-2]), 3, 1)
42     SumLikt <- (y[t] - (mu + phi1*y[t-1] + phi2*y[t-2]))*xt
43     SumLik2t <- (y[t] - (mu + phi1*y[t-1] + phi2*y[t-2]))^2
44     SumLik <- rowSums(cbind(SumLik, SumLikt))
45     SumLik2 <- sum(SumLik2, SumLik2t)
46   }
47   Grad_mu <- SumLik[1]/sig2 - (1/MU0)*(mu - mu0)
48   Grad_phi1 <- SumLik[2]/exp(tau) - 1/Phi0[1,1]*(phi1 - phi0
49     [1])
50   Grad_phi2 <- SumLik[3]/exp(tau) - 1/Phi0[2,2]*(phi2 - phi0
51     [2])
52   Grad_tau <- -(T-2)/2 + SumLik2/(2*exp(tau)) - (a0/2 + 1) +
53     d0/(2*exp(tau)) + 1
54   Grad <- c(Grad_mu, Grad_phi1, Grad_phi2, Grad_tau)
55   return(Grad)
56 }

```

***R code. AR(2) model: Hamiltonian Monte Carlo
with stationary restrictions***

```

1 StatRest <- function(phi1, phi2){
2   if(abs(phi2) < 1 & phi1 + phi2 < 1 & phi2 - phi1 < 1){
3     check <- 1
4   }else{
5     check <- 0
6   }
7   return(check)
8 }
9 HMC <- function(theta, y, epsilon, M){
10  L <- ceiling(1/epsilon)
11  Minv <- solve(M)
12  thetat <- theta
13  K <- length(thetat)
14  mom <- t(mvtnorm::rmvnorm(1, rep(0, K), M))
15  logPost_Mom_t <- -LogPost(thetat, y) + mvtnorm::dmvnorm(t
16    (mom), rep(0, K), M, log = TRUE)
17  for(l in 1:L){
18    if(l == L){
19      mom <- mom + 0.5*epsilon*GradientTheta(theta, y)
20      theta <- theta + epsilon*Minv%%mom
21    }else{
22      mom <- mom + epsilon*GradientTheta(theta, y)
23      theta <- theta + epsilon*Minv%%mom
24    }
25  }
26  logPost_Mom_star <- -LogPost(theta, y) + mvtnorm::dmvnorm
27    (t(mom), rep(0, K), M, log = TRUE)
28  alpha <- min(1, exp(logPost_Mom_star-logPost_Mom_t))
29  u <- runif(1)
30  if(u <= alpha & StatRest(phi1 = theta[2], phi2 = theta[3])
31    == 1){
32    thetaNew <- c(theta)
33  }else{
34    thetaNew <- thetat
35  }
36  rest <- list(theta = thetaNew, Prob = alpha)
37  return(rest)
38 }

```

*R code. AR(2) model: Hamiltonian Monte Carlo
with stationary restrictions*

```

1 # Posterior draws
2 S <- 1000; burnin <- 1000; thin <- 2; tot <- S + burnin
3 thetaPost <- matrix(NA, tot, K)
4 ProbAccept <- rep(NA, tot)
5 # theta0 <- theta0
6 theta0 <- c(mean(y), 0, 0, exp(var(y)))
7 M <- solve(VarPost); epsilon0 <- 0.4
8 pb <- txtProgressBar(min = 0, max = tot, style = 3)
9 for(s in 1:tot){
10   epsilon <- runif(1, 0, 2*epsilon0)
11   L <- ceiling(1/epsilon)
12   HMCs <- HMC(theta = theta0, y, epsilon, M)
13   theta0 <- HMCs$theta
14   thetaPost[s,] <- HMCs$theta
15   ProbAccept[s] <- HMCs$Prob
16   setTxtProgressBar(pb, s)
17 }
18 close(pb)
19 keep <- seq((burnin+1), tot, thin)
20 thetaF <- coda::mcmc(thetaPost[keep,])
21 summary(thetaF)
22 plot(thetaF)
23 summary(exp(thetaF[,K]))
24 plot(exp(thetaF[,K]))
25 ProbAcceptF <- coda::mcmc(ProbAccept[keep])
26 summary(ProbAcceptF)

```

7. Stochastic volatility model

- Program a sequential importance sampling (SIS) from scratch in the vanilla stochastic volatility model setting $\mu = -10$, $\phi = 0.95$, $\sigma = 0.3$ and $T = 250$. Check what happen with its performance.
- Modify the sequential Monte Carlo (SMC) to perform multinomial resampling when the effective sample size is lower than 50% the number of particles.

Answer

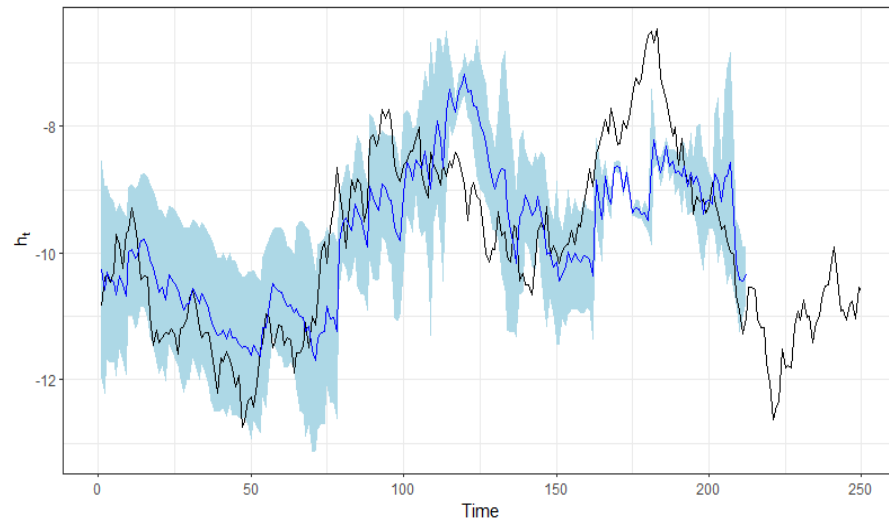
We observe that, initially, sequential importance sampling performs relatively well. However, particle degeneracy occurs around ($t = 50$), causing the algorithm's performance to deteriorate rapidly (see Figure 8.3). We can see in Figure 8.4 how the effective sample size (ESS) of the particles decreases rapidly.

R code. Stochastic volatility model: Sequential importance sampling

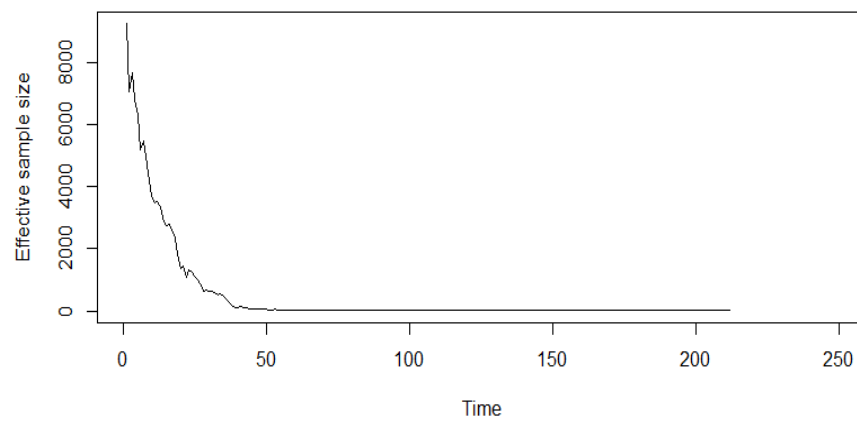
```

1 rm(list = ls()); set.seed(010101)
2 T <- 250; K <- 2; mu <- -10; phi <- 0.95; sigma <- 0.3
3 h <- numeric(T); y <- numeric(T)
4 h[1] <- rnorm(1, mu, sigma / sqrt(1 - phi^2))
5 y[1] <- rnorm(1, 0, exp(h[1] / 2))
6 for (t in 2:T) {
7   h[t] <- mu + phi*(h[t-1]-mu) + rnorm(1, 0, sigma)
8   y[t] <- rnorm(1, 0, sd = exp(0.5*h[t]))
9 }
10 N <- 10000
11 log_Weights <- matrix(NA, N, T) # Log weights
12 Weights <- matrix(NA, N, T) # Weights
13 WeightsST <- matrix(NA, N, T) # Normalized weights
14 particles <- matrix(NA, N, T) # Particles
15 logalphas <- matrix(NA, N, T) # Incremental importance
    weights
16 ESS <- rep(NA, T) # Effective sample size
17 particles[, 1] <- rnorm(N, mu, sigma / sqrt(1 - phi^2))
18 log_Weights[, 1] <- dnorm(y[1], 0, sd = exp(0.5*particles
    [,1]), log = TRUE)
19 Weights[, 1] <- exp(log_Weights[, 1])
20 WeightsST[, 1] <- Weights[, 1] / sum(Weights[, 1])
21 ESS[1] <- (sum(WeightsST[, 1]^2))^(1)
22 for (t in 2:T) {
23   particles[, t] <- rnorm(N, mu + phi*(particles[, t - 1] -
    mu), sigma) # Sample from proposal
24   logalphas[, t] <- dnorm(y[t], 0, sd = exp(0.5*particles[,t
    ]), log = TRUE)
25   log_Weights[, t] <- log_Weights[, t - 1] + logalphas[, t]
26   Weights[, t] <- exp(log_Weights[, t])
27   WeightsST[, t] <- Weights[, t] / sum(Weights[, t])
28   ESS[t] <- (sum(WeightsST[, t]^2))^(1)
29 }
30 FilterDist <- colSums(particles * WeightsST)
31 SDFilterDist <- (colSums(particles^2 * WeightsST) -
    FilterDist^2)^0.5
32 MargLik <- colMeans(Weights)
33 library(dplyr); library(ggplot2); require(latex2exp)
34 ggplot2::theme_set(theme_bw())
35 df <- tibble(t = seq(1, T), mean = FilterDist, lower =
    FilterDist - 1.96*SDFilterDist, upper = FilterDist +
    1.96*SDFilterDist, x_true = h)
36 plot_filtering_estimates <- function(df) {
37   p <- ggplot(data = df, aes(x = t)) + geom_ribbon(aes(ymin
    = lower, ymax = upper), alpha = 1, fill = "lightblue") +
    geom_line(aes(y = x_true), colour = "black", alpha = 1,
    linewidth = 0.5) + geom_line(aes(y = mean), colour = "
    blue", linewidth = 0.5) +
38   ylab(TeX("$h_{t}$")) + xlab("Time")
39   print(p)
40 }
41 plot_filtering_estimates(df)
42 plot(ESS, type = "l", ylab = "Effective sample size", xlab =
    "Time")

```

**FIGURE 8.3**

Stochastic volatility model: Sequential importance sampling (SIS).

**FIGURE 8.4**

Stochastic volatility model: Effective sample size (ESS) of the particles.

This code shows how to program the SMC algorithm performing multinomial resampling when the effective sample size is lower than 50% the initial number of particles. It seems that in this example is better to perform re-

sampling every time period as the mean recursion (blue and purple lines) are not good following in state (black line). However, the bands corresponding to plus/minus two standard deviations (light blue shading) encompass the state vector; although, it is too wide.

R code. Stochastic volatility model: Sequential Monte Carlo

```

1 rm(list = ls()); set.seed(010101)
2 T <- 1250; mu <- -10; phi <- 0.95; sigma <- 0.3
3 h <- numeric(T); y <- numeric(T)
4 h[1] <- rnorm(1, mu, sigma / sqrt(1 - phi^2))
5 y[1] <- rnorm(1, 0, exp(h[1] / 2))
6 for (t in 2:T) {
7   h[t] <- mu + phi*(h[t-1]-mu) + rnorm(1, 0, sigma)
8   y[t] <- rnorm(1, 0, sd = exp(0.5*h[t]))
9 }
10 N <- 10000
11 log_Weights <- matrix(NA, N, T) # Log weights
12 Weights <- matrix(NA, N, T) # Weights
13 WeightsST <- matrix(NA, N, T) # Normalized weights
14 WeightsSTT <- matrix(1/N, N, T) # Normalized weights bar
15 particles <- matrix(NA, N, T) # Particles
16 particlesT <- matrix(NA, N, T) # Particles bar
17 logalphas <- matrix(NA, N, T) # Incremental import. weig.
18 ESS <- rep(NA, T) # Effective sample size
19 cutoff <- 0.5
20 particles[, 1] <- rnorm(N, mu, sigma / sqrt(1 - phi^2))
21 log_Weights[, 1] <- dnorm(y[1], 0, sd = exp(0.5*particles
  [,1]), log = TRUE)
22 Weights[, 1] <- exp(log_Weights[, 1])
23 WeightsST[, 1] <- Weights[, 1] / sum(Weights[, 1])
24 ESS[1] <- (sum(WeightsST[, 1]^2))^(-1)
25 ind <- sample(1:N, size = N, replace = TRUE, prob =
  WeightsST[, 1]) # Resample
26 particles[, 1] <- particles[, 1] # Resampled particles
27 particlesT[, 1] <- particles[ind, 1] # Resampled particles
28 WeightsST[, 1] <- rep(1/N, N) # Resampled weights
29 pb <- txtProgressBar(min = 0, max = T, style = 3)
30 for (t in 2:T) {
31   particles[, t] <- rnorm(N, mu + phi*(particles[, t - 1] -
    mu), sigma) # Sample from proposal
32   logalphas[, t] <- dnorm(y[t], 0, sd = exp(0.5*particles[,t
    ]), log = TRUE)
33   Weights[, t] <- exp(logalphas[, t])
34   WeightsST[, t] <- Weights[, t] / sum(Weights[, t])
35   ESS[t] <- (sum(WeightsST[, t]^2))^(-1)
36   particlesT[, t] <- particles[, t]
37   if(ESS[t] < N*cutoff){
38     ind <- sample(1:N, size = N, replace = TRUE, prob =
      WeightsST[, t])
39     particles[, 1:t] <- particles[ind, 1:t]
40     WeightsST[, t] <- 1/N
41   }else{if(t == T & ESS[t] < N*cutoff)
42     ind <- sample(1:N, size = N, replace = TRUE, prob =
      WeightsST[, t])
43     particlesT[, 1:t] <- particles[ind, 1:t]
44   }
45   setTxtProgressBar(pb, t)
46 }
47 close(pb)

```

R code. Stochastic volatility model: Sequential Monte Carlo

```

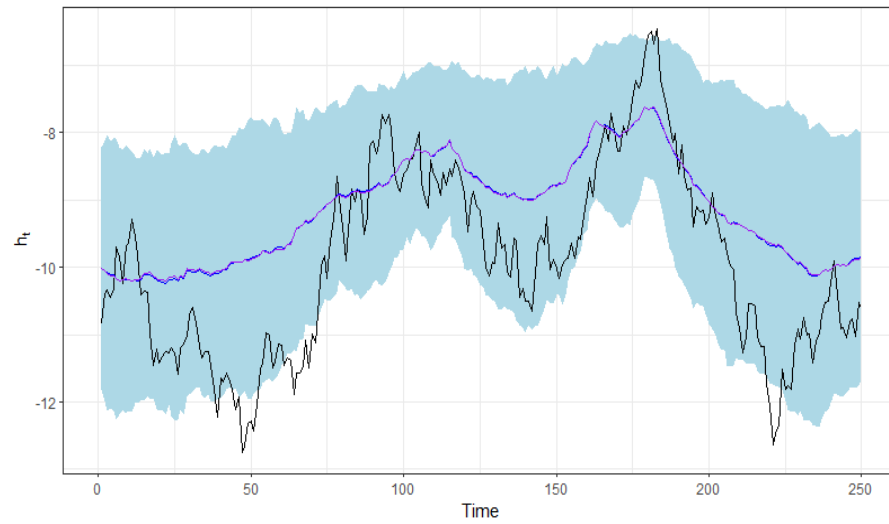
1 FilterDist <- colSums(particles * WeightsST)
2 SDFilterDist <- (colSums(particles^2 * WeightsST) -
  FilterDist^2)^0.5
3 FilterDistT <- colSums(particlesT * WeightsSTT)
4 SDFilterDistT <- (colSums(particlesT^2 * WeightsSTT) -
  FilterDistT^2)^0.5
5 MargLik <- colMeans(Weights)
6 plot(MargLik, type = "l")
7 plot(ESS, type = "l")
8 library(dplyr)
9 library(ggplot2)
10 require(latex2exp)
11 ggplot2::theme_set(theme_bw())
12 Tfig <- 250
13 keepFig <- 1:Tfig
14 df <- tibble(t = keepFig,
15 mean = FilterDist[keepFig],
16 lower = FilterDist[keepFig] - 2*SDFilterDist[keepFig],
17 upper = FilterDist[keepFig] + 2*SDFilterDist[keepFig],
18 meanT = FilterDistT[keepFig],
19 lowerT = FilterDistT[keepFig] - 2*SDFilterDistT[keepFig],
20 upperT = FilterDistT[keepFig] + 2*SDFilterDistT[keepFig],
21 x_true = h[keepFig])
22 plot_filtering_estimates <- function(df) {
23   p <- ggplot(data = df, aes(x = t)) +
24     geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 1,
25     fill = "lightblue") +
26     geom_line(aes(y = x_true), colour = "black", alpha = 1,
27     linewidth = 0.5) +
28     geom_line(aes(y = mean), colour = "blue", linewidth = 0.5)
29     +
30     geom_line(aes(y = meanT), colour = "purple", linewidth =
31     0.5) +
32     ylab(TeX("$h_{t}$")) + xlab("Time")
33   print(p)
34 }
35 plot_filtering_estimates(df)

```

8. Estimate the vanilla stochastic volatility model using the dataset *17Ex-cRate.csv* provided by [41] of the exchange rate log daily returns from USD/EUR, USD/GBP and GBP/EUR one year before and after the WHO declared the COVID-19 pandemic on 11 March 2020.

Answer

The following code shows the results of modeling the exchange rate log

**FIGURE 8.5**

Stochastic volatility model: Sequential Monte Carlo (SMC).

daily returns from USD/EUR using a stochastic volatility model. Similar code can be used to estimate the other two models.

R code. Stochastic volatility model: Exchange rate log daily returns from USD/EUR

```

1 rm(list = ls()); set.seed(010101)
2 DataExcRate <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/17ExcRate.csv",
  sep = ",", header = TRUE, quote = "")
3 attach(DataExcRate)
4 MCMC <- 10000; burnin <- 10000; thin <- 5
5 y <- USDEUR - mean(USDEUR)
6 plot(y, type = "l")
7 res <- stochvol::svsample(y, draws = MCMC, burnin = burnin,
  thin = thin, priormu = c(0, 100), priorsigma = c(1),
  priorphi = c(5, 1.5), priorbeta = c(0, 10000))
8 summary(res[["para"]][[1]][, -c(4,5)])
9 Iterations = 10005:20000
10 Thinning interval = 5
11 Number of chains = 1
12 Sample size per chain = 2000
13 1. Empirical mean and standard deviation for each variable,
14 plus standard error of the mean:
15 Mean      SD Naive SE Time-series SE
16 mu      -11.2886 0.28916 0.0064657      0.006466
17 phi       0.9618 0.02202 0.0004923      0.001421
18 sigma    0.1664 0.04501 0.0010064      0.003722
19 2. Quantiles for each variable:
20      2.5%      25%      50%      75%      97.5%
21 mu      -11.81067 -11.4287 -11.2902 -11.1448 -10.7699
22 phi       0.90719  0.9512  0.9657  0.9774  0.9913
23 sigma    0.09615  0.1327  0.1607  0.1926  0.2666
24 plot(res)
25 ht <- res[["latent"]][[1]]
26 library(dplyr)
27 library(ggplot2)
28 require(latex2exp)
29 ggplot2::theme_set(theme_bw())
30 x_means <- colMeans(ht)
31 x_quantiles <- apply(ht, 2, function(x) quantile(x, probs =
  c(0.025, 0.975)))
32 df <- tibble(t = 1:length(y),
33 mean = x_means,
34 lower = x_quantiles[1, ],
35 upper = x_quantiles[2, ])
36 plot_filtering_estimates <- function(df) {
37   p <- ggplot(data = df, aes(x = t)) +
38     geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 1,
39     fill = "lightblue") +
40     geom_line(aes(y = mean), colour = "blue", linewidth = 0.5)
41   +
42   ylab(TeX("$h_{t}$")) + xlab("Time")
43   print(p)
44 }
45 plot_filtering_estimates(df)

```

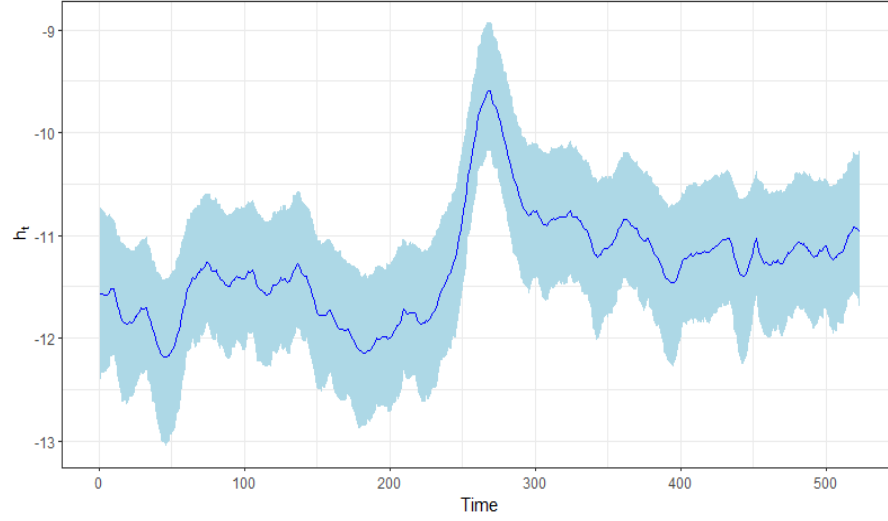


FIGURE 8.6

Stochastic volatility model: Exchange rate log daily returns from USD/EUR.

9. Simulate the VAR(1) process

$$\begin{bmatrix} y_{1t} \\ y_{2t} \\ y_{3t} \end{bmatrix} = \begin{bmatrix} 2.8 \\ 2.2 \\ 1.3 \end{bmatrix} + \begin{bmatrix} 0.5 & 0 & 0 \\ 0.1 & 0.1 & 0.3 \\ 0 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} y_{1t-1} \\ y_{2t-1} \\ y_{3t-1} \end{bmatrix} + \begin{bmatrix} \mu_{1t} \\ \mu_{2t} \\ \mu_{3t} \end{bmatrix},$$

$$\text{where } \Sigma = \begin{bmatrix} 2.25 & 0 & 0 \\ 0 & 1 & 0.5 \\ 0 & 0.5 & 0.74 \end{bmatrix}.$$

- Use vague independent priors setting $\beta_0 = \mathbf{0}$, $B_0 = 100\mathbf{I}$, $V_0 = 5\mathbf{I}$ and $\alpha_0 = 3$, and estimate a VAR(1) model using the *rsurGibbs* function from the package *bayesm*. Then, program from scratch algorithms to perform inference of the *forecast error* and *ortogonalized* impulse response functions, and compare with the population impulse response functions, that is, using the population parameters.
- Using the previous setting perform inference of the *forecast error* and the *ortogonalized* impulse responses using the package *bvartools*.

Answer

Using the recursion $\Phi_s = \sum_{l=1}^s \Phi_{s-l} A_l$, $A_l = \mathbf{0}$, $l > p$ and $s = 1, 2, \dots$, we get $\Phi_s = A_1^s$. In addition, we know that $\Theta_s = \Phi_s P$. Then, we can get these impulse response functions.

The following code shows how to perform from scratch the impulse response

functions. Figure 8.7 shows the *forecast error impulse response* of y_{2t} to shocks in μ_{3t} . We see that the 95% (blue shadow) and 90% (light blue shadow) credible intervals of the impulse response functions embrace the true impulse response (black line); although the mean impulse response (blue line) overestimates the true impulse. In addition, we can see that the results from the *bvartools* package have the same patterns compared to the ones that we get programming the algorithm from scratch. Remember that the *bvartools* package uses the inverse Wishart distribution as prior for Σ , where the hyperparameters are the degrees of freedom of the error term, and the prior error variance of endogenous variables.

R code. Vector autoregressive model: Simulation

```

1 rm(list = ls()); set.seed(010101)
2 T <- 300; M <- 3
3 SIGMA <- matrix(c(2.25, 0, 0, 0, 1, 0.5, 0, 0.5, 0.74), M, M
4 )
5 U <- MASS::mvrnorm(T, rep(0, M), SIGMA)
6 A <- matrix(c(0.5, 0.1, 0, 0, 0.1, 0.2, 0, 0.3, 0.3), M, M )
7 v <- c(1.4, 1.9, 1.1); Y <- matrix(NA, T, M)
8 Y[1, ] <- solve(diag(M)-A)%*v + U[1,]
9 for(t in 2:T){
10   Y[t, ] <- v + A%*Y[t-1,] + U[t,]
11 }
12 plot(Y[,1], type = "l"); plot(Y[,2], type = "l"); plot(Y
13   [,3], type = "l")
14 y1 <- Y[-1,1]; y2 <- Y[-1,2]; y3 <- Y[-1,3]
15 X1 <- cbind(1, lag(Y)); X1 <- X1[-1,]
16 X2 <- cbind(1, lag(Y)); X2 <- X2[-1,]
17 X3 <- cbind(1, lag(Y)); X3 <- X3[-1,]
18 regdata <- NULL
19 regdata[[1]] <- list(y = y1, X = X1); regdata[[2]] <- list(y
20   = y2, X = X2); regdata[[3]] <- list(y = y3, X = X3)
21 M <- length(regdata); K1 <- dim(X1)[2]; K2 <- dim(X2)[2]; K3
22   <- dim(X3)[2]
23 K <- K1 + K2 + K3
24 # Hyperparameters
25 b0 <- rep(0, K); c0 <- 100
26 B0 <- c0*diag(K); V <- 5*diag(M); a0 <- M
27 Prior <- list(betabar = b0, A = solve(B0), nu = a0, V = V)
28 #Posterior draws
29 MCMC <- 10000; thin <- 5; burnin <- 1000; tot <- MCMC +
30   burnin
31 Mcmc <- list(R = tot, keep = thin)
32 PosteriorDraws <- bayesm::rsurGibbs(Data = list(regdata =
33   regdata), Mcmc = Mcmc, Prior = Prior)
34 keep <- seq(round(burnin/thin)+1, round(tot/thin))
35 Bs <- PosteriorDraws[["betadraw"]][keep,]
36 SIGMAs <- PosteriorDraws[["Sigmadraw"]][keep,]
37 S <- dim(Bs)[1]
38 As <- array(NA, c(M, M, S))
39 for(s in 1:S){
40   As[,s] <- matrix(Bs[s, -c(1, 5, 9)], M, M, byrow = TRUE)
41 }
42 # Impulse response functions
43 H <- 10; Ptrue <- t(chol(SIGMA))
44 Phistrue <- array(NA, c(M, M, H))
45 Phistrue[,1] <- diag(M); PhistrueAccum <- Phistrue
46 PhistrueAccum[,1] <- diag(M)
47 Thetatrue <- Phistrue; Thetatrue[,1] <- Ptrue
48 ThetatrueAccum <- Thetatrue; ThetatrueAccum[,1] <-
49   Ptrue
50 for(h in 2:H){
51   Phistrue[,h] <- Phistrue[,h-1]%*A
52   PhistrueAccum[,h] <- PhistrueAccum[,h-1] + Phistrue[,h]
53   Thetatrue[,h] <- Phistrue[,h]%*Ptrue
54   PhistrueAccum[,h] <- ThetatrueAccum[,h-1] + Thetatrue
55     [,h]
56 }
57 P <- array(NA, c(M, M, S))

```

R code. Vector autoregressive model: Simulation

```

1 for(s in 1:S){
2   P[,s] <- t(chol(matrix(SIGMAs[s,], M, M)))
3 }
4 Phis <- list(); PhisAcum <- list()
5 Thetas <- list(); ThetasAcum <- list()
6 for(h in 0:H){
7   if(h == 0){
8     Phis[[h+1]] <- array(diag(M), c(M,M,S))
9     Thetas <- Phis
10    for(s in 1:S){
11      Thetas[[h+1]][,s] <- Phis[[h+1]][,s]*%P[,s]
12    }
13    PhisAcum[[h+1]] <- Phis[[h+1]]
14    ThetasAcum[[h+1]] <- Thetas[[h+1]]
15  }else{
16    Phis[[h+1]] <- array(diag(M), c(M,M,S))
17    Thetas[[h+1]] <- Phis[[h+1]]
18    for(s in 1:S){
19      Phis[[h+1]][,s] <- Phis[[h]][,s]*%As[,s]
20      Thetas[[h+1]][,s] <- Phis[[h+1]][,s]*%P[,s]
21    }
22    PhisAcum[[h+1]] <- PhisAcum[[h]] + Phis[[h+1]]
23    ThetasAcum[[h+1]] <- ThetasAcum[[h]] + Thetas[[h+1]]
24  }
25 }
26 library(dplyr); library(ggplot2); require(latex2exp)
27 ggplot2::theme_set(theme_bw())
28 plot_filtering_estimates <- function(df) {
29   p <- ggplot(data = df, aes(x = t)) +
30     geom_ribbon(aes(ymin = lower1, ymax = upper1), alpha = 1,
31               fill = "blue") +
32     geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 1,
33               fill = "lightblue") +
34     geom_line(aes(y = x_true), colour = "black", alpha = 1,
35              linewidth = 0.5) +
36     geom_line(aes(y = mean), colour = "blue", linewidth = 0.5)
37   +
38   ylab("Impulse response") + xlab("Time") + xlim(0,H-1)
39   print(p)
40 }

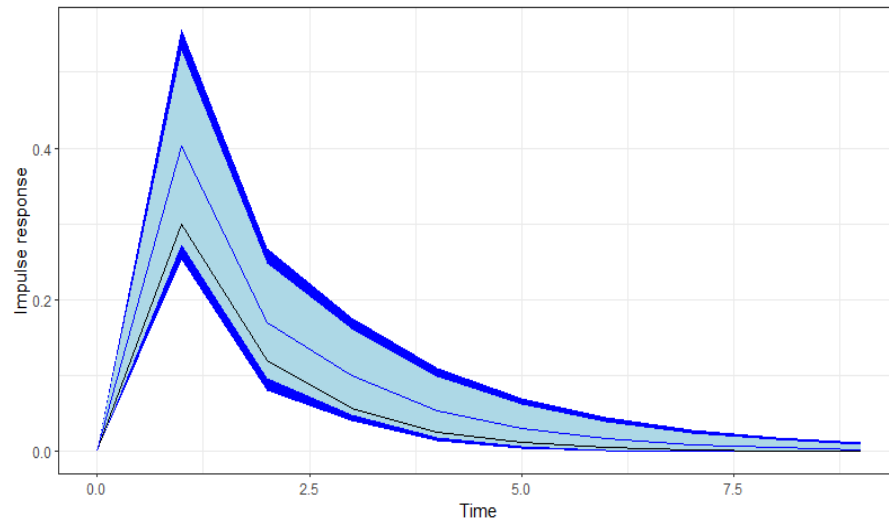
```

R code. Vector autoregressive model: Simulation

```

1 IR <- function(m, j, Accumulated = "FALSE", Type = "Ordinary
  "){
2   if(Type == "Cholesky"){
3     IRtrue <- Thetastrue[m,j,]
4     if(Accumulated == FALSE){
5       RES <- Thetas; IRtrue <- IRtrue
6     }else{
7       RES <- ThetasAcum; IRtrue <- cumsum(IRtrue)
8     }
9     Mean <- sapply(1:H, function(h){mean(RES[[h]][m,j,])})
10    LimInf1 <- sapply(1:H, function(h){quantile(RES[[h]][m,j
11    ,], 0.025)})
12    LimSup1 <- sapply(1:H, function(h){quantile(RES[[h]][m,j
13    ,], 0.975)})
14    LimInf <- sapply(1:H, function(h){quantile(RES[[h]][m,j
15    ,], 0.05)})
16    LimSup <- sapply(1:H, function(h){quantile(RES[[h]][m,j
17    ,], 0.95)})
18  }else{
19    IRtrue <- Phistrue[m,j,]
20    if(Accumulated == FALSE){
21      RES <- Phis; IRtrue <- IRtrue
22    }else{
23      RES <- PhisAcum; IRtrue <- cumsum(IRtrue)
24    }
25    Mean <- sapply(1:H, function(h){mean(RES[[h]][m,j,])})
26    LimInf1 <- sapply(1:H, function(h){quantile(RES[[h]][m,j
27    ,], 0.025)})
28    LimSup1 <- sapply(1:H, function(h){quantile(RES[[h]][m,j
29    ,], 0.975)})
30    LimInf <- sapply(1:H, function(h){quantile(RES[[h]][m,j
31    ,], 0.05)})
32    LimSup <- sapply(1:H, function(h){quantile(RES[[h]][m,j
33    ,], 0.95)})
34  }
35  df <- tibble(t = 0:(H-1), mean = Mean, lower1 = LimInf1,
36    upper1 = LimSup1, lower = LimInf, upper = LimSup, x_true
37    = IRtrue)
38  Fig <- plot_filtering_estimates(df)
39  return(Fig)
40 }
41 m <- 2; j <- 3
42 IR(m,j, Accumulated = "FALSE", Type = "Cholesky")
43 Ynew <- ts(Y)
44 model <- bvartools::gen_var(Ynew, p = 1, deterministic = "
45   const", iterations = MCMC, burnin = burnin) # Model
46 model <- bvartools::add_priors(model, coef = list(v_i = b0
47   [1], v_i_det = b0[1]), sigma = list(df = a0, scale = V
48   [1,1]/a0)) # Prior
49 object <- bvartools::draw_posterior(model) # Pposterior
50 ir <- bvartools::irf.bvar(object, impulse = "Series 2",
51   response = "Series 3") # Calculate IR
52 plot(ir) # Plot IR

```

**FIGURE 8.7**

Forecast error impulse response: Response in y_{2t} to shock in μ_{3t} .

9

Longitudinal/Panel data models

Solutions of Exercises

1. Show that the posterior distribution of $\beta|\sigma^2, \mathbf{D}$ is $N(\beta_n, \mathbf{B}_n)$, where $\mathbf{B}_n = (\mathbf{B}_0^{-1} + \sum_{i=1}^N \mathbf{X}_i^\top \mathbf{V}_i^{-1} \mathbf{X}_i)^{-1}$ and $\beta_n = \mathbf{B}_n(\mathbf{B}_0^{-1}\beta_0 + \sum_{i=1}^N \mathbf{X}_i^\top \mathbf{V}_i^{-1} \mathbf{y}_i)$.

Answer

$$\begin{aligned}
 \pi(\beta|\sigma^2, \mathbf{D}, \mathbf{y}, \mathbf{X}, \mathbf{W}) &\propto \exp \left\{ -\frac{1}{2} \sum_{i=1}^N (\mathbf{y}_i - \mathbf{X}_i \beta)^\top \mathbf{V}_i^{-1} (\mathbf{y}_i - \mathbf{X}_i \beta) \right\} \\
 &\quad \times \exp \left\{ -\frac{1}{2} (\beta - \beta_0)^\top \mathbf{B}_0^{-1} (\beta - \beta_0) \right\} \\
 &\propto \exp \left\{ -\frac{1}{2} \left(-2\beta^\top \left(\sum_{i=1}^N \mathbf{X}_i^\top \mathbf{V}_i^{-1} \mathbf{y}_i + \mathbf{B}_0^{-1} \beta_0 \right) + \beta^\top \left(\sum_{i=1}^N \mathbf{X}_i^\top \mathbf{V}_i^{-1} \mathbf{X}_i + \mathbf{B}_0^{-1} \right) \beta \right) \right\} \\
 &= \exp \left\{ -\frac{1}{2} (-2\beta^\top \mathbf{B}_n^{-1} \mathbf{B}_n \left(\sum_{i=1}^N \mathbf{X}_i^\top \mathbf{V}_i^{-1} \mathbf{y}_i + \mathbf{B}_0^{-1} \beta_0 \right) + \beta^\top \mathbf{B}_n^{-1} \beta) \right\} \\
 &= \exp \left\{ -\frac{1}{2} (-2\beta^\top \mathbf{B}_n^{-1} \beta_n + \beta^\top \mathbf{B}_n^{-1} \beta) \right\}.
 \end{aligned}$$

We can complete the square in this expression by adding and subtracting $\beta_n^\top \mathbf{B}_n^{-1} \beta_n$. Thus,

$$\begin{aligned}
 \pi(\beta|\sigma^2, \mathbf{D}, \mathbf{y}, \mathbf{X}, \mathbf{W}) &\propto \exp \left\{ -\frac{1}{2} (-2\beta^\top \mathbf{B}_n^{-1} \beta_n + \beta^\top \mathbf{B}_n^{-1} \beta + \beta_n^\top \mathbf{B}_n^{-1} \beta_n - \beta_n^\top \mathbf{B}_n^{-1} \beta_n) \right\} \\
 &\propto \exp \left\{ -\frac{1}{2} (\beta - \beta_n)^\top \mathbf{B}_n^{-1} (\beta - \beta_n) \right\}.
 \end{aligned}$$

This is the kernel of a multivariate random variable with mean β_n and variance matrix $\mathbf{B}_n$.

2. **The relation between productivity and public investment example continues**

- Perform inference of this example using our GUI.

- Program from scratch a Gibbs sampling algorithm to perform this application. Set $\beta_0 = \mathbf{0}_5$, $B_0 = I_5$, $\alpha_0 = \delta_0 = 0.001$, $d_0 = 5$ and $D_0 = I_1$.
- Perform inference in this example assuming that $\mu_{it}|\tau_{it} \sim N(0, \sigma^2/\tau_{it})$ and $\tau_{it} \sim G(v/2, v/2)$ setting $v = 5$.

Answer

R code. The relationship between productivity and public investment, programming from scratch the Gibbs sampler

```

1 rm(list = ls())
2 set.seed(12345)
3 DataGSP <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/8PublicCap.
  csv", sep = ",", header = TRUE, quote = "")
4 attach(DataGSP)
5 N <- length(unique(id))
6 y <- log(gsp)
7 NT <- length(y)
8 X <- cbind(1, log(pcap), log(pc), log(emp), unemp)
9 K1 <- dim(X)[2]
10 W <- matrix(rep(1, NT), NT, 1)
11 K2 <- dim(W)[2]
12 mcmc <- 10000; burnin <- 5000; thin <- 1; tot <- mcmc +
  burnin
13 b0 <- rep(0, K1); B0 <- diag(K1); B0i <- solve(B0)
14 r0 <- 5; R0 <- diag(K2); a0 <- 0.001; d0 <- 0.001
15 PostBeta <- function(sig2, D){
16   XVX <- matrix(0, K1, K1)
17   XVy <- matrix(0, K1, 1)
18   for(i in 1:N){
19     ids <- which(id == i)
20     Ti <- length(ids)
21     Wi <- W[ids, ]
22     Vi <- sig2*diag(Ti) + Wi*%D%*t(Wi)
23     ViInv <- solve(Vi)
24     Xi <- X[ids, ]
25     XVXi <- t(Xi)*%ViInv%*Xi
26     XVX <- XVX + XVXi
27     yi <- y[ids]
28     XVyi <- t(Xi)*%ViInv%*yi
29     XVy <- XVy + XVyi
30   }
31   Bn <- solve(B0i + XVX)
32   bn <- Bn*%(B0i*%b0 + XVy)
33   Beta <- MASS::mvrnorm(1, bn, Bn)
34   return(Beta)
35 }

```

R code. The relationship between productivity and public investment, programming from scratch the Gibbs sampler

```

1 Postb <- function(Beta, sig2, D){
2   Di <- solve(D)
3   bis <- matrix(0, N, K2)
4   for(i in 1:N){
5     ids <- which(id == i)
6     Wi <- W[ids, ]
7     Xi <- X[ids, ]
8     yi <- y[ids]
9     Wtei <- sig2^(-1)*t(Wi)%*(yi - Xi%*Beta)
10    Bni <- solve(sig2^(-1)*t(Wi)%*Wi + Di)
11    bni <- Bni%*Wtei
12    bi <- MASS::mvrnorm(1, bni, Bni)
13    bis[i, ] <- bi
14  }
15  return(as.matrix(bis))
16 }
17 PostSig2 <- function(Beta, bs){
18   an <- a0 + 0.5*NT
19   ete <- 0
20   for(i in 1:N){
21     ids <- which(id == i)
22     Xi <- X[ids, ]
23     yi <- y[ids]
24     Wi <- W[ids, ]
25     ei <- yi - Xi%*Beta - Wi*bs[i, ]
26     etei <- t(ei)%*ei
27     ete <- ete + etei
28   }
29   dn <- d0 + 0.5*ete
30   sig2 <- MCMCpack::rinvgamma(1, shape = an, scale = dn)
31   return(sig2)
32 }
33 PostD <- function(bs){
34   rn <- r0 + N
35   btb <- matrix(0, K2, K2)
36   for(i in 1:N){
37     bsi <- bs[i, ]
38     btbi <- bsi%*t(bsi)
39     btb <- btb + btbi
40   }
41   Rn <- d0*R0 + btb
42   Sigma <- MCMCpack::riwish(v = rn, S = Rn)
43   return(Sigma)
44 }

```

R code. The relationship between productivity and public investment, programming from scratch the Gibbs sampler

```

1 PostBetas <- matrix(0, tot, K1); PostDs <- matrix(0, tot, K2
  *(K2+1)/2)
2 PostSig2s <- rep(0, tot); Postbs <- array(0, c(N, K2, tot))
3 RegLS <- lm(y ~ X - 1); SumLS <- summary(RegLS)
4 Beta <- SumLS[["coefficients"]][,1]
5 sig2 <- SumLS[["sigma"]]^2; D <- diag(K2)
6 pb <- txtProgressBar(min = 0, max = tot, style = 3)
7 for(s in 1:tot){
8   bs <- Postb(Beta = Beta, sig2 = sig2, D = D)
9   D <- PostD(bs = bs)
10  Beta <- PostBeta(sig2 = sig2, D = D)
11  # Beta <- PostBetaNew(sig2 = sig2, D = D)
12  sig2 <- PostSig2(Beta = Beta, bs = bs)
13  PostBetas[s,] <- Beta
14  PostDs[s,] <- matrixcalc::vech(D)
15  PostSig2s[s] <- sig2
16  Postbs[, , s] <- bs
17  setTxtProgressBar(pb, s)
18 }
19 close(pb)
20 keep <- seq((burnin+1), tot, thin)
21 Bs <- PostBetas[keep,]; Ds <- PostDs[keep,]
22 bs <- Postbs[, , keep]; sig2s <- PostSig2s[keep]
23 summary(coda::mcmc(Bs))
24 Quantiles for each variable:
25      2.5%      25%      50%      75%      97.5%
26 var1  1.727227  1.934457  2.037411  2.138829  2.325961
27 var2 -0.033664  0.001015  0.018974  0.037977  0.076626
28 var3  0.274521  0.303619  0.318302  0.332881  0.363323
29 var4  0.652937  0.692479  0.712605  0.730885  0.768415
30 var5 -0.008724 -0.007265 -0.006548 -0.005844 -0.004508
31 summary(coda::mcmc(Ds))
32 summary(coda::mcmc(sig2s))
33 # Convergence diagnostics
34 coda::geweke.diag(Bs)
35 coda::raftery.diag(Bs,q=0.5,r=0.05,s = 0.95)
36 coda::heidel.diag(Bs)

```

The likelihood function in the case with heteroskedasticity is proportional

to

$$p(\boldsymbol{\beta}, \mathbf{b}, \sigma^2, \boldsymbol{\tau} | \mathbf{y}, \mathbf{X}, \mathbf{W}) \propto \prod_{i=1}^N |\sigma^2 \boldsymbol{\Psi}_i|^{-1/2} \\ \times \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y}_i - \mathbf{X}_i \boldsymbol{\beta} - \mathbf{W}_i \mathbf{b}_i)^\top \boldsymbol{\Psi}_i^{-1} (\mathbf{y}_i - \mathbf{X}_i \boldsymbol{\beta} - \mathbf{W}_i \mathbf{b}_i) \right\},$$

where $\mathbf{b} = [\mathbf{b}_1^\top, \mathbf{b}_2^\top, \dots, \mathbf{b}_N^\top]^\top$, $\boldsymbol{\tau} = [\tau_{it}]^\top$ and $\boldsymbol{\Psi}_i = \text{diag} \{ \tau_{it}^{-1} \}$.

Following the same procedure as in Section 9.1 of the book, and Exercise 1 of this chapter, we have that

$$\boldsymbol{\beta} | \sigma^2, \boldsymbol{\tau}, \mathbf{D}, \mathbf{y}, \mathbf{X}, \mathbf{W} \sim N(\boldsymbol{\beta}_n, \mathbf{B}_n),$$

where $\mathbf{B}_n = (\mathbf{B}_0^{-1} + \sum_{i=1}^N \mathbf{X}_i^\top \mathbf{V}_i^{-1} \mathbf{X}_i)^{-1}$, $\boldsymbol{\beta}_n = \mathbf{B}_n (\mathbf{B}_0^{-1} \boldsymbol{\beta}_0 + \sum_{i=1}^N \mathbf{X}_i^\top \mathbf{V}_i^{-1} \mathbf{y}_i)$ and $\mathbf{V}_i = \sigma^2 \boldsymbol{\Psi}_i + \sigma_b^2 \mathbf{i}_{T_i} \mathbf{i}_{T_i}^\top$.

$$\mathbf{b}_i | \boldsymbol{\beta}, \sigma^2, \boldsymbol{\tau}, \mathbf{D}, \mathbf{y}, \mathbf{X}, \mathbf{W} \sim N(\mathbf{b}_{ni}, \mathbf{B}_{ni}),$$

where $\mathbf{B}_{ni} = (\sigma^{-2} \mathbf{W}_i^\top \boldsymbol{\Psi}_i^{-1} \mathbf{W}_i + \mathbf{D}^{-1})^{-1}$ and $\mathbf{b}_{ni} = \mathbf{B}_{ni} (\sigma^{-2} \mathbf{W}_i^\top \boldsymbol{\Psi}_i^{-1} (\mathbf{y}_i - \mathbf{X}_i \boldsymbol{\beta}))$.

$$\sigma^2 | \boldsymbol{\beta}, \mathbf{b}, \boldsymbol{\tau}, \mathbf{y}, \mathbf{X}, \mathbf{W} \sim IG(\alpha_n, \delta_n),$$

where $\alpha_n = \alpha_0 + \frac{1}{2} \sum_{i=1}^N T_i$ and $\delta_n = \delta_0 + \frac{1}{2} \sum_{i=1}^N (\mathbf{y}_i - \mathbf{X}_i \boldsymbol{\beta} - \mathbf{W}_i \mathbf{b}_i)^\top \boldsymbol{\Psi}_i^{-1} (\mathbf{y}_i - \mathbf{X}_i \boldsymbol{\beta} - \mathbf{W}_i \mathbf{b}_i)$.

$$\mathbf{D} | \mathbf{b} \sim IW(d_n, \mathbf{D}_n),$$

where $d_n = d_0 + N$ and $\mathbf{D}_n = d_0 \mathbf{D}_0 + \sum_{i=1}^N \mathbf{b}_i \mathbf{b}_i^\top$. And

$$\tau_{it} | \sigma^2, \boldsymbol{\beta}, \mathbf{b}, \mathbf{y}, \mathbf{X}, \mathbf{W} \sim G(v_{1n}/2, v_{2ni}/2),$$

where $v_{1n} = v + 1$ and $v_{2ni} = v + \sigma^{-2} (\mathbf{y}_{it} - \mathbf{x}_{it}^\top \boldsymbol{\beta} - \mathbf{w}_{it}^\top \mathbf{b}_i)^2$.

The following code shows to implement this for the productivity application. The results of the *fixed effects* are very similar compared to the results without taken into account heteroskedasticity.

R code. The relationship between productivity and public investment, programming from scratch the Gibbs sampler with heteroskedasticity

```

1 rm(list = ls())
2 set.seed(12345)
3 DataGSP <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/8PublicCap.
  csv", sep = ",", header = TRUE, quote = "")
4 attach(DataGSP)
5 N <- length(unique(id))
6 y <- log(gsp)
7 NT <- length(y)
8 X <- cbind(1, log(pcap), log(pc), log(emp), unemp)
9 K1 <- dim(X)[2]
10 W <- matrix(rep(1, NT), NT, 1)
11 K2 <- dim(W)[2]
12 mcmc <- 10000; burnin <- 5000; thin <- 1; tot <- mcmc +
  burnin
13 b0 <- rep(0, K1); B0 <- diag(K1); B0i <- solve(B0)
14 r0 <- 5; R0 <- diag(K2); a0 <- 0.001; d0 <- 0.001; v <- 5
15 # Gibbs by hand
16 PostBeta <- function(sig2, D, tau){
17   XVX <- matrix(0, K1, K1)
18   XVy <- matrix(0, K1, 1)
19   for(i in 1:N){
20     ids <- which(id == i)
21     Ti <- length(ids)
22     Wi <- W[ids, ]
23     tau_i <- tau[ids]
24     Vi <- sig2*solve(diag(1/tau_i)) + Wi%*%D%*%t(Wi)
25     ViInv <- solve(Vi)
26     Xi <- X[ids, ]
27     XVXi <- t(Xi)%*%ViInv%*%Xi
28     XVX <- XVX + XVXi
29     yi <- y[ids]
30     XVyi <- t(Xi)%*%ViInv%*%yi
31     XVy <- XVy + XVyi
32   }
33   Bn <- solve(B0i + XVX)
34   bn <- Bn%*%(B0i%*%b0 + XVy)
35   Beta <- MASS::mvrnorm(1, bn, Bn)
36   return(Beta)
37 }

```

R code. The relationship between productivity and public investment, programming from scratch the Gibbs sampler with heteroskedasticity

```

1 Postb <- function(Beta, sig2, D, tau){
2   Di <- solve(D);   bis <- matrix(0, N, K2)
3   for(i in 1:N){
4     ids <- which(id == i)
5     Wi <- W[ids, ]; Xi <- X[ids, ]
6     yi <- y[ids]; tau_i <- tau[ids]
7     Tau_i <- solve(diag(1/tau_i))
8     Wtei <- sig2^(-1)*t(Wi)%*%Tau_i%*(yi - Xi%*Beta)
9     Bni <- solve(sig2^(-1)*t(Wi)%*%Tau_i%*Wi + Di)
10    bni <- Bni%*Wtei
11    bi <- MASS::mvrnorm(1, bni, Bni)
12    bis[i, ] <- bi
13  }
14  return(bis)
15 }
16 PostSig2 <- function(Beta, bs, tau){
17   an <- a0 + 0.5*NT; ete <- 0
18   for(i in 1:N){
19     ids <- which(id == i)
20     Xi <- X[ids, ]; yi <- y[ids]
21     Wi <- W[ids, ]; tau_i <- tau[ids]
22     Tau_i <- solve(diag(1/tau_i))
23     ei <- yi - Xi%*Beta - Wi*bs[i]
24     etei <- t(ei)%*%Tau_i%*ei
25     ete <- ete + etei
26   }
27   dn <- d0 + 0.5*ete
28   sig2 <- MCMCpack::rinvgamma(1, shape = an, scale = dn)
29   return(sig2)
30 }
31 PostD <- function(bs){
32   rn <- r0 + N
33   btb <- matrix(0, K2, K2)
34   for(i in 1:N){
35     bsi <- bs[i, ]
36     btbi <- bsi%*%t(bsi)
37     btb <- btb + btbi
38   }
39   Rn <- d0*R0 + btb
40   Sigma <- MCMCpack::riwish(v = rn, S = Rn)
41   return(Sigma)
42 }
43 PostTau <- function(sig2, Beta, bs){
44   v1n <- v + 1
45   v2n <- NULL
46   for(i in 1:NT){
47     Xi <- X[i, ]; yi <- y[i]
48     Wi <- W[i, ]; bi <- bs[id[i],]
49     v2ni <- v + sig2^(-1)*(yi - Xi%*Beta - Wi%*bi)^2
50     v2n <- c(v2n, v2ni)
51   }
52   tau <- rgamma(NT, shape = rep(v1n/2, NT), rate = v2n/2)
53   return(tau)
54 }

```


R code. The relationship between productivity and public investment, programming from scratch the Gibbs sampler with heteroskedasticity

```

1 PostBetas <- matrix(0, tot, K1); PostDs <- matrix(0, tot, K2
  *(K2+1)/2)
2 PostSig2s <- rep(0, tot); Postbs <- array(0, c(N, K2, tot))
3 PostTaus <- matrix(0, tot, NT); RegLS <- lm(y ~ X - 1)
4 SumLS <- summary(RegLS); Beta <- SumLS[["coefficients"]][,1]
5 sig2 <- SumLS[["sigma"]]^2; D <- diag(K2)
6 tau <- rgamma(NT, shape = v/2, rate = v/2)
7 pb <- txtProgressBar(min = 0, max = tot, style = 3)
8 for(s in 1:tot){
9   bs <- Postb(Beta = Beta, sig2 = sig2, D = D, tau = tau)
10  D <- PostD(bs = bs)
11  Beta <- PostBeta(sig2 = sig2, D = D, tau = tau)
12  sig2 <- PostSig2(Beta = Beta, bs = bs, tau = tau)
13  tau <- PostTau(sig2 = sig2, Beta = Beta, bs = bs)
14  PostBetas[s,] <- Beta
15  PostDs[s,] <- matrixcalc::vech(D)
16  PostSig2s[s] <- sig2
17  Postbs[, , s] <- bs
18  PostTaus[s,] <- tau
19  setTxtProgressBar(pb, s)
20 }
21 close(pb)
22 keep <- seq((burnin+1), tot, thin)
23 Bs <- PostBetas[keep,]
24 Ds <- PostDs[keep,]
25 bs <- Postbs[, , keep]
26 sig2s <- PostSig2s[keep]
27 taus <- PostTaus[keep,]
28 summary(coda::mcmc(Bs))
29 Quantiles for each variable:
30      2.5%      25%      50%      75%      97.5%
31 var1  1.61764  1.891242  2.029985  2.165677  2.427349
32 var2 -0.07088 -0.016921  0.013080  0.044681  0.107360
33 var3  0.27259  0.312197  0.333106  0.353678  0.395123
34 var4  0.59507  0.664490  0.699526  0.734612  0.796977
35 var5 -0.01046 -0.008533 -0.007586 -0.006666 -0.004918
36 summary(coda::mcmc(Ds))
37 summary(coda::mcmc(sig2s))

```

3. Simulation exercise of the longitudinal normal model continues

Assume that $y_{it} = \beta_1 x_{it1} + \beta_2 x_{it2} + \beta_3 x_{it3} + \beta_4 z_{i1} + b_i + w_{it1} b_{i1} + \mu_{it}$ where $x_{itk} \sim N(0, 1)$, $k = 1, 2, 3$, $z_{i1} \sim B(0.5)$, $w_{it1} \sim N(0, 1)$, $b_i \sim N(0, 0.7^{1/2})$, $b_{i1} \sim N(0, 0.6^{1/2})$, $\mu_{it} \sim N(0, 0.1^{1/2})$ $\beta = [0.4 \ 0.6 \ -0.6 \ 0.7]^\top$, $i = 1, 2, \dots, 50$, and the sample size is 2000 in an *unbalanced panel structure*.

In addition, we assume that $\mathbf{b}_i$ depends on $\mathbf{z}_i = [1 \ z_{i1}]^\top$ such that $\mathbf{b}_i \sim N(\mathbf{Z}_i \boldsymbol{\gamma}, \mathbf{D})$ where $\mathbf{Z}_i = \mathbf{I}_{K_2} \otimes \mathbf{z}_i^\top$, and $\boldsymbol{\gamma} = [1 \ 1 \ 1]^\top$. The prior for $\boldsymbol{\gamma}$ is $N(\boldsymbol{\gamma}_0, \boldsymbol{\Gamma}_0)$ where we set $\boldsymbol{\gamma}_0 = \mathbf{0}_4$ and $\boldsymbol{\Gamma}_0 = \mathbf{I}_4$. In addition, Set $\boldsymbol{\beta}_0 = \mathbf{0}_4$, $\mathbf{B}_0 = \mathbf{I}_4$, $\alpha_0 = \delta_0 = 0.001$, $d_0 = 2$ and $\mathbf{D}_0 = \mathbf{I}_2$.

- Perform inference in this model without taking into account the dependence between $\mathbf{b}_i$ and z_{i1} , and compare the posterior estimates with the population parameters.
- Perform inference in this model taking into account the dependence between $\mathbf{b}_i$ and z_{i1} , and compare the posterior estimates with the population parameters.

Answer

Following the same procedure as in Section 9.1 of the book, and Exercise 1 of this chapter, we have that

$$\boldsymbol{\beta} | \sigma^2, \boldsymbol{\gamma}, \mathbf{D}, \mathbf{y}, \mathbf{X}, \mathbf{W}, \mathbf{Z} \sim N(\boldsymbol{\beta}_n, \mathbf{B}_n),$$

where $\mathbf{B}_n = (\mathbf{B}_0^{-1} + \sum_{i=1}^N \mathbf{X}_i^\top \mathbf{V}_i^{-1} \mathbf{X}_i)^{-1}$, $\boldsymbol{\beta}_n = \mathbf{B}_n(\mathbf{B}_0^{-1} \boldsymbol{\beta}_0 + \sum_{i=1}^N \mathbf{X}_i^\top \mathbf{V}_i^{-1} (\mathbf{y}_i - \mathbf{W}_i \mathbf{Z}_i \boldsymbol{\gamma}))$ and $\mathbf{V}_i = \sigma^2 \mathbf{I}_{T_i} + \sigma_b^2 \mathbf{i}_{T_i} \mathbf{i}_{T_i}^\top$.

$$\mathbf{b}_i | \boldsymbol{\beta}, \sigma^2, \boldsymbol{\gamma}, \mathbf{D}, \mathbf{y}, \mathbf{X}, \mathbf{W}, \mathbf{Z} \sim N(\mathbf{b}_{ni}, \mathbf{B}_{ni}),$$

where $\mathbf{B}_{ni} = (\sigma^{-2} \mathbf{W}_i^\top \mathbf{W}_i + \mathbf{D}^{-1})^{-1}$ and $\mathbf{b}_{ni} = \mathbf{B}_{ni}(\sigma^{-2} \mathbf{W}_i^\top (\mathbf{y}_i - \mathbf{X}_i \boldsymbol{\beta}) + \mathbf{D}^{-1} \mathbf{Z}_i \boldsymbol{\gamma})$.

$$\sigma^2 | \boldsymbol{\beta}, \mathbf{b}, \boldsymbol{\tau}, \mathbf{y}, \mathbf{X}, \mathbf{W} \sim IG(\alpha_n, \delta_n),$$

where $\alpha_n = \alpha_0 + \frac{1}{2} \sum_{i=1}^N T_i$ and $\delta_n = \delta_0 + \frac{1}{2} \sum_{i=1}^N (\mathbf{y}_i - \mathbf{X}_i \boldsymbol{\beta} - \mathbf{W}_i \mathbf{b}_i)^\top (\mathbf{y}_i - \mathbf{X}_i \boldsymbol{\beta} - \mathbf{W}_i \mathbf{b}_i)$.

$$\mathbf{D} | \mathbf{b}, \boldsymbol{\gamma}, \mathbf{Z} \sim IW(d_n, \mathbf{D}_n),$$

where $d_n = d_0 + N$ and $\mathbf{D}_n = d_0 \mathbf{D}_0 + \sum_{i=1}^N (\mathbf{b}_i - \mathbf{Z}_i \boldsymbol{\gamma})(\mathbf{b}_i - \mathbf{Z}_i \boldsymbol{\gamma})^\top$. And

$$\boldsymbol{\gamma} | \mathbf{b}, \mathbf{D}, \mathbf{Z} \sim N(\boldsymbol{\gamma}_n, \boldsymbol{\Gamma}_n),$$

where $\boldsymbol{\Gamma}_n = (\sum_{i=1}^N \mathbf{Z}_i^\top \mathbf{D}^{-1} \mathbf{Z}_i + \boldsymbol{\Gamma}_0^{-1})^{-1}$ and $\boldsymbol{\gamma}_n = \boldsymbol{\Gamma}_n(\sum_{i=1}^N \mathbf{Z}_i^\top \mathbf{D}^{-1} \mathbf{b}_i + \boldsymbol{\Gamma}_0^{-1} \boldsymbol{\gamma}_0)$.

The following code shows how to implement this longitudinal normal model where *random effects* are correlated with regressors. We set MCMC iterations, burn-in and thinning parameters equal to 15000, 5000 and 5, respectively. In addition, $\boldsymbol{\beta}_0 = \mathbf{0}_5$, $\mathbf{B}_0 = \mathbf{I}_5$, $\alpha_0 = \delta_0 = 0.001$, $d_0 = 2$, $\mathbf{D}_0 = \mathbf{I}_2$, $\boldsymbol{\gamma}_0 = \mathbf{0}_4$ and $\boldsymbol{\Gamma}_0 = \mathbf{I}_4$.

R code. Simulation exercise: Longitudinal normal model with fixed effects correlated to regressors

```

1 rm(list = ls())
2 set.seed(010101)
3 NT <- 2000
4 N <- 50
5 id <- c(1:N, sample(1:N, NT - N, replace=TRUE))
6 table(id)
7 x1 <- rnorm(NT); x2 <- rnorm(NT)
8 x3 <- rnorm(NT); z1 <- rbinom(N, 1, 0.5)
9 zdata <- NULL
10 for(i in 1:NT){
11   zdatai <- z1[id[i]]
12   zdata <- c(zdata, zdatai)
13 }
14 X <- cbind(x1, x2, x3, zdata)
15 K1 <- dim(X)[2]
16 w1 <- rnorm(NT)
17 W <- cbind(1, w1)
18 K2 <- dim(W)[2]
19 B <- c(0.5, 0.4, 0.6, -0.6, 0.7)
20 D <- c(0.7, 0.6)
21 sig2 <- 0.1
22 u <- rnorm(NT, 0, sd = sig2^0.5)
23 Z <- cbind(1, z1)
24 K3 <- dim(Z)[2]
25 G <- rep(1, K3*K2)
26 b <- matrix(0, N, K2)
27 for(i in 1:N){
28   ZGi <- t(kronecker(diag(K2), Z[i, ]))%*%G
29   b[i, ] <- MASS::mvrnorm(1, ZGi, diag(D))
30 }
31 y <- NULL
32 for(i in 1:NT){
33   yi <- X[i,]%*%B + W[i,]%*%b[id[i],] + u[i]
34   y <- c(y, yi)
35 }
36 Data <- as.data.frame(cbind(y, x1, x2, x3, zdata, w1, id))
37 mcmc <- 15000; burnin <- 5000; thin <- 10; tot <- mcmc +
   burnin
38 b0 <- rep(0, K1+1); B0 <- diag(K1+1); B0i <- solve(B0)
39 r0 <- K2; R0 <- diag(K2); a0 <- 0.001; d0 <- 0.001
40 g0 <- rep(0, K2*K3); G0 <- diag(K2*K3); G0i <- solve(G0)
41 Resultshreg <- MCMCpack::MCMChregress(fixed = y~x1 + x2 + x3
   + zdata, random = ~w1, group="id", data = Data, burnin
   = burnin, mcmc = mcmc, thin = thin,
42 mubeta = b0, Vbeta = B0, r = r0, R = R0, nu = a0, delta = d0
   )
43 Betas <- Resultshreg[["mcmc"]][,1:(K1+1)]
44 Sigma2RanEff <- Resultshreg[["mcmc"]][,c(K2*N+K1+2, 2*N+K1
   +1+K2^2)]
45 Sigma2 <- Resultshreg[["mcmc"]][,K2*N+K1+K2^2+2]
46 summary(Betas)
47 summary(Sigma2RanEff)
48 summary(Sigma2)

```

R code. Simulation exercise: Longitudinal normal model with fixed effects correlated to regressors

```

1 PostBeta <- function(sig2, Gamma, D){
2   XVX <- matrix(0, K1, K1)
3   XVy <- matrix(0, K1, 1)
4   for(i in 1:N){
5     ids <- which(id == i)
6     Ti <- length(ids)
7     Wi <- W[ids, ]
8     Vi <- sig2*diag(Ti) + Wi%*%D%*%t(Wi)
9     ViInv <- solve(Vi)
10    Xi <- X[ids, ]
11    XVXi <- t(Xi)%*%ViInv%*%Xi
12    XVX <- XVX + XVXi
13    Zi <- Z[i, ]
14    yi <- y[ids]
15    ZGi <- t(kronecker(diag(K2), Zi))%*%Gamma
16    XVyi <- t(Xi)%*%ViInv%*%(yi - Wi%*%ZGi)
17    XVy <- XVy + XVyi
18  }
19  Bn <- solve(B0i + XVX)
20  bn <- Bn%*%(B0i%*%b0 + XVy)
21  Beta <- MASS::mvrnorm(1, bn, Bn)
22  return(Beta)
23 }
24 Postb <- function(Beta, sig2, Gamma, D){
25   Di <- solve(D)
26   bis <- matrix(0, N, K2)
27   for(i in 1:N){
28     ids <- which(id == i)
29     Wi <- W[ids, ]
30     Xi <- X[ids, ]
31     yi <- y[ids]
32     Wtei <- sig2^(-1)*t(Wi)%*%(yi - Xi%*%Beta)
33     Bni <- solve(sig2^(-1)*t(Wi)%*%Wi + Di)
34     Zi <- Z[i, ]
35     ZGi <- t(kronecker(diag(K2), Zi))%*%Gamma
36     bni <- Bni%*%(Wtei + Di%*%ZGi)
37     bi <- MASS::mvrnorm(1, bni, Bni)
38     bis[i, ] <- bi
39   }
40   return(bis)
41 }
42 PostSig2 <- function(Beta, bs){
43   an <- a0 + 0.5*NT
44   ete <- 0
45   for(i in 1:N){
46     ids <- which(id == i)
47     Xi <- X[ids, ]
48     yi <- y[ids]
49     Wi <- W[ids, ]
50     ei <- yi - Xi%*%Beta - Wi%*%bs[i, ]
51     etei <- t(ei)%*%ei
52     ete <- ete + etei
53   }
54   dn <- d0 + 0.5*ete
55   sig2 <- MCMCpack::rinvgamma(1, shape = an, scale = dn)
56   return(sig2)
57 }

```

R code. Simulation exercise: Longitudinal normal model with fixed effects correlated to regressors

```

1 PostGamma <- function(bs, D){
2   Di <- solve(D)
3   ZDZ <- matrix(0, K2*K3, K2*K3)
4   ZDb <- matrix(0, K2*K3, 1)
5   for(i in 1:N){
6     Zi <- Z[i, ]
7     ZZi <- t(kronecker(diag(K2), Zi))
8     ZDZi <- t(ZZi)%*%Di%*%ZZi
9     ZDZ <- ZDZ + ZDZi
10    bi <- bs[i,]
11    ZDbi <- t(ZZi)%*%Di%*%bi
12    ZDb <- ZDb + ZDbi
13  }
14  Gn <- solve(G0i + ZDZ); gn <- Gn%*%(G0i%*%g0 + ZDb)
15  Gamma <- MASS::mvrnorm(1, gn, Gn)
16  return(Gamma)
17 }
18 PostD <- function(bs, Gamma){
19   rn <- r0 + N
20   btb <- matrix(0, K2, K2)
21   for(i in 1:N){
22     bsi <- bs[i, ]
23     Zi <- Z[i, ]
24     ZGi <- t(kronecker(diag(K2), Zi))%*%Gamma
25     btbi <- (bsi-ZGi)%*%t(bsi-ZGi)
26     btb <- btb + btbi
27   }
28   Rn <- d0*R0 + btb
29   Sigma <- MCMCpack::riwish(v = rn, S = Rn)
30   return(Sigma)
31 }
32 PostBetas <- matrix(0, tot, K1); PostGammas <- matrix(0, tot,
33   , K2*K3)
34 PostDs <- matrix(0, tot, K2*(K2+1)/2); PostSig2s <- rep(0,
35   tot)
36 Postbs <- array(0, c(N, K2, tot)); RegLS <- lm(y ~ X - 1)
37 SumLS <- summary(RegLS); Beta <- SumLS[["coefficients"]][,1]
38 sig2 <- SumLS[["sigma"]]2; Gamma <- rep(0, K2*K3)
39 D <- diag(K2); b0 <- rep(0, K1); B0 <- diag(K1); B0i <-
40   solve(B0)
41 pb <- txtProgressBar(min = 0, max = tot, style = 3)
42 for(s in 1:tot){
43   bs <- Postb(Beta = Beta, sig2 = sig2, Gamma = Gamma, D = D
44   )
45   D <- PostD(bs = bs, Gamma = Gamma)
46   Beta <- PostBeta(sig2 = sig2, Gamma = Gamma, D = D)
47   sig2 <- PostSig2(Beta = Beta, bs = bs)
48   Gamma <- PostGamma(bs = bs, D = D)
49   PostBetas[s,] <- Beta; PostDs[s,] <- matrixcalc::vech(D)
50   PostSig2s[s] <- sig2; Postbs[, , s] <- bs
51   PostGammas[s,] <- Gamma
52   setTxtProgressBar(pb, s)
53 }

```

R code. Simulation exercise: Longitudinal normal model with fixed effects correlated to regressors

```

1 close(pb)
2 keep <- seq((burnin+1), tot, thin)
3 Bs <- PostBetas[keep,]
4 Ds <- PostDs[keep,]
5 bs <- Postbs[, , keep]
6 sig2s <- PostSig2s[keep]
7 Gs <- PostGammas[keep, ]
8 summary(coda::mcmc(Bs))
9 Quantiles for each variable:
10      2.5%      25%      50%      75%      97.5%
11 var1  0.3629  0.3812  0.3880  0.3945  0.4120
12 var2  0.5844  0.6032  0.6098  0.6176  0.6381
13 var3 -0.6114 -0.5950 -0.5880 -0.5809 -0.5605
14 var4 -0.4455  0.4155  0.8598  1.3047  2.0654
15 summary(coda::mcmc(Ds))
16 Quantiles for each variable:
17      2.5%      25%      50%      75%      97.5%
18 var1  0.5858  0.7798  0.9173  1.13381  2.0153
19 var2 -0.3914 -0.1901 -0.1104 -0.03585  0.1250
20 var3  0.4285  0.5495  0.6314  0.72342  0.9662
21 summary(coda::mcmc(sig2s))
22 Quantiles for each variable:
23 Quantiles for each variable:
24      2.5%      25%      50%      75%      97.5%
25 0.09609 0.11486 0.17790 0.35215 1.12537
26 summary(coda::mcmc(Gs))
27 Quantiles for each variable:
28      2.5%      25%      50%      75%      97.5%
29 var1  0.6552  0.8998  1.0288  1.162  1.402
30 var2 -0.4463  0.4056  0.7882  1.242  2.144
31 var3  0.8583  1.0599  1.1623  1.271  1.482
32 var4  0.5525  0.8430  1.0038  1.147  1.416

```

In one hand, the 95% credible intervals of the model without taking into account the dependence between the *random effects* and z_{i1} overestimates the posterior mean of β_4 and the variance of b_{i1} . In addition, the credible intervals are very narrow. However, the variance of μ_i is well estimated. On the other hand, all the 95% credible intervals of the model taking into account the dependence between the *random effects* and z_{i1} encompass the population parameters. However, there is a lot of variability in β_4 posterior estimates. The posterior mean estimate of the variance of b_{i1} is close to the population value, and the location parameters associated with z_{i1} in the mean of the *random effects* are also well estimated by the posterior mean. However, the variance of the model is overestimated.

4. Doctor visits in Germany continues I

Replicate this example using our GUI, which by default does not fix the over-dispersion parameter (σ^2), and compare the results with the results of this example in Section 9.2.

Answer

This code replicates what we get using our GUI.

R code. Doctor visits in Germany, results

```

1 rm(list = ls())
2 set.seed(12345)
3 Data <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/9VisitDoc.csv",
  sep = ",", header = TRUE, quote = "\"")
4 attach(Data)
5 K1 <- 7; K2 <- 2; N <- 9197
6 b0 <- rep(0, K1); B0 <- diag(K1)
7 r0 <- 5; R0 <- diag(K2)
8 a0 <- 0.001; d0 <- 0.001
9 RegLogit <- glm(DocVis ~ Age + Male + Sport + LogInc +
  GoodHealth + BadHealth, family = binomial(link = "logit"
  ))
10 SumLogit <- summary(RegLogit)
11 Beta0 <- SumLogit[["coefficients"]][,1]
12 mcmc <- 10000; burnin <- 1000; thin <- 10
13 # MCMChlogit
14 Resultshlogit <- MCMCpack::MCMChlogit(fixed = DocVis ~ Age +
  Male + Sport + LogInc + GoodHealth + BadHealth, random
  = ~Sozh, group="id", data = Data, burnin = burnin, mcmc
  = mcmc, thin = thin, mubeta = b0, Vbeta = B0, r = r0, R
  = R0, nu = a0, delta = d0, beta.start = Beta0, FixOD =
  0)
15 Betas <- Resultshlogit[["mcmc"]][,1:K1]
16 Sigma2RanEff <- Resultshlogit[["mcmc"]][,c(K2*N+K1+1, 2*N+K1
  +K2^2)]
17 summary(Betas)
18 summary(Sigma2RanEff)
19 summary(Sigma2)
20 Quantiles for each variable:
21      2.5%      25%      50%      75%      97.5%
22 Intercept -9.48e-01 -5.27e-01 -2.86e-01 -6.28e-02 2.99e-01
23 Age        4.88e-03 7.27e-03 8.35e-03 9.31e-03 1.09e-02
24 Male      -9.90e-01 -9.52e-01 -9.31e-01 -9.10e-01 -8.75e-01
25 Sport      1.80e-01 2.25e-01 2.55e-01 2.85e-01 3.47e-01
26 LogInc     1.38e-01 1.85e-01 2.15e-01 2.49e-01 3.01e-01
27 GoodHealth -9.56e-01 -9.18e-01 -8.97e-01 -8.75e-01 -8.36e-01
28 BadHealth  1.08e+00 1.14e+00 1.19e+00 1.23e+00 1.28e+00
29 D11        1.43e+00 1.53e+00 1.58e+00 1.62e+00 1.70e+00
30 D21       -2.67e-01 1.94e-02 1.85e-01 4.08e-01 5.81e-01
31 D12       -2.67e-01 1.94e-02 1.85e-01 4.08e-01 5.81e-01
32 D22        6.05e-01 7.99e-01 9.75e-01 1.21e+00 1.62e+00
33 sigma2     6.37e-02 7.76e-02 1.01e-01 1.15e-01 1.58e-01

```

We see in this example running our GUI that we get qualitatively same results as the results fixing the over-dispersion parameter in Section 9.2 in the book. However, there small numerical differences in the *fixed effects*,

and the differences are greater in the posterior results of the covariance matrix of the *random effects*.

5. Simulation exercise of the longitudinal logit model

Perform a simulation exercise to assess the performance of the hierarchical longitudinal logit model. The point of departure is to assume that $y_{it}^* = \beta_0 + \beta_1 x_{it1} + \beta_2 x_{it2} + \beta_3 x_{it3} + b_i + w_{it1} b_{i1}$ where $x_{itk} \sim N(0, 1)$, $k = 1, 2, 3$, $w_{it1} \sim N(0, 1)$, $b_i \sim N(0, 0.7^{1/2})$, $b_{i1} \sim N(0, 0.6^{1/2})$, $\beta = [0.5 \ 0.4 \ 0.6 \ -0.6]^\top$, $i = 1, 2, \dots, 50$, and $y_{it} \sim B(\pi_{it})$, where $\pi_{it} = 1/(1 + \exp(-y_{it}^*))$. The sample size is 1000 in an *unbalanced panel structure*.

- Perform inference using the command *MCMChlogit* fixing the over-dispersion parameter, and using $\beta_0 = \mathbf{0}_4$, $\mathbf{B}_0 = \mathbf{I}_4$, $\alpha_0 = \delta_0 = 0.001$, $d_0 = 2$ and $\mathbf{D}_0 = \mathbf{I}_2$.
- Program from scratch a Metropolis-within-Gibbs algorithm to perform inference in this simulation.

Answer

The following code shows the simulation setting, and the results using the *MCMChlogit* command. We see that all posterior estimates encompass the population values.

R code. Simulation exercise: Hierarchical longitudinal logit model using MCMChlogit

```

1  rm(list = ls()); set.seed(010101)
2  NT <- 1000; N <- 50
3  id <- c(1:N, sample(1:N, NT - N, replace=TRUE))
4  x1 <- rnorm(NT); x2 <- rnorm(NT); x3 <- rnorm(NT)
5  X <- cbind(1, x1, x2, x3); K1 <- dim(X)[2]
6  w1 <- rnorm(NT); W <- cbind(1, w1); K2 <- dim(W)[2]
7  B <- c(0.5, 0.4, 0.6, -0.6); D <- c(0.7, 0.6)
8  sig2 <- 0.1
9  b1 <- rnorm(N, 0, sd = D[1]^0.5)
10 b2 <- rnorm(N, 0, sd = D[2]^0.5)
11 b <- cbind(b1, b2)
12 yl <- NULL
13 for(i in 1:NT){
14   ylmeani <- X[i,]%*%B + W[i,]%*%b[id[i],]
15   yli <- rnorm(1, ylmeani, sig2^0.5)
16   yl <- c(yl, yli)
17 }
18 pit <- 1/(1+exp(-yl))
19 y <- rbinom(NT, 1, prob = pit)
20 Data <- as.data.frame(cbind(y, x1, x2, x3, w1, id))
21 mcmc <- 15000; burnin <- 5000; thin <- 10; tot <- mcmc +
   burnin
22 b0 <- rep(0, K1); B0 <- diag(K1); B0i <- solve(B0)
23 r0 <- K2; R0 <- diag(K2); a0 <- 0.001; d0 <- 0.001
24 RegLogit <- glm(y ~ X - 1, family = binomial(link = "logit")
   )
25 SumLogit <- summary(RegLogit)
26
27 Beta0 <- SumLogit[["coefficients"]][,1]
28 sig20 <- sum(SumLogit[["deviance.resid"]]^2)/SumLogit[["df.
   residual"]]
29 Resultshlogit <- MCMCpack::MCMChlogit(fixed = y~x1 + x2 + x3
   , random = ~w1, group="id", data = Data, burnin = burnin
   , mcmc = mcmc, thin = thin,
30   mubeta = b0, Vbeta = B0, r = r0, R = R0, nu = a0, delta = d0
   ,
31   beta.start = Beta0, FixOD = 1, sigma2.start = sig20)
32 Betas <- Resultshlogit[["mcmc"]][,1:K1]
33 Sigma2RanEff <- Resultshlogit[["mcmc"]][,c(K2*N+K1+1, 2*N+K1
   +K2^2)]
34 Sigma2 <- Resultshlogit[["mcmc"]][,K2*N+K1+K2^2+1]
35 summary(Betas)
36 Quantiles for each variable:
37      2.5%      25%      50%      75%      97.5%
38 beta.(Intercept)  0.4258  0.6165  0.7140  0.8155  1.0041
39 beta.x1           0.3294  0.4372  0.4999  0.5617  0.6684
40 beta.x2           0.5555  0.6763  0.7388  0.8062  0.9377
41 beta.x3          -0.9327 -0.7957 -0.7276 -0.6591 -0.5383
42 summary(Sigma2RanEff)
43 Quantiles for each variable:
44      2.5%      25%      50%      75% 97.5%
45 D11 0.3394 0.5366 0.6962 0.8652 1.338
46 D22 0.4101 0.6478 0.8091 1.0039 1.611

```

The following code shows the algorithm step by step.

R code. Simulation exercise: Hierarchical longitudinal logit model from scratch

```

1 LatentMHV1 <- function(tuning, Beta, bs, sig2){
2   ylhat <- rep(0, NT)
3   accept <- NULL
4   for(i in 1:NT){
5     ids <- which(id == i)
6     yi <- y[i]
7     ylhatmean1 <- X[i,]%*Beta + W[i,]%*bs[id[i],]
8     ylhati <- rnorm(1, ylhatmean1, sd = sig2^0.5)
9     pihati <- 1/(1+exp(-ylhati))
10    ei <- rnorm(1, 0, sd = tuning)
11    ylpropi <- ylhati + ei; pipropi <- 1/(1+exp(-ylpropi))
12    logPosthati <- sum(dbinom(yi, 1, prob = pihati, log =
13    TRUE) + dnorm(ylhati, ylhatmean1, sig2^0.5, log = TRUE))
14    logPostpropi <- sum(dbinom(yi, 1, prob = pipropi, log =
15    TRUE) + dnorm(ylpropi, ylhatmean1, sig2^0.5, log = TRUE)
16    )
17    alphai <- min(1, exp(logPostpropi - logPosthati))
18    ui <- runif(1)
19    if(ui <= alphai){
20      ylhati <- ylpropi; accepti <- 1
21    }else{
22      ylhati <- ylhati; accepti <- 0
23    }
24    ylhat[i] <- ylhati; accept <- c(accept, accepti)
25  }
26  res <- list(ylhat = ylhat, accept = mean(accept))
27  return(res)
28 }
29
30 PostBeta <- function(D, ylhat, sig2){
31   XVX <- matrix(0, K1, K1); XVy <- matrix(0, K1, 1)
32   for(i in 1:N){
33     ids <- which(id == i); Ti <- length(ids)
34     Wi <- W[ids, ]; Vi <- diag(Ti)*sig2 + Wi%*D%*t(Wi)
35     ViInv <- solve(Vi); Xi <- X[ids, ]
36     XVXi <- t(Xi)%*ViInv%*Xi; XVX <- XVX + XVXi
37     yi <- ylhat[ids]; XVyi <- t(Xi)%*ViInv%*yi
38     XVy <- XVy + XVyi
39   }
40   Bn <- solve(B0i + XVX)
41   bn <- Bn%*(B0i%*b0 + XVy)
42   Beta <- MASS::mvrnorm(1, bn, Bn)
43   return(Beta)
44 }
45
46 Postb <- function(Beta, D, ylhat, sig2){
47   Di <- solve(D); bis <- matrix(0, N, K2)
48   for(i in 1:N){
49     ids <- which(id == i)
50     Wi <- W[ids, ]; Xi <- X[ids, ]
51     yi <- ylhat[ids]
52     Wtei <- sig2^(-1)*t(Wi)%*(yi - Xi%*Beta)
53     Bni <- solve(sig2^(-1)*t(Wi)%*Wi + Di)
54     bni <- Bni%*Wtei; bi <- MASS::mvrnorm(1, bni, Bni)
55     bis[i, ] <- bi
56   }
57   return(bis)
58 }

```

R code. Simulation exercise: Hierarchical longitudinal logit model from scratch

```

1 PostD <- function(bs){
2   rn <- r0 + N
3   btb <- matrix(0, K2, K2)
4   for(i in 1:N){
5     bsi <- bs[i, ]; btb_i <- bsi%*%t(bsi)
6     btb <- btb + btb_i
7   }
8   Rn <- d0*R0 + btb
9   Sigma <- MCMCpack::riwish(v = rn, S = Rn)
10  return(Sigma)
11 }
12 PostSig2 <- function(Beta, bs, ylhat, ss){
13   an <- a0 + 0.5*NT; ete <- 0
14   for(i in 1:N){
15     ids <- which(id == i)
16     Xi <- X[ids, ]; yi <- ylhat[ids]
17     Wi <- W[ids, ]
18     ei <- yi - Xi%*%Beta - Wi%*%bs[i, ]
19     etei <- t(ei)%*%ei; ete <- ete + etei
20   }
21   dn <- d0 + 0.5*ete
22   sig2 <- MCMCpack::rinvgamma(1, shape = an, scale = dn)
23   if(sig2 > ss){
24     sig2 <- ss
25   }else{
26     sig2 <- sig2
27   }
28   return(sig2)
29 }
30 PostBetas <- matrix(0, tot, K1)
31 PostDs <- matrix(0, tot, K2*(K2+1)/2)
32 Postbs <- array(0, c(N, K2, tot))
33 PostSig2s <- rep(0, tot)
34 Accepts <- rep(NULL, tot)
35 RegLogit <- glm(y ~ X - 1, family = binomial(link = "logit")
36 )
37 SumLogit <- summary(RegLogit)
38 Beta <- SumLogit[["coefficients"]][,1]
39 sig2 <- sum(SumLogit[["deviance.resid"]]^2)/SumLogit[["df.
40 residual"]]
41 ss0 <- sig2; D <- diag(K2)
42 bs1 <- rnorm(N, 0, sd = D[1]^0.5)
43 bs2 <- rnorm(N, 0, sd = D[2]^0.5)
44 bs <- cbind(bs1, bs2)
45 tuning <- 0.1; ropt <- 0.44
46 tuneapariter <- seq(round(tot/10, 0), tot, round(tot/10, 0));
47   l <- 1
48 pb <- txtProgressBar(min = 0, max = tot, style = 3)

```

R code. Simulation exercise: Hierarchical longitudinal logit model from scratch

```

1 for(s in 1:tot){
2   LatY <- LatentMHV1(tuning = tuning, Beta = Beta, bs = bs,
3     sig2 = sig2)
4   ylhat <- LatY[["ylhat"]]
5   bs <- Postb(Beta = Beta, D = D, ylhat = ylhat, sig2 = sig2)
6   D <- PostD(bs = bs)
7   Beta <- PostBeta(D = D, ylhat = ylhat, sig2 = sig2)
8   sig2 <- PostSig2(Beta = Beta, bs = bs, ylhat = ylhat, ss =
9     ss0)
10  PostBetas[s,] <- Beta
11  PostDs[s,] <- matrixcalc::vech(D)
12  Postbs[, , s] <- bs
13  PostSig2s[s] <- sig2
14  AcceptRate <- LatY[["accept"]]
15  Accepts[s] <- AcceptRate
16  if(AcceptRate > ropt){
17    tuning = tuning*(2-(1-AcceptRate)/(1-ropt))
18  }else{
19    tuning = tuning/(2-AcceptRate/ropt)
20  }
21  if(s == tunepariter[1]){
22    print(AcceptRate)
23    l <- l + 1
24  }
25  setTxtProgressBar(pb, s)
26 }
27 close(pb)
28 keep <- seq((burnin+1), tot, thin)
29 Bs <- PostBetas[keep,]
30 Ds <- PostDs[keep,]
31 bs <- Postbs[, , keep]
32 sig2s <- PostSig2s[keep]
33 summary(coda::mcmc(Bs))
34 Quantiles for each variable:
35      2.5%      25%      50%      75%      97.5%
36 var1  0.3195  0.5266  0.6325  0.7452  0.9427
37 var2  0.1780  0.3563  0.4521  0.5436  0.7352
38 var3  0.3596  0.5197  0.6140  0.7165  0.9149
39 var4 -0.8994 -0.7153 -0.6125 -0.5135 -0.3240
40 summary(coda::mcmc(Ds))
41 Quantiles for each variable:
42      2.5%      25%      50%      75%      97.5%
43 var1  0.0002003  0.0007473  0.0022547  0.0124378  0.09575
44 var2 -0.0534437 -0.0029207 -0.0001433  0.0009905  0.02296
45 var3  0.0002150  0.0009101  0.0026712  0.0119391  0.14863
46 summary(coda::mcmc(sig2s))
47 Quantiles for each variable:
48      2.5%      25%      50%      75%      97.5%
49 0.1321  0.5174  0.8674  1.0788  1.1909

```

The 95% credible intervals of the *fixed effects* encompass all the population values. However, we do not get good posterior estimates of the covariance matrix and over-dispersion parameters.

6. Doctor visits in Germany continues II

Take a sub-sample of the first 500 individuals of the dataset *9VisitDoc.csv* to perform inference in the number of visits to doctors (*DocNum*) with the same specification of the example of **Doctor visits in Germany** of Section 9.2.

Answer

This algorithm shows how to perform inference in this application programming from scratch the algorithm.

R code. Doctor visits in Germany: Hierarchical longitudinal Poisson model from scratch

```

1 rm(list = ls())
2 set.seed(12345)
3 Data <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/9VisitDoc.csv",
  sep = ",", header = TRUE, quote = "")
4 library(dplyr)
5 Data <- Data %>%
6 filter(id <= 500)
7 attach(Data)
8 K1 <- 7; K2 <- 2; N <- 9197
9 b0 <- rep(0, K1); B0 <- diag(K1)
10 r0 <- 5; R0 <- diag(K2)
11 a0 <- 0.001; d0 <- 0.001
12 NT <- dim(Data)[1]
13 N <- length(unique(id))
14 X <- cbind(1, Age, Male, Sport, LogInc, GoodHealth,
  BadHealth)
15 K1 <- dim(X)[2]
16 W <- cbind(1, Sozh)
17 K2 <- dim(W)[2]
18 y <- DocNum
19 mcmc <- 15000; burnin <- 5000; thin <- 10; tot <- mcmc +
  burnin
20 b0 <- rep(0, K1); B0 <- diag(K1); B0i <- solve(B0)
21 r0 <- K2; R0 <- diag(K2); a0 <- 0.001; d0 <- 0.001
22 LatentMHV1 <- function(tuning, Beta, bs, sig2){
23   ylhat <- rep(0, NT)
24   accept <- NULL
25   for(i in 1:NT){
26     ids <- which(id == i)
27     yi <- y[ids]
28     ylhatmean1 <- X[ids,]%*%Beta + W[ids,]%*%bs[id[ids],]
29     ylhati <- rnorm(1, ylhatmean1, sd = sig2^0.5)
30     lambdahati <- exp(ylhati)
31     ei <- rnorm(1, 0, sd = tuning)
32     ylpropi <- ylhati + ei
33     lambdapropi <- exp(ylpropi)
34     logPosthati <- sum(dpois(yi, lambdahati, log = TRUE) +
35       dnorm(ylhati, ylhatmean1, sig2^0.5, log = TRUE))
36     logPostpropi <- sum(dpois(yi, lambdapropi, log = TRUE) +
37       dnorm(ylpropi, ylhatmean1, sig2^0.5, log = TRUE))
38     alphas <- min(1, exp(logPostpropi - logPosthati))
39     ui <- runif(1)
40     if(ui <= alphas){
41       ylhati <- ylpropi; accepti <- 1
42     }else{
43       ylhati <- ylhati; accepti <- 0
44     }
45     ylhat[i] <- ylhati
46     accept <- c(accept, accepti)
47   }
48   res <- list(ylhat = ylhat, accept = mean(accept))
49   return(res)
50 }

```


R code. Doctor visits in Germany: Hierarchical longitudinal Poisson model from scratch

```

1 PostBeta <- function(D, ylhat, sig2){
2   XVX <- matrix(0, K1, K1); XVy <- matrix(0, K1, 1)
3   for(i in 1:N){
4     ids <- which(id == i); Ti <- length(ids)
5     Wi <- matrix(W[ids, ], Ti, K2)
6     Vi <- diag(Ti)*sig2 + Wi%*%D%*%t(Wi)
7     ViInv <- solve(Vi); Xi <- matrix(X[ids, ], Ti, K1)
8     XVXi <- t(Xi)%*%ViInv%*%Xi
9     XVX <- XVX + XVXi; yi <- ylhat[ids]
10    XVyi <- t(Xi)%*%ViInv%*%yi; XVy <- XVy + XVyi
11  }
12  Bn <- solve(B0i + XVX); bn <- Bn%*%(B0i%*%b0 + XVy)
13  Beta <- MASS::mvrnorm(1, bn, Bn)
14  return(Beta)
15 }
16 Postb <- function(Beta, D, ylhat, sig2){
17  Di <- solve(D); bis <- matrix(0, N, K2)
18  for(i in 1:N){
19    ids <- which(id == i); Ti <- length(ids)
20    Wi <- matrix(W[ids, ], Ti, K2)
21    Xi <- matrix(X[ids, ], Ti, K1)
22    yi <- ylhat[ids]
23    Wtei <- sig2^(-1)*t(Wi)%*%(yi - Xi%*%Beta)
24    Bni <- solve(sig2^(-1)*t(Wi)%*%Wi + Di)
25    bni <- Bni%*%Wtei
26    bi <- MASS::mvrnorm(1, bni, Bni)
27    bis[i, ] <- bi
28  }
29  return(bis)
30 }
31 PostD <- function(bs){
32  rn <- r0 + N; btb <- matrix(0, K2, K2)
33  for(i in 1:N){
34    bsi <- bs[i, ]; btbi <- bsi%*%t(bsi)
35    btb <- btb + btbi
36  }
37  Rn <- d0*R0 + btb; Sigma <- MCMCpack::riwish(v = rn, S =
    Rn)
38  return(Sigma)
39 }
40 PostSig2 <- function(Beta, bs, ylhat){
41  an <- a0 + 0.5*NT; ete <- 0
42  for(i in 1:N){
43    ids <- which(id == i)
44    Ti <- length(ids)
45    Xi <- matrix(X[ids, ], Ti, K1); yi <- ylhat[ids]
46    Wi <- matrix(W[ids, ], Ti, K2)
47    ei <- yi - Xi%*%Beta - Wi%*%bs[i, ]
48    etei <- t(ei)%*%ei; ete <- ete + etei
49  }
50  dn <- d0 + 0.5*ete
51  sig2 <- MCMCpack::rinvgamma(1, shape = an, scale = dn)
52  return(sig2)
53 }

```

R code. Doctor visits in Germany: Hierarchical longitudinal Poisson model from scratch

```

1 PostBetas <- matrix(0, tot, K1)
2 PostDs <- matrix(0, tot, K2*(K2+1)/2)
3 Postbs <- array(0, c(N, K2, tot))
4 PostSig2s <- rep(0, tot); ccepts <- rep(NULL, tot)
5 RegPois <- glm(DocNum ~ Age + Male + Sport + LogInc +
  GoodHealth + BadHealth, family = poisson(link = "log"))
6 SumPois <- summary(RegPois)
7 Beta <- SumPois[["coefficients"]][,1]
8 sig2 <- sum(SumPois[["deviance.resid"]]^2)/SumPois[["df.
  residual"]]
9 D <- diag(K2)
10 bs1 <- rnorm(N, 0, sd = D[1,1]^0.5)
11 bs2 <- rnorm(N, 0, sd = D[2,2]^0.5)
12 bs <- cbind(bs1, bs2); tuning <- 0.1; ropt <- 0.44
13 tunepriter <- seq(round(tot/10, 0), tot, round(tot/10, 0));
  l <- 1
14 pb <- txtProgressBar(min = 0, max = tot, style = 3)
15 for(s in 1:tot){
16   LatY <- LatentMHV1(tuning = tuning, Beta = Beta, bs = bs,
    sig2 = sig2)
17   ylhat <- LatY[["ylhat"]]
18   bs <- Postb(Beta = Beta, D = D, ylhat = ylhat, sig2 = sig2)
19   D <- PostD(bs = bs)
20   Beta <- PostBeta(D = D, ylhat = ylhat, sig2 = sig2)
21   sig2 <- PostSig2(Beta = Beta, bs = bs, ylhat = ylhat)
22   PostBetas[s,] <- Beta; PostDs[s,] <- matrixcalc::vech(D)
23   Postbs[, , s] <- bs; PostSig2s[s] <- sig2
24   AcceptRate <- LatY[["accept"]]; Accepts[s] <- AcceptRate
25   if(AcceptRate > ropt){
26     tuning = tuning*(2-(1-AcceptRate)/(1-ropt))
27   }else{
28     tuning = tuning/(2-AcceptRate/ropt)
29   }
30   if(s == tunepriter[l]){
31     print(AcceptRate)
32     l <- l + 1
33   }
34   setTxtProgressBar(pb, s)
35 }
36 close(pb)
37 keep <- seq((burnin+1), tot, thin)
38 Bs <- PostBetas[keep,]; Ds <- PostDs[keep,]
39 bs <- Postbs[, , keep]; sig2s <- PostSig2s[keep]
40 summary(coda::mcmc(Bs))
41 Quantiles for each variable:
42 2.5%      25%      50%      75%      97.5%
43 var1 -1.250036 -0.269211  0.1995811  0.69139  1.59234
44 var2 -0.001542  0.004852  0.0083935  0.01147  0.01815
45 var3 -0.484097 -0.331748 -0.2586891 -0.19011 -0.04787
46 var4 -0.023677  0.123336  0.1973927  0.26883  0.41314
47 var5 -0.196732 -0.071421 -0.0001036  0.06601  0.19356
48 var6 -0.815499 -0.657182 -0.5784964 -0.49969 -0.36882
49 var7  0.613990  0.799534  0.8982645  1.00161  1.18880

```

We see that males with good perception of health status visit less frequently the physicians, and women with bad perception of health status visit more frequently the physicians.



10

Bayesian model average

Solutions of Exercises

1. The Gaussian linear model specifies $\mathbf{y} = \alpha \mathbf{i}_N + \mathbf{X}_m \boldsymbol{\beta}_m + \boldsymbol{\mu}_m$ such that $\boldsymbol{\mu}_m \sim N(\mathbf{0}, \sigma^2 \mathbf{I}_n)$, and $\mathbf{X}_m$ does not have the column of ones. Assuming that $\pi(\sigma^2) \propto 1/\sigma^2$, $\pi(\alpha) \propto 1$, and $\boldsymbol{\beta}_m | \sigma^2 \sim N(\mathbf{0}_{k_m}, \sigma^2 (g_m \mathbf{X}_m^\top \mathbf{X}_m)^{-1})$.
 - Show that the conditional posterior distribution of $\boldsymbol{\beta}_m$ is $N(\boldsymbol{\beta}_{mn}, \sigma^2 \mathbf{B}_{mn})$, where $\boldsymbol{\beta}_{mn} = \mathbf{B}_{mn} \mathbf{X}_m^\top \mathbf{y}$ and $\mathbf{B}_{mn} = ((1 + g_m) \mathbf{X}_m^\top \mathbf{X}_m)^{-1}$.
 - Show that the marginal the marginal likelihood associated with model $\mathcal{M}_m$ is proportional to

$$p(\mathbf{y} | \mathcal{M}_m) \propto \left(\frac{g_m}{1 + g_m} \right)^{k_m/2} \left[(\mathbf{y} - \bar{y} \mathbf{i}_N)^\top (\mathbf{y} - \bar{y} \mathbf{i}_N) - \frac{1}{1 + g_m} (\mathbf{y}^\top \mathbf{P}_{X_m} \mathbf{y}) \right]^{-(N-1)/2},$$

where all parameter are indexed to model $\mathcal{M}_m$, $\mathbf{P}_{X_m} = \mathbf{X}_m (\mathbf{X}_m^\top \mathbf{X}_m)^{-1} \mathbf{X}_m^\top$ is the projection matrix on the space generated by the columns of $\mathbf{X}_m$, and $\bar{y}$ is the sample mean of $\mathbf{y}$.

Hint: Take into account that $\mathbf{i}_N^\top \mathbf{X}_m = \mathbf{0}_{k_m}$ due to all columns being centered with respect to their means.

Answer

The marginal likelihood of this model is

$$\begin{aligned} p(\mathbf{y}) &= \int_0^\infty \int_{R^K} \int_R \pi(\boldsymbol{\beta} | \sigma^2) \pi(\sigma^2) \pi(\alpha) p(\mathbf{y} | \boldsymbol{\beta}, \sigma^2, \alpha) d\alpha d\boldsymbol{\beta} d\sigma^2 \\ &\propto \int_0^\infty \int_{R^K} \int_R (\sigma^2)^{-k_m/2} |g_m \mathbf{X}_m^\top \mathbf{X}_m|^{1/2} \exp \left\{ -\frac{1}{2\sigma^2} (\boldsymbol{\beta}^\top (g_m \mathbf{X}_m^\top \mathbf{X}_m) \boldsymbol{\beta}) \right\} (\sigma^2)^{-1} \\ &\quad \times (\sigma^2)^{-N/2} \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y} - \alpha \mathbf{i}_N - \mathbf{X}_m \boldsymbol{\beta})^\top (\mathbf{y} - \alpha \mathbf{i}_N - \mathbf{X}_m \boldsymbol{\beta}) \right\} d\alpha d\boldsymbol{\beta} d\sigma^2. \end{aligned}$$

Taking into account that $(\mathbf{y} - \alpha \mathbf{i}_N - \mathbf{X}_m \boldsymbol{\beta})^\top (\mathbf{y} - \alpha \mathbf{i}_N - \mathbf{X}_m \boldsymbol{\beta}) = (\mathbf{y} - \mathbf{X}_m \boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}_m \boldsymbol{\beta}) + N(\alpha - \bar{y})^2 - N\bar{y}^2$,

$$\begin{aligned}
p(\mathbf{y}) &\propto \int_0^\infty \int_{R^K} (\sigma^2)^{-k_m/2} |g_m \mathbf{X}_m^\top \mathbf{X}_m|^{1/2} \exp \left\{ -\frac{1}{2\sigma^2} (\boldsymbol{\beta}^\top (g_m \mathbf{X}_m^\top \mathbf{X}_m) \boldsymbol{\beta}) \right\} (\sigma^2)^{-1} \\
&\quad \times (\sigma^2)^{-N/2} \exp \left\{ -\frac{1}{2\sigma^2} [(\mathbf{y} - \mathbf{X}_m \boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}_m \boldsymbol{\beta}) - N\bar{y}^2] \right\} \\
&\quad \times \int_R \exp \left\{ -\frac{1}{2\sigma^2} (N(\alpha - \bar{y})^2) \right\} d\alpha d\boldsymbol{\beta} d\sigma^2.
\end{aligned}$$

The last term is the kernel of a normal density function with mean $\bar{y}$ and variance σ^2/N , then

$$\begin{aligned}
p(\mathbf{y}) &\propto \int_0^\infty \int_{R^K} (\sigma^2)^{-k_m/2} |g_m \mathbf{X}_m^\top \mathbf{X}_m|^{1/2} \exp \left\{ -\frac{1}{2\sigma^2} (\boldsymbol{\beta}^\top (g_m \mathbf{X}_m^\top \mathbf{X}_m) \boldsymbol{\beta}) \right\} (\sigma^2)^{-1} \\
&\quad \times (\sigma^2)^{-N/2} (\sigma^2)^{1/2} \exp \left\{ -\frac{1}{2\sigma^2} [(\mathbf{y} - \mathbf{X}_m \boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}_m \boldsymbol{\beta}) - N\bar{y}^2] \right\} d\boldsymbol{\beta} d\sigma^2.
\end{aligned}$$

Collecting terms for $\boldsymbol{\beta}$, we have $(\mathbf{y} - \mathbf{X}_m \boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}_m \boldsymbol{\beta}) - N\bar{y}^2 + \boldsymbol{\beta}^\top (g_m \mathbf{X}_m^\top \mathbf{X}_m) \boldsymbol{\beta} = (\boldsymbol{\beta} - \boldsymbol{\beta}_{mn})^\top \mathbf{B}_{mn}^{-1} (\boldsymbol{\beta} - \boldsymbol{\beta}_{mn}) + \mathbf{y}^\top \mathbf{y} - N\bar{y}^2 - \boldsymbol{\beta}_{mn}^\top \mathbf{B}_{mn} \boldsymbol{\beta}_{mn}$ where $\boldsymbol{\beta}_{mn} = \mathbf{B}_{mn} \mathbf{X}^\top \mathbf{y}$ and $\mathbf{B}_{mn} = ((1 + g_m) \mathbf{X}^\top \mathbf{X})^{-1}$. Then,

$$\begin{aligned}
p(\mathbf{y}) &\propto \int_0^\infty (\sigma^2)^{-k_m/2} |g_m \mathbf{X}_m^\top \mathbf{X}_m|^{1/2} \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y}^\top \mathbf{y} - N\bar{y}^2 - \boldsymbol{\beta}_{mn}^\top \mathbf{B}_{mn} \boldsymbol{\beta}_{mn}) \right\} (\sigma^2)^{-1} \\
&\quad \times (\sigma^2)^{-N/2} (\sigma^2)^{1/2} \int_{R^K} \exp \left\{ -\frac{1}{2\sigma^2} (\boldsymbol{\beta} - \boldsymbol{\beta}_{mn})^\top \mathbf{B}_{mn}^{-1} (\boldsymbol{\beta} - \boldsymbol{\beta}_{mn}) \right\} d\boldsymbol{\beta} d\sigma^2.
\end{aligned}$$

The last term is the kernel of a multivariate normal density with mean $\boldsymbol{\beta}_{mn}$ and variance $\mathbf{B}_{mn}$ (proof of the first bullet). Then,

$$p(\mathbf{y}) \propto \int_0^\infty \left(\frac{g_m}{1 + g_m} \right)^{k_m/2} (\sigma^2)^{-(N-1)/2-1} \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y}^\top \mathbf{y} - N\bar{y}^2 - \boldsymbol{\beta}_{mn}^\top \mathbf{B}_{mn} \boldsymbol{\beta}_{mn}) \right\} d\sigma^2.$$

This is the kernel of an inverse-gamma density with parameters $\alpha_n = (N-1)/2$ and $\delta_n = \mathbf{y}^\top \mathbf{y} - N\bar{y}^2 - \boldsymbol{\beta}_{mn}^\top \mathbf{B}_{mn} \boldsymbol{\beta}_{mn}$, where $\mathbf{y}^\top \mathbf{y} - N\bar{y}^2 - \boldsymbol{\beta}_{mn}^\top \mathbf{B}_{mn} \boldsymbol{\beta}_{mn} = (\mathbf{y} - \mathbf{i}_N \bar{y})^\top (\mathbf{y} - \mathbf{i}_N \bar{y}) - (1 + g_m)^{-1} \mathbf{y}^\top (\mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top) \mathbf{y}$. Then,

$$p(\mathbf{y} | \mathcal{M}_m) \propto \left(\frac{g_m}{1 + g_m} \right)^{k_m/2} \left[(\mathbf{y} - \bar{y} \mathbf{i}_N)^\top (\mathbf{y} - \bar{y} \mathbf{i}_N) - \frac{1}{1 + g_m} (\mathbf{y}^\top \mathbf{P}_{X_m} \mathbf{y}) \right]^{-(N-1)/2},$$

2. Determinants of export diversification I

[20] use BMA to study the determinants of export diversification. Use the dataset *10ExportDiversificationHHI.csv* to perform BMA using the BIC approximation and MC3 with 10000 iterations to check if these two approaches agree.

Answer

The first aspect to note is that the BIC approximation is faster by far than the MC3 algorithm. We see from the results that the two approaches show that the most relevant variables to determine export diversification are *avgedu5* (primary education) and *avgnatres* (natural resources). See [20] for details of all variables. The model with the highest PMP using MC3 includes these two variables (PMP = 0.16), while the model with the highest PMP using the BIC approximation in addition to these two variables also includes Portugal former colony and population (PMP = 0.03). Both methods agree that export diversification increases with primary education, and decreases with natural resources.

R code. Determinants of export diversification

```

1 rm(list = ls()); set.seed(010101)
2 Data <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/10
  ExportDiversificationHHI.csv", sep = ",", header = TRUE,
  quote = "\"")
3 attach(Data)
4 y <- Data[,1]; X <- as.matrix(Data[, -1]); K <- dim(X)[2]
5 BMAglm <- BMA::bicreg(X, y, strict = FALSE, OR = 50)
6 summary(BMAglm)
7 BMAreg <- BMA::MC3.REG(y, X, num.its=10000)
8 Models <- unique(BMAreg[["variables"]])
9 nModels <- dim(Models)[1]
10 nVistModels <- dim(BMAreg[["variables"]])[1]
11 PMPmc3 <- NULL
12 for(m in 1:nModels){
13   idModm <- NULL
14   for(j in 1:nVistModels){
15     if(sum(Models[m,] == BMAreg[["variables"]][j,]) == K){
16       idModm <- c(idModm, j)
17     }else{
18       idModm <- idModm
19     }
20   }
21   PMPm <- sum(BMAreg[["post.prob"]][idModm])
22   PMPmc3 <- c(PMPmc3, PMPm)
23 }
24 PMPmc3
25 PIPmc3 <- NULL
26 for(k in 1:K){
27   PIPk <- sum(PMPmc3[which(Models[,k] == 1)])
28   PIPmc3 <- c(PIPmc3, PIPk)
29 }
30 plot(PIPmc3)
31 Meansmc3 <- matrix(0, nModels, K)
32 Varsmc3 <- matrix(0, nModels, K)
33 for(m in 1:nModels){
34   idXs <- which(Models[m,] == 1)
35   if(length(idXs) == 0){
36     Regm <- lm(y ~ 1)
37   }else{
38     Xm <- X[, idXs]
39     Regm <- lm(y ~ Xm)
40     SumRegm <- summary(Regm)
41     Meansmc3[m, idXs] <- SumRegm[["coefficients"]][-1,1]
42     Varsmc3[m, idXs] <- SumRegm[["coefficients"]][-1,2]^2
43   }
44 }
45 BMAmeansmc3 <- colSums(Meansmc3*PMPmc3)
46 BMAstdmc3 <- (colSums(PMPmc3*Varsmc3) + colSums(PMPmc3*(
  Meansmc3-matrix(rep(BMAmeansmc3, each = nModels),
  nModels, K))^2))^0.5
47 plot(BMAmeansmc3)
48 plot(BMAstdmc3)
49 Ratio <- BMAmeansmc3/BMAstdmc3
50 ResselMC3 <- as.data.frame(cbind(PIPmc3, Models[1,],
  BMAmeansmc3, BMAstdmc3, Ratio))
51 PMPmc3

```


3. Simulation exercise of the Markov chain Monte Carlo model composition continues

Program an algorithm to perform MC3 where the final S models are unique. Use the simulation setting of Section 10.2 of the book increasing the number of regressors to 40, this implies approximately 1.1×10^{12} models.

Answer

The following code shows how to perform BMA using MC3 with the consideration that all final S models should be different.

After running the algorithm 50000 ($\ll 2^{40}$) times, we can see that the PIP is 1 for variables x_1 , x_5 and x_{10} , which are the variables in the data generating process (population statistical model). However, we can see that variable x_{23} has a high PIP (0.49), this makes that the PMP of the model including x_1 , x_5 , x_{10} and x_{23} is the highest, followed by the model including x_1 , x_5 and x_{10} , which is the population statistical model. This highlights the relevance of performing BMA; selecting just one model based on the highest PMP would induce a mistake. Although selecting the median probability model would uncover the population statistical model.

Estimating the BMA mean shows that we get values very close to the population values, a remarkable results is all other BMA posterior means are close to 0, including the mean coefficient of x_{23} despite that its PIP is almost 0.5. We calculate the posterior ratio between the mean and standard deviation, and get values higher than 2 just for x_1 , x_5 and x_{10} , again given evidence for the data generating process.

R code. Markov chain Monte Carlo model composition

```

1 rm(list = ls()); set.seed(010101)
2 N <- 1000
3 K1 <- 6; K2 <- 4; K3 <- 30; K <- K1 + K2 + K3
4 X1 <- matrix(rnorm(N*K1,1,1), N, K1)
5 X2 <- matrix(rbinom(N*K2, 1, 0.5), N, K2)
6 X3 <- matrix(rnorm(N*K3,1,1), N, K3)
7 X <- cbind(X1, X2, X3); e <- rnorm(N, 0, 0.5)
8 B <- c(1,0,0,0,0.5,0,0,0,0,-0.7, rep(0, 30))
9 y <- 1 + X%B + e
10 LogMLfunt <- function(Model){
11   indr <- Model == 1
12   kr <- sum(indr)
13   if(kr > 0){
14     gr <- ifelse(N > kr^2, 1/N, kr^(-2))
15     Xr <- matrix(Xnew[, indr], ncol = kr)
16     # PX <- diag(N) - Xr%*%solve(t(Xr)%*%Xr)%*%t(Xr)
17     # s2pos <- c(t(y)%*%PX%*%y/(1 + gr) + gr*(t(y - mean(y))
18     # %*%(y - mean(y)))/(1 + gr))
19     PX <- Xr%*%solve(t(Xr)%*%Xr)%*%t(Xr)
20     s2pos <- c((t(y - mean(y))%*%(y - mean(y))) - t(y)%*%PX%
21     # %*%y/(1 + gr))
22     mllMod <- (kr/2)*log(gr/(1+gr))-(N-1)/2*log(s2pos)
23   }else{
24     gr <- ifelse(N > kr^2, 1/N, kr^(-2))
25     # PX <- diag(N)
26     # s2pos <- c(t(y)%*%PX%*%y/(1 + gr) + gr*(t(y - mean(y))
27     # %*%(y - mean(y)))/(1 + gr))
28     s2pos <- c((t(y - mean(y))%*%(y - mean(y))))
29     mllMod <- (kr/2)*log(gr/(1+gr))-(N-1)/2*log(s2pos)
30   }
31   return(mllMod)
32 }
33 Xnew <- apply(X, 2, scale); M <- 100
34 Models <- matrix(rbinom(K*M, 1, p = 0.5), ncol = K, nrow = M
35 + 800)
36 Models <- unique(Models)[1:M,]
37 mllnew <- sapply(1:M, function(s){LogMLfunt(matrix(Models[s
38 ], 1, K))})
39 oind <- order(mllnew, decreasing = TRUE)
40 mllnew <- mllnew[oind, ]
41 Models <- Models[oind, ]
42 iter <- 50000; s <- 1
43 pb <- txtProgressBar(min = 0, max = iter, style = 3)

```

*R code. Markov chain Monte Carlo model
composition*

```

1 while(s <= iter){
2   ActModel <- Models[M,]
3   idK <- which(ActModel == 1)
4   Kact <- length(idK)
5   Continue <- 0
6   while(Continue == 0){
7     if(Kact < K & Kact > 1){
8       CardMol <- K
9       opt <- sample(1:3, 1)
10      if(opt == 1){ # Same
11        CandModel <- ActModel
12      }else{
13        if(opt == 2){ # Add
14          All <- 1:K
15          NewX <- sample(All[-idK], 1)
16          CandModel <- ActModel
17          CandModel[NewX] <- 1
18        }else{ # Subtract
19          LessX <- sample(idK, 1)
20          CandModel <- ActModel
21          CandModel[LessX] <- 0
22        }
23      }
24    }else{
25      CardMol <- K + 1
26      if(Kact == K){
27        opt <- sample(1:2, 1)
28        if(opt == 1){ # Same
29          CandModel <- ActModel
30        }else{ # Subtract
31          LessX <- sample(1:K, 1)
32          CandModel <- ActModel
33          CandModel[LessX] <- 0
34        }
35      }else{
36        if(K == 1){
37          opt <- sample(1:3, 1)
38          if(opt == 1){ # Same
39            CandModel <- ActModel
40          }else{
41            if(opt == 2){ # Add
42              All <- 1:K
43              NewX <- sample(All[-idK], 1)
44              CandModel <- ActModel
45              CandModel[NewX] <- 1
46            }else{ # Subtract
47              LessX <- sample(idK, 1)
48              CandModel <- ActModel
49              CandModel[LessX] <- 0
50            }
51          }
52        }else{ # Add
53          NewX <- sample(1:K, 1)
54          CandModel <- ActModel
55          CandModel[NewX] <- 1
56        }
57      }
58    }

```

R code. Markov chain Monte Carlo model composition

```

1      check <- NULL
2      for(j in 1:M){
3          if(sum(Models[j,] == CandModel) == K){
4              checkj <- 0
5              check <- c(check, checkj)
6          }else{
7              checkj <- 1
8              check <- c(check, checkj)
9          }
10     }
11     dimUniModels <- sum(check)
12     if(dimUniModels == M){
13         Continue <- 1
14     }else{
15         Continue <- 0
16     }
17 }
18 LogMLact <- LogMLfunt(matrix(ActModel, 1, K))
19 LogMLcand <- LogMLfunt(matrix(CandModel, 1, K))
20 alpha <- min(1, exp(LogMLcand-LogMLact)); u <- runif(1)
21 if(u <= alpha){
22     mllnew[M] <- LogMLcand
23     Models[M, ] <- CandModel
24     oind <- order(mllnew, decreasing = TRUE)
25     mllnew <- mllnew[oind]
26     Models <- Models[oind, ]
27 }else{
28     mllnew <- mllnew
29     Models <- Models
30 }
31 s <- s + 1
32 setTxtProgressBar(pb, s)
33 }
34 close(pb)
35 ModelsUni <- unique(Models)
36 mllnewUni <- sapply(1:dim(ModelsUni)[1], function(s){
37     LogMLfunt(matrix(ModelsUni[s,], 1, K))})
38 StMarLik <- exp(mllnewUni-mllnewUni[1])
39 PMP <- StMarLik/sum(StMarLik)
40 PIP <- NULL
41 for(k in 1:K){
42     PIPk <- sum(PMP[which(ModelsUni[,k] == 1)])
43     PIP <- c(PIP, PIPk)
44 }
45 PIP

```

R code. Markov chain Monte Carlo model composition

```

1 Means <- matrix(0, M, K)
2 Vars <- matrix(0, M, K)
3 for(m in 1:M){
4   idXs <- which(ModelsUni[m,] == 1)
5   if(length(idXs) == 0){
6     Regm <- lm(y ~ 1)
7   }else{
8     Xm <- X[, idXs]
9     Regm <- lm(y ~ Xm)
10    SumRegm <- summary(Regm)
11    Means[m, idXs] <- SumRegm[["coefficients"]][-1,1]
12    Vars[m, idXs] <- SumRegm[["coefficients"]][-1,2]^2
13  }
14 }
15 BMMeans <- colSums(Means*PMP)
16 BMAsd <- (colSums(PMP*Vars) + colSums(PMP*(Means-matrix(rep
  (BMMeans, each = M), M, K))^2))^0.5
17 plot(BMMeans)
18 plot(BMAsd)
19 plot(BMMeans/BMAsd)

```

4. Simulation exercise of IV BMA continues

Use the simulation setting with endogeneity in Section 10.2 to perform BMA based on the BIC approximation and MC3.

Answer

The following code shows how to perform BMA using the BIC approximation and MC3 in this simulation setting with endogeneity. We see from the results that the BIC approximation and MC3 do a good job with the PMP and the PIP, as the data generating process gets the highest PMP using both approaches, and the PIPs of the variables in the data generating process are equal 1. The critical point is the BMA posterior means of the endogenous regressors, as these are far from the population values. The population values of x_{i1} and x_{i2} are 0.5 and -1, whereas the posterior means are 0.97 and -0.52.

R code. BIC and MC3 in model with endogeneity

```

1 rm(list = ls())
2 set.seed(010101)
3 simIV <- function(delta1,delta2,beta0,betas1,betas2,beta2,
4   Sigma,n,z) {
5   eps <- matrix(rnorm(3*n),ncol=3) %%% chol(Sigma)
6   xs1 <- z%%delta1 + eps[,1]
7   xs2 <- z%%delta2 + eps[,2]
8   x2 <- rnorm(dim(z)[1])
9   y <- beta0+betas1*xs1+betas2*xs2+beta2*x2 + eps[,3]
10  X <- as.matrix(cbind(xs1,xs2,1,x2))
11  colnames(X) <- c("x1en","x2en","cte","xex")
12  y <- matrix(y,dim(z)[1],1)
13  colnames(y) <- c("y")
14  list(X=X,y=y)
15 }
16 n <- 1000 ; p <- 3
17 z <- matrix(runif(n*p),ncol=p)
18 rho31 <- 0.8; rho32 <- 0.5;
19 Sigma <- matrix(c(1,0,rho31,0,1,rho32,rho31,rho32,1),ncol=3)
20 delta1 <- c(4,-1,2); delta2 <- c(-2,3,-1); betas1 <- .5;
21 betas2 <- -1; beta2 <- 1; beta0 <- 2
22 simiv <- simIV(delta1,delta2,beta0,betas1,betas2,beta2,Sigma,
23   n,z)
24 nW <- 18
25 W <- matrix(rnorm(nW*dim(z)[1]),dim(z)[1],nW)
26 YXW<-cbind(simiv$y, simiv$X, W)
27 y <- YXW[,1]; X <- YXW[,2:3]; W <- YXW[,-c(1:4)]
28 Xnew <- cbind(X, W)
29 BMAglm <- BMA::bicreg(Xnew, y, strict = FALSE, OR = 50)
30 summary(BMAglm)
31 BMAREg <- BMA::MC3.REG(y, Xnew, num.its=10000)
32 Models <- unique(BMAREg[["variables"]])
33 nModels <- dim(Models)[1]
34 nVistModels <- dim(BMAREg[["variables"]])[1]
35 K <- dim(Xnew)[2]
36 PMP <- NULL
37 for(m in 1:nModels){
38   idModm <- NULL
39   for(j in 1:nVistModels){
40     if(sum(Models[m,] == BMAREg[["variables"]][j,]) == K){
41       idModm <- c(idModm, j)
42     }else{
43       idModm <- idModm
44     }
45   }
46   PMPm <- sum(BMAREg[["post.prob"]][idModm])
47   PMP <- c(PMP, PMPm)
48 }
49 PMP
50 PIP <- NULL
51 for(k in 1:K){
52   PIPk <- sum(PMP[which(Models[,k] == 1)])
53   PIP <- c(PIP, PIPk)
54 }
55 plot(PIP)

```

R code. BIC and MC3 in model with endogeneity

```

1 Means <- matrix(0, nModels, K)
2 Vars <- matrix(0, nModels, K)
3 for(m in 1:nModels){
4   idXs <- which(Models[m,] == 1)
5   if(length(idXs) == 0){
6     Regm <- lm(y ~ 1)
7   }else{
8     Xm <- Xnew[, idXs]
9     Regm <- lm(y ~ Xm)
10    SumRegm <- summary(Regm)
11    Means[m, idXs] <- SumRegm[["coefficients"]][-1,1]
12    Vars[m, idXs] <- SumRegm[["coefficients"]][-1,2]^2
13  }
14 }
15 BMAMEANS <- colSums(Means*PMP)
16 BMASD <- (colSums(PMP*Vars) + colSums(PMP*(Means-matrix(rep
17   (BMAMEANS, each = nModels), nModels, K))^2))^0.5
18 plot(BMAMEANS)
19 plot(BMASD)
20 plot(BMAMEANS/BMASD)

```

The previous results are intuitive, as the PMP are calculated based on fit (and penalty for complexity), for instance, $BIC = k_m \log(N) - 2 \log(p(\hat{\theta}_m | \mathbf{y}))$, where $\hat{\theta}_m$ is the maximum likelihood estimator. Observe that the fit is not affected by endogeneity. Thus, the PMP are well calculated. However, the coefficients are not well identified.

This suggests a simple strategy to perform BMA taking into account endogeneity, calculate the PMPs using standard BMA approaches, for instance, BIC approximation, and then estimate the BMA means using these PMPs, but estimating the different models using instrumental variables. This approach is easily implemented using packages from **R**. The following code does this:

R code. Easy IV BMA

```

1 BMAglm <- BMA::bicreg(Xnew, y, strict = FALSE, OR = 50)
2 summary(BMAglm)
3 PMPBIC <- BMAglm[["postprob"]]
4 ModelsBIC <- BMAglm[["which"]]
5 nModels <- dim(ModelsBIC)[1]
6 K <- dim(Xnew)[2]
7 Means <- matrix(0, nModels, K)
8 Vars <- matrix(0, nModels, K)
9 for(m in 1:nModels){
10   idXs <- which(ModelsBIC[m,] == 1)
11   if(length(idXs) == 0){
12     Regm <- lm(y ~ 1)
13   }else{
14     Xm <- Xnew[, idXs]
15     Regm <- ivreg::ivreg(y ~ Xm | z + W)
16     SumRegm <- summary(Regm)
17     Means[m, idXs] <- SumRegm[["coefficients"]][-1,1]
18     Vars[m, idXs] <- SumRegm[["coefficients"]][-1,2]^2
19   }
20 }
21 BMAMEANS <- colSums(Means*PMPBIC)
22 BMASD <- (colSums(PMPBIC*Vars) + colSums(PMPBIC*(Means -
23   matrix(rep(BMAMEANS, each = nModels), nModels, K))^2))
24   ^0.5
25 BMAMEANS
26 5.589366e-01 -9.431664e-01 1.035593e+00 -1.967444e-05
27 -1.429801e-04 -3.310215e-04 4.036846e-04 4.651796e-03
28 -2.673730e-03 6.462503e-05 4.286529e-04 -1.829889e-04
29 5.073229e-03 -1.007356e-04 -5.377972e-04 1.644475e-03
30 -4.029205e-04 -1.457381e-04 6.421582e-04 2.211994e-04
31 2.216503e-04
32
33 plot(BMASD)
34 plot(BMAMEANS/BMASD)
35 17.764095075 -23.233601133 34.734610916 -0.005600369
36 -0.039168667 -0.071895067 0.080909448 0.274994618
37 -0.210854131 0.018790592 0.061271650 -0.047117887
38 0.305639285 -0.028830411 -0.092463544 0.169455952
39 -0.058299940 -0.038099792 0.102373567 0.056074670
40 0.055500343

```

We observe that the Easy IV BMA means of the endogenous regressors are 0.59 and -0.94, which are closer to the population values (0.5 and -1) than the exogenous BMA means (0.97 and -0.52) and align closely with the IV BMA means calculated using conditional Bayes factors (0.51 and -0.98; see Section 10.2 in the book). Additionally, the ratios between the Easy IV

BMA means and their standard deviations exceed 2 in absolute value for these variables, and the t-intervals encompass the population values

5. Determinants of export diversification II

Use the datasets *11ExportDiversificationHHI.csv* and *12ExportDiversificationHHIInstr.csv* to perform IV BMA assuming that the log of per capita gross domestic product is endogenous (*avglgdp*). See [20] for details.

Answer

The results show that primary education (*avgedu5*) and natural resources (*avgnatres*) have again the highest PIP, 0.77 and 0.92, respectively. The former variable increases export diversification, and the latter decreases export diversification. However, it seems that endogeneity is not a concern in this application, as the 95% credible interval of σ_{12} is (-0.014, 0.024).

R code. IV BMA in export diversification

```

1 rm(list = ls())
2 set.seed(010101)
3 DataMain <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/11
  ExportDiversificationHHI.csv", sep = ",", header = TRUE,
  quote = "")
4 DataInst <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/12
  ExportDiversificationHHIInstr.csv", sep = ",", header =
  TRUE, quote = "")
5 attach(DataMain)
6 attach(DataInst)
7 y <- DataMain[,1]
8 X <- as.matrix(DataMain[,2])
9 W <- as.matrix(DataMain[,c(1:2)])
10 Z <- as.matrix(DataInst)
11 S <- 10000; burnin <- 1000
12 regivBMA <- ivbma::ivbma(Y = y, X = X, Z = Z, W = W, s = S+
  burnin, b = burnin, odens = S, print.every = round(S/10)
  , run.diagnostics = FALSE)
13 PIPmain <- regivBMA[["L.bar"]] # PIP outcome
14 PIPmain
15 EVmain <- regivBMA[["rho.bar"]] # Posterior mean outcome
16 EVmain
17 PIPaux <- regivBMA[["M.bar"]] # PIP auxiliary
18 PIPaux
19 EVaux <- regivBMA[["lambda.bar"]] # Posterior mean auxiliary
20 plot(EVaux[,1])
21 EVsigma <- regivBMA[["Sigma.bar"]] # Posterior mean variance
  matrix
22 EVsigma
23 summary(coda::mcmc(regivBMA[["Sigma"]][1,2,]))
24 Iterations = 1:10000
25 Thinning interval = 1
26 Number of chains = 1
27 Sample size per chain = 10000
28 1. Empirical mean and standard deviation for each variable,
29 plus standard error of the mean:
30 Mean          SD          Naive SE Time-series SE
31 3.944e-03      9.592e-03      9.592e-05      4.043e-04
32 2. Quantiles for each variable:
33 2.5%          25%          50%          75%          97.5%
34 -0.014155     -0.002415     0.003592     0.009988     0.024047

```

6. Show that the link function in the case of the Bernoulli distribution is $\log\left(\frac{\theta}{1-\theta}\right)$.

Answer

$$\begin{aligned}
 p(\mathbf{y}|\theta) &= \theta^y(1-\theta)^{1-y} \\
 &= (1-\theta) \exp \left\{ y \log \left(\frac{\theta}{1-\theta} \right) \right\} \\
 &= \exp \left\{ y \log \left(\frac{\theta}{1-\theta} \right) + \log(1-\theta) \right\},
 \end{aligned}$$

then $\eta(\theta) = \log \left(\frac{\theta}{1-\theta} \right)$, and this density in the canonical form is $p(y|\eta) = \exp \{ y\eta - \log(1 + \exp(\eta)) \}$ consequently, $\mathbb{E}[Y|\mathbf{x}] = \nabla (\log(1 + \exp(\eta))) = \frac{\exp(\eta)}{1 + \exp(\eta)} = \theta = \frac{\exp(\mathbf{x}^\top \boldsymbol{\beta})}{1 + \exp(\mathbf{x}^\top \boldsymbol{\beta})}$. Then, the link function in the Bernoulli case is the *logit* function.

7. [39, 40] perform variable selection using the file *13InternetMed.csv*. In this data set, the dependent variable is an indicator of Internet adoption (internet) for 5000 households in Medellín (Colombia) during the period 2006–2014. This dataset contains information about 18 potential determinants, which means 262144 (2^{18}) potential models just taking into account variable uncertainty (see these papers for details about the data set). Perform BMA using the logit link function using this data set.

Answer

R code. BMA logit in internet adoption

```

1 rm(list = ls())
2 set.seed(010101)
3 DataMain <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/13InternetMed
  .csv", sep = ",", header = TRUE, quote = "")
4 attach(DataMain)
5 ### BIC approximation
6 y <- DataMain[,1]
7 X <- as.matrix(DataMain[,-1])
8 BMAGlm <- BMA::bic.glm(X, y, strict = FALSE, OR = 50, glm.
  family = binomial(link="logit"))
9 summary(BMAGlm)
10

```

The results show that the best model has a posterior model probability equal to 31%, and the second best model has a PMP equal to 30%. It seems that age, squared age, years of education of the head of the household, total expenses, having pay TV, any household member studying, and number of children in the household are relevant determinants of Internet adoption.

8. [47] use the file *14ValueFootballPlayers.csv* to analyze the market value of soccer players in the most important leagues in Europe. In particular, there are 26 potential determinants of the market value (dependent variable) of a stratified sample of 335 soccer players in the five most important leagues in Europe (see [47] for details). Use this data set to perform BMA using the gamma distribution with the log link function, and setting default values for Occam's window.

Answer

R code. BMA gamma in market value of soccer players in Europe

```

1 rm(list = ls())
2 set.seed(010101)
3 DataMain <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/14
  ValueFootballPlayers.csv", sep = ",", header = TRUE,
  quote = "")
4 attach(DataMain)
5 ### BIC approximation
6 y <- DataMain[,1]
7 X <- as.matrix(DataMain[,-1])
8 BMAGlm <- BMA::bic.glm(X, y, strict = FALSE, OR = 50, glm.
  family = Gamma(link="log"))
9 summary(BMAGlm)

```

The results show that performance, age, squared age, participation in national team, scored goals, and appearance in the UEFA champions league have PIPs equal 1. All these variables increase the market value of soccer players, except squared age.

9. Use the dataset *15Fertile2.csv* from [53, p. 547] to perform BMA using the Poisson model with the log link. This data set has information about 1,781 women from Botswana in 1988 (for details, see <https://rdrr.io/cran/wooldridge/man/fertil2.html>, and take into account that we deleted some variables and omitted observations with NA values). The dependent variable is the number of children ever born (ceb), which is a count variable, as a function of 19 potential determinants.

Answer

*R code. BMA Poisson in determinants of fertility
in Botswana*

```

1 rm(list = ls())
2 set.seed(010101)
3 DataMain <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/15Fertil2.csv",
  sep = ",", header = TRUE, quote = "")
4 attach(DataMain)
5 ### BIC approximation
6 y <- DataMain[,1]
7 X <- as.matrix(DataMain[,-1])
8 BMAGlm <- BMA::bic.glm(X, y, strict = FALSE, OR = 50, glm.
  family = poisson(link="log"))
9 summary(BMAGlm)

```

We found that the best model, for which the PMP is equal to 27%, has as regressors age, squared age, age at first birth, use birth control, husband's years of education, woman's years of education, and living in an urban area. The first five variables have PIPs equal to 100. For instance, we found that women using birth control have approximately 15% fewer children on average than women who are not using birth control.

10. Perform BMA in the logit model using MC3 and the BIC approximation using the simulation setting of Section 10.3.

Answer

R code. BMA Logit model

```

1 rm(list = ls()); set.seed(010101)
2 n<-1000; B<-c(0.5,0.8,-1.2)
3 X<-matrix(cbind(rep(1,n),rnorm(n,0,1),rnorm(n,0,1)),n,length
  (B))
4 p <- exp(X%*%B)/(1+exp(X%*%B))
5 y <- rbinom(n, 1, p); table(y)
6 nXgar<-25; Xgar<-matrix(rnorm(nXgar*n),n,nXgar)
7 df<-as.data.frame(cbind(y,X[,-1],Xgar))
8 colnames(df) <- c("y", "x1", "x2", "x3", "x4", "x5", "x6", "
  x7", "x8", "x9", "x10", "x11", "x12", "x13", "x14", "x15
  ", "x16", "x17", "x18", "x19", "x20", "x21", "x22", "x23
  ", "x24", "x25", "x26", "x27")
9 Xnew <- apply(df[,-1], 2, scale)
10 BICfunt <- function(Model){
11   indr <- Model == 1
12   kr <- sum(indr)
13   if(kr > 0){
14     Xr <- as.matrix(Xnew[ , indr])
15     model <- glm(y ~ Xr, family = binomial(link = "logit"))
16     model_bic <- BIC(model)
17     mllMod <- -model_bic/2
18   }else{
19     model <- glm(y ~ 1, family = binomial(link = "logit"))
20     model_bic <- BIC(model)
21     mllMod <- -model_bic/2
22   }
23   return(mllMod)
24 }
25 M <- 500; K <- dim(df)[2] - 1
26 Models <- matrix(rbinom(K*M, 1, p = 0.5), ncol = K, nrow = M
  )
27 mllnew <- sapply(1:M, function(s){BICfunt(matrix(Models[s,],
  1, K))})
28 oind <- order(mllnew, decreasing = TRUE)
29 mllnew <- mllnew[oind]; Models <- Models[oind, ]
30 iter <- 25000; s <- 1
31 pb <- txtProgressBar(min = 0, max = iter, style = 3)
32 while(s <= iter){
33   ActModel <- Models[M,]
34   idK <- which(ActModel == 1)
35   Kact <- length(idK)
36   if(Kact < K & Kact > 1){
37     CardMol <- K; opt <- sample(1:3, 1)
38     if(opt == 1){ # Same
39       CandModel <- ActModel
40     }else{
41       if(opt == 2){ # Add
42         All <- 1:K; NewX <- sample(All[-idK], 1)
43         CandModel <- ActModel; CandModel[NewX] <- 1
44       }else{ # Subtract
45         LessX <- sample(idK, 1)
46         CandModel <- ActModel; CandModel[LessX] <- 0
47       }
48     }
  }
}

```

R code. BMA Logit model

```

1   }else{
2   CardMol <- K + 1
3   if(Kact == K){
4     opt <- sample(1:2, 1)
5     if(opt == 1){ # Same
6       CandModel <- ActModel
7     }else{ # Subtract
8       LessX <- sample(1:K, 1)
9       CandModel <- ActModel
10      CandModel[LessX] <- 0
11    }
12  }else{
13    if(K == 1){
14      opt <- sample(1:3, 1)
15      if(opt == 1){ # Same
16        CandModel <- ActModel
17      }else{
18        if(opt == 2){ # Add
19          All <- 1:K
20          NewX <- sample(All[-idK], 1)
21          CandModel <- ActModel
22          CandModel[NewX] <- 1
23        }else{ # Subtract
24          LessX <- sample(idK, 1)
25          CandModel <- ActModel
26          CandModel[LessX] <- 0
27        }
28      }
29    }else{ # Add
30      NewX <- sample(1:K, 1)
31      CandModel <- ActModel
32      CandModel[NewX] <- 1
33    }
34  }
35  }
36  LogMLact <- BICfunt(matrix(ActModel, 1, K))
37  LogMLcand <- BICfunt(matrix(CandModel, 1, K))
38  alpha <- min(1, exp(LogMLcand-LogMLact)) # Let's reasonably
39  assume same prior model probability for candidate and
40  actual, and same carnality of neighbor models
41  u <- runif(1)
42  if(u <= alpha){
43    mllnew[M] <- LogMLcand
44    Models[M, ] <- CandModel
45    oind <- order(mllnew, decreasing = TRUE)
46    mllnew <- mllnew[oind]
47    Models <- Models[oind, ]
48  }else{
49    mllnew <- mllnew
50    Models <- Models
51  }
52  s <- s + 1
53  setTxtProgressBar(pb, s)
54  }
55  close(pb)
56  }

```

R code. BMA Logit model

```

1 ModelsUni <- unique(Models)
2 mllnewUni <- sapply(1:dim(ModelsUni)[1], function(s){BICfunt
  (matrix(ModelsUni[s,], 1, K))})
3 StMarLik <- exp(mllnewUni-mllnewUni[1])
4 PMP <- StMarLik/sum(StMarLik) # PMP based on unique selected
  models
5 plot(PMP)
6 ModelsUni[1,]
7 PIP <- NULL
8 for(k in 1:K){
9   PIPk <- sum(PMP[which(ModelsUni[,k] == 1)])
10  PIP <- c(PIP, PIPk)
11 }
12 plot(PIP)
13 Xnew <- df[,-1]
14 nModels <- dim(ModelsUni)[1]
15 Means <- matrix(0, nModels, K)
16 Vars <- matrix(0, nModels, K)
17 for(m in 1:nModels){
18   idXs <- which(ModelsUni[m,] == 1)
19   if(length(idXs) == 0){
20     Regm <- glm(y ~ 1, family = binomial(link = "logit"))
21   }else{
22     Xm <- as.matrix(Xnew[, idXs])
23     Regm <- glm(y ~ Xm, family = binomial(link = "logit"))
24     SumRegm <- summary(Regm)
25     Means[m, idXs] <- SumRegm[["coefficients"]][-1,1]
26     Vars[m, idXs] <- SumRegm[["coefficients"]][-1,2]^2
27   }
28 }
29 BMAMEANS <- colSums(Means*PMP)
30 BMASD <- (colSums(PMP*Vars) + colSums(PMP*(Means-matrix(rep
  (BMAMEANS, each = nModels), nModels, K))^2))^0.5
31 plot(BMAMEANS)
32 plot(BMASD)
33 plot(BMAMEANS/BMASD)

```

The simulation setting implies 2^{27} models, which implies approximately 135 million models in the model space. We run our MC3 algorithm using the BIC approximation with 25000 iterations. This takes by far more time than the BIC approximation from the *BMA* package, but it seems to do a good job finding the data generating process as the highest PMP is associated with it, the PIPs of x_{i1} and x_{i2} are 1, the posterior means are 0.85 and -1.1, which are very close to the population values (0.8 and -1.2), and the t-ratios are by far higher than 2. The PIPs of the other regressors are lower than 20%, and the BMA means are close to 0.

11. Use the dataset *19ExchangeRateCOPUSD.csv* to estimate four different *state-space models* to explain the annual variation of the COP to USD exchange rate:

- Interest rate parity

$$\Delta e_t = \beta_{1t}^{IRP} + \beta_{2t}^{IRP}(i_{t-1}^{Col} - i_{t-1}^{USA}) + \mu_t^{IRP}$$
- Purchasing power parity

$$\Delta e_t = \beta_{1t}^{PPP} + \beta_{2t}^{PPP}(\pi_{t-1}^{Col} - \pi_{t-1}^{USA}) + \mu_t^{PPP}$$
- Taylor rule

$$\Delta e_t = \beta_{1t}^{Taylor} + \beta_{2t}^{Taylor}(\pi_{t-1}^{Col} - \pi_{t-1}^{USA}) + \beta_{2t}^{Taylor}(g_{t-1}^{Col} - g_{t-1}^{USA}) + \mu_t^{IRP}$$
- Money supply

$$\Delta e_t = \beta_{1t}^{Money} + \beta_{2t}^{Money}(g_{t-1}^{Col} - g_{t-1}^{USA}) + \beta_{2t}^{Money}(m_{t-1}^{Col} - m_{t-1}^{USA}) + \mu_t^{Money}$$

where $varTRM(\Delta e_t)$ is the annual variation rate of the exchange rate COP to USD, $TES_COL10(i_t^{Col})$ and $TES_USA10(i_t^{USA})$ are the annual return rates of Colombian and USA public debts in 10 years, $inflation_COL(\pi_t^{Col})$ and $inflation_USA(\pi_t^{USA})$ are the annual inflation rates, $varISE_COL(g_t^{Col})$ and $varISE_USA(g_t^{USA})$ are annual variation of economic activity indices, and $varCOL_M3(m_t^{Col})$ and $varUSA_M3(m_t^{USA})$ are annual variations of money supply. We have monthly variations between January 2006 and November 2023.

Perform Bayesian model averaging using these four models to explain the annual variation of the exchange rate, get the posterior model probabilities, and plot the posterior mean and credible interval of β_{2t}^{Money} .

Answer

*R code. Dynamic Bayesian model average:
Theories to explain (forecast) exchange rate*

```

1 rm(list = ls()); set.seed(010101)
2 DataMain <- read.csv("https://raw.githubusercontent.com/
  besmarter/BSTApp/refs/heads/master/DataApp/19
  ExchangeRateCOPUSD.csv", sep = ",", header = TRUE, quote
    = "")
3 X <- DataMain[-c(1:12,227),c(4, 5, 7, 9, 11, 13, 14, 15)]
4 DifIntRat <- X[,8] - X[,7]; DifInf <- X[,1] - X[,2]
5 DifGrowth <- X[,3] - X[,4]; DifM3 <- X[,6] - X[,5]
6 XDif <- cbind(DifIntRat, DifInf, DifGrowth, DifM3)
7 matplot(XDif, type = "l")
8 y <- DataMain[-c(1:13),17] # Var exchange rate depends on
  lag of other variables
9 plot(y, type = "l")
10 IRP <- c(1, 0, 0, 0); PPP <- c(0, 1, 0, 0)
11 Taylor <- c(0, 1, 1, 0); Money <- c(0, 0, 1, 1)
12 Models <- rbind(IRP, PPP, Taylor, Money)
13 T0 <- 50
14 dma.test <- dma::dma(XDif, y, Models, lambda=.99, gamma=.99,
  initialperiod = T0)
15 plot(ts(dma.test[["pmp"]][,1], start = 2006, end = 2023,
  frequency = 12), type = "l", main = "Exchange rate
  models", ylab = "Posterior model probability", ylim = c
    (0,1)) lines(ts(dma.test[["pmp"]][,2], start = 2006, end
    = 2023, frequency = 12), col = "red") lines(ts(dma.test
    [["pmp"]][,3], start = 2006, end = 2023, frequency = 12)
    , col = "green") lines(ts(dma.test[["pmp"]][,4], start =
    2006, end = 2023, frequency = 12), col = "blue") legend
    (x = 2009, y = 0.6, legend = c("IRP", "PPP", "Taylor
    rule", "Money supply"), col = c("black", "red", "green",
    "blue"), lty=1:1, cex=0.8)
16 require(latex2exp)
17 plot_filtering_estimates <- function(df) {
18   p <- ggplot(data = df, aes(x = t)) + geom_ribbon(aes(ymin
    = lower, ymax = upper), alpha = 0.5, fill = "blue") +
    geom_line(aes(y = mean), colour = "black", linewidth =
    0.8) + geom_hline(yintercept=0, colour = "red") + ylab(
    TeX("$\\beta_{Growth_t}^{\\{Money\\}}")) + xlab("Time")
19   print(p)
20 }
21 menathetaIRP <- dma.test[["thetahat"]][4,,4]
22 SDthetaIRP <- (dma.test[["Vtheta"]][4,,4])^0.5
23 LimInf <- menathetaIRP - 2*SDthetaIRP
24 LimSup <- menathetaIRP + 2*SDthetaIRP
25 library(ggplot2,tibble,dplyr)
26 df <- tibble(t = 1:214, mean = menathetaIRP, lower = LimInf,
  upper = LimSup)
27 Fig <- plot_filtering_estimates(df)

```

Figure 10.1 shows the posterior model probabilities. We see that at the beginning, between 2006 and 2007, the purchasing power parity theory (PPP) had the highest PMP (70% on average) to explain the annual variations of the COP/USD exchange rate. Then, the money supply theory (Money) got the highest PMP (80% on average) between 2008 and 2017. Finally, the interest rate parity got the highest PMP (80% on average) between 2018 and 2023.

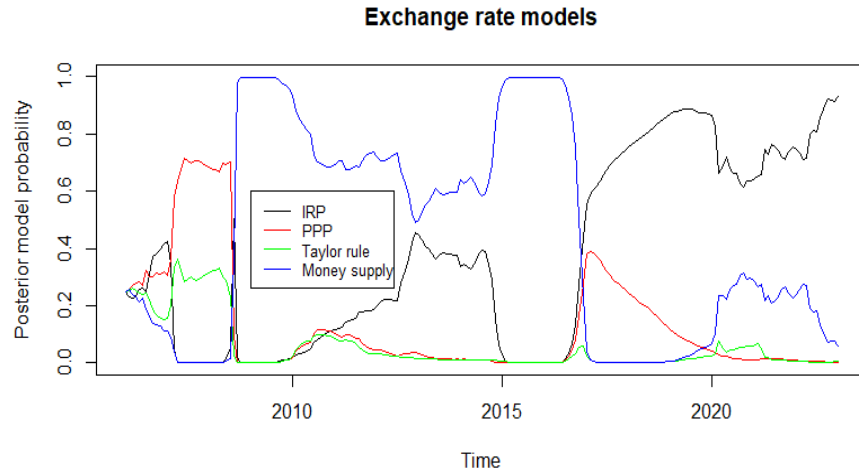
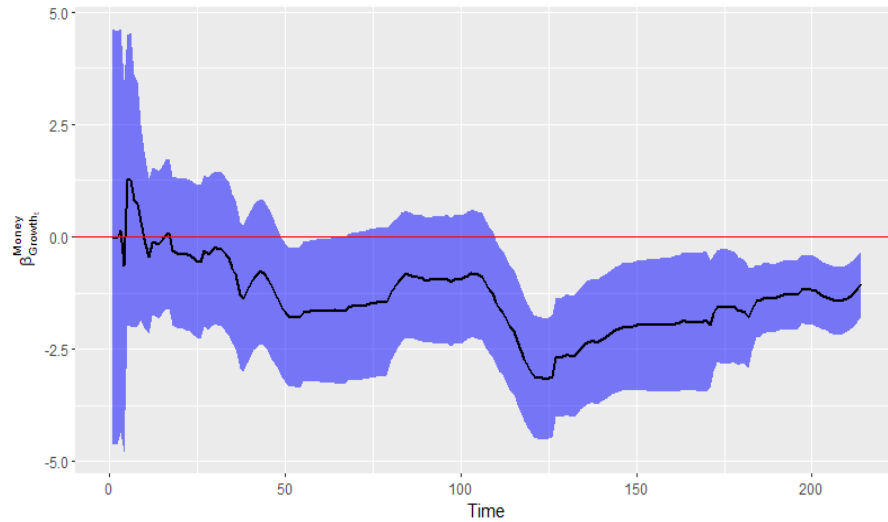


FIGURE 10.1

Dynamic posterior model probability: Models to explain variations of the COP to USD exchange rate.

Figure 10.2 shows the dynamic posterior mean (black line) and credible interval (blue shadow) of the coefficient related to the differential of economic growths (Colombia vs the USA) in the money supply equation to explain the annual variations of the exchange rate. We observe that increasing this economic growth differential implies appreciation of the COP relatively to USD.

12. Perform a simulation of the dynamic logistic model, where there are 7 ($2^3 - 1$, the model without regressors is excluded) competing models originated from 3 regressors, $x_{tk} \sim N(0.5, 0.8^2)$, $k = 2, 3, 4$, and $\beta_1 = 0.5$, β_{2t} is a sequence from 1 to 2 in steps given by $1/T$, $\beta_{3t} = \begin{cases} -1, & 1 < t \leq 0.5T \\ 0, & 0.5T < t \leq T \end{cases}$, and $\beta_4 = 1.2$. Then, $\mathbf{x}_t^\top \boldsymbol{\beta}_t = \beta_1 + \beta_{2t}x_{2t} + \beta_{3t}x_{3t} + \beta_4x_{4t}$, where $P[Y_t = 1 | \mathbf{x}_t, \boldsymbol{\beta}_t] = \exp(\mathbf{x}_t^\top \boldsymbol{\beta}_t) / (1 + \exp(\mathbf{x}_t^\top \boldsymbol{\beta}_t))$, $t = 1, 2, \dots, 1100$. Use the function *logistic.dma* from the *dma* package to get the posterior model probabilities

**FIGURE 10.2**

State coefficient money supply theory: Differential of economic growths (Colombia vs the USA).

setting the forgetting parameter of the models equal to 0.99, and then to 0.95. Compare the results.

Answer

The following code shows the simulation exercise, and the results of the dynamic Bayesian model average of logistic models.

R code. Dynamic Bayesian model average: Logit model

```

1 rm(list = ls()); set.seed(010101)
2 T <- 1100; K <- 3
3 X <- matrix(rnorm(T*K, mean = 0.5, sd = 0.8), T, K)
4 combs <- expand.grid(c(0,1), c(0,1), c(0,1))
5 B1 <- 0.5; B2t <- seq(1, 2, length.out=T)
6 a <- 0.5
7 B3t <- c(rep(-1, round(a*T)), rep(0, round((1-a)*T)))
8 B4 <- 1.2
9 y1 <- B1 + X[,1]*B2t + X[,2]*B3t + X[,3]*B4
10 py <- exp(y1)/(1 + exp(y1))
11 y <- rbinom(T, 1, prob = py)
12 table(y)
13 T0 <- 100
14 dma.test <- dma::logistic.dma(X, y, combs[-1,], lambda =
    0.99, alpha = 0.99, initialsamp = T0)
15 plot(dma.test[["pmp"]][7, -c(1:T0)], type = "l", col = "
    green", main = "Posterior model probability: Model all
    regressors vs model regressors 1 and 3", xlab = "Time",
    ylab = "PMP", ylim = c(0, 1))
16 lines(dma.test[["pmp"]][5, -c(1:T0)], col = "red")
17 legend(x = 0, y = 0.9, legend = c("Model: All regressors", "
    Model: Regressors 1 and 3"), col = c("green", "red"),
    lty=1:1, cex=0.8)
18 dma.test1 <- dma::logistic.dma(X, y, combs[-1,], lambda =
    0.99, alpha = 0.95, initialsamp = T0)
19 plot(dma.test1[["pmp"]][7, -c(1:T0)], type = "l", col = "
    green", main = "Posterior model probability: Model all
    regressors vs model regressors 1 and 3", xlab = "Time",
    ylab = "PMP", ylim = c(0, 1))
20 lines(dma.test1[["pmp"]][5, -c(1:T0)], col = "red")
21 legend(x = 0, y = 0.9, legend = c("Model: All regressors", "
    Model: Regressors 1 and 3"), col = c("green", "red"),
    lty=1:1, cex=0.8)

```

Figure 10.3 shows the posterior model probabilities of the model with all the regressors (green line), and the model with regressors 2 and 4 (red line), setting the forgetting parameter of the models equal to 0.99. In one hand, we see that the model with all regressors, which is the data generating process in the first period ($t \leq 0.5T$), gets high posterior model probabilities. We see that the PMPs decrease around $T = 700$, whereas the change in the data generating process is at $T = 550$. This means that there is a lack to identify the data generating process, which is a consequence of the high forgetting parameter of the models. Consequently, the PMPs of the model with regressors 2 and 4 increases around $T = 700$, but it is lower than the

PMPs of the model with all regressors, despite that in this time period the data generating process is given by the model with regressors 2 and 4.

Figure 10.4 shows the posterior model probabilities using a forgetting parameter equal to 0.95. We can see that as the forgetting parameter decreases, the process of identifying the data generating process is faster, and consequently, the PMP of the model with regressors 2 and 4 increases around $T = 550$, which is the turning point in the data generating process. This does not mean that we should set small forgetting parameters. These parameters should be set based on model performance, for instance, small mean squared error in a cross validation period.

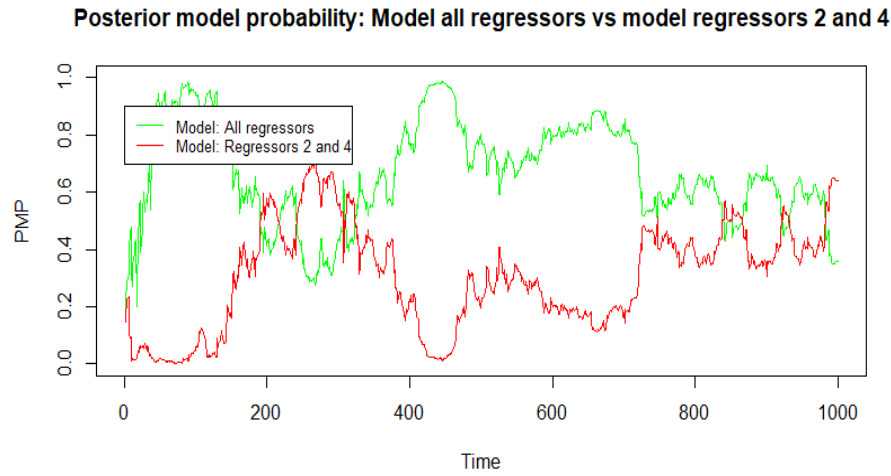


FIGURE 10.3

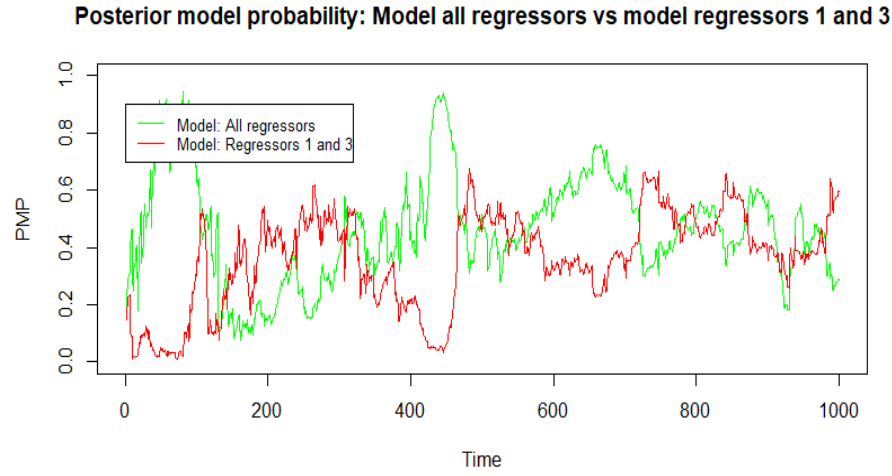
Posterior model probability: Dynamic Bayesian model average, forgetting parameter equal to 0.99.

13. Show that

$$\mathbb{E} \left[\frac{q(\boldsymbol{\theta})}{\pi(\boldsymbol{\theta}|\mathcal{M}_m)p(\mathbf{y}|\boldsymbol{\theta}_m, \mathcal{M}_m)} \middle| \mathbf{y}, \mathcal{M}_m \right] = \frac{1}{p(\mathbf{y}|\mathcal{M}_m)},$$

where the expected value is with respect to the posterior distribution given the model $\mathcal{M}_m$, and $q(\boldsymbol{\theta})$ is the proposal distribution whose support is $\boldsymbol{\Theta}$.

Answer

**FIGURE 10.4**

Posterior model probability: Dynamic Bayesian model average, forgetting parameter equal to 0.95.

Given a probability density function $q(\theta)$, whose support is in Θ , then

$$\begin{aligned}
 \mathbb{E} \left[\frac{q(\theta)}{\pi(\theta|\mathcal{M}_m)p(\mathbf{y}|\theta_m, \mathcal{M}_m)} \middle| \mathbf{y}, \mathcal{M}_m \right] &= \int_{\Theta} \left[\frac{q(\theta)}{\pi(\theta|\mathcal{M}_m)p(\mathbf{y}|\theta_m, \mathcal{M}_m)} \right] \pi(\theta|\mathbf{y}, \mathcal{M}_m) d\theta \\
 &= \int_{\Theta} \left[\frac{q(\theta)}{\pi(\theta|\mathcal{M}_m)p(\mathbf{y}|\theta_m, \mathcal{M}_m)} \right] \frac{\pi(\theta|\mathcal{M}_m)p(\mathbf{y}|\theta_m, \mathcal{M}_m)}{p(\mathbf{y}|\mathcal{M}_m)} d\theta \\
 &= \frac{1}{p(\mathbf{y}|\mathcal{M}_m)} \int_{\Theta} q(\theta) d\theta \\
 &= \frac{1}{p(\mathbf{y}|\mathcal{M}_m)}.
 \end{aligned}$$



Part III

Advanced methods: A brief introduction



11

Semi-parametric and non-parametric models

Solutions of Exercises

1. Simulate a semi-parametric regression where

$$y_i = 0.5x_{i1} - 1.2x_{i2} + \mu_i,$$
$$p(\mu_i) = 0.7\phi(\mu_i \mid -0.5, 0.5^2) + 0.3\phi(\mu_i \mid 1, 0.8^2).$$

Assume that x_{i1} and x_{i2} follow a standard normal distribution and that the sample size is 1,000. Perform inference in this model assuming that the number of components is unknown. Start with $H = 5$ and use non-informative priors, setting $\alpha_{h0} = \delta_{h0} = 0.01$, $\beta_0 = \mathbf{0}_2$, $\mathbf{B}_0 = \mathbf{I}_2$, $\mu_{h0} = 0$, $\sigma_{h0}^2 = 10$, and $\boldsymbol{\alpha}_0 = [1/H \ \dots \ 1/H]^\top$. Use 6,000 MCMC iterations, a burn-in period of 4,000, and a thinning parameter of 2. Compare the population parameters with the posterior estimates and plot the population density along with the posterior density estimate of $\boldsymbol{\mu}$ (the mean, and the 95% credible interval).

Answer

We can see from the posterior estimates that three components disappear after the burn-in iterations. The 95% credible intervals encompass the population values of all parameters, and the posterior density estimate closely matches the population density. See Figure 11.1, where the black line represents the population density, the blue line denotes the posterior mean density estimate, and the shaded light blue area corresponds to the 95% credible interval.

R code. Simulation: Semi-parametric Gaussian mixture

```

1 rm(list = ls()); set.seed(010101)
2 # Simulate data from a 2-component mixture model
3 n <- 1000
4 x1 <- rnorm(n); x2 <- rnorm(n)
5 B <- c(0.5, -1.2); X <- cbind(x1, x2)
6 z <- rbinom(n, 1, 0.3) # Latent class indicator
7 mu <- ifelse(z == 0, rnorm(n, -0.5, 0.5), rnorm(n, 1, 0.8))
8 plot(density(mu))
9 y <- X%*%B + mu
10 # Hyperparameters
11 d0 <- 0.001; a0 <- 0.001
12 b0 <- rep(0, 2); B0 <- diag(2); B0i <- solve(B0)
13 mu0 <- 0; sig2mu0 <- 10
14 H <- 5; a0h <- rep(1/H, H)
15 # MCMC parameters
16 mcmc <- 2000
17 burnin <- 4000
18 tot <- mcmc + burnin
19 thin <- 2
20 # Gibbs sampling functions
21 PostSig2 <- function(Beta, muh, Xh, yh){
22   Nh <- length(yh)
23   an <- a0 + Nh
24   dn <- d0 + t(yh - muh - Xh%*%Beta)%*(yh - muh - Xh%*%Beta)
25   sig2 <- invgamma::rinvgamma(1, shape = an/2, rate = dn/2)
26   return(sig2)
27 }
28 PostBeta <- function(sig2, mu, X, y, Psi){
29   XtX <- matrix(0, 2, 2)
30   Xty <- matrix(0, 2, 1)
31   Hs <- length(mu)
32   for(h in 1:Hs){
33     idh <- which(Psi == h)
34     if(length(idh) == 1){
35       Xh <- matrix(X[idh,], 1, 2)
36       XtXh <- sig2[h]^(-1)*t(Xh)%*%Xh
37       yh <- y[idh]
38       Xtyh <- sig2[h]^(-1)*t(Xh)%*(yh - mu[h])
39     }else{
40       Xh <- X[idh,]
41       XtXh <- sig2[h]^(-1)*t(Xh)%*%Xh
42       yh <- y[idh]
43       Xtyh <- sig2[h]^(-1)*t(Xh)%*(yh - mu[h])
44     }
45     XtX <- XtX + XtXh
46     Xty <- Xty + Xtyh
47   }
48   Bn <- solve(B0i + XtX)
49   bn <- Bn%*(B0i%*%b0 + Xty)
50   Beta <- MASS::mvrnorm(1, bn, Bn)
51   return(Beta)
52 }

```

R code. Simulation: Semi-parametric Gaussian mixture

```

1 Postmu <- function(sig2h, Beta, Xh, yh){
2   Nh <- length(yh)
3   sig2mu <- (1/sig2mu0 + Nh/sig2h)^(-1)
4   mun <- sig2mu*(mu0/sig2mu0 + sum((yh - Xh%*Beta))/sig2h)
5   mu <- rnorm(1, mun, sig2mu^0.5)
6   return(mu)
7 }
8 PostBetas <- matrix(0, mcmc+burnin, 2)
9 PostPsi <- matrix(0, mcmc+burnin, n)
10 PostSigma2 <- list(); PostMu <- list()
11 PostLambda <- list(); Reg <- lm(y ~ X)
12 Beta <- Reg$coefficients[2:3]; Res <- Reg$residuals
13 plot(density(Res))
14 Resq <- quantile(Res, c(0.2, 0.4, 0.6, 0.8))
15 Id1 <- which(Res <= Resq[1])
16 Id2 <- which(Res > Resq[1] & Res <= Resq[2])
17 Id3 <- which(Res > Resq[2] & Res <= Resq[3])
18 Id4 <- which(Res > Resq[3] & Res <= Resq[4])
19 Id5 <- which(Res > Resq[4])
20 Nh <- rep(n/H, H); Lambda <- rep(1/H, H)
21 MU <- c(mean(Res[Id1]), mean(Res[Id2]), mean(Res[Id3]), mean(
  Res[Id4]), mean(Res[Id5]))
22 Sig2 <- c(var(Res[Id1]), var(Res[Id2]), var(Res[Id3]), var(
  Res[Id4]), var(Res[Id5]))
23 Psi <- rep(NA, n); Hs <- length(MU)
24 pb <- txtProgressBar(min = 0, max = tot, style = 3)
25 for(s in 1:tot){
26   for(i in 1:n){
27     lambdai <- NULL
28     for(h in 1:Hs){
29       lambdaih <- Lambda[h]*dnorm(y[i] - X[i,]%*Beta, MU[h]
30       ], Sig2[h]^0.5)
31       lambdai <- c(lambdai, lambdaih)
32     }
33     Psi[i] <- sample(1:Hs, 1, prob = lambdai)
34   }
35   PostPsi[s, ] <- Psi
36   Hs <- length(table(Psi))
37   for(h in 1:Hs){
38     idh <- which(Psi == h)
39     Sig2[h] <- PostSig2(Beta = Beta, muh = MU[h], Xh = X[idh
40     ], yh = y[idh])
41     MU[h] <- Postmu(sig2h = Sig2[h], Beta = Beta, Xh = X[idh
42     ], yh = y[idh])
43   }
44   PostSigma2[[s]] <- Sig2
45   PostMu[[s]] <- MU
46   Beta <- PostBeta(sig2 = Sig2, mu = MU, X = X, y = y, Psi =
47   Psi)
48   PostBetas[s,] <- Beta
49   Lambda <- sort(MCMCpack::rdirichlet(1, a0h[1:Hs] + table(
50   Psi)), decreasing = TRUE)
51   PostLambda[[s]] <- Lambda
52   setTxtProgressBar(pb, s)
53 }
54 close(pb)

```

R code. Simulation: Semi-parametric Gaussian mixture

```

1 keep <- seq(burnin, tot, thin)
2 PosteriorBetas <- coda::mcmc(PostBetas[keep,])
3 summary(PosteriorBetas); plot(PosteriorBetas)
4 PosteriorPsi <- PostPsi[keep,]
5 Clusters <- sapply(1:length(keep), function(i){length(table(
  PosteriorPsi[i,]))})
6 PosteriorSIGMA <- matrix(NA, length(keep), 2)
7 PosteriorMU <- matrix(NA, length(keep), 2)
8 PosteriorLAMBDA <- matrix(NA, length(keep), 2)
9 l <- 1
10 for (s in keep){
11   PosteriorSIGMA[l,] <- PostSigma2[[s]][1:2]
12   PosteriorMU[l,] <- PostMu[[s]][1:2]
13   PosteriorLAMBDA[l,] <- PostLambda[[s]][1:2]
14   l <- l + 1
15 }
16 summary(coda::mcmc(PosteriorSIGMA))
17 summary(coda::mcmc(PosteriorMU))
18 summary(coda::mcmc(PosteriorLAMBDA))
19 mus <- seq(-3, 4, 0.01)
20 DensityEst <- function(par, eval){
21   lambda1 <- par[1]; lambda2 <- par[2]
22   mu1 <- par[3]; mu2 <- par[4]
23   sd1 <- par[5]; sd2 <- par[6]
24   Dens <- lambda1 * dnorm(eval, mu1, sd1) + lambda2 * dnorm(
    eval, mu2, sd2)
25   return(Dens)
26 }
27 par <- cbind(PosteriorLAMBDA, PosteriorMU, PosteriorSIGMA
  ^0.5)
28 DensEval <- matrix(NA, length(keep), length(mus))
29 DensPop <- rep(NA, length(mus))
30 for(r in 1:length(mus)){
31   for(l in 1:length(keep)){
32     DensEval[l, r] <- DensityEst(par = par[l,], eval = mus[r]
  )
33   }
34   DensPop[r] <- DensityEst(par = c(0.7, 0.3, -0.5, 1, 0.5,
    0.8), eval = mus[r])
35 }
36 library(dplyr); library(ggplot2); require(latex2exp)
37 DataDens <- tibble(t = mus, Pop = DensPop, lower = apply(
  DensEval, 2, quantile, probs = 0.025), upper = apply(
  DensEval, 2, quantile, probs = 0.975), meanT = colMeans(
  DensEval))
38 plot_filtering_estimates <- function(df) {
39   p <- ggplot(data = df, aes(x = t)) + geom_ribbon(aes(ymin
    = lower, ymax = upper), alpha = 1, fill = "lightblue")
    + geom_line(aes(y = Pop, color = "True Value"),
    linewidth = 0.5) + geom_line(aes(y = meanT, color = "
    Estimate"), linewidth = 0.5) +
40   scale_color_manual(values = c("True Value" = "black", "
    Estimate" = "blue")) + xlab(TeX("$\\mu$")) + ylab("
    Density") + labs(title = "Density: Stochastic Errors",
    color = "") + theme_minimal()
41   print(p)
42 }
43 plot_filtering_estimates(DataDens)

```

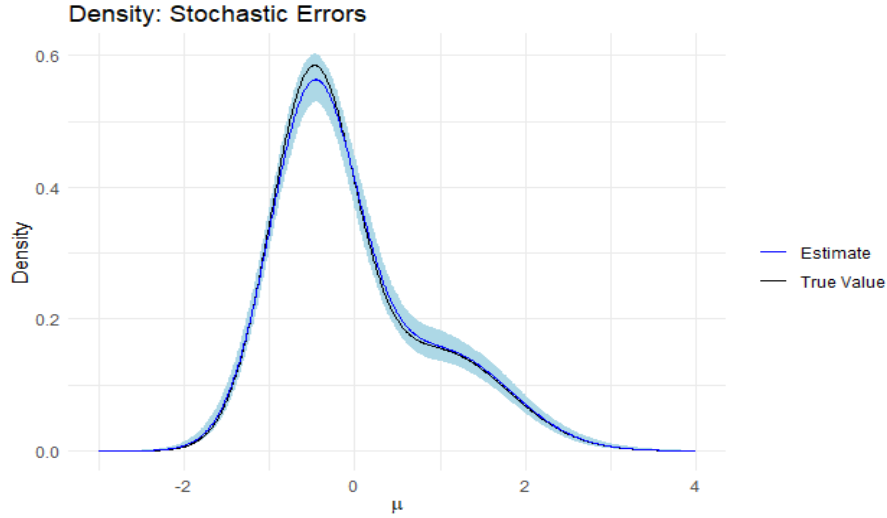


FIGURE 11.1

Posterior density estimate: Stochastic errors.

2. Use the dataset *MarijuanaColombia.csv* from our GitHub repository to perform inference on the demand for marijuana in Colombia. This dataset contains information on the (log) monthly demand in 2019 from the National Survey of the Consumption of Psychoactive Substances. It includes variables such as the presence of a drug dealer in the neighborhood (*Dealer*), gender (*Female*), indicators of good physical and mental health (*PhysicalHealthGood* and *MentalHealthGood*), age (*Age* and *Age2*), years of schooling (*YearsEducation*), and (log) prices of marijuana, cocaine, and crack by individual (*LogPriceMarijuana*, *LogPriceCocaine*, and *LogPriceCrack*). The sample size is 1,156.

Estimate a finite Gaussian mixture regression using non-informative priors, that is, $\alpha_0 = \delta_0 = 0.01$, $\beta_0 = \mathbf{0}_K$, $\mathbf{B}_0 = \mathbf{I}_K$, and $\alpha_0 = [1/H \dots 1/H]^\top$, K is the number of regressors, 11 including the intercept. The number of MCMC iterations is 5,000, the burn-in is 1,000, and the thinning parameter is 2. Start with five potential clusters. Obtain the posterior distribution of the own-price elasticity of marijuana and the cross-price elasticities of marijuana demand with respect to the prices of cocaine and crack.

Answer:

The following code demonstrates how to perform inference in this model. The results indicate the presence of four clusters, with mean probabilities of 0.64, 0.14, 0.12 and 0.10, and 95% credible intervals of (0.60, 0.67), (0.12, 0.16), (0.10, 0.14) and (0.08, 0.12), respectively.

The 95% posterior credible interval for the own-price elasticities of the most representative group, which comprises two-thirds of the sample, is $(-0.55, -0.26)$. Additionally, the 95% credible interval for the cross-price elasticities with respect to cocaine is $(0.01, 0.35)$, while that with respect to crack is $(0.07, 0.43)$.

These results suggest that marijuana is an inelastic good for these individuals and that there is substitution between marijuana and both cocaine and crack: an increase in the prices of the latter leads to higher marijuana demand in the most representative cluster of individuals. However, these findings should be interpreted with caution due to potential endogeneity issues that must be addressed for a rigorous scientific analysis. The primary goal of this exercise is pedagogical.

R code. Marijuana in Colombia: Gaussian mixture

```

1 rm(list = ls()); set.seed(010101)
2 Data <- read.csv("https://raw.githubusercontent.com/
   BEsmarter-consultancy/BSTApp/refs/heads/master/DataApp/
   MarijuanaColombia.csv")
3 attach(Data)
4 y <- LogMarijuana; X <- as.matrix(cbind(1, Data[, -1]))
5 Reg <- lm(y ~ X - 1); summary(Reg)
6 k <- dim(X)[2]; N <- dim(X)[1]
7 # Plot
8 library(ggplot2)
9 ggplot(Data, aes(x = LogMarijuana)) + geom_density(fill = "
   blue", alpha = 0.3) + labs(title = "Density Plot", x = "
   y", y = "Density") + theme_minimal()
10 # Hyperparameters
11 d0 <- 0.001; a0 <- 0.001
12 b0 <- rep(0, k); B0 <- diag(k); B0i <- solve(B0)
13 H <- 5; a0h <- rep(1/H, H)
14 # MCMC parameters
15 mcmc <- 2000; burnin <- 500
16 tot <- mcmc + burnin; thin <- 2
17 # Gibbs sampling functions
18 PostSig2 <- function(Betah, Xh, yh){
19   Nh <- length(yh); an <- a0 + Nh
20   dn <- d0 + t(yh - Xh%*%Betah)%*(yh - Xh%*%Betah)
21   sig2 <- invgamma::rinvgamma(1, shape = an/2, rate = dn/2)
22   return(sig2)
23 }
24 PostBeta <- function(sig2h, Xh, yh){
25   if(length(yh) == 1){
26     Xh <- matrix(Xh, k, 1)
27     Bn <- solve(B0i + sig2h^(-1)*Xh%*%t(Xh))
28     bn <- Bn%*(B0i%*b0 + sig2h^(-1)*Xh%*%yh)
29   }else{
30     Bn <- solve(B0i + sig2h^(-1)*t(Xh)%*%Xh)
31     bn <- Bn%*(B0i%*b0 + sig2h^(-1)*t(Xh)%*%yh)
32   }
33   Beta <- MASS::mvrnorm(1, bn, Bn)
34   return(Beta)
35 }
36 PostBetas <- list(); PostSigma2 <- list(); PostLambda <-
   list()
37 PostPsi <- matrix(0, tot, N); Psi <- sample(1:H, N, replace
   = TRUE, prob = a0h)
38 Beta <- matrix(rep(Reg$coefficients, H), k, H); Hs <- H

```

R code. Marijuana in Colombia: Gaussian mixture

```

1 pb <- txtProgressBar(min = 0, max = tot, style = 3)
2 for(s in 1:tot){
3   Sig2 <- rep(NA, Hs)
4   for(h in 1:Hs){
5     idh <- which(Psi == h)
6     Sig2[h] <- PostSig2(Betah = Beta[, h], Xh = X[idh,], yh
7       = y[idh])
8   }
9   Hs <- length(Sig2)
10  Beta <- matrix(NA, k, Hs)
11  for(h in 1:Hs){
12    idh <- which(Psi == h)
13    Beta[, h] <- PostBeta(sig2h = Sig2[h], Xh = X[idh,], yh
14      = y[idh])
15  }
16  Lambda <- sort(MCMCpack::rdirichlet(1, a0h[1:Hs] + table(
17    Psi)), decreasing = TRUE)
18  for(i in 1:N){
19    lambdai <- NULL
20    for(h in 1:Hs){
21      idh <- which(Psi == h)
22      lambdaih <- Lambda[h]*dnorm(y[i], X[i,]%*%Beta[,h],
23        Sig2[h]^0.5)
24      lambdai <- c(lambdai, lambdaih)
25    }
26    Psi[i] <- sample(1:Hs, 1, prob = lambdai)
27  }
28  Hs <- length(table(Psi))
29  PostBetas[[s]] <- Beta; PostSigma2[[s]] <- Sig2
30  PostLambda[[s]] <- Lambda; PostPsi[s, ] <- Psi
31  setTxtProgressBar(pb, s)
32 }
33 close(pb)
34 keep <- seq((burnin+1), tot, thin)
35 PosteriorPsi <- PostPsi[keep,]
36 Clusters <- sapply(1:length(keep), function(i){length(table(
37   PosteriorPsi[i,]))})
38 NClus <- 4
39 PosteriorSIGMA <- matrix(NA, length(keep), NClus)
40 PosteriorBETA <- array(NA, c(k,NClus,length(keep)))
41 PosteriorLAMBDA <- matrix(NA, length(keep), NClus)
42 l <- 1
43 for (s in keep){
44   PosteriorSIGMA[l,] <- PostSigma2[[s]][1:NClus]
45   PosteriorLAMBDA[l,] <- PostLambda[[s]][1:NClus]
46   PosteriorBETA[,l,] <- PostBetas[[s]]
47   l <- l + 1
48 }
49 summary(coda::mcmc(PosteriorSIGMA)); summary(coda::mcmc(
50   PosteriorLAMBDA))
51 for(l in 1:NClus){
52   PosteriorBeta <- t(PosteriorBETA[,l,])
53   colnames(PosteriorBeta) <- c("Ct", names(Data[, -1]))
54   print(summary(coda::mcmc(PosteriorBeta)))
55 }

```

3. Get the posterior sampler in the semi-parametric setting using a Dirichlet process mixture:

$$y_i = \mathbf{x}_i^\top \boldsymbol{\beta} + e_i$$

$$e_i \mid \mu_i, \sigma_i^2 \stackrel{iid}{\sim} N(\mu_i, \sigma_i^2),$$

Do not include the intercept in $\boldsymbol{\beta}$ to get flexibility in the distribution of the stochastic errors.

Let's assume $\boldsymbol{\beta} \sim N(\boldsymbol{\beta}_0, \mathbf{B}_0)$, $\sigma_i^2 \sim IG(\alpha_0/2, \delta_0/2)$, $\mu_i \sim N(\mu_0, \sigma_i^2/\beta_0)$, $\alpha \sim G(a, b)$ such that introducing the latent variable $\xi \mid \alpha, N \sim Be(\alpha+1, N)$, allows to easily sample the posterior draws of $\alpha \mid \xi, H, \pi_\xi \sim \pi_\xi G(a+H, b - \log(\xi)) + (1 - \pi_\xi)G(a+H-1, b - \log(\xi))$, where $\frac{\pi_\xi}{1-\pi_\xi} = \frac{a+H-1}{N(b-\log(\xi))}$, H is the number of atoms (mixture components).

Answer:

This mixture model can be expressed in a hierarchical structure:

$$e_i \mid \boldsymbol{\theta}_i \stackrel{ind}{\sim} f_{\boldsymbol{\theta}_i}$$

$$\boldsymbol{\theta}_i \mid G \stackrel{iid}{\sim} G$$

$$G \mid \alpha, G_0 \sim DP(\alpha G_0),$$

where $e_i = y_i - \mathbf{x}_i^\top \boldsymbol{\beta}$ and $\boldsymbol{\theta}_i = [\mu_i \ \sigma_i^2]^\top$.

Thus,

$$s_i \sim \sum_{h=0}^{\infty} \lambda_h \delta_h,$$

$$e_i \mid s_i, \boldsymbol{\theta}_{s_i} \sim N(\mu_{s_i}, \sigma_{s_i}^2),$$

where $\lambda_h = P(\boldsymbol{\theta}_i = \boldsymbol{\theta}_h^*)$.

The conditional posterior distribution of $\boldsymbol{\theta}_i$ is

$$\boldsymbol{\theta}_i \mid \{\boldsymbol{\theta}_{i'}, \mathbf{s}_{i'} : i' \neq i\}, e_i, \alpha \sim \sum_{i' \neq i} \frac{N_h^{(i)}}{\alpha + N - 1} f_N(e_i \mid \mu_h, \sigma_h^2)$$

$$+ \frac{\alpha}{\alpha + N - 1} \int_{\mathcal{R}} \int_0^\infty f_N(e_i \mid \mu, \sigma^2) f_N\left(\mu \mid \mu_0, \frac{\sigma^2}{\beta_0}\right) f_{IG}(\sigma^2 \mid \alpha_0, \delta_0) d\sigma^2 d\mu,$$

where $N_h^{(i)}$ is the number of observations such that $s_{i'} = h$, $i' \neq i$.

Observe that the integral in the previous equation has exactly the same form as in the marginal likelihood presented in Section 3.2 in the book in the normal-normal/inverse gamma model. Thus,

$$p(e_i) = \int_{\mathcal{R}} \int_0^\infty f_N(e_i \mid \mu, \sigma^2) f_N\left(\mu \mid \mu_0, \frac{\sigma^2}{\beta_0}\right) f_{IG}(\sigma^2 \mid \alpha_0, \delta_0) d\sigma^2 d\mu$$

$$= \frac{\Gamma\left(\frac{\alpha_n}{2}\right) (\delta_0/2)^{\alpha_0/2}}{\Gamma\left(\frac{\alpha_0}{2}\right) (\delta_n/2)^{\alpha_n/2}} \left(\frac{\beta_0}{\beta_n}\right)^{1/2} (\pi)^{-1/2},$$

where $\alpha_n = 1 + \alpha_0$, $\delta_n = \delta_0 + \frac{\beta_0}{1+\beta_0}(e_i - \mu_0)^2$ and $\beta_n = 1 + \beta_0$. Therefore, we sample s_i as follows,

$$s_i | \{\mu_{i'}, \sigma_{i'}^2, \mathbf{s}_{i'} : i' \neq i\}, e_i, \alpha \sim \left\{ \begin{array}{l} P(s_i = 0 | \cdot) = q_0^* \\ P(s_i = h | \cdot) = q_h^*, h = 1, 2, \dots, H^{(i)} \end{array} \right\},$$

where $H^{(i)}$ is the number of clusters excluding i , which may have its own cluster (singleton cluster), $q_c^* = \frac{q_c}{q_0 + \sum_h q_h}$, $q_c = \{q_0, q_h\}$, $q_h = \frac{N_h^{(i)}}{\alpha + N - 1} f_N(e_i | \mu_h, \sigma_h^2)$ and $q_0 = \frac{\alpha}{\alpha + N - 1} p(e_i)$.

If $s_i = 0$ is sampled, then $s_i = H + 1$, and a new σ_h^2 is sampled from $IG(\alpha_n/2, \delta_n/2)$, a new μ_h is sampled from $N(\mu_i, \sigma_h^2/\beta_n)$, where $\mu_i = \frac{e_i + \beta_0 \mu_0}{1 + \beta_0}$.

Discarding θ_h 's from last step, we use $\mathbf{s}$ and total number of components to sample σ_h^2 from

$$IG\left(\frac{\alpha_0 + N_h}{2}, \frac{\delta_0 + \sum_{i \in \mathcal{C}_h} (e_i - \bar{e})^2 + \frac{\beta_0 N_h}{\beta_0 + N_h} (\bar{e} - \mu_0)^2}{2}\right),$$

where $\bar{e} = \frac{\sum_{i \in \mathcal{C}_h} e_i}{N_h}$, that is, the mean uses individuals in cluster $\mathcal{C}_h$. Note that when $N_h = 1$, $\delta_0 + \sum_{i \in \mathcal{C}_h} (e_i - \bar{e})^2 + \frac{\beta_0 N_h}{\beta_0 + N_h} (\bar{e} - \mu_0)^2$ is equal to δ_n .

We sample $\mu_h | \sigma_h^2, \mathbf{e} \sim N\left(\mu_{hn}, \frac{\sigma_h^2}{\beta_0 + N_h}\right)$, where $\mu_{hn} = \frac{\beta_0 \mu_0 + N_h \bar{e}}{\beta_0 + N_h}$, $h = 1, 2, \dots, H$. Note that μ_{hn} is equal to μ_i when $N_h = 1$.

The posterior distribution of β is given by

$$\beta \sim N(\beta_n, \mathbf{B}_n),$$

where $\mathbf{B}_n = (\mathbf{B}_0^{-1} + \sum_{h=1}^H \sum_{i \in \mathcal{C}_h} \sigma_h^{-2} \mathbf{x}_i \mathbf{x}_i^\top)^{-1}$ and $\beta_n = \mathbf{B}_n (\mathbf{B}_0^{-1} \beta_0 + \sum_{h=1}^H \sum_{i \in \mathcal{C}_h} \sigma_h^{-2} \mathbf{x}_i (y_i - \mu_h))$.

4. Example: Exercise 1 and 3 continue

Perform inference in the simulation of the semi-parametric model of Exercise 1 using the sampler of Exercise 3. Use non-informative priors, setting $\alpha_0 = \delta_0 = 0.01$, $\beta_0 = \mathbf{0}_2$, $\mathbf{B}_0 = \mathbf{I}_2$, and $a = b = 0.1$. The number of MCMC iterations is 5,000, the burn-in is 1,000, and the thinning parameter is 2.

Answer:

The following code implements the Gibbs sampling algorithm to perform inference using the DPM in the semi-parametric simulation. From the summary statistics of the posterior chains, we can see that the 95% credible intervals encompass the population values. Figure 11.2 shows the posterior

density of the intercept associated with the second cluster. We observe bimodality, with the two modes located around the population means of the clusters.

R code. Simulation: Semi-parametric Dirichlet process mixture

```

1 rm(list = ls())
2 set.seed(010101)
3 # Simulate data from a 2-component mixture model
4 n <- 1000
5 x1 <- rnorm(n); x2 <- rnorm(n)
6 B <- c(0.5, -1.2)
7 X <- cbind(x1, x2)
8 z <- rbinom(n, 1, 0.3) # Latent class indicator
9 mu <- ifelse(z == 0, rnorm(n, -0.5, 0.5), rnorm(n, 1, 0.8))
10 y <- X%*%B + mu
11 k <- dim(X)[2]
12 N <- dim(X)[1]
13 # Hyperparameters
14 a0 <- 0.001; d0 <- 0.001
15 b0 <- rep(0, k); B0 <- diag(k)
16 B0i <- solve(B0)
17 a <- 0.1; b <- 0.1
18 mu0 <- 0
19 sig2mu0 <- 10
20 beta0 <- 0.1
21 # MCMC parameters
22 mcmc <- 5000; burnin <- 1000
23 tot <- mcmc + burnin; thin <- 2
24 # Gibbs sampling functions
25 PostSig2 <- function(Xh, yh, Beta){
26   Nh <- length(yh)
27   if(Nh == 1){
28     Xh <- matrix(Xh, k, 1)
29     XhB <- t(Xh)%*%Beta
30   }else{
31     Xh <- matrix(Xh, Nh, k)
32     XhB <- Xh%*%Beta
33   }
34   eh <- matrix(yh, Nh, 1) - XhB
35   an <- a0 + Nh
36   dn <- d0 + sum((eh - mean(eh))^2) + beta0*Nh/(beta0 + Nh)*
37     (mean(eh) - mu0)^2
38   sig2 <- invgamma::rinvgamma(1, shape = an/2, rate = dn/2)
39   return(sig2)
40 }
41 Postmu <- function(sig2h, Beta, Xh, yh){
42   Nh <- length(yh)
43   if(Nh == 1){
44     Xh <- matrix(Xh, k, 1)
45     XhB <- t(Xh)%*%Beta
46   }else{
47     Xh <- matrix(Xh, Nh, k)
48     XhB <- Xh%*%Beta
49   }
50   eh <- matrix(yh, Nh, 1) - XhB
51   sig2mu <- sig2h/(beta0 + Nh)
52   mun <- (beta0*mu0 + Nh*mean(eh))/(beta0 + Nh)
53   mu <- rnorm(1, mun, sig2mu^0.5)
54   return(mu)
55 }

```

*R code. Simulation: Semi-parametric Dirichlet
process mixture*

```

1 PostBeta <- function(sig2, mu, X, y, Clust){
2   XtX <- matrix(0, k, k); Xty <- matrix(0, k, 1)
3   Hs <- length(mu)
4   for(h in 1:Hs){
5     idh <- which(Clust == h)
6     if(length(idh) == 1){
7       Xh <- matrix(X[idh,], k, 1); XtXh <- sig2[h]^(-1)*Xh%*
8       %t(Xh)
9       yh <- y[idh]; Xtyh <- sig2[h]^(-1)*Xh%*(yh - mu[h])
10      }else{
11        Xh <- X[idh,]; XtXh <- sig2[h]^(-1)*t(Xh)%*Xh
12        yh <- y[idh]; Xtyh <- sig2[h]^(-1)*t(Xh)%*(yh - mu[h]
13        ])
14      }
15      XtX <- XtX + XtXh; Xty <- Xty + Xtyh
16    }
17    Bn <- solve(B0i + XtX); bn <- Bn%*(B0i%*b0 + Xty)
18    Beta <- MASS::mvrnorm(1, bn, Bn)
19    return(Beta)
20  }
21 PostAlpha <- function(Clust, alpha){
22   H <- length(unique(Clust))
23   psi <- rbeta(1, alpha + 1, N)
24   pi.ratio <- (a + H - 1) / (N * (b - log(psi)))
25   pi <- pi.ratio / (1 + pi.ratio)
26   components <- sample(1:2, prob = c(pi, (1 - pi)), size =
27   1)
28   cs <- c(a + H, a + H - 1); ds <- b - log(psi)
29   alpha <- rgamma(1, cs[components], ds)
30   return(alpha)
31 }
32 LogMarLikLM <- function(xh, yh, Beta){
33   xh <- matrix(xh, k, 1); ei <- c(yh - t(xh)%*Beta)
34   betan <- beta0 + 1; an <- a0 + 1
35   dn <- d0 + beta0/(beta0 + 1)*(ei - mu0)^2
36   logpy <- (1/2)*log(1/pi)+(a0/2)*log(d0)-(an/2)*log(dn) +
37   0.5*log(beta0/betan) + lgamma(an/2)-lgamma(a0/2)
38   return(logpy)
39 }
40 PostS <- function(Beta, SIGMA, MU, Alpha, Clust, i){
41   N1 <- table(Clust[-i]); H <- length(N1)
42   ei <- c(y[i] - t(X[i,])%*Beta)
43   qh <- sapply(1:H, function(h){(N1[h]/(N+Alpha-1))*dnorm(ei
44   , mean = MU[h], sd = SIGMA[h])})
45   q0 <- (Alpha/(N+Alpha-1))*exp(LogMarLikLM(xh = X[i,], yh =
46   y[i], Beta = Beta))
47   qh <- c(q0, qh)
48   Clust <- as.numeric(names(N1))
49   si <- sample(c(0, Clust), 1, prob = qh)
50   if(si == 0){si <- Clust[H] + 1
51     Sig2New <- PostSig2(Xh = X[i,], yh = y[i], Beta = Beta)
52     SIGMA <- c(SIGMA, Sig2New^0.5)
53     MuNew <- Postmu(sig2h = Sig2New, Beta = Beta, Xh = X[i
54     ], yh = y[i])
55     MU <- cbind(MU, MuNew)
56   }else{si == si}
57   return(list(si = si, MU = MU, SIGMA = SIGMA))
58 }

```

*R code. Simulation: Semi-parametric Dirichlet
process mixture*

```

1 PostBetas <- matrix(NA, tot, k)
2 PostSigma <- list(); PostMu <- list()
3 Posts <- matrix(0, tot, N); PostAlphas <- rep(0, tot)
4 Clust <- sample(1:3, N, replace=T, prob = c(0.5, 0.3, 0.2))
5 Reg <- lm(y ~ X)
6 SumReg <- summary(Reg)
7 BETA <- Reg$coefficients[-1]
8 SIGMA <- rep(SumReg$sigma, 3)
9 MU <- rep(Reg$coefficients[1], 3)
10 Alpha <- rgamma(1, a, b)
11 pb <- txtProgressBar(min = 0, max = tot, style = 3)
12 for(s in 1:tot){
13   for(i in 1:N){
14     Rests <- PostS(Beta = BETA, SIGMA = SIGMA, MU = MU,
15       Alpha = Alpha, Clust = Clust, i = i)
16     Clust[i] <- Rests$si
17     MU <- Rests$MU; SIGMA <- Rests$SIGMA
18   }
19   sFreq <- table(Clust)
20   lt <- 1
21   for(li in as.numeric(names(sFreq))){
22     Index <- which(Clust == li)
23     if(li == lt){Clust[Index] <- li
24     } else {Clust[Index] <- lt
25     }
26     lt <- lt + 1
27   }
28   Alpha <- PostAlpha(Clust = Clust, alpha = Alpha)
29   N1 <- table(Clust); H <- length(N1)
30   SIGMA <- rep(NA, H)
31   MU <- rep(NA, H)
32   l <- 1
33   for(h in unique(Clust)){
34     Idh <- which(Clust == h)
35     SIGMA[l] <- (PostSig2(Beta = BETA, Xh = X[Idh, ], yh = y
36       [Idh]))^0.5
37     MU[l] <- Postmu(Beta = BETA, sig2h = SIGMA[l]^2, Xh = X[
38       Idh, ], yh = y[Idh])
39     l <- l + 1
40   }
41   BETA <- PostBeta(sig2 = SIGMA, mu = MU, X = X, y = y,
42     Clust = Clust)
43   PostBetas[s,] <- BETA
44   PostSigma[[s]] <- SIGMA
45   PostMu[[s]] <- MU
46   Posts[s, ] <- Clust
47   PostAlphas[s] <- Alpha
48   setTxtProgressBar(pb, s)
49 }
50 close(pb)

```


R code. Simulation: Semi-parametric Dirichlet process mixture

```

1 keep <- seq(burnin, tot, thin)
2 PosteriorBetas <- coda::mcmc(PostBetas[keep,])
3 summary(PosteriorBetas)
4 plot(PosteriorBetas)
5 PosteriorPsi <- Posts[keep,]
6 Clusters <- sapply(1:length(keep), function(i){length(table(
  PosteriorPsi[i,]))})
7 table(Clusters)
8 PosteriorSIGMA <- matrix(NA, length(keep), 2)
9 PosteriorMU <- matrix(NA, length(keep), 2)
10 l <- 1
11 for (s in keep){
12   PosteriorSIGMA[l,] <- PostSigma[[s]][1:2]
13   PosteriorMU[l,] <- PostMu[[s]][1:2]
14   l <- l + 1
15 }
16 summary(coda::mcmc(PosteriorSIGMA))
17 summary(coda::mcmc(PosteriorMU))
18 plot(coda::mcmc(PosteriorMU))
19 DataMU2 <- data.frame(Posterior = PosteriorMU[,2])
20 library(ggplot2)
21 densMU <- ggplot(DataMU2, aes(x = Posterior)) + geom_density
  (fill = "blue", alpha = 0.3) + labs(x = "Intercept", y =
    "Density") + theme_minimal()
22 densMU

```

5. Example: Simulation exercise continues

Fix the label-switching problem of the simulation exercise of the DPM using *random permutation of latent classes*.

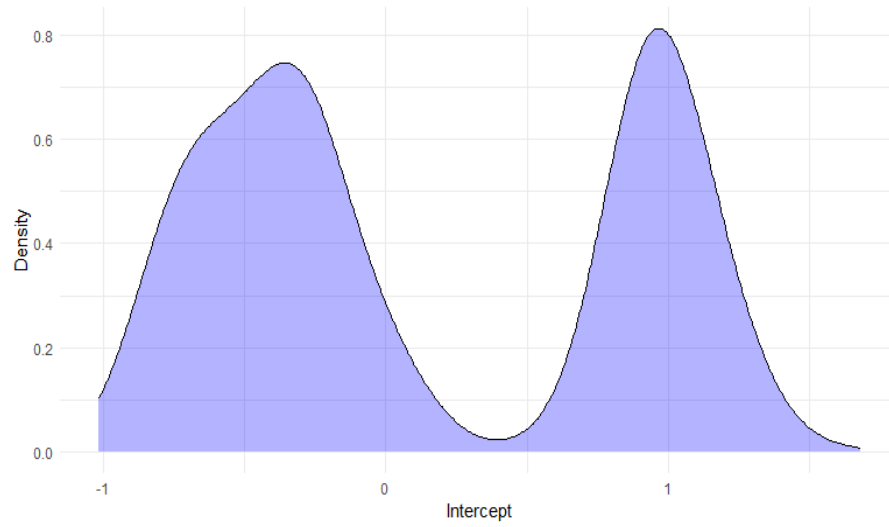
Answer:

A simple strategy to address label switching in this simulation exercise is to fix the number of clusters to three, given that this is the typical number of clusters obtained after performing the DPM. Recall that, according to the simulation setup, there are only two clusters:

$$p(y_i | \mathbf{x}_i) = 0.5\phi(y_i | 2 + 1.5x_i, 1^2) + 0.5\phi(y_i | -1 + 0.5x_i, 0.8^2).$$

Thus, we can use a finite Gaussian mixture model with $H = 3$.¹

¹Another approach is to use finite Dirichlet processes [32, Chap. 2] and set a random permutation of latent classes.

**FIGURE 11.2**

Posterior density estimate: Mean of the stochastic errors.

The following code illustrates the algorithm.

*R code. Simulation: Random permutation of
latent classes*

```

1 rm(list = ls()); set.seed(010101)
2 # Simulate data from a 2-component mixture model
3 n <- 1000; x <- rnorm(n)
4 z <- rbinom(n, 1, 0.5)
5 y <- ifelse(z == 0, rnorm(n, 2 + 1.5*x, 1), rnorm(n, -1 +
6   0.5*x, 0.8))
7 data <- data.frame(y, x)
8 # Hyperparameters
9 d0 <- 0.001; a0 <- 0.001
10 b0 <- rep(0, 2); B0 <- diag(2); B0i <- solve(B0)
11 H <- 3; a0h <- rep(1/H, H)
12 # MCMC parameters
13 mcmc <- 10000; burnin <- 2000
14 tot <- mcmc + burnin; thin <- 2
15 # Gibbs sampling functions
16 PostSig2 <- function(Betah, Xh, yh){
17   Nh <- length(yh); an <- a0 + Nh
18   dn <- d0 + t(yh - Xh%*%Betah)%*(yh - Xh%*%Betah)
19   sig2 <- invgamma::rinvgamma(1, shape = an/2, rate = dn/2)
20   return(sig2)
21 }
22 PostBeta <- function(sig2h, Xh, yh){
23   Bn <- solve(B0i + sig2h^(-1)*t(Xh)%*%Xh)
24   bn <- Bn%*(B0i%*%b0 + sig2h^(-1)*t(Xh)%*%yh)
25   Beta <- MASS::mvrnorm(1, bn, Bn)
26   return(Beta)
27 }
28 PostBetas1 <- matrix(0, tot, 2)
29 PostBetas2 <- matrix(0, tot, 2)
30 PostBetas3 <- matrix(0, tot, 2)
31 PostSigma2 <- matrix(0, tot, H)
32 PostPsi <- matrix(0, tot, n)
33 PostLambda <- matrix(0, tot, H)
34 Reg <- lm(y ~ x)
35 SumReg <- summary(Reg)
36 BETA <- cbind(Reg$coefficients, Reg$coefficients, Reg$
37   coefficients)
38 SIG2 <- rep(SumReg$sigma, H)
39 X <- cbind(1, x); Psi <- rep(NA, n)
40 Lambda1 <- 1/H; Lambda2 <- 1/H; Lambda3 <- 1/H
41 perutatms <- gtools::permutations(3, 3, 1:H)

```

R code. Simulation: Random permutation of latent classes

```

1 pb <- txtProgressBar(min = 0, max = tot, style = 3)
2 for(s in 1:tot){
3   for(i in 1:n){
4     lambdai1 <- Lambda1*dnorm(y[i], X[i,]%*%BETA[,1], SIG2
5       [1]^0.5)
6     lambdai2 <- Lambda2*dnorm(y[i], X[i,]%*%BETA[,2], SIG2
7       [2]^0.5)
8     lambdai3 <- Lambda3*dnorm(y[i], X[i,]%*%BETA[,3], SIG2
9       [3]^0.5)
10    Psi[i] <- sample(1:H, 1, prob = c(lambdai1, lambdai2,
11      lambdai3))
12  }
13  NH <- NULL
14  for(h in 1:H){
15    Idh <- which(Psi == h)
16    Nh <- length(Idh)
17    NH <- c(NH, Nh)
18    SIG2[h] <- PostSig2(Betah = BETA[,h], Xh = X[Idh, ], yh
19      = y[Idh])
20    BETA[,h] <- PostBeta(sig2h = SIG2[h], Xh = X[Idh, ], yh
21      = y[Idh])
22  }
23  Lambda <- sort(MCMCpack::rdirichlet(1, a0h + NH),
24    decreasing = TRUE)
25  IndPer <- sample(1:dim(perutatms)[1], 1)
26  Clust <- perutatms[IndPer, ]
27  SIG2 <- SIG2[Clust]
28  BETA <- BETA[,Clust]
29  Lambda <- Lambda[Clust]
30  for(i in 1:n){
31    if(Psi[i] == 1){
32      Psi[i] <- Clust[1]
33    }else{
34      if(Psi[i] == 2){
35        Psi[i] <- Clust[2]
36      }else{
37        Psi[i] <- Clust[3]
38      }
39    }
40  }
41  }
42  Lambda1 <- Lambda[1]; Lambda2 <- Lambda[2]; Lambda3 <-
43    Lambda[3]
44  PostPsi[s, ] <- Psi
45  PostSigma2[s, ] <- SIG2
46  PostBetas1[s,] <- BETA[,1]
47  PostBetas2[s,] <- BETA[,2]
48  PostBetas3[s,] <- BETA[,3]
49  PostLambda[s,] <- Lambda
50  setTxtProgressBar(pb, s)
51 }
52 close(pb)

```

R code. Simulation: Random permutation of latent classes

```

1 keep <- seq((burnin+1), tot, thin)
2 PosteriorBetas1 <- coda::mcmc(PostBetas1[keep,])
3 PosteriorBetas2 <- coda::mcmc(PostBetas2[keep,])
4 PosteriorBetas3 <- coda::mcmc(PostBetas3[keep,])
5 PosteriorSigma2 <- coda::mcmc(PostSigma2[keep,])
6 summary(PosteriorSigma2)
7 PosteriorLambda <- coda::mcmc(PostLambda[keep,])
8 summary(PosteriorLambda)
9 DataBetas <- data.frame(Cluster1 = as.numeric(
    PosteriorBetas1[,2]), Cluster2 = as.numeric(
    PosteriorBetas2[,2]), Cluster3 = as.numeric(
    PosteriorBetas3[,2]))
10 library(ggplot2)
11 dens1 <- ggplot(DataBetas, aes(x = Cluster1)) + geom_density
    (fill = "blue", alpha = 0.3) + labs(x = "Cluster 1", y =
    "Density") + theme_minimal()
12 dens2 <- ggplot(DataBetas, aes(x = Cluster2)) + geom_density
    (fill = "blue", alpha = 0.3) + labs(x = "Cluster 2", y =
    "Density") + theme_minimal()
13 dens3 <- ggplot(DataBetas, aes(x = Cluster3)) + geom_density
    (fill = "blue", alpha = 0.3) + labs(x = "Cluster 3", y =
    "Density") + theme_minimal()
14 library(ggpubr)
15 ggarrange(dens1, dens2, dens3, labels = c("A", "B", "C"),
    ncol = 3, nrow = 1, legend = "bottom", common.legend =
    TRUE)
16

```

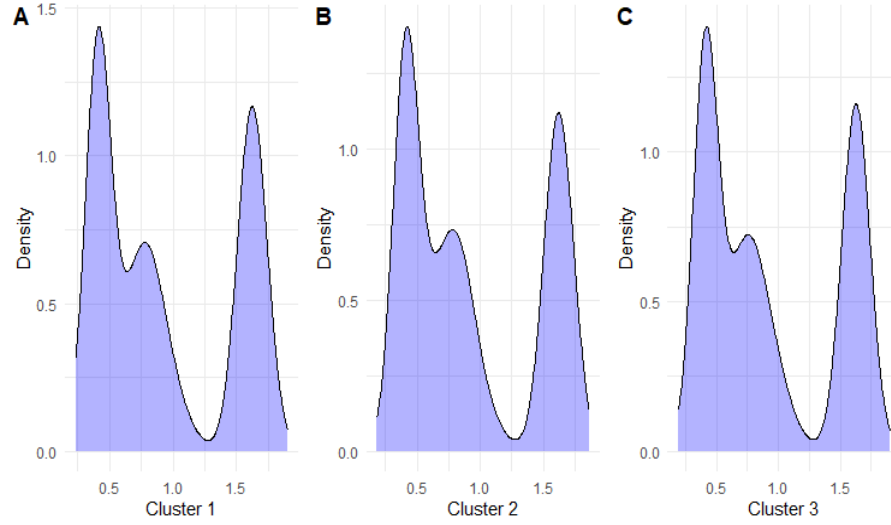
Figure 11.3 shows the posterior density estimates of the three clusters. We observe that they have similar shapes, as they are targeting the same posterior distributions. There are two modes with large values around the population parameters, and an additional mode around 0. This occurs because we are forcing three clusters, even though the data-generating process contains only two.

6. Example: Simulation exercise continues II

Obtain the density estimate of y in the simulation exercise of the DPM, evaluated at $x = 0$.

Answer:

The following code shows how to perform inference, and get the density estimate using the DPM. Note that the density estimate is given by

**FIGURE 11.3**

Posterior density estimates slope parameter: Population values are 1.5 and 0.5.

$$y \mid \boldsymbol{\theta}_h, \mathbf{s}, \alpha \sim \sum_{h=1}^H \frac{N_h}{\alpha + N + 1} f_N(y \mid \mathbf{x}_h^\top \boldsymbol{\beta}_h, \sigma_h^2) + \frac{\alpha}{\alpha + N + 1} p(y),$$

where N_h is the number of individuals in cluster h , H is the total observed clusters after estimation, and $p(y)$ is the marginal likelihood given in Section 11.1.2 in the book.

Figure 11.4 shows the density plots, population (black line), mean (blue line), and 95% credible interval (shaded blue). We observe that the DPM performs very well despite having more clusters than the actual data generating process.

*R code. Simulation: Density from Dirichlet
process mixture*

```

1 rm(list = ls()); set.seed(010101)
2 # Simulate data from a 2-component mixture model
3 N <- 1000; x <- rnorm(N); z <- rbinom(N, 1, 0.5)
4 y <- ifelse(z == 0, rnorm(N, 2 + 1.5*x, 1), rnorm(N, -1 +
5   0.5*x, 0.8))
6 X <- cbind(1, x); k <- 2
7 data <- data.frame(y, x); Reg <- lm(y ~ x)
8 SumReg <- summary(Reg)
9 # Hyperparameters
10 a0 <- 0.001; d0 <- 0.001
11 b0 <- rep(0, k); B0 <- diag(k)
12 B0i <- solve(B0)
13 a <- 0.1; b <- 0.1
14 # MCMC parameters
15 mcmc <- 5000; burnin <- 1000
16 tot <- mcmc + burnin; thin <- 2
17 # Gibbs sampling functions
18 PostSig2 <- function(Xh, yh){
19   Nh <- length(yh)
20   yh <- matrix(yh, Nh, 1)
21   if(Nh == 1){
22     Xh <- matrix(Xh, k, 1)
23     Bn <- solve(Xh%*%t(Xh) + B0i)
24     bn <- Bn%*(B0i%*%b0 + Xh%*%yh)
25   }else{
26     Xh <- matrix(Xh, Nh, k)
27     Bn <- solve(t(Xh)%*%Xh + B0i)
28     bn <- Bn%*(B0i%*%b0 + t(Xh)%*%yh)
29   }
30   Bni <- solve(Bn)
31   an <- a0 + Nh
32   dn <- d0 + t(yh)%*%yh + t(b0)%*%B0i%*%b0 - t(bn)%*%Bni%*%
33     bn
34   sig2 <- invgamma::rinvgamma(1, shape = an/2, rate = dn/2)
35   return(sig2)
36 }
37 PostBeta <- function(sig2h, Xh, yh){
38   Nh <- length(yh)
39   yh <- matrix(yh, Nh, 1)
40   if(Nh == 1){
41     Xh <- matrix(Xh, k, 1)
42     Bn <- solve(Xh%*%t(Xh) + B0i)
43     bn <- Bn%*(B0i%*%b0 + Xh%*%yh)
44   }else{
45     Xh <- matrix(Xh, Nh, k)
46     Bn <- solve(t(Xh)%*%Xh + B0i)
47     bn <- Bn%*(B0i%*%b0 + t(Xh)%*%yh)
48   }
49   Beta <- MASS::mvrnorm(1, bn, sig2h*Bn)
50   return(Beta)
51 }

```

*R code. Simulation: Density from Dirichlet
process mixture*

```

1 PostAlpha <- function(s, alpha){
2   H <- length(unique(s))
3   psi <- rbeta(1, alpha + 1, N)
4   pi.ratio <- (a + H - 1) / (N * (b - log(psi)))
5   pi <- pi.ratio / (1 + pi.ratio)
6   components <- sample(1:2, prob = c(pi, (1 - pi)), size =
7     1)
8   cs <- c(a + H, a + H - 1)
9   ds <- b - log(psi)
10  alpha <- rgamma(1, cs[components], ds)
11  return(alpha)
12 }
13 LogMarLikLM <- function(xh, yh){
14   xh <- matrix(xh, k, 1)
15   Bn <- solve(xh%*%t(xh) + B0i)
16   Bni <- solve(Bn)
17   bn <- Bn%*%(B0i%*%b0 + xh%*%yh)
18   an <- a0 + 1
19   dn <- d0 + yh^2 + t(b0)%*%B0i%*%b0 - t(bn)%*%Bni%*%bn
20   # Log marginal likelihood
21   logpy <- (1/2)*log(1/pi)+(a0/2)*log(d0)-(an/2)*log(dn) +
22     0.5*log(det(Bn)/det(B0)) + lgamma(an/2)-lgamma(a0/2)
23   return(logpy)
24 }
25 PostS <- function(BETA, SIGMA, Alpha, s, i){
26   N1 <- table(s[-i]); H <- length(N1)
27   qh <- sapply(1:H, function(h){(N1[h]/(N+Alpha-1))*dnorm(y[
28     i], mean = t(X[i,])%*%BETA[,h], sd = SIGMA[h])})
29   q0 <- (Alpha/(N+Alpha-1))*exp(LogMarLikLM(xh = X[i,], yh =
30     y[i]))
31   qh <- c(q0, qh)
32   Clust <- as.numeric(names(N1))
33   si <- sample(c(0, Clust), 1, prob = qh)
34   if(si == 0){
35     si <- Clust[H] + 1
36     Sig2New <- PostSig2(Xh = X[i,], yh = y[i])
37     SIGMA <- c(SIGMA, Sig2New^0.5)
38     BetaNew <- PostBeta(sig2h = Sig2New, Xh = X[i,], yh = y[
39       i])
40     BETA <- cbind(BETA, BetaNew)
41   }else{si == si}
42   }
43   return(list(si = si, BETA = BETA, SIGMA = SIGMA))
44 }
45 PostBetas <- list(); PostSigma <- list()
46 Posts <- matrix(0, tot, N); PostAlphas <- rep(0, tot)
47 S <- sample(1:3, N, replace = T, prob = c(0.5, 0.3, 0.2))
48 BETA <- cbind(Reg$coefficients, Reg$coefficients, Reg$
49   coefficients)
50 SIGMA <- rep(SumReg$sigma, 3)
51 Alpha <- rgamma(1, a, b)

```


*R code. Simulation: Density from Dirichlet
process mixture*

```

1 pb <- txtProgressBar(min = 0, max = tot, style = 3)
2 for(s in 1:tot){
3   for(i in 1:N){
4     Rests <- PostS(BETA = BETA, SIGMA = SIGMA, Alpha = Alpha
5       , s = S, i = i)
6     S[i] <- Rests$si
7     BETA <- Rests$BETA; SIGMA <- Rests$SIGMA
8   }
9   sFreq <- table(S)
10  lt <- 1
11  for(li in as.numeric(names(sFreq))){
12    Index <- which(S == li)
13    if(li == lt){S[Index] <- li
14    } else {S[Index] <- lt
15    }
16    lt <- lt + 1
17  }
18  Alpha <- PostAlpha(s = S, alpha = Alpha)
19  N1 <- table(S); H <- length(N1)
20  SIGMA <- rep(NA, H)
21  BETA <- matrix(NA, k, H)
22  l <- 1
23  for(h in unique(S)){
24    Idh <- which(S == h)
25    SIGMA[l] <- (PostSig2(Xh = X[Idh, ], yh = y[Idh]))^0.5
26    BETA[l,1] <- PostBeta(sig2h = SIGMA[l]^2, Xh = X[Idh, ],
27      yh = y[Idh])
28    l <- l + 1
29  }
30  PostBetas[[s]] <- BETA
31  PostSigma[[s]] <- SIGMA
32  Posts[s, ] <- S
33  PostAlphas[s] <- Alpha
34  setTxtProgressBar(pb, s)
35 }
36 close(pb)
37 keep <- seq((burnin+1), tot, thin)
38 PosteriorS <- Posts[keep,]
39 Clusters <- sapply(1:length(keep), function(i){length(table(
40   PosteriorS[i,]))})
41 table(Clusters)
42 Clusters <- sapply(1:length(keep), function(i){table(
43   PosteriorS[i,]))})
44 PosteriorBeta <- PostBetas[keep]
45 PosteriorSigma <- PostSigma[keep]
46 PosteriorAlpha <- PostAlphas[keep]

```

R code. Simulation: Density from Dirichlet process mixture

```

1 DensityFunc <- function(yobs = 0, xobs = c(1, 0), s){
2   dens <- NULL
3   for(j in 1:length(Clusters[[s]])){
4     densj <- Clusters[[s]][j]/(PosteriorAlpha[s] + N + 1) *
       dnorm(yobs, t(PosteriorBeta[[s]][,j])%*%xobs,
       PosteriorSigma[[s]][j])
5     dens <- c(dens, densj)
6   }
7   dens <- c(dens, PosteriorAlpha[s]/(PosteriorAlpha[s] + N +
       1)*exp(LogMarLikLM(yh = yobs, xh = xobs)))
8   return(sum(dens))
9 }
10 xobs <- c(1, 0)
11 ys <- seq(-3, 5, 0.05)
12 pdfy <- 0.5*dnorm(ys, 2 + 1.5*xobs[2], 1) + 0.5*dnorm(ys, -1
       + 0.5*xobs[2], 0.8)
13 DensEval <- matrix(NA, length(keep), length(ys))
14 for(r in 1:length(ys)){
15   for(l in 1:length(keep)){
16     DensEval[l, r] <- DensityFunc(yobs = ys[r], xobs = xobs,
       s = 1)
17   }
18 }
19 library(dplyr)
20 library(ggplot2)
21 require(latex2exp)
22 DataDens <- tibble(t = ys, Pop = pdfy, lower = apply(
       DensEval, 2, quantile, probs = 0.025), upper = apply(
       DensEval, 2, quantile, probs = 0.975), meanT = colMeans(
       DensEval))
23 plot_filtering_estimates <- function(df) {
24   p <- ggplot(data = df, aes(x = t)) + geom_ribbon(aes(ymin
       = lower, ymax = upper), alpha = 1, fill = "lightblue") +
       geom_line(aes(y = Pop, color = "True Value"), linewidth
       = 0.5) + geom_line(aes(y = meanT, color = "Estimate"),
       linewidth = 0.5) + scale_color_manual(values = c("True
       Value" = "black", "Estimate" = "blue")) + xlab(TeX("y"))
       + ylab("Density") + labs(title = "Density: Dependent
       variable", color = "") + theme_minimal()
25   print(p)
26 }
27 plot_filtering_estimates(DataDens)

```

7. Example: Consumption of marijuana in Colombia continues II

Perform the application of marijuana consumption with the following spec-

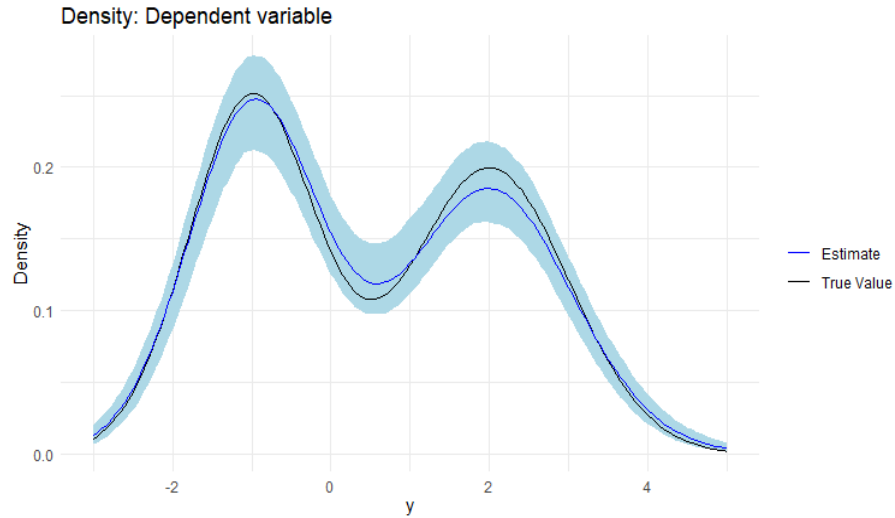


FIGURE 11.4
Density from Dirichlet process mixture.

ification:

$$y_i = \mathbf{z}_i^\top \boldsymbol{\gamma} + f(\text{Age}_i) + \mu_i,$$

where y_i is the (log) marijuana monthly consumption, $\mathbf{z}_i$ represents the presence of a drug dealer in the neighborhood (*Dealer*), gender (*Female*), indicators of good physical and mental health (*PhysicalHealthGood* and *MentalHealthGood*), years of education (*YearsEducation*), and the (log) prices of marijuana, cocaine, and crack by individual.

Initially, set the knots as the percentiles $\{0, 0.05, \dots, 0.95, 1\}$ of age and use cubic B-splines. Then, apply the BIC approximation to perform variable selection in this model with non-informative conjugate priors, 5,000 MCMC iterations, and 5,000 burn-in iterations.

Do you think that using a linear regression with a second-degree polynomial in age provides a good approximation to the relationship found using splines in this application?

Answer:

The following code illustrates this exercise. We use the function *bicreg* from the package *BMA* to perform variable selection. Initially, we construct the design matrix, excluding the B-splines that cause perfect multicollinearity, and then select the variables with a posterior inclusion probability greater than 50%. This also corresponds to the model with the highest posterior probability.

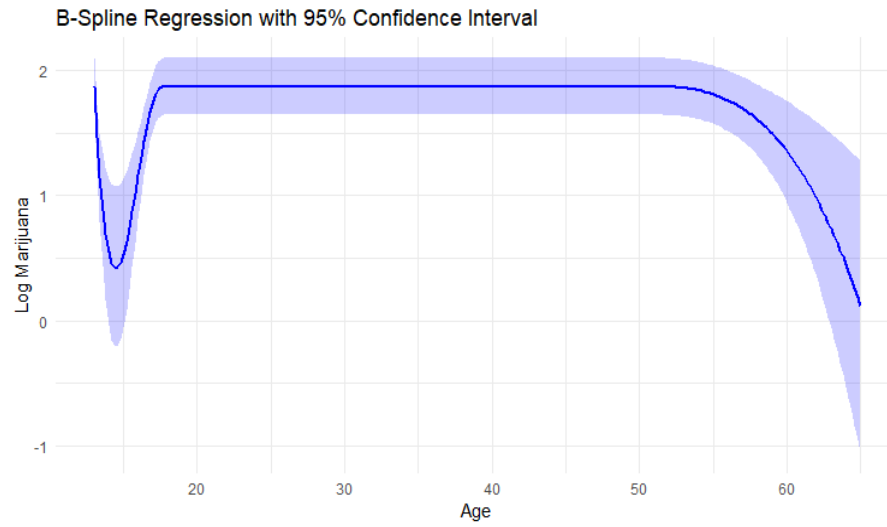
After this, we proceed with inference using a linear regression with conjugate prior distributions. Figure 11.5 shows the fit, with the mean (blue line) and 95% credible intervals (shaded blue). This curve is simpler than the one using all the B-splines: there appears to be a hook-shaped pattern at the beginning, where teenagers consume less marijuana than individuals in their 20s to 50s. After age 55, consumption decreases significantly. Given this pattern, assuming a second-degree polynomial appears to be a reasonable choice.

*R code. Simulation: Semi-parametric Dirichlet
process mixture*

```

1 rm(list = ls()); set.seed(010101)
2 library(splines); library(ggplot2)
3 Data <- read.csv("https://raw.githubusercontent.com/
   BEsmarter-consultancy/BSTApp/refs/heads/master/DataApp/
   MarijuanaColombia.csv")
4 attach(Data)
5 IdOrd <- order(Age); y <- LogMarijuana[IdOrd]
6 Z <- as.matrix(cbind(Data[IdOrd,-c(1, 6, 7)]))
7 x <- Age[IdOrd]
8 knots <- quantile(x, seq(0, 1, 0.05))
9 BS <- bs(x, knots = knots, degree = 3, Boundary.knots =
   range(x), intercept = FALSE)
10 matplot(x, BS, type = "l", lty = 1, col = rainbow(ncol(BS)))
11 # Function to check if a column is constant
12 is_constant <- function(col) {
13   return(length(unique(col)) == 1)
14 }
15 constant_columns <- apply(BS, 2, is_constant)
16 X <- cbind(BS[, -c(which(constant_columns == TRUE))], Z)
17 BMAGlm <- BMA::bicreg(X, y, strict = FALSE, OR = 50)
18 summary(BMAGlm)
19 idReg <- c(2, 23, 24, 27:29) # Relevant regressors, PIP >
   0.5
20 Xnew <- cbind(1, X[, idReg])
21 N <- dim(Xnew)[1]; k <- dim(Xnew)[2]
22 # Hyperparameters
23 d0 <- 0.001; a0 <- 0.001
24 b0 <- rep(0, k); c0 <- 1000
25 B0 <- c0*diag(k); B0i <- solve(B0)
26 # MCMC parameters
27 mcmc <- 5000; burnin <- 5000
28 tot <- mcmc + burnin; thin <- 1
29 posterior <- MCMCpack::MCMCregress(y~Xnew-1, b0=b0, B0 =
   B0i, c0 = a0, d0 = d0, burnin = burnin, mcmc = mcmc,
   thin = thin)
30 summary(coda::mcmc(posterior))
31 # Predict values with 95% credible intervals
32 xfit <- seq(min(x), max(x), 0.2); H <- length(xfit)
33 i <- 675 # Pick a particular individual
34 idfit <- sample(1:N, H)
35 BSfit <- bs(xfit, knots = knots, degree = 3, Boundary.knots
   = range(x), intercept = FALSE)
36 Xfit <- cbind(1, BSfit[, c(2,23)], Z[rep(i, H), c(1,4:6)]) #
   Relevant regressors, PIP > 0.5
37 Fit <- matrix(NA, mcmc, H)
38 for(s in 1:mcmc){
39   Fit[s,] <- Xfit%*%posterior[s,1:7]
40 }
41 plot_data <- data.frame(x = xfit, fit = colMeans(Fit),
   liminf = apply(Fit, 2, quantile, 0.025), limsup = apply(
   Fit, 2, quantile, 0.975))
42
43 ggplot() + geom_line(data = plot_data, aes(x, fit), color =
   "blue", linewidth = 1) + geom_ribbon(data = plot_data,
   aes(x, ymin = liminf, ymax = limsup), fill = "blue",
   alpha = 0.2) + labs(title = "B-Spline Regression with
   95% Confidence Interval", x = "Age",
44 y = "Log Marijuana") + theme_minimal()

```

**FIGURE 11.5**

Spline: Marijuana monthly consumption vs age.

12

Bayesian machine learning

Solutions of Exercises

1. Simulation exercise: the Bayesian LASSO continues

Program the Gibbs sampler for the Bayesian LASSO from scratch, assuming a hierarchical structure for the global shrinkage parameter, where both the shape and rate parameters are set to 1. Perform inference using this sampler in the Bayesian LASSO simulation exercise and compare the results with those obtained using the *monomvn* package.

Answer

The following code illustrates how to implement the Gibbs sampler for the Bayesian LASSO. Overall, the results are very similar to those obtained using the *monomvn* package (see Figure 12.1).

R code. Simulation: The Bayesian LASSO

```

1 rm(list = ls()); set.seed(10101)
2 library(monomvn)
3 # Parameters
4 n <- 500 # sample size
5 k <- 100 # number of predictors
6 s <- 10 # number of non-zero coefficients
7 # Generate design matrix
8 X <- matrix(rnorm(n * k), nrow = n, ncol = k)
9 # True beta: first s coefficients are non-zero, rest are
  zero
10 beta_true <- c(runif(s, -3, 3), rep(0, k - s))
11 # Generate response with some noise
12 sigma <- 1
13 y <- X %>% beta_true + rnorm(n, sd = sigma)
14 df <- data.frame(X,y)
15 ### Using monomvn ###
16 library(monomvn)
17 library(statmod)
18 # Fit model using Bayesian LASSO
19 fit <- monomvn::blasso(X, y, T = 5000, verb = 0)
20 # burn-in removal
21 burnin <- 1000
22 beta_samples <- fit$beta[(burnin + 1):5000, ] # exclude
  burn-in
23 intercept_samples <- fit$mu[(burnin + 1):5000] # intercept
  posterior draws
24 # Posterior mean of coefficients
25 beta_post_mean <- colMeans(beta_samples)
26 # Intercept summary
27 mean(intercept_samples)
28 quantile(intercept_samples, c(0.025, 0.975))
29 # Local shrinkage parameters
30 colMeans(is.na(fit$tau2i)) # proportion of NA values per
  coefficient
31
32 #### Gibss sampler ####
33 # Hyperparameters
34 r0 <- 1; d0 <- 1
35 # MCMC parameters
36 mcmc <- 5000; burnin <- 5000
37 tot <- mcmc + burnin; thin <- 1
38 # Posterior distributions programming the Gibbs sampling
39 # Standardized variables
40 W <- scale(X)
41 yh <- y - mean(y)
42 # Gibbs sampling functions
43 PostBeta <- function(sig2, tau2){
44   Bn <- solve(diag(1/tau2) + t(W)%*%W)
45   bn <- Bn%*%t(W)%*%yh
46   Beta <- MASS::mvrnorm(1, bn, sig2*Bn)
47   return(Beta)
48 }

```


R code. Simulation: The Bayesian LASSO

```

1 PostSig2 <- function(Beta, tau2){
2   an <- n - 1 + k
3   dn <- t(yh - W%%Beta)%*(yh - W%%Beta) + t(Beta)%*diag
      (1/tau2)%*Beta
4   sig2 <- invgamma::rinvgamma(1, shape = an/2, rate = dn/2)
5   return(sig2)
6 }
7 PostTau2 <- function(sig2, Beta, lambda, p){
8   mukn <- (lambda^2*sig2/Beta[p]^2)^0.5
9   lambdan <- lambda^2
10  tauk2k <- rinvgauss(1, mean = mukn, shape = lambdan)
11  return(tauk2k)
12 }
13 PostLambda2 <- function(tau2){
14   sh <- k + r0
15   rat <- 0.5 * sum(tau2) + d0
16   lambda2 <- rgamma(1, shape = sh, rate = rat)
17   return(lambda2^0.5)
18 }
19 PostBeta0 <- function(sig2){
20   beta0 <- rnorm(1, mean = mean(y), sd = (sig2/n)^0.5)
21   return(beta0)
22 }
23 PostBetas <- matrix(0, mcmc+burnin, k)
24 PostSigma2 <- rep(0, mcmc+burnin)
25 PostTaus2 <- matrix(0, mcmc+burnin, k)
26 PostLambda <- rep(0, mcmc+burnin)
27 PostBetas0 <- rep(0, mcmc+burnin)
28 Beta <- rep(0, k); tau2 <- rep(1, k); lambda <- 1
29 # create progress bar in case that you want to see
      iterations progress
30 pb <- txtProgressBar(min = 0, max = tot, style = 3)
31 for(s in 1:tot){
32   sig2 <- PostSig2(Beta = Beta, tau2 = tau2)
33   PostSigma2[s] <- sig2
34   Beta <- PostBeta(sig2 = sig2, tau2 = tau2)
35   PostBetas[s,] <- Beta
36   tau2 <- sapply(1:k, function(i){PostTau2(sig2 = sig2, Beta
      = Beta, lambda = lambda, p = i)})
37   PostTaus2[s,] <- tau2
38   lambda <- PostLambda2(tau2 = tau2)
39   PostLambda[s] <- lambda
40   beta0 <- PostBeta0(sig2 = sig2)
41   PostBetas0[s] <- beta0
42   setTxtProgressBar(pb, s)
43 }
44 close(pb)
45 keep <- seq((burnin+1), tot, thin)
46 PosteriorBetas <- PostBetas[keep,]
47 summary(coda::mcmc(PosteriorBetas))
48 PosteriorBeta0 <- PostBetas0[keep]
49 summary(coda::mcmc(PosteriorBeta0))
50 PosteriorSigma2 <- PostSigma2[keep]
51 summary(coda::mcmc(PosteriorSigma2))
52 PosteriorTau2s <- PostTaus2[keep,]
53 summary(coda::mcmc(PosteriorTau2s))
54 PosteriorLambda <- PostLambda[keep]
55 summary(coda::mcmc(PosteriorLambda))

```

R code. Simulation: The Bayesian LASSO

```

1 # Posterior mean of coefficients
2 beta_post_meanNew <- colMeans(PosteriorBetas)
3 # Plot true vs estimated betas
4 plot(beta_true, beta_post_mean, pch = 19, col = "steelblue",
5 xlab = "True beta", ylab = "Posterior mean of beta",
6 main = "Bayesian LASSO (blasso): Shrinkage Effect")
7 abline(0, 1, col = "red", lty = 2)
8 # Add another set of posterior means
9 points(beta_true, beta_post_meanNew, pch = 17, col = "
    darkorange")
10 # Add legend
11 legend("topleft", legend = c("blasso", "Our Gibbs"),
12 col = c("steelblue", "darkorange"),
13 pch = c(19, 17), bty = "n")
14

```

Bayesian LASSO (blasso): Shrinkage Effect

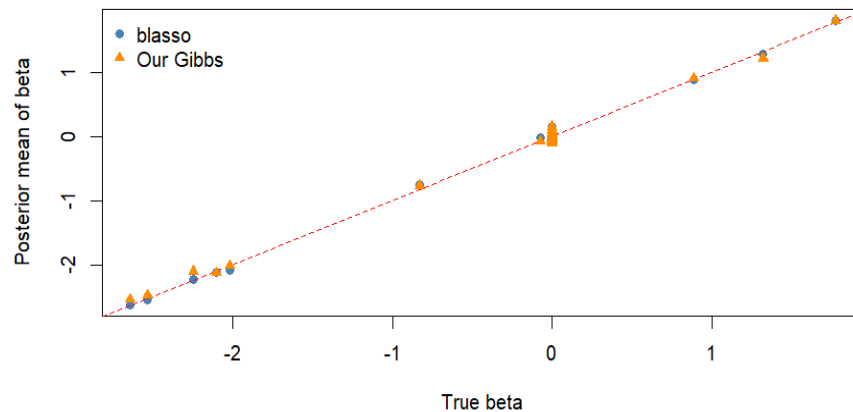


FIGURE 12.1

Comparison of LASSO performance: Package vs Gibbs from scratch.

- [21] employ stochastic search variable selection (SSVS) to identify the main drivers of civil conflict in the post-Cold War era, considering a set of 35 potential determinants across 175 countries worldwide. We use a subset of their dataset provided in *Conflict.xlsx*, where the dependent variable

is *conflictcw*, a binary indicator of civil conflict. Perform SSVS using the *BoomSpikeSlab* package, specifically the *lm.spike* function, to identify the best subset of models.

Answer

The following code illustrates how to implement SSVS in this application, based on [21], using 15,000 iterations and the default settings for the hyperparameters. We find that infant mortality rate, population, peace, and political rights are the most relevant drivers of the incidence of civil conflict. *Peace* exhibits a negative 95% credible interval, while the other variables have positive credible intervals. The model with the highest posterior probability (30%) includes the first three regressors.

R code. Application: Drivers of civil conflict in the post-Cold War era using a linear model

```

1 rm(list = ls())
2 set.seed(010101)
3 Data <- read.csv("https://raw.githubusercontent.com/
  BEsmarter-consultancy/BSTApp/refs/heads/master/DataApp/
  Conflict.csv", sep = ",", header = TRUE, quote = "")
4
5 # Scale regressors
6 W <- as.matrix(scale(Data[, -c(1, 2)]))
7 niter <- 15000
8 y <- unlist(Data[, 2])
9
10 # Linear model
11 SSBoomLinear <- lm.spike(y ~ W, niter = niter)
12 Models <- SSBoomLinear$beta != 0
13 PIP <- colMeans(SSBoomLinear$beta != 0)
14 # Convert the logical matrix to a data frame and then to a
  tibble
15 df <- as.data.frame(Models); df_tbl <- as_tibble(df)
16 # Count identical rows
17 row_counts <- df_tbl %>% count(across(everything()), name =
  "frequency") %>% arrange(desc(frequency))
18 colnames(W)
19 # Get indices of best model
20 for(l in 1:5) {
21   print(which(row_counts[l,] == 1))
22   print(row_counts[l, dim(row_counts)[2]]/niter)
23 }
24 # Coefficients
25 SummarySS <- summary(coda::mcmc(SSBoomLinear$beta))
26 SummarySS

```

3. [51] proposes an SSVS approach for binary response models. Use the dataset *Conflict.xlsx*, where the dependent variable is *conflict_{cw}*, to perform SSVS using the *BoomSpikeSlab* package, specifically the `logit.spike` function, in order to identify the best subset of models. Compare the results with those obtained in Exercise 2.

Answer

The following code illustrates how to implement SSVS in this application, based on [21], using 15,000 iterations and the default settings for the hyperparameters with a logit model.

We find that the results using a logit model are very similar to those obtained with a linear model. Once again, infant mortality rate, population,

peace, and political rights emerge as the most relevant drivers of the incidence of civil conflict. *Peace* exhibits a negative 95% credible interval, while the other variables have positive credible intervals. In this case, the model with the highest posterior probability (35%) includes infant mortality rate and peace. However, the model with the second-highest posterior probability (17%) includes the first three drivers.

R code. Application: Drivers of civil conflict in the post-Cold War era using a logit model

```

1 rm(list = ls())
2 set.seed(010101)
3 Data <- read.csv("https://raw.githubusercontent.com/
  BEsmarter-consultancy/BSTApp/refs/heads/master/DataApp/
  Conflict.csv", sep = ",", header = TRUE, quote = "")
4
5 # Scale regressors
6 W <- as.matrix(scale(Data[, -c(1, 2)]))
7 niter <- 15000
8 y <- unlist(Data[, 2])
9
10 # Logit model
11 SSBoomLogit <- logit.spike(y ~ W, niter = niter)
12 ModelsLogit <- SSBoomLogit$beta != 0
13 PIPLogit <- colMeans(SSBoomLogit$beta != 0)
14 # Convert the logical matrix to a data frame and then to a
  tibble
15 df <- as.data.frame(ModelsLogit); df_tbl <- as_tibble(df)
16 # Count identical rows
17 row_counts <- df_tbl %>% count(across(everything()), name =
  "frequency") %>% arrange(desc(frequency))
18 colnames(W)
19 # Get indices of best model
20 for(l in 1:5) {
21   print(which(row_counts[l,] == 1))
22   print(row_counts[l, dim(row_counts)[2]]/niter)
23 }
24 # Coefficients
25 SummarySSlogit <- summary(coda::mcmc(SSBoomLogit$beta))
26 SummarySSlogit
27

```

4. Example: Simulation exercise $K > N$

Use the simulation setting from the Bayesian LASSO and SSVS examples, but now assume there are 600 features. This setup implies that the number of features exceeds the sample size. In such a scenario, there is no unique solution to the least squares estimator because the determinant of $\mathbf{W}^\top \mathbf{W}$

is zero. This means the matrix is not invertible, and consequently, standard inference procedures based on the least squares estimator cannot be applied. On the other hand, Bayesian inference in this setup is well-defined because the prior helps regularize the problem, which is a key motivation for these methods.

Answer

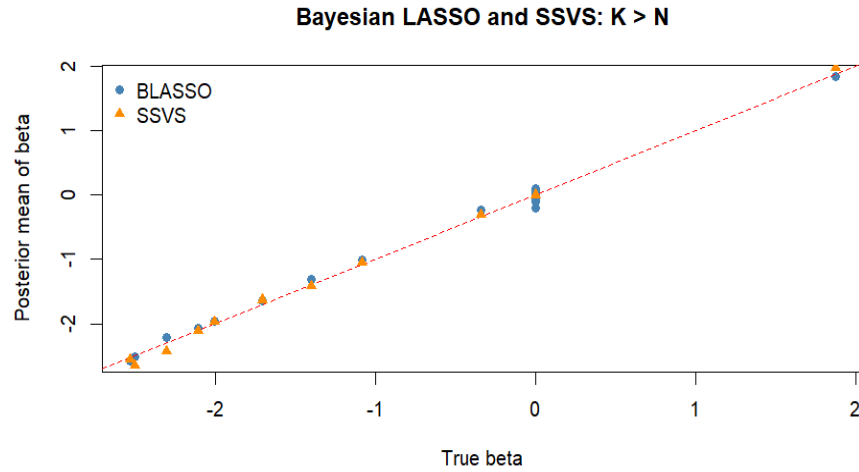
The following code shows how to perform this exercise using the *bayesreg* and *BoomSpikeSlab* packages. We observe that both the Bayesian LASSO and SSVS perform well (see Figure 12.2) and allow for valid inference.

R code. Simulation: Performance of the Bayesian LASSO and SSVS when $K > N$

```

1 rm(list = ls()); set.seed(10101)
2 library(BoomSpikeSlab)
3 library(dplyr)
4 library(tibble)
5 library(bayesreg)
6 # Parameters
7 n <- 500 # sample size
8 k <- 600 # number of predictors
9 s <- 10 # number of non-zero coefficients
10 # Generate design matrix
11 X <- matrix(rnorm(n * k), nrow = n, ncol = k)
12 # True beta: first s coefficients are non-zero, rest are
    zero
13 beta_true <- c(runif(s, -3, 3), rep(0, k - s))
14 # Generate response with some noise
15 sigma <- 1
16 y <- X %*% beta_true + rnorm(n, sd = sigma)
17 df <- data.frame(X, y)
18 ### Using BoomSpikeSlab ###
19 #Scale regressors
20 W <- scale(X); yh <- y - mean(y)
21 niter <- 5000
22 #####Estimate model#####
23 # Least squared
24 Reg <- lm(y ~ X)
25 summary(Reg)
26 ## BLASSO ##
27 # Fit the model
28 fit <- bayesreg::bayesreg(y ~ X, data = df, model = "
    gaussian", prior = "lasso",
29 n.samples = niter, burnin = 1000)
30 # Check summary
31 summary(fit)
32 # Extract posterior means of beta
33 beta_post_meanLASSO <- rowMeans(fit$beta)
34
35 ## SSVS ##
36 SSBoomNew <- lm.spike(yh ~ W - 1, niter = niter)
37 # Coefficients
38 SummarySS <- summary(coda::mcmc(SSBoomNew$beta))
39 # Extract posterior means of beta
40 beta_post_meanSSVS <- SummarySS$statistics[, 1]
41
42 # Compare true vs estimated
43 plot(beta_true, beta_post_meanLASSO, pch = 19, col = "
    steelblue",
44 xlab = "True beta", ylab = "Posterior mean of beta",
45 main = "Bayesian LASSO and SSVS")
46 abline(0, 1, col = "red", lty = 2)
47 # Add another set of posterior means
48 points(beta_true, beta_post_meanSSVS, pch = 17, col = "
    darkorange")
49 # Add legend
50 legend("topleft", legend = c("BLASSO", "SSVS"),
51 col = c("steelblue", "darkorange"),
52 pch = c(19, 17), bty = "n")

```

**FIGURE 12.2**

Performance of the Bayesian LASSO and SSVS: $K > N$.

5. Simulation exercise: the BART model continues

Compute Friedman's partial dependence functions [10] for all variables in the BART model simulation example, and plot the posterior mean along with the 95% credible intervals.

Answer

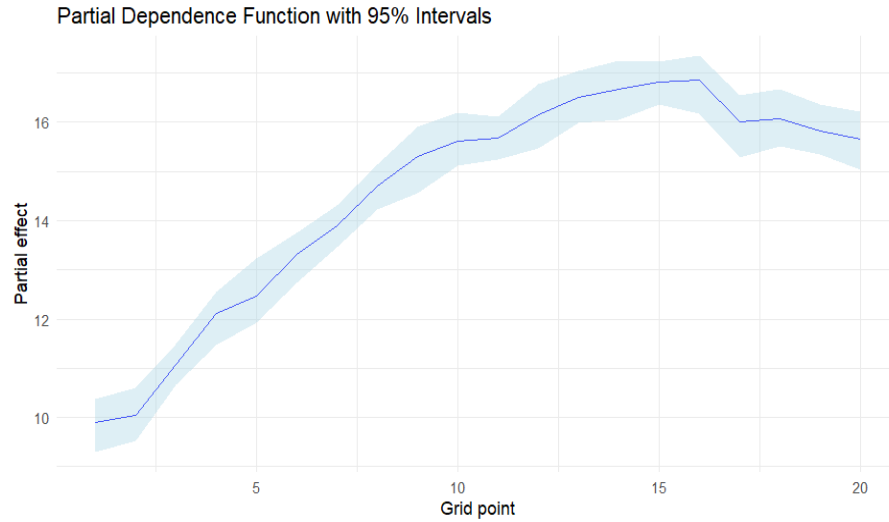
The following code illustrates how to compute the posterior distribution of Friedman's partial dependence functions. Figures 12.3 and 12.4 display the results for variables 1 and 10. From the simulation setup, we know that variable 1 is relevant, whereas variable 10 is irrelevant. This is reflected in the partial dependence functions: the function for variable 10 is flat, indicating no effect, while the function for variable 1 shows a positive relationship with the outcome variable.

R code. Simulation: Partial dependence functions from BART models

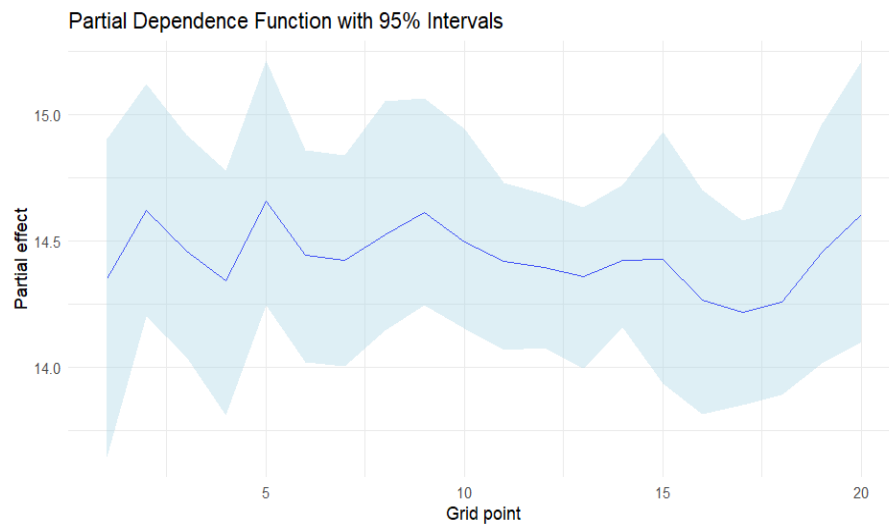
```

1 rm(list = ls()); set.seed(10101)
2 library(BART); library(ggplot2); library(dplyr); library(
  tidyr)
3 N <- 500; K <- 10
4 # Simulate the dataset
5 MeanFunc <- function(x){
6   f <- 10*sin(pi*x[,1]*x[,2]) + 20*(x[,3]-.5)^2+10*x[,4]+5*x
7     [,5]
8   return(f)
9 }
10 sig2 <- 1
11 e <- rnorm(N, 0, sig2^0.5)
12 X <- matrix(runif(N*K),N,K)
13 y <- MeanFunc(X[,1:5]) + e
14 # Hyperparameters
15 alpha <- 0.95; beta <- 2; k <- 2
16 v <- 3; q <- 0.9; J <- 200
17 # MCMC parameters
18 MCMCiter <- 1000; burnin <- 100; thinning <- 1
19 # Partial dependence function
20 BARTfit <- wbart(x.train = X, y.train = y, base = alpha,
21   power = beta, k = k, sigdf = v, sigquant = q, ntree = J,
22   ndpost = MCMCiter, nskip = burnin, keepevery = thinning)
23 k <- 1; L <- 20
24 x <- seq(min(X[, k]), max(X[, k]), length.out = L)
25 X.test <- X
26 X.test[,k] <- x[1]
27 for(j in 2:L){
28   X.k <- X
29   X.k[,k] <- x[j]
30   X.test <- rbind(X.test, X.k)
31 }
32 pred <- predict(BARTfit, X.test)
33 partial <- matrix(nrow = MCMCiter, ncol = L)
34 for(j in 1:L) {
35   h <- (j - 1) * N + 1:N
36   partial[, j] <- apply(pred[, h], 1, mean)
37 }
38 partial_df <- as.data.frame(partial)
39 partial_df$draw <- 1:nrow(partial_df)
40 long_df <- pivot_longer(partial_df, cols = -draw, names_to =
41   "x", values_to = "value")
42 long_df$x <- as.numeric(gsub("V", "", long_df$x))
43 summary_df <- long_df %>% group_by(x) %>%
44   summarise(mean = mean(value), lower = quantile(value, 0.025),
45     upper = quantile(value, 0.975))
46 ggplot(summary_df, aes(x = x, y = mean)) + geom_line(color =
47   "blue") + geom_ribbon(aes(ymin = lower, ymax = upper),
48   fill = "lightblue", alpha = 0.4) + labs(x = "Grid point",
49   y = "Partial effect", title = "Partial Dependence
50   Function with 95% Intervals") + theme_minimal()

```

**FIGURE 12.3**

Partial dependence function: Variable 1.

**FIGURE 12.4**

Partial dependence function: Variable 10.

6. [51] proposes an SSVS approach for binary response models. Use the dataset *Conflict.csv*, where the dependent variable is *conflict_{cw}*, to perform SSVS using the *BoomSpikeSlab* package, specifically the `logit.spike`

function, in order to identify the best subset of models. Compare the results with those obtained in Exercise 2.

Answer

The following code demonstrates how to perform probit BART for binary classification. According to the results obtained using 10 trees, the most relevant regressors are *infant mortality rate*, *peace*, *political rights*, and *population*. Note that this agrees with previous exercises using this dataset.

R code. Application: Probit BART for binary classification

```

1 rm(list = ls()); set.seed(10101)
2 library(BART); library(tidyr)
3 library(pROC); library(caret)
4 # Load and prepare data
5 Data <- read.csv("https://raw.githubusercontent.com/
    BEsmarter-consultancy/BSTApp/refs/heads/master/DataApp/
    Conflict.csv", sep = ",", header = TRUE, quote = "")
6 X <- as.matrix(Data[, -c(1, 2)])
7 y <- unlist(Data[, 2])
8 # MCMC parameters
9 niter <- 1000; burnin <- 200; J <- 200
10 # Create 5-fold cross-validation indices
11 folds <- createFolds(y, k = 5, list = TRUE, returnTrain =
    FALSE)
12 thresholds <- seq(0.01, 0.99, by = 0.01)
13 scores <- matrix(0, nrow = 5, ncol = length(thresholds))
14 for (i in 1:5) {
15   test_idx <- folds[[i]]; train_idx <- setdiff(1:length(y),
    test_idx)
16   X.train <- X[train_idx, ]; y.train <- y[train_idx]
17   X.test <- X[test_idx, ]; y.test <- y[test_idx]
18   pf <- pbart(x.train = X.train, y = y.train, x.test = X.
    train, ntree = J, ndpost = niter, nskip = burnin)
19   prob_train <- pf$prob.test.mean
20   for (j in 1:length(thresholds)) {
21     thresh <- thresholds[j]
22     preds <- ifelse(prob_train > thresh, 1, 0)
23     TP <- sum(preds == 1 & y.train == 1)
24     TN <- sum(preds == 0 & y.train == 0)
25     FP <- sum(preds == 1 & y.train == 0)
26     FN <- sum(preds == 0 & y.train == 1)
27     TPR <- TP / (TP + FN + 1e-8)
28     TNR <- TN / (TN + FP + 1e-8)
29     scores[i, j] <- TPR + TNR
30   }
31 }
32 mean_scores <- colMeans(scores)
33 best_thresh <- thresholds[which.max(mean_scores)]
34 cat("Optimal threshold by CV:", best_thresh, "\n")
35 # Train/test split to evaluate final performance
36 N <- nrow(X); K <- ncol(X)
37 N.train <- floor(0.8 * N); X.train <- X[1:N.train, ]; y.
    train <- y[1:N.train]
38 X.test <- X[(N.train + 1):N, ]; y.test <- y[(N.train + 1):N]
39 pf_final <- pbart(x.train = X.train, y = y.train, x.test = X.
    test, ntree = 200, ndpost = 1000, nskip = 200)
40 prob_test <- pf_final$prob.test.mean
41 y_pred_test <- ifelse(prob_test > best_thresh, 1, 0)
42 conf_mat <- table(Predicted = y_pred_test, Actual = y.test)
43 accuracy <- mean(y_pred_test == y.test)
44 print(conf_mat)
45 cat("Test set accuracy:", round(accuracy, 3), "\n")

```

R code. Application: Probit BART for binary classification

```

1 # Relevant regressors
2 Js <- c(10, 20, 50, 100, 200)
3 VarImportance <- matrix(0, length(Js), K)
4 l <- 1
5 for (j in Js){
6   BARTfit <- BART::pbart(x.train = X.train, y = y.train, x.
7     test = X.test, ntree = j, ndpost = niter, nskip = burnin
8   )
9   VarImportance[l, ] <- BARTfit[["varcount.mean"]]/j
10  l <- l + 1
11 }
12 # Assign row and column names for clarity
13 rownames(VarImportance) <- as.character(Js)
14 colnames(VarImportance) <- as.character(1:K)
15 # Convert to long format
16 importance_df <- as.data.frame(VarImportance) %>%
17 mutate(trees = rownames(.)) %>%
18 pivot_longer(
19   cols = -trees,
20   names_to = "variable",
21   values_to = "percent_used"
22 )
23 # Make variable numeric for plotting
24 importance_df$variable <- as.numeric(importance_df$variable)
25 importance_df$trees <- factor(importance_df$trees, levels =
26   c("10", "20", "50", "100", "200"))
27
28 ggplot(importance_df, aes(x = variable, y = percent_used,
29   color = trees, linetype = trees)) +
30 geom_line() + geom_point() + scale_color_manual(values = c("
31   10" = "red", "20" = "green", "50" = "blue", "100" = "
32   cyan", "200" = "magenta")) + scale_x_continuous(breaks =
33   1:K) + labs(x = "variable", y = "percent used", color =
34   "#trees", linetype = "#trees") + theme_minimal()
35
36 data.frame(Nanes = colnames(X), Frequency = VarImportance
37   [1,])

```

7. Simulation exercise: The Gaussian Process simulation continues

Simulate the process

$$f_i = \sin(2\pi x_{i1}) + \cos(2\pi x_{i2}) + \sin(x_{i1}x_{i2}) + \mu_i,$$

where $\mu_i \stackrel{\text{i.i.d.}}{\sim} N(0, 0.1^2)$, $x_{ik} \sim U(0, 1)$ for $k = 1, 2$, and the sample size is 500.

Define a grid of 20 evenly spaced values between 0 and 1 for each covariate x_{ik} , and use this grid to perform prediction.

Estimate the hyperparameters of the Gaussian Process by maximizing the log marginal likelihood. Then, use the *km* function from the *DiceKriging* package to fit the Gaussian Process, fixing the noise variance at the value that maximizes the log marginal likelihood.

Finally, use the fitted model to predict the outputs on the grid points, and produce a 3D plot showing the predicted surface along with the training data points.

Answer

We use the Cholesky decomposition to optimize time calculating the inverse of the covariance matrix. In particular, let $\mathbf{K} \in \mathbb{R}^{N \times N}$ be a symmetric positive definite matrix. Then, it admits a Cholesky decomposition:

$$\mathbf{K} = \mathbf{L}\mathbf{L}^\top,$$

where $\mathbf{L}$ is a lower triangular matrix with positive diagonal entries.

The determinant of $\mathbf{K}$ can be computed using the Cholesky factor:

$$\det(\mathbf{K}) = \det(\mathbf{L}\mathbf{L}^\top) = \det(\mathbf{L})^2.$$

Since $\mathbf{L}$ is lower triangular, its determinant is the product of its diagonal elements:

$$\det(\mathbf{K}) = \left(\prod_{i=1}^N L_{ii} \right)^2.$$

Equivalently, the log-determinant is:

$$\log |\mathbf{K}| = 2 \sum_{i=1}^N \log L_{ii}.$$

The inverse of $\mathbf{K}$ can also be computed using the Cholesky factor:

$$\mathbf{K}^{-1} = (\mathbf{L}\mathbf{L}^\top)^{-1} = (\mathbf{L}^\top)^{-1}\mathbf{L}^{-1}.$$

This is useful computationally because triangular systems can be solved efficiently via forward and backward substitution:

- (a) Solve $\mathbf{L}\mathbf{z} = \mathbf{b}$ for $\mathbf{z}$ (forward solve),
- (b) Solve $\mathbf{L}^\top \mathbf{x} = \mathbf{z}$ for $\mathbf{x}$ (backward solve),

then, $\mathbf{x} = (\mathbf{L}^\top)^{-1}\mathbf{z} = (\mathbf{L}^\top)^{-1}\mathbf{L}^{-1}\mathbf{b} = (\mathbf{L}\mathbf{L}^\top)^{-1}\mathbf{b} = \mathbf{K}^{-1}\mathbf{b}$. Thus, we avoid computing the matrix inverse, which requires $O(N^3)$ operations. Instead, it is more efficient and numerically stable to solve triangular systems of equations, which arise naturally from the Cholesky decomposition.

The following code shows how to perform this exercise, and Figure 12.5 displays the 3D plot.

R code. Simulation:
 $f_i = \sin(2\pi x_{i1}) + \cos(2\pi x_{i2}) + \sin(x_{i1}x_{i2}) + \mu_i$ and
Gaussian process

```

1 rm(list = ls()); set.seed(10101)
2 library(fields); library(DiceKriging); library(ggplot2)
3 # Simulate training data
4 n_train <- 500
5 x1 <- runif(n_train); x2 <- runif(n_train)
6 X_train <- data.frame(x1 = x1, x2 = x2)
7 sig <- 0.1; u <- rnorm(n_train, mean = 0, sd = sig)
8 # True function without noise
9 f_train <- sin(2 * pi * X_train$x1) + cos(2 * pi * X_train$x2) + sin(X_train$x1 * X_train$x2)
10 y_train <- scale(f_train + u)
11 # Empirical Bayes: Optimize the log marginal likelihood
12 log_marginal_likelihood <- function(par, X, y) {
13   sigma2_f <- par[1]; l2 <- par[2]; sigma2_n <- par[3]; n <-
     nrow(X)
14   # Ensure parameters are positive
15   if (sigma2_f <= 0 | l2 <= 0 | sigma2_n <= 0) return(Inf)
16   # Squared exponential kernel
17   dists <- rdist(X)
18   K <- sigma2_f * exp(-0.5 * (dists^2) / l2)
19   # Add noise variance + jitter for numerical stability
20   K <- K + (sigma2_n + 1e-8) * diag(n)
21   # Cholesky decomposition
22   L <- tryCatch(chol(K), error = function(e) return(NULL))
23   if (is.null(L)) return(Inf)
24   alpha <- backsolve(t(L), forwardsolve(L, y)) #--> K^{-1}y
25   log_det_K <- 2 * sum(log(diag(L)))
26   lml <- -0.5 * t(y) %*% alpha - 0.5 * log_det_K - 0.5 * n *
     log(2 * pi)
27   return(-as.numeric(lml)) # Negative log-marginal
     likelihood
28 }
29 par0 <- rep(1, 3)
30 ResOpt <- optim(par = par0, fn = log_marginal_likelihood,
31   method = "L-BFGS-B", lower = c(1e-5, 1e-5), X = X_train,
32   y = y_train)
33 theta_hat <- ResOpt$par
34 sigma2_f_hat <- theta_hat[1]; l2_hat <- theta_hat[2]; sigma2
   _n_hat <- theta_hat[3]
35 # Grid for prediction
36 grid_points <- 20
37 x1_seq <- seq(0, 1, length.out = grid_points)
38 x2_seq <- seq(0, 1, length.out = grid_points)
39 X_new <- expand.grid(x1 = x1_seq, x2 = x2_seq)
40 # Fit Gaussian Process
41 fit_km <- km(design = X_train, response = y_train, covtype =
42   "gauss", noise.var = rep(sigma2_n_hat, n_train))
43 # Predict GP surface
44 pred <- predict(fit_km, newdata = X_new, type = "UK")
45 z_pred <- matrix(pred$mean, nrow = grid_points, ncol = grid_
46   points)
47 # Plot
48 persp3d(x = x1_seq, y = x2_seq, z = z_pred, col = "lightblue",
49   alpha = 0.7, xlab = "x1", ylab = "x2", zlab = "GP
     Mean")
50 points3d(x = X_train$x1, y = X_train$x2, z = y_train, col =
51   "red", size = 8)

```

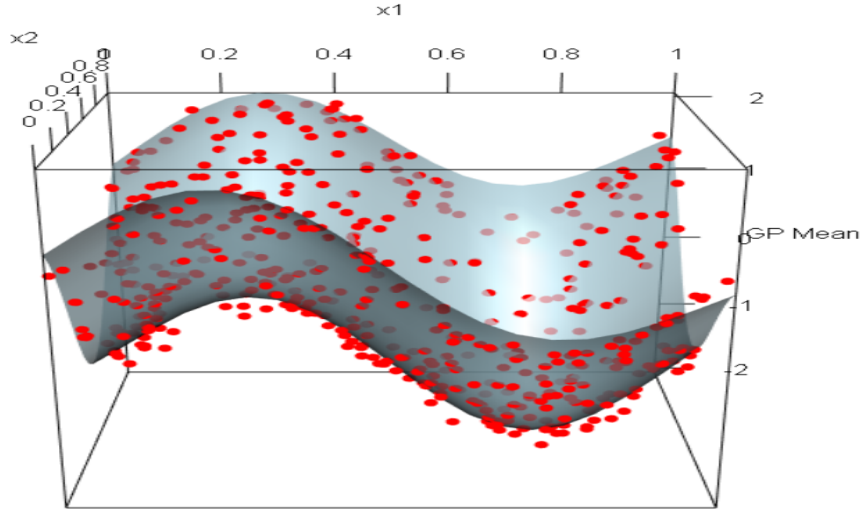



FIGURE 12.5

Simulation: $f_i = \sin(2\pi x_{i1}) + \cos(2\pi x_{i2}) + \sin(x_{i1}x_{i2}) + \mu_i$ and Gaussian process

8. Simulation exercise: Stochastic gradient MCMC continues

Program from scratch the stochastic gradient Langevin dynamics algorithm for the logit simulation exercise implementing the control variate version performing 1,500 stochastic gradient descent iterations to locate the posterior mode, which should be then used as the initial value for 1,000 subsequent MCMC iterations using a step size set to 1×10^{-4} .

Answer

The following code illustrates how to implement the Stochastic Gradient Langevin Dynamics (SGLD) algorithm with a control variate. Figure 12.6 shows the posterior distributions obtained from the SGLD algorithm with a control variate and the Metropolis-Hastings algorithm. We observe that the former is well centered, but exhibits overdispersion; however, the control variate helps reduce the variability compared to the naive SGLD algorithm (see Figure 12.10 in the book).

*R code. Stochastic Gradient Langevin Dynamics
control variate (SGLDcv): Logit regression*

```

1 rm(list = ls()); set.seed(10101)
2 library(mvtnorm); library(MCMCpack)
3 library(ggplot2); library(dplyr)
4 #--- Generate correlated covariates
5 genCovMat <- function(K, rho = 0.4) {
6   Sigma0 <- matrix(1, K, K)
7   for (i in 2:K) {
8     for (j in 1:(i - 1)) {
9       Sigma0[i, j] <- runif(1, -rho, rho)^(i - j)
10    }
11  }
12  Sigma0 <- Sigma0 * t(Sigma0)
13  diag(Sigma0) <- 1
14  return(Sigma0)
15 }
16 #--- Simulate logistic regression data
17 simulate_logit_data <- function(K, N, beta_true) {
18   Sigma0 <- genCovMat(K)
19   X <- rmvnorm(N, mean = rep(0, K), sigma = Sigma0)
20   linpred <- X %*% beta_true
21   p <- 1 / (1 + exp(-linpred))
22   y <- rbinom(N, 1, p)
23   list(y = y, X = X)
24 }
25 #--- One SGD step
26 SGD_step <- function(beta, y, X, lr, batch_size, prior_var =
27   10) {
28   N <- nrow(X)
29   K <- length(beta)
30   ids <- sample(1:N, size = batch_size, replace = FALSE)
31   grad <- rep(0, K)
32   for (i in ids) {
33     xi <- X[i, ]
34     eta <- sum(xi * beta)
35     pi <- 1 / (1 + exp(-eta))
36     grad_i <- -(y[i] - pi) * xi
37     grad <- grad + grad_i
38   }
39   grad <- grad / batch_size + beta / prior_var * 1 / N
40   beta_new <- beta - lr * grad
41   return(beta_new)
42 }
43 #--- Parameters
44 K <- 10; N <- 100000
45 beta_true <- rep(0.5, K); batch_prop <- 0.01
46 batch_size <- round(N*batch_prop); n_iter <- 1000
47 n_iterSGD <- 1500; kappa <- 0.55
48 stepsize <- 1e-4; prior_var <- 10
49 k_target <- 5 # beta5
50
51 sim_data <- simulate_logit_data(K, N, beta_true)
52 y <- sim_data$y
53 X <- scale(sim_data$X)

```

*R code. Stochastic Gradient Langevin Dynamics
control variate (SGLDcv): Logit regression*

```

1 beta_mat <- matrix(0, n_iterSGD, K)
2 beta_mat[1, ] <- rep(0, K)
3 for (s in 2:n_iterSGD) {
4   beta_mat[s, ] <- SGD_step(beta_mat[s - 1, ], y = y, X = X,
5     lr = s^(-kappa), batch_size = batch_size, prior_var =
6     prior_var)
7 }
8 matplot(beta_mat, type = "l"); betahat <- beta_mat[s, ]
9 gradhat <- rep(0, K)
10 for (i in 1:N) {
11   xi <- X[i, ]
12   etahat <- sum(xi * betahat)
13   pihat <- 1 / (1 + exp(-etahat))
14   grad_ihat <- -(y[i] - pihat) * xi
15   gradhat <- gradhat + grad_ihat
16 }
17 #--- One SGLD control variate step
18 SGLDcv_step <- function(beta, betahat, gradhat, y, X,
19   stepsize, batch_size, prior_var = 10) {
20   N <- nrow(X)
21   K <- length(beta)
22   ids <- sample(1:N, size = batch_size, replace = FALSE)
23   grad_dif <- rep(0, K)
24   for (i in ids) {
25     xi <- X[i, ]
26     eta <- sum(xi * beta)
27     pi <- 1 / (1 + exp(-eta))
28     etahat <- sum(xi * betahat)
29     pihat <- 1 / (1 + exp(-etahat))
30     grad_dif_i <- (pi - pihat) * xi
31     grad_dif <- grad_dif + grad_dif_i
32   }
33   grad <- gradhat + grad_dif / batch_size * N
34   grad <- grad + beta / prior_var # gradient of log-prior
35   noise <- rnorm(K, 0, sqrt(stepsize))
36   beta_new <- beta - 0.5 * stepsize * grad + noise
37   return(beta_new)
38 }
39 #--- SGLD algorithm
40 run_SGLDcv <- function(y, X, stepsize, batch_prop, n_iter,
41   beta_init = NULL, prior_var = prior_var) {
42   N <- nrow(X)
43   K <- ncol(X)
44   batch_size <- round(batch_prop * N)
45   beta_mat <- matrix(0, n_iter, K)
46   beta_mat[1, ] <- if (is.null(beta_init)) rep(0, K) else
47     beta_init
48   for (s in 2:n_iter) {
49     beta_mat[s, ] <- SGLDcv_step(beta_mat[s - 1, ], betahat
50       = betahat, gradhat = gradhat, y, X, stepsize, batch_size
51     )
52   }
53   return(beta_mat)
54 }
55 #--- Run SGLD cv
56 posterior_sgldcv <- run_SGLDcv(y, X, stepsize, batch_prop, n
57   _iter)

```

***R code. Stochastic Gradient Langevin Dynamics
control variate (SGLDcv): Logit regression***

```

1 #--- Run MCMCpack logit
2 df <- as.data.frame(X)
3 colnames(df) <- paste0("X", 1:K)
4 df$y <- y
5 formula <- as.formula(paste("y ~", paste(colnames(df)[1:K],
6 collapse = " + "), "-1"))
7 posterior_mh <- MCMClogit(formula, data = df, b0 = 0, B0 =
8 0.1,
9 burnin = n_iterSGD, mcmc = n_iter)
10 #--- Compare densities for beta5
11 df_plot <- data.frame(
12 value = c(posterior_sgldcv[, k_target], posterior_mh[, k_
13 target]),
14 method = rep(c("SGLDcv", "MCMC"), each = n_iter)
15 )
16 ggplot(df_plot, aes(x = value, fill = method, color = method
17 )) + geom_density(alpha = 0.4) + geom_vline(xintercept =
18 beta_true[k_target], linetype = "dashed", color = "
19 black") + labs(title = expression(paste("Posterior
20 density of ", beta[k_target])), x = expression(beta[k_
21 target]), y = "Density") + theme_minimal() + xlim(0.45,
22 0.55)

```

9. Perform the simulation according to the model $y_i = 1 - 2x_{i1} + 0.5x_{i2} + \mu_i$, where $\mu_i \sim N(0, 1)$, the sample size is 100,000, and the covariates $\mathbf{x}_i \sim N(\mathbf{0}, \mathbf{I}_2)$. Use 5,000 MCMC iterations and a batch size of 1,000 to implement the SGLD algorithm. Set a learning rate schedule that yields sensible results.

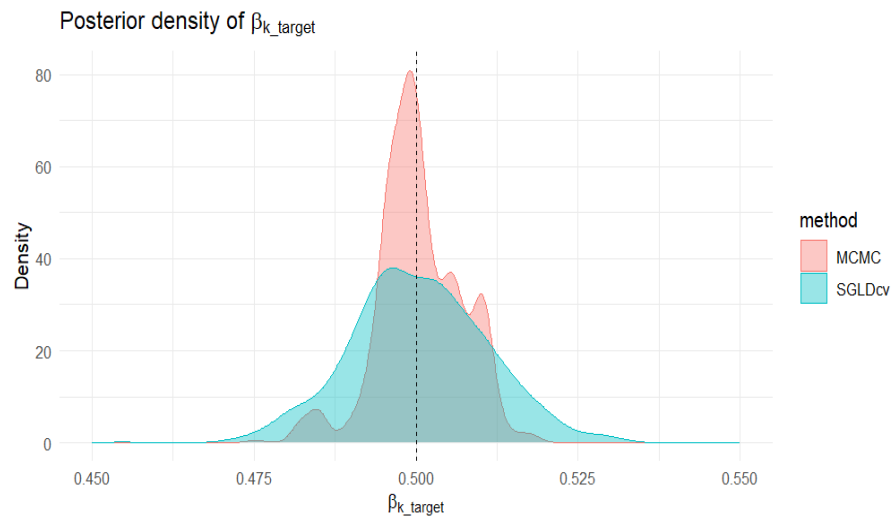
Assume independent priors $\pi(\boldsymbol{\beta}, \sigma^2) = \pi(\boldsymbol{\beta}) \times \pi(\sigma^2)$, with $\boldsymbol{\beta} \sim N(\mathbf{0}, \mathbf{I}_3)$ and $\sigma^2 \sim IG(\alpha_0/2, \delta_0/2)$, where $\alpha_0 = \delta_0 = 0.01$.

Answer

Take into account that SGLD was originally developed for parameter spaces in $\mathbb{R}^K$. In this exercise, we use SGLD to perform inference on $\boldsymbol{\beta}$, while obtaining draws from the conditional posterior distribution of $\sigma^2 \sim IG(\alpha_n/2, \delta_n/2)$, where $\alpha_n = N + \alpha_0$ and $\delta_n = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + \delta_0$.

Additionally, recall the gradients involved in the SGLD updates:

$$\begin{aligned}\nabla_{\boldsymbol{\beta}} \log p(\mathbf{y} \mid \boldsymbol{\beta}, \sigma^2) &= \frac{1}{\sigma^2} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}), \\ \nabla_{\boldsymbol{\beta}} \log \pi(\boldsymbol{\beta}) &= -\boldsymbol{\beta}.\end{aligned}$$

**FIGURE 12.6**

Simulation: Posterior distributions from logit simulation exercise

The following code illustrates how to implement this procedure. We note that the results for the location parameters β are satisfactory; however, the algorithm appears to exhibit a positive bias in the estimation of σ^2 .

***R code. Stochastic Gradient MCMC
(SG-MCMC): Linear regression***

```

1 rm(list = ls()); set.seed(10101)
2 # Simulated data
3 N <- 100000; K <- 3
4 X <- cbind(1, matrix(rnorm(N * (K - 1)), N, K - 1)) #
   design matrix
5 beta_true <- c(1, -2, 0.5); sigma2_true <- 1
6 y <- X %>% beta_true + rnorm(N, 0, sqrt(sigma2_true))
7 X[, 2:3] <- scale(X[, 2:3])
8 # SGLD settings
9 iterations <- 5000
10 batch_size <- 1000
11 initial_stepsize <- 1e-4
12 decay_rate <- 0.55
13 schedule <- 1000
14 # Priors
15 a0 <- 0.01; b0 <- 0.01
16 # Storage
17 trace_beta <- matrix(NA, nrow = iterations, ncol = K)
18 trace_sigma2 <- rep(NA, iterations)
19 # Initialization
20 beta <- rep(0, K)
21 sigma2 <- 0.5
22 # Gradient of log-posterior w.r.t. beta (scaled)
23 grad_log_post <- function(beta, X_batch, y_batch, N, sigma2)
24 {
25   grad_loglik <- t(X_batch) %>% (y_batch - X_batch %>% beta)
26   / sigma2
27   grad_logprior <- -beta # derivative of log N(0, I)
28   return((N / nrow(X_batch)) * grad_loglik + grad_logprior)
29 }

```

*R code. Stochastic Gradient MCMC
(SG-MCMC): Linear regression*

```

1 # SGLD algorithm
2 for (t in 1:iterations) {
3   # Step size
4   stepsize_t <- initial_stepsize / (1 + t / schedule)^decay_
      rate
5   # Sample mini-batch
6   idx <- sample(1:N, batch_size)
7   X_batch <- X[idx, ]
8   y_batch <- y[idx]
9   # Beta update via SGLD
10  grad <- grad_log_post(beta, X_batch, y_batch, N, sigma2)
11  noise <- rnorm(K, 0, sqrt(stepsize_t))
12  beta <- beta + 0.5 * stepsize_t * grad + noise
13  # Sigma2 update via inverse-gamma (full conditional)
14  resid <- y - X %*% beta
15  shape <- (a0 + N) / 2
16  rate <- (b0 + sum(resid^2)) / 2
17  sigma2 <- 1 / rgamma(1, shape = shape, rate = rate)
18  # Store draws
19  trace_beta[t, ] <- beta
20  trace_sigma2[t] <- sigma2
21 }
22 # Plot trace of beta[2]
23 plot(trace_beta[4001:5000, 2], type = "l", col = "blue", lwd
      = 2,
24 main = expression("SGLD: Evolution of " * beta[2]),
25 xlab = "Iteration", ylab = expression(beta[2]))
26 abline(h = beta_true[2], col = "red", lty = 2)
27 # Plot trace of sigma2
28 plot(trace_sigma2[4001:5000], type = "l", col = "darkgreen",
      lwd = 2,
29 main = expression("Trace plot of " * sigma^2),
30 xlab = "Iteration", ylab = expression(sigma^2))
31 abline(h = sigma2_true, col = "red", lty = 2)

```



13

Causal inference

Solutions of Exercises

1. Show that the Average Treatment Effect (ATE) in the simple linear regression framework

$$Y_i = \beta_0 + \tau D_i + \mu_i,$$

assuming non-informative prior distributions, so that the posterior mean of the location parameter coincides with the maximum likelihood estimator, is equal to

$$\bar{y}_1 - \bar{y}_0.$$

Answer

Consider the simple linear regression model

$$Y_i = \beta_0 + \tau D_i + \mu_i, \quad i = 1, \dots, N,$$

where $D_i \in \{0, 1\}$ is a treatment indicator. In matrix notation:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\mu}, \quad \mathbf{X} = \begin{pmatrix} 1 & D_1 \\ \vdots & \vdots \\ 1 & D_n \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \tau \end{pmatrix}.$$

Under the usual Gaussian likelihood and a non-informative prior $\pi(\boldsymbol{\beta}, \sigma^2) \propto \sigma^{-2}$, the posterior mean of $\boldsymbol{\beta}$ coincides with the maximum likelihood estimator (MLE), which is the ordinary least squares (OLS) estimator:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Compute $\mathbf{X}^\top \mathbf{X}$ and $\mathbf{X}^\top \mathbf{y}$:

$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} N & N_1 \\ N_1 & N_1 \end{pmatrix}, \quad \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} \sum_{i=1}^N y_i \\ \sum_{i:D_i=1} y_i \end{pmatrix},$$

where $N_1 = \sum_i D_i$ and $N_0 = N - N_1$. Let $\bar{y}_1 = \frac{1}{N_1} \sum_{i:D_i=1} y_i$ and $\bar{y}_0 = \frac{1}{N_0} \sum_{i:D_i=0} y_i$.

Compute the inverse:

$$(\mathbf{X}^\top \mathbf{X})^{-1} = \frac{1}{N_0 N_1} \begin{pmatrix} N_1 & -N_1 \\ -N_1 & N \end{pmatrix}.$$

Thus:

$$\hat{\boldsymbol{\beta}} = \frac{1}{N_0 N_1} \begin{pmatrix} N_1 & -N_1 \\ -N_1 & N \end{pmatrix} \begin{pmatrix} \sum_{i=1}^N y_i \\ \sum_{i:D_i=1} y_i \end{pmatrix} = \begin{pmatrix} \bar{y}_0 \\ \bar{y}_1 - \bar{y}_0 \end{pmatrix}.$$

Therefore, the intercept equals the mean outcome for the control group ($\bar{y}_0$), and the slope equals the difference in means between treated and control groups:

$$\hat{\tau} = \bar{y}_1 - \bar{y}_0.$$

Since under a non-informative prior the posterior mean equals the ML estimator, the posterior mean of τ coincides with the sample difference in means:

$$\text{ATE} = \mathbb{E}[\tau \mid \mathbf{y}] = \bar{y}_1 - \bar{y}_0.$$

2. Some readers may question the assumption that potential outcomes are normally distributed. However, it is important to note that the normal distribution is the *maximum entropy* continuous distribution given a specified mean μ and finite variance σ^2 . In other words, among all distributions with the same mean and variance, the normal distribution represents the one with the greatest level of uncertainty or unpredictability [9].

Show that the normal distribution is the *maximum entropy* continuous distribution given a specified mean μ and finite variance σ^2 by considering the formal definition of entropy:

$$H(f) = - \int_{-\infty}^{\infty} f(y) \log f(y) dy,$$

where $f(y)$ is a probability density function.

Answer

We want to maximize $H(f)$ subject to the constraints:

$$\int_{-\infty}^{\infty} f(y) dy = 1, \quad \int_{-\infty}^{\infty} y f(y) dy = \mu, \quad \int_{-\infty}^{\infty} (y - \mu)^2 f(y) dy = \sigma^2.$$

Form the Lagrangian:

$$\mathcal{L}(f) = - \int f(y) \log f(y) dy + \lambda_0 \left(\int f(y) dy - 1 \right) + \lambda_1 \left(\int y f(y) dy - \mu \right) + \lambda_2 \left(\int (y - \mu)^2 f(y) dy - \sigma^2 \right).$$

Taking the functional derivative with respect to $f(y)$:

$$\frac{\delta \mathcal{L}}{\delta f} = -(\log f(y) + 1) + \lambda_0 + \lambda_1 y + \lambda_2 (y - \mu)^2 = 0.$$

Thus:

$$\log f(y) = \lambda'_0 + \lambda_1 y + \lambda_2 (y - \mu)^2,$$

where λ'_0 absorbs constants. Exponentiating:

$$f(y) \propto \exp(\lambda_1 y + \lambda_2 (y - \mu)^2).$$

For integrability, $\lambda_2 < 0$. Completing the square:

$$f(y) \propto \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right),$$

which is the kernel of a normal density. Normalizing:

$$f(y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right).$$

Therefore, the maximum entropy distribution for a given mean μ and variance σ^2 is

$$Y \sim N(\mu, \sigma^2).$$

3. Use the package *dagitty* to construct the DAG in Figure 13.5, verify that it is acyclic, and check whether the causal effect of D on Y is identifiable by controlling for $\mathbf{X}$.

Answer

*R code. Simple exercise in dagitty package:
Conditional independence assumption*

```

1 library(dagitty)
2 library(ggdag)
3
4 Gd = dagitty('dag{
5   X [pos="-1,1"]
6   D [exposure, pos="0,0"]
7   Y [outcome, pos="1,1"]
8   X -> D
9   X -> Y
10  D -> Y
11 }')
12
13 ggdag(Gd) + theme_dag()
14
15 isAcyclic(Gd)
16
17 for( n in names(Gd) ){
18   for( m in children(Gd,n) ){
19     a <- adjustmentSets(Gd, n, m, effect = c("direct"), type
20       = c("minimal"))
21     if( length(a) > 0 ){
22       cat("The effect ",n,"->",m,
23         " is identifiable by controlling for:\n",sep="")
24       print( a, prefix=" * " )
25     }
26   }
27 }

```

4. Use the package *dagitty* to construct the DAG in Figure 13.8, verify that it is acyclic, and check that the causal effect of *D* on *Y* is identifiable by controlling for *X* but not for *C*.

Answer

*R code. Simple exercise in dagitty package:
Collider bias*

```

1 ##### DAG: Collider bias #####
2 library(dagitty)
3 library(ggdag)
4
5 Gd = dagitty('dag{
6   X [pos="-1,1"]
7   D [exposure, pos="0,0"]
8   Y [outcome, pos="1,1"]
9   C [pos="0,0.5"]
10  X -> D
11  X -> Y
12  X -> C
13  D -> Y
14  D -> C
15 }')
16
17 ggdag(Gd) + theme_dag()
18
19 isAcyclic(Gd)
20
21 adjustmentSets(Gd, exposure = "D", outcome = "Y")
22
23 for( n in names(Gd) ){
24   for( m in children(Gd,n) ){
25     a <- adjustmentSets(Gd, n, m, effect = c("direct"), type
26       = c("minimal"))
27     if( length(a) > 0 ){
28       cat("The effect ",n,"->",m,
29         " is identifiable by controlling for:\n",sep="")
30       print( a, prefix=" * " )
31     }
32   }
33 }

```

5. Use the package *dagitty* to construct the DAG in Figure 13.11, taking into account that **U** is unobserved (latent). Verify that it is acyclic, check that **Z** is a valid instrument, and determine whether the causal effect of **D** on **Y** is identifiable.

Answer

*R code. Simple exercise in dagitty package:
Instrumental variable*

```

1 ##### DAG: Instruments #####
2 library(dagitty)
3 library(ggdag)
4
5 Gd = dagitty('dag{
6   U [latent, pos="-1,1"]
7   D [exposure, pos="0,0"]
8   Y [outcome, pos="1,1"]
9   Z [pos="0,0.5"]
10  U -> D
11  U -> Y
12  D -> Y
13  Z -> D
14 }')
15
16 ggdag(Gd) + theme_dag()
17
18 isAcyclic(Gd)
19
20 instrumentalVariables(Gd)
21
22 adjustmentSets(Gd, exposure = "D", outcome = "Y")

```

6. 401(k) participation on net financial assets continues I

Apply the framework from this example to compute the intention-to-treat effect, the local average treatment effect, and the effect of eligibility on participation.

Answer

Taking into account that

$$\begin{aligned}
 \tau_{LATE} &= \mathbb{E}[Y_i(1) - Y_i(0) \mid D(1) = 1, D(0) = 0] \\
 &= \frac{\mathbb{E}[Y_i \mid Z_i = 1] - \mathbb{E}[Y_i \mid Z_i = 0]}{\mathbb{E}[D_i \mid Z_i = 1] - \mathbb{E}[D_i \mid Z_i = 0]},
 \end{aligned}$$

we can recover the ITT effect by multiplying the LATE by the effect of eligibility on participation. The following code illustrates this procedure and plots the ITT. The posterior distribution of the ITT is shown in Figure 13.1, which closely resembles the posterior distribution of the effect of eligibility reported in the main text.

R code. Treatment effect: 401(k) participation on net financial assets

```

1 rm(list = ls()); set.seed(10101)
2 library(coda); library(ggplot2)
3 mydata <- read.csv("https://raw.githubusercontent.com/
  BEsmarter-consultancy/BSTApp/refs/heads/master/DataApp/
  401k.csv", sep = ",", header = TRUE, quote = "")
4 # Attach variables
5 attach(mydata)
6 y <- net_tfa/1000 # Outcome: net financial assets
7 x <- as.vector(p401) # Endogenous regressor: participation
8 w <- as.matrix(cbind(1, age, inc, fsize, educ, marr, twoearn
  , db, pira, hown)) # Exogenous regressors with
  intercept
9 z <- as.matrix(e401) # Instrument: eligibility (NO
  intercept here)
10 X <- cbind(x, w); Z <- cbind(z, w)
11 # Dimensions
12 k <- ncol(X); kz <- ncol(Z)
13 # Priors
14 b0 <- rep(0, k); B0i <- diag(1e-5, k)
15 g0 <- rep(0, kz); G0i <- diag(1e-5, kz)
16 nu <- 3; Psi0 <- nu * 1000 * diag(2); Psi0i <- solve(Psi0)
17 # MCMC parameters
18 mcmc <- 5000; burnin <- 1000
19 tot <- mcmc + burnin; thin <- 1
20 # Auxiliary elements
21 XtX <- t(X)%*%X; ZtZ <- t(Z)%*%Z; nun <- nu + length(y)
22 # Gibbs sampling
23 PostBeta <- function(Sigma, Gamma){
24   w1 <- Sigma[1,1] - Sigma[1,2]^2/Sigma[2,2]
25   Bn <- solve(w1^(-1)*XtX + B0i)
26   yaux <- y - (Sigma[1,2]/Sigma[2,2])*(x - Z%*%Gamma)
27   bn <- Bn%*%(B0i%*%b0 + w1^(-1)*t(X)%*%yaux)
28   Beta <- MASS::mvrnorm(1, bn, Bn)
29   return(Beta)
30 }
31 PostGamma <- function(Sigma, Beta){
32   w2 <- Sigma[2,2] - Sigma[1,2]^2/Sigma[1,1]
33   Gn <- solve(w2^(-1)*ZtZ + G0i)
34   xaux <- x - (Sigma[1,2]/Sigma[1,1])*(y - X%*%Beta)
35   gn <- Gn%*%(G0i%*%g0 + w2^(-1)*t(Z)%*%xaux)
36   Gamma <- MASS::mvrnorm(1, gn, Gn)
37   return(Gamma)
38 }
39 PostSigma <- function(Beta, Gamma){
40   Uy <- y - X%*%Beta; Ux <- x - Z%*%Gamma
41   U <- cbind(Uy, Ux)
42   Psin <- solve(Psi0i + t(U)%*%U)
43   Sigmai <- rWishart::rWishart(1, df = nun, Sigma = Psin)
44   Sigma <- solve(Sigmai[, ,1])
45   return(Sigma)
46 }

```

R code. Treatment effect: 401(k) participation on net financial assets

```

1 ITT <- Bs[,1]*Gs[,1]
2 summary(coda::mcmc(ITT))
3 # Convert to data frame for ggplot
4 df_ITT <- data.frame(ITT = as.vector(ITT))
5
6 # Plot posterior distribution of treatment effect
7 ggplot(df_ITT, aes(x = ITT)) + geom_density(fill = "
  steelblue", alpha = 0.6) +
8 geom_vline(xintercept = mean(ITT), color = "red", linetype =
  "dashed", linewidth = 1) + labs(title = "Posterior
  Distribution of 401(k) ITT", x = expression(beta["ITT"])
  , y = "Density") + theme_minimal(base_size = 14)

```

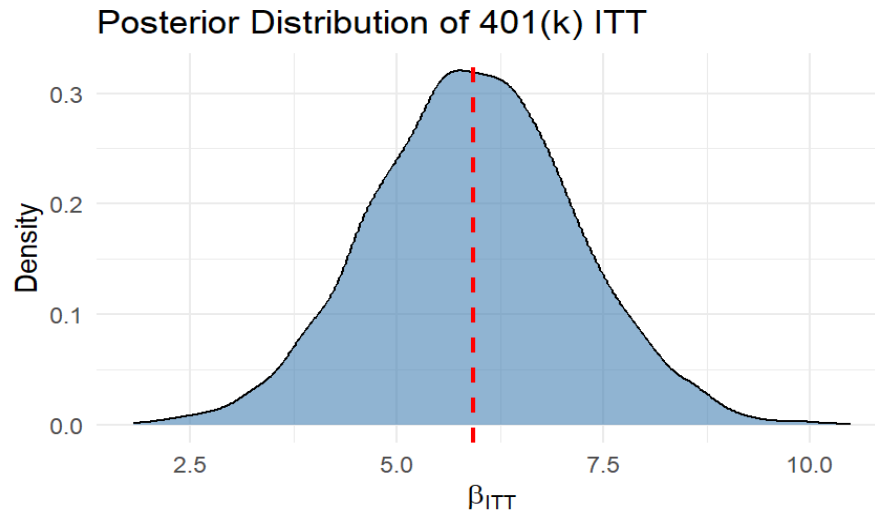


FIGURE 13.1

Posterior distribution: Intention-to-treat effect 401k participation on net financial assets.

7. 401(k) participation on net financial assets continues II

Perform inference in this example assuming that the stochastic errors follow a Dirichlet process, using the function *rivDP* from the *bayesm* package to

analyze 401(k) participation and its effect on net financial assets under the same specification as in the main text, and plot the LATE.

Answer

The following code implements the exercise, and Figure 13.2 display the posterior distribution of the LATE. The results are very similar to the ones in the main text assuming a normal distribution; however, the degree of dispersion of higher using a Dirichlet process for the stochastic errors.

R code. Treatment effect: 401(k) participation on net financial assets using Dirichlet process

```

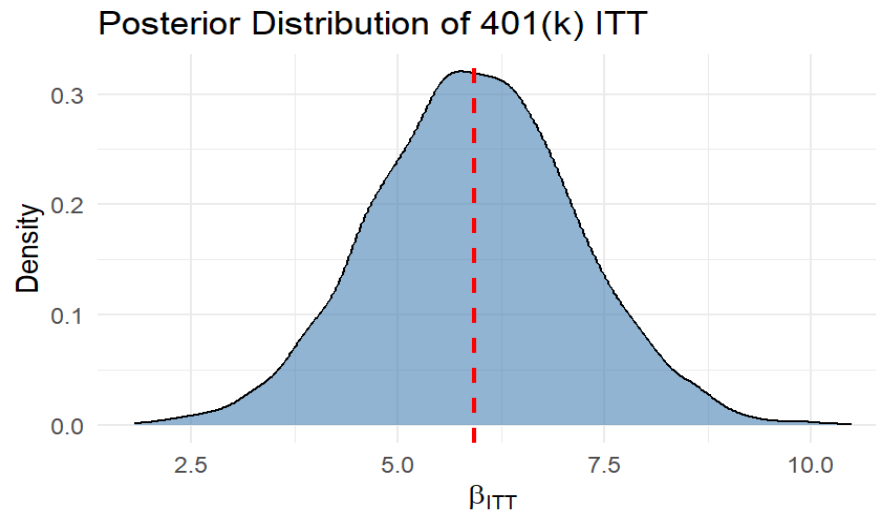
1 rm(list = ls()); set.seed(10101)
2 library(ggplot2); library(bayesm)
3 # Load data
4 mydata <- read.csv("https://raw.githubusercontent.com/
    BEsmarter-consultancy/BSTApp/refs/heads/master/DataApp/
    401k.csv",
5 sep = ",", header = TRUE, quote = "")
6 # Attach variables
7 attach(mydata)
8 y <- net_tfa/1000 # Outcome: net financial assets
9 x <- as.vector(p401) # Endogenous regressor: participation
10 w <- as.matrix(cbind(age, inc, fsize, educ, marr, twoearn,
    db, pira, hown)) # Exogenous regressors with intercept
11 z <- as.matrix(e401) # Instrument: eligibility (NO
    intercept here)
12
13 # specify data input and mcmc parameters
14 Data = list();
15 Data$z <- z
16 Data$x <- x
17 Data$y <- y
18 Data$w <- w
19
20 Mcmc = list()
21 Mcmc$maxuniq <- 100
22 Mcmc$R <- 5000
23 keep <- seq((1000+1), Mcmc$R)
24
25 out <- rivDP(Data=Data, Mcmc=Mcmc)
26
27 # Plot posterior distribution of treatment effect
28 df_ITT <- data.frame(ITT = out[["betadraw"]][kepp])
29 ggplot(df_ITT, aes(x = ITT)) + geom_density(fill = "
    steelblue", alpha = 0.6) + geom_vline(xintercept = mean(
    ITT), color = "red", linetype = "dashed", linewidth = 1)
    + labs(title = "Posterior Distribution of 401(k) ITT:
    Dirichlet process", x = expression(beta["ITT"]), y = "
    Density") + theme_minimal(base_size = 14)

```

8. Difference-in-Differences simulation continues

Perform the simulation of the DiD example, and perform inference using the specification:

$$Y_{it} = \alpha + \alpha_i + \phi_t + \tau_2 [D_i \cdot \mathbb{1}(t = 2)] + \epsilon_{it}.$$

**FIGURE 13.2**

Posterior distribution: Local average treatment effect 401k participation on net financial assets using a Dirichlet process.

Answer

You can check that we get same results as in example in the main book.

R code. Difference-in-Differences: Simulation exercise

```

1 rm(list = ls()); set.seed(10101)
2 library(ggplot2); library(dplyr); library(fastDummies)
3 # Parameters
4 N_per_group <- 200           # units per group
5 T_periods   <- 2             # keep 2x2 for clarity
6 tau_true    <- 1             # ATT
7 sigma_eps   <- 0.5           # noise SD
8 # Panel index
9 id <- rep(1:(2*N_per_group), each = T_periods)
10 t <- rep(1:T_periods, times = 2*N_per_group)
11 # Group: treated (D=1) vs control (D=0)
12 D <- rep(c(rep(0, N_per_group), rep(1, N_per_group)), each
13         = T_periods)
14 # Post indicator (t=2 is post)
15 post <- as.integer(t == 2)
16 # Unit fixed effects (random heterogeneity)
17 alpha_i <- rnorm(2*N_per_group, 0, 0.8)
18 alpha <- alpha_i[id]
19 # Time effects
20 phi_t <- c(0, -1.8) # common decline from t=1 to t=2
21 phi <- phi_t[t]
22 treat_effect <- tau_true * (D * post)
23 # Outcome
24 eps <- rnorm(length(id), 0, sigma_eps)
25 Y <- alpha + phi + treat_effect + eps
26 did <- data.frame(id, t, D, post, Y)
27 plot_data <- did %>%
28   group_by(D, t) %>%
29   summarise(meanY = mean(Y), .groups = "drop") %>%
30   mutate(Group = ifelse(D == 1, "Treated", "Control"))
31 ggp <- ggplot(plot_data, aes(x = t, y = meanY, group = Group
32   , linetype = Group)) +
33   geom_line(linewidth = 1) + geom_point() + scale_x_continuous
34   (breaks = c(1, 2), labels = c("t = 1 (pre)", "t = 2 (
35   post)")) + labs(x = "Time", y = "Mean outcome", title =
36   "Synthetic DiD: Parallel Trends & No Anticipation") +
37   theme_minimal(base_size = 12)
38 print(ggp)
39 # Bayesian inference: Model with interaction treatment x
40   post period
41 post_fit1 <- MCMCpack::MCMCregress(Y ~ 1 + factor(id) +
42   factor(t) + I(D * post), data = did, burnin = 100, mcmc
43   = 1000)
44 tau_draws1 <- post_fit1[, "I(D * post)"]
45 quantile(tau_draws1, c(.025,.5,.975))

```

9. Difference-in-Differences simulation continues II

Note that another strategy to perform inference on the ATT is to estimate the saturated model

$$Y_{it} = \sum_{t,d} \mu_{tl} [D_{il} \cdot \mathbb{1}(t = t)] + \epsilon_{it}, \quad t = 1, 2, \quad d = 1, 0,$$

and then use the posterior draws to compute

$$\tau_2 = (\mu_{21} - \mu_{11}) - (\mu_{20} - \mu_{10}).$$

Explain why inference on τ_2 using this approach in the simulation setting shows that the posterior mean is similar to that from the previous approaches, but the level of uncertainty is higher.

Answer

There is more uncertainty because performing inference on τ_2 uses four parameters in this setting, whereas in the previous we use only one parameter.

R code. Difference-in-Differences: Simulation exercise

```

1 rm(list = ls()); set.seed(10101)
2 library(ggplot2); library(dplyr); library(fastDummies)
3 # Parameters
4 N_per_group <- 200           # units per group
5 T_periods   <- 2             # keep 2x2 for clarity
6 tau_true    <- 1             # ATT
7 sigma_eps   <- 0.5           # noise SD
8 # Panel index
9 id <- rep(1:(2*N_per_group), each = T_periods)
10 t <- rep(1:T_periods, times = 2*N_per_group)
11 # Group: treated (D=1) vs control (D=0)
12 D <- rep(c(rep(0, N_per_group), rep(1, N_per_group)), each
13         = T_periods)
14 # Post indicator (t=2 is post)
15 post <- as.integer(t == 2)
16 # Unit fixed effects (random heterogeneity)
17 alpha_i <- rnorm(2*N_per_group, 0, 0.8)
18 alpha <- alpha_i[id]
19 # Time effects
20 phi_t <- c(0, -1.8) # common decline from t=1 to t=2
21 phi <- phi_t[t]
22 # No anticipation: effect is zero in pre, equals tau in post
23 # *only for treated*
24 treat_effect <- tau_true * (D * post)
25 # Outcome: Y_it(0) = alpha_i + phi_t + linear trend if you
26 # want; here just FE + shock
27 eps <- rnorm(length(id), 0, sigma_eps)
28 Y <- alpha + phi + treat_effect + eps
29 did <- data.frame(id, t, D, post, Y)
30 # Calculating the means at each level, and then ATT
31 # ---- Build 2x2 cell indicators (no intercept model) ----
32 did$c_pre <- as.integer(did$D == 0 & did$t == 1)
33 did$c_post <- as.integer(did$D == 0 & did$t == 2)
34 did$t_pre <- as.integer(did$D == 1 & did$t == 1)
35 did$t_post <- as.integer(did$D == 1 & did$t == 2)
36 # Sanity check: each row must belong to exactly one cell
37 stopifnot(all(did$c_pre + did$c_post + did$t_pre + did$t_post == 1))
38 # ---- Bayesian saturated cell-mean model (compatible priors) ----
39 # No intercept: each coefficient IS a cell mean.
40 fit_cells <- MCMCpack::MCMCregress(
41   Y ~ 0 + c_pre + c_post + t_pre + t_post, data = did, burnin
42   = 1000, mcmc = 10000, thin = 5)
43 draws_cells <- as.matrix(fit_cells)
44 colnames(draws_cells) # should be c("c_pre", "c_post", "t_pre",
45   "t_post", "sigma2")
46 # Posterior of the four cell means
47 mu_c_pre_draw <- draws_cells[, "c_pre"]; mu_c_post_draw <-
48   draws_cells[, "c_post"]
49 mu_t_pre_draw <- draws_cells[, "t_pre"]; mu_t_post_draw <-
50   draws_cells[, "t_post"]
51 # ATT = (mu_t,post - mu_t,pre) - (mu_c,post - mu_c,pre)
52 ATT_cells_draw <- (mu_t_post_draw - mu_t_pre_draw) - (mu_c_post_draw - mu_c_pre_draw)
53 quantile(ATT_cells_draw, c(.025, .5, .975))

```

10. Perform a simulation exercise to assess the ability of BETEL to identify the causal effect when an instrument is used to address the omission of relevant regressors. Illustrate the consequences of varying the dependence between the omitted regressor and the observed regressor.

Answer

We simulate the observed process $X_i = 0.5Z_i + e_i$, with the unobserved regressor defined as $W_i = H_i + \nu_i$, where

$$\begin{bmatrix} e_i \\ \nu_i \end{bmatrix} \sim N \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix} \right).$$

Thus, ρ defines the degree of correlation between X_i and W_i , and we perform two exercises with $\rho \in \{0.0, 0.7\}$. The variables Z_i and H_i are standard normal. The dependent variable is specified as

$$Y_i = 1 + 1.2X_i - 0.7W_i + \mu_i,$$

where μ_i follows a mixture of two normal distributions with means 0.5 and -0.5 , and standard deviations 0.5 and 1.2. The sample size is 2,000, the burn-in is 1,000, and the number of MCMC draws retained after burn-in is 10,000. We adopt the default hyperparameter values provided in the *betel* package. For comparison, we also perform the analysis under the assumption of no endogeneity using the package *MCMCpack* using default values for the hyperparameters.

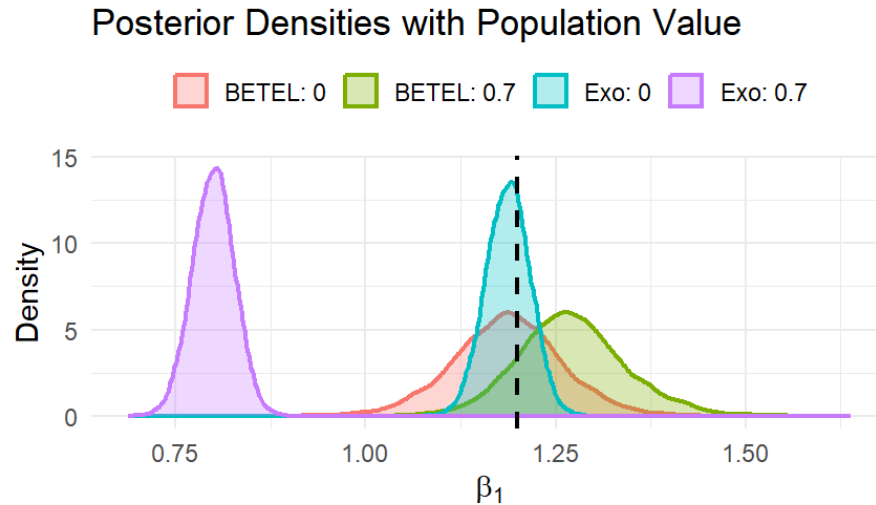
The following code shows the implementation, and Figure 13.3 displays the posterior distributions. When $\rho = 0$, implying no association between the regressors, there are no endogeneity issues, and assuming exogeneity yields more efficient posterior estimates. In contrast, when $\rho = 0.7$, endogeneity arises, and assuming exogeneity produces a biased posterior distribution. BETEL performs relatively well in both settings, although it is less efficient.

R code. BETEL: Omission of a relevant regressor

```

1 rm(list = ls()); set.seed(10101)
2 library(betel); library(ucminf)
3 # Simulate data
4 N <- 2000; d <- 2; k <- 2
5 gamma <- 0.5; beta <- c(1, 1.2, -0.7); rho <- 0.7
6 # Mixture
7 mum1 <- 1/2; mum2 <- -1/2
8 mu1 <- rnorm(N, mum1, 0.5); mu2 <- rnorm(N, mum2, 1.2)
9 mu <- sapply(1:N, function(i){sample(c(mu1[i], mu2[i]), 1,
   prob = c(0.5, 0.5))})
10 z <- rnorm(N) # Instrument
11 E <- MASS::mvrnorm(n = N, mu = c(0, 0), Sigma = matrix(c(1,
   rho, rho, 1), 2, 2))
12 x <- gamma*z + E[,1] # Observed regressor
13 w <- rnorm(N) + E[,2] # Unobserved correlated regressor
14 X <- cbind(1, x, w)
15 y <- X%*%beta + mu
16 dat <- cbind(1, x, z) # Data
17 # Function g_i by row in BETEL
18 gfunc <- function(psi = psi, y = y, dat = dat) {
19   X <- dat[,1:2]
20   e <- y - X %*% psi
21   E <- e %*% rep(1,d)
22   Z <- dat[,c(1,3)]
23   G <- E * Z;
24   return(G)
25 }
26 nt <- round(N * 0.1, 0); # training sample size for prior
27 psi0 <- lm(y[1:nt]~x[1:nt])$coefficients # Starting value of
   psi = (theta, v), v is the slack parameter in CSS
   (2018)
28 names(psi0) <- c("alpha", "beta")
29 psi0_ <- as.matrix(psi0) # Prior mean of psi
30 Psi0_ <- 5*rep(1,k) # Prior dispersions of psi
31 lam0 <- .5*rnorm(d) # Starting value of lambda
32 nu <- 2.5 # df of the prior student-t
33 nuprop <- 15 # df of the student-t proposal
34 n0 <- 1000 # burn-in
35 m <- 10000 # iterations beyond burn-in
36 # MCMC ESTIMATION BY THE CSS (2018) method
37 psim2 <- betel::bayesetel(gfunc = gfunc, y = y[-(1:nt)], dat
   = dat[-(1:nt),], psi0 = psi0, lam0 = lam0, psi0_ = psi0_
   , Psi0_ = Psi0_, nu = nu, nuprop = nuprop,
38 controlpsi = list(maxiterpsi = 50, mingrpsi = 1.0e-8), #
   list of parameters in maximizing likelihood over psi
39 controllam = list(maxiterlam = 50, # list of parameters in
   minimizing dual over lambda
40 mingrlam = 1.0e-7),
41 n0 = n0, m = m)
42 MCMCreg2 <- MCMCpack::MCMCregress(y~x)

```


**FIGURE 13.3**

Posterior distribution slope parameter given omission of relevant regressor: betel and MCMCregress

11. Simultaneous causality continues

Use the demand–supply simulation and the moment conditions to infer the causal effect of a 10% tax, implementing a BETEL algorithm from scratch.

Answer

The following code shows how to program a BETEL from scratch for the demand–supply simulation exercise. Figure 13.4 displays the posterior distribution of the causal effect. The 95% credible interval contains the population value, and the posterior mean lies close to it.

R code. BETEL: Simultaneous causality

```

1 # Simulation
2 rm(list = ls()); set.seed(12345)
3 # Population parameters demand
4 B1 <- 5; B2 <- -0.5; B3 <- 0.8; B4 <- -0.4; B5 <- 0.7; SD <-
  0.5
5 # Population parameters supply
6 A1 <- -2; A2 <- 0.5; A3 <- -0.4; SS <- 0.5
7 # Reduced form parameters
8 P0 <- (A1-B1)/(B2-A2); P2 <- -B3/(B2-A2); P3 <- -B4/(B2-A2);
  P1 <- A3/(B2-A2); P4 <- -B5/(B2-A2)
9 T0 <- B1+B2*P0; T2 <- B3+B2*P2; T3 <- B4+B2*P3; T1 <- B2*P1;
  T4 <- B5+B2*P4;
10 n <- 5000
11 ED <- rnorm(n, 0, SD); ES <- rnorm(n, 0, SS)
12 VP <- (ES-ED)/(B2-A2); UQ <- B2*VP+ED
13 y <- rnorm(n, 10, 1); pc <- rnorm(n, 5, 1); er <- rnorm(n,
  15, 1); ps <- rnorm(n, 5, 1);
14 p <- P0+P1*er+P2*y+P3*pc+P4*ps+VP
15 q <- T0+T1*er+T2*y+T3*pc+T4*ps+UQ
16
17 ### Supply ###
18 p_s <- as.numeric(scale(p, TRUE, TRUE))
19 er_s <- as.numeric(scale(er, TRUE, TRUE))
20
21 X <- cbind(1, p_s, er_s) # k = 3 (STRUCTURAL: intercept, p
  , er)
22 Z <- cbind(1,
23 as.numeric(scale(y, TRUE, TRUE)),
24 as.numeric(scale(pc, TRUE, TRUE)),
25 as.numeric(scale(ps, TRUE, TRUE)),
26 er_s) # EXCLUDED instruments y,pc,ps, plus er
27
28 k <- ncol(X); d <- ncol(Z)
29
30 # Objective function
31 lambdafunc <- function(lambda, theta, q, X, Z){
32   e <- as.numeric(q - X %*% theta); if (!all(is.finite(e)))
33     return(1e300)
34   G <- Z * e
35   eta <- as.numeric(G %*% lambda); if (!all(is.finite(eta)))
36     return(1e300)
37   m <- max(eta); m + log(mean(exp(eta - m)))
38 }
39
40 # ----- Profile ETEL log-likelihood (use this in MH) -----
41 loglik_etel <- function(theta, q, X, Z){
42   e <- as.numeric(q - X %*% theta); if (!all(is.finite(e)))
43     return(-Inf)
44   G <- Z * e
45   op <- optim(par = rep(0, ncol(G)), fn = function(l){
46     eta <- as.numeric(G %*% l); m <- max(eta)
47     m + log(mean(exp(eta - m)))
48   }, method="BFGS", control=list(maxit=2000))
49   if (!is.finite(op$value)) return(-Inf)
50   lam <- op$par
51   eta <- as.numeric(G %*% lam)
52   sum(eta) - length(e) * log(sum(exp(eta)))
53 }

```

R code. BETEL: Simultaneous causality

```

1 theta0 <- coef(lm(q ~ p_s + er_s))
2 # ----- MH using profile ETEL -----
3 b0 <- rep(0, k); B0 <- 1000*diag(k)
4 S <- 10000; burnin <- 2000; thin <- 5; tot <- S + burnin
5 BETA <- matrix(NA, tot, k); accept <- logical(tot)
6 step <- c(0.05, 0.02, 0.02) #
7   preconditioned steps
8 BETA[1,] <- theta0
9 LL <- loglik_etel(BETA[1,], q, X, Z)
10 pb <- txtProgressBar(min=0, max=tot, style=3)
11 for(s in 2:tot){
12   cand <- BETA[s-1,] + rnorm(k, 0, step)
13   LLc <- loglik_etel(cand, q, X, Z)
14   priorRat <- mvtnorm::dmvnorm(cand, b0, B0, log=TRUE) -
15   mvtnorm::dmvnorm(BETA[s-1,], b0, B0, log=TRUE)
16   loga <- (LLc - LL) + priorRat
17   if (is.finite(loga) && log(runif(1)) <= loga) {
18     BETA[s,] <- cand; LL <- LLc; accept[s] <- TRUE
19   } else {
20     BETA[s,] <- BETA[s-1,]; accept[s] <- FALSE
21   }
22   if (s %% 200 == 0) { # gentle adaptation
23     acc <- mean(accept[(s-199):s])
24     if (acc > 0.4) step <- step * 1.25
25     if (acc < 0.15) step <- step / 1.25
26   }
27   setTxtProgressBar(pb, s)
28 }
29 close(pb)
30 cat("\nAcceptance rate:", mean(accept), "\n")
31 keep <- seq(burnin, tot, by = thin)
32 post_s <- BETA[keep, , drop=FALSE] # posterior draws in
33   scaled space
34 ## ----- Back-transform each draw to original scale
35   -----
36 p_mu <- mean(p); p_sd <- sd(p); er_mu <- mean(er); er_sd
37   <- sd(er)
38 alpha_draws <- cbind(
39   alpha0 = post_s[,1] - post_s[,2] * (p_mu/p_sd) - post_s[,3]
40   * (er_mu/er_sd),
41   alphap = post_s[,2] / p_sd,
42   alphaer = post_s[,3] / er_sd
43 )
44 ## ----- Summaries -----
45 summ <- function(x) c(mean=mean(x), sd=sd(x), quantile(x, c
46   (.025,.25,.5,.75,.975)))
47 cat("\nPosterior (original scale):\n")
48 print(rbind(
49   alpha0 = summ(alpha_draws[, "alpha0"]), alphap = summ(alpha
50   _draws[, "alphap"]), alphaer = summ(alpha_draws[, "alphaer
51   "]))))
52 cat("\nTrue (original scale):\n")
53 print(c(alpha0=A1, alphap=A2, alphaer=A3))
54 ElastSupPrice <- alpha_draws[,2]
55 #### Demand ####
56 y_s <- as.numeric(scale(y)); pc_s <- as.numeric(scale(pc))
57 ps_s <- as.numeric(scale(ps)); Xdem <- cbind(1, p_s, y_s, pc
58   _s, ps_s)
59 k <- ncol(Xdem); d <- ncol(Z); theta0 <- coef(lm(q ~ p_s + y
60   _s + pc_s + ps_s))

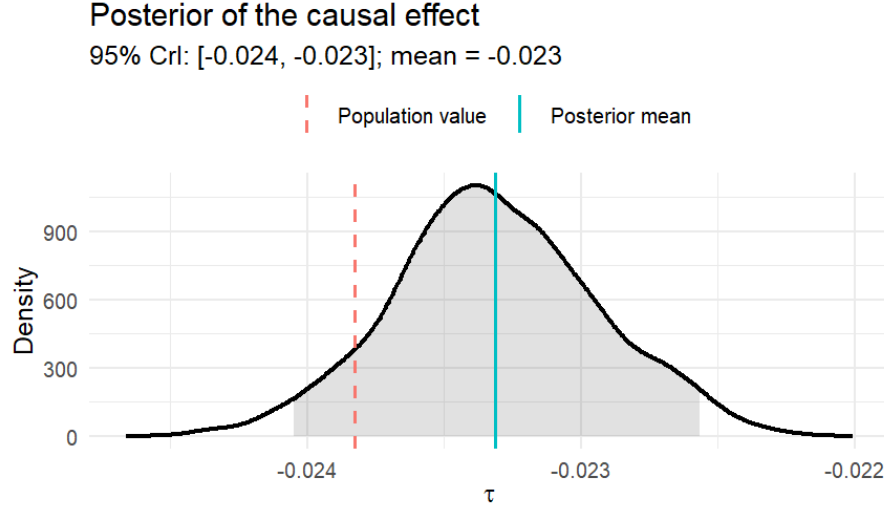
```

R code. BETEL: Simultaneous causality

```

1 # ----- MH using profile ETEL -----
2 b0 <- rep(0, k); B0 <- 1000*diag(k)
3 S <- 10000; burnin <- 2000; thin <- 5; tot <- S + burnin
4 BETAdem <- matrix(NA, tot, k); accept <- logical(tot)
5 step <- c(0.05, 0.02, 0.02); BETAdem[1,] <- theta0
6 LL <- loglik_etel(BETAdem[1,], q, X = Xdem, Z)
7 pb <- txtProgressBar(min=0, max=tot, style=3)
8 for(s in 2:tot){
9   cand <- BETAdem[s-1,] + rnorm(k, 0, step)
10  LLc <- loglik_etel(cand, q, X = Xdem, Z)
11  priorRat <- mvtnorm::dmvnorm(cand, b0, B0, log=TRUE) -
12  mvtnorm::dmvnorm(BETAdem[s-1,], b0, B0, log=TRUE)
13  loga <- (LLc - LL) + priorRat
14  if (is.finite(loga) && log(runif(1)) <= loga) {
15    BETAdem[s,] <- cand; LL <- LLc; accept[s] <- TRUE
16  } else {
17    BETAdem[s,] <- BETAdem[s-1,]; accept[s] <- FALSE
18  }
19  if (s %% 200 == 0) { # gentle adaptation
20    acc <- mean(accept[(s-199):s])
21    if (acc > 0.4) step <- step * 1.25
22    if (acc < 0.15) step <- step / 1.25
23  }
24  setTxtProgressBar(pb, s)
25 }
26 close(pb)
27 cat("\nAcceptance rate:", mean(accept), "\n")
28 keep <- seq(burnin, tot, by = thin)
29 postDem_s <- BETAdem[keep, , drop=FALSE]
30 p_mu <- mean(p); p_sd <- sd(p); y_mu <- mean(y); y_sd <-
  sd(y)
31 pc_mu <- mean(pc); pc_sd <- sd(pc); ps_mu <- mean(ps); ps_sd
  <- sd(ps)
32 alphaDem_draws <- cbind(
33 alpha0 = postDem_s[,1] - postDem_s[,2] * (p_mu/p_sd) -
  postDem_s[,3] * (y_mu/y_sd)
34 - postDem_s[,4] * (pc_mu/pc_sd) - postDem_s[,5] * (pc_mu/pc_
  sd),
35 alphap = postDem_s[,2] / p_sd, alphay = postDem_s[,3] / y_
  sd, alphapc = postDem_s[,4] / pc_sd, alphaps = postDem_s
  [,5] / ps_sd)
36 summ <- function(x) c(mean=mean(x), sd=sd(x), quantile(x, c
  (.025,.25,.5,.75,.975)))
37 cat("\nPosterior (original scale):\n")
38 print(rbind(
39 alpha0 = summ(alphaDem_draws[, "alpha0"]), alphap = summ(
  alphaDem_draws[, "alphap"]), alphay = summ(alphaDem_draws
  [, "alphay"]),
40 alphapc = summ(alphaDem_draws[, "alphapc"]), alphaps = summ(
  alphaDem_draws[, "alphaps"])))
41 cat("\nTrue (original scale):\n")
42 print(c(alpha0=B1, alphap=B2, alphay=B3, alphpc=B4, alphaps=
  B5))
43 ElastDemPrice <- alphaDem_draws[,2]; tax <- 0.1
44 CausalEffect <- (ElastSupPrice*ElastDemPrice)*log(1+tax)/(
  ElastSupPrice-ElastDemPrice)
45 popCauEff <- (A2 * B2)/(A2 - B2) * log(1 + tax)

```

**FIGURE 13.4**

Posterior distribution causal effect: BETEL

12. Perform a simulation of the instrumental variable quantile regression, and then perform inference using the general Bayes posterior framework. Use the Laplace asymmetric distribution to simulate the stochastic error, such that

$$\int_{-\infty}^0 f_{\tau}(\mu_i) d\mu_i = \tau,$$

which implies that the τ -th quantile of μ_i is 0. In addition, let

$$q_{\tau}(Y_i | X_i, D_i) = \beta_0 + \beta_1 X_i + \beta_2 D_i$$

denote the τ -th quantile regression function of Y_i given X_i and D_i , with $0 < \tau < 1$. Specifically, consider the model

$$Y_i = \beta_{0\tau} + \beta_{1\tau} X_i + \beta_{2\tau} D_i + \mu_i,$$

together with

$$D_i = \gamma Z_i + \delta \mu_i, \quad Z_i \sim N(0, 1).$$

Answer:

The following code shows how to perform inference in this simulation. The 95% credible intervals encompass the population values, and the posterior means are also close to these values.

R code. Instrumental Variable Quantile Regression: General Bayes posteriors

```

1 rm(list = ls()); set.seed(10101); library(quantreg)
2 ### Simulation ###
3 # Asymmetric Laplace
4 rALD <- function(n, tau) {
5   stopifnot(tau > 0, tau < 1)
6   theta <- (1 - 2*tau) / (tau * (1 - tau))
7   psi <- 2 / (tau * (1 - tau))
8   v <- rexp(n, rate = 1)
9   z <- rnorm(n)
10  mu <- theta * v + sqrt(psi * v) * z
11  return(mu)
12 }
13 ## --- Simulator for a given tau ---
14 simulate_qr <- function(n = 5000, tau = 0.5,
15 beta0 = 0.7,
16 beta1_tau = 1.5,
17 beta2_tau = 1,
18 gamma = 1,
19 delta = 0.5) {
20   # Nonnegative regressor stabilizes intercept & preserves
21   # tau-ordering of slopes
22   x <- runif(n, 0, 1)
23   # optional: anchor a portion at 0 to pin intercept
24   x[sample.int(n, size = round(0.1*n))] <- 0
25   mu <- rALD(n, tau = tau) # ALD error
26   z <- rnorm(n) # Instrument
27   d <- gamma * z + delta * mu
28   y <- beta0 + beta1_tau * x + beta2_tau * d + mu
29   df <- data.frame(y, x, d, z)
30 }
31 tau <- 0.75
32 df <- simulate_qr(tau = tau, beta0 = 1, beta1_tau = 1,
33 beta2_tau = 1, gamma = 1, delta = 1, n = 5000)
34 Reg <- MCMCpack::MCMCquantreg(y~x+d, data = df, tau = tau)
35 summary(Reg)
36 summary(rq(y ~ x + d, tau = tau, data = df))
37 LossFunct <- function(par, tau, y, z, x, d){
38   n <- length(y)
39   X <- cbind(1, x, d)
40   Z <- cbind(1, x, z)
41   Ind <- as.numeric(y <= X%%par)
42   gn <- colMeans((tau - Ind) * Z)
43   Wni <- lapply(1:n, function(i) {Z[i,] %% t(Z[i,])})
44   Wn <- 1 / (tau * (1 - tau)) * solve(Reduce("+", Wni)/n)
45   Ln <- - 0.5 * n * t(gn) %% Wn %% gn
46   return(Ln)
47 }
48 par0 <- rnorm(3)
49 LossFunct(par = par0, tau = tau, y = df$y, z = df$z, x = df$x, d = df$d)

```

*R code. Instrumental Variable Quantile
Regression: General Bayes posteriors*

```

1 # ----- MH using Ln -----
2 k <- 3
3 b0 <- rep(0, k); B0 <- 1000*diag(k)
4 S <- 5000; burnin <- 6000; thin <- 5; tot <- S + burnin
5 BETA <- matrix(NA, tot, k); accept <- logical(tot)
6 step <- 0.05
7 BETA[1,] <- par0
8 LL <- LossFunc(par = BETA[1,], tau = tau, y = df$y, z = df$
   z, x = df$x, d = df$d)
9 pb <- txtProgressBar(min=0, max=tot, style=3)
10 for(s in 2:tot){
11   cand <- BETA[s-1,] + rnorm(k, 0, step)
12   LLc <- LossFunc(par = cand, tau = tau, y = df$y, z = df$
   z, x = df$x, d = df$d)
13   priorRat <- mvtnorm::dmvnorm(cand, b0, B0, log=TRUE) -
   mvtnorm::dmvnorm(BETA[s-1,], b0, B0, log=TRUE)
14   loga <- (LLc - LL) + priorRat
15   if (is.finite(loga) && log(runif(1)) <= loga) {
16     BETA[s,] <- cand; LL <- LLc; accept[s] <- TRUE
17   } else {
18     BETA[s,] <- BETA[s-1,]; accept[s] <- FALSE
19   }
20   if (s <= burnin && s %% 200 == 0) {
21     acc <- mean(accept[(s-199):s])
22     if (acc > 0.4) step <- step * 1.25
23     if (acc < 0.15) step <- step / 1.25
24   }
25   setTxtProgressBar(pb, s)
26 }
27 close(pb)
28 cat("\nAcceptance rate:", mean(accept), "\n")
29 keep <- seq(burnin, tot, by = thin)
30 post <- BETA[keep, , drop=FALSE] # posterior draws in
   scaled space
31 colnames(post) <- c("beta0", "beta_x", "beta_d")
32 # Posterior summaries
33 post_mean <- colMeans(post)
34 post_ci <- apply(post, 2, quantile, c(0.025, 0.975))
35 round(post_mean, 3); round(post_ci, 3)

```

13. Example: Doubly robust Bayesian inference continues

Perform a simulation exercise to assess the consequences of misspecifying the propensity score function and the outcome regression in doubly robust Bayesian inference.

Answer:

Let us simulate the process

$$\eta_i^\pi = -0.3 + 0.8X_{i1} - 0.5X_{i2} + 0.7X_{i3} + 0.7X_{i1}X_{i3} + 0.5\sin(X_{i1}),$$

$$\eta_i^m = -0.2 + 0.6D_i + 0.5X_{i1} - 0.4X_{i2} + 0.3X_{i3} + 0.6X_{i1}^2 - 0.5X_{i1}X_{i2},$$

where $X_1 \sim N(0, 1)$, $X_2 \sim \text{Bernoulli}(0.5)$, and $X_3 \sim U(-1, 1)$, and the sample size is 2,000.

The following code implements the algorithm.

*R code. Doubly robust Bayesian inference:
Misspecified simulation*

```

1 rm(list = ls()); set.seed(123); library(glmnet)
2
3 ## simulate covariates
4 n <- 2000
5 X1 <- rnorm(n)                # continuous
6 X2 <- rbinom(n, 1, 0.5)        # binary
7 X3 <- runif(n, -1, 1)          # continuous
8 Z <- cbind(X1, X2, X3)
9 # True propensity and outcome
10 logit <- function(t) 1/(1+exp(-t))
11 true_pi <- function(X){ logit(-0.3 + 0.8*X[,1] -0.5*X[,2] +
12   0.7*X[,3] + 0.7*X[,1]*X[,3] + 0.5*sin(X[,1])) }
12 true_eta <- function(d,X){ -0.2 + 0.6*d + 0.5*X[,1] -0.4*X
13   [,2] + 0.3*X[,3] + 0.6*(X[,1]^2) - 0.5*X[,1]*X[,2] }
13 true_p <- function(d,X){ logit(true_eta(d,X)) }
14 ATE <- mean(true_p(1,Z) - true_p(0,Z))
15 pi <- true_pi(Z)
16 D <- rbinom(n,1,pi)
17 P <- true_p(D,Z)
18 Y <- rbinom(n,1,P)
19 dat <- data.frame(Y, D, X1, X2 = factor(X2), X3)
20 ##### Doubly Robust Bayesian: Implementation #####
21 # OK specification propensity score
22 Z <- cbind(X1, X2, X3, X1*X3, sin(X1)); p <- dim(Z)[2]
23 covars <- c("X1","X2","X3", "X1X3", "sinX1")
24 # OK specification outcome regression
25 # Z <- cbind(X1, X2, X3, X1*X2, X1^2); p <- dim(Z)[2]
26 # covars <- c("X1","X2","X3", "X1X2", "X12")
27 cvfit <- cv.glmnet(Z, D, family = "binomial", alpha=1,
28   nfolds = 10) # lasso,min
29 ps.coef <- coef(cvfit, s = "lambda.min")[0:p+1]
30 ps_hat <- predict(cvfit, newx = Z, s = "lambda.min", type = "
31   response")
32 ps_hat <- pmin(pmax(ps_hat, 1e-6), 1 - 1e-6)
33 # Riesz representer
34 riesz_fun <- function(d, ps) d/ps - (1 - d)/(1 - ps)
35 # GP model
36 make_gp <- function(X_train, y_bin, use_correction = FALSE,
37   gamma_scaled = NULL,
38   approx_method = gplite::approx_laplace(),
39   init_lscale = 0.3) {
40   stopifnot(length(y_bin) == nrow(X_train))
41   cf_se <- gplite::cf_sexp(vars = colnames(X_train), lscale
42     = init_lscale, magn = 1, normalize = TRUE)
43   cfs <- list(cf_se)
44   if (use_correction) {
45     stopifnot(!is.null(gamma_scaled))
46     cf_lin <- gplite::cf_lin(vars = "gamma", magn = 1,
47       normalize = FALSE)
48     cfs <- list(cf_se, cf_lin)
49   }
50   gp <- gplite::gp_init(cfs = cfs, lik = gplite::lik_
51     bernoulli(), approx = approx_method)
52   gp <- gplite::gp_optim(gp, x = X_train, y = y_bin, maxiter
53     = 1000, restarts = 3, tol = 1e-05)
54   return(gp)
55 }

```

*R code. Doubly robust Bayesian inference:
Simulation exercise*

```

1 posterior_draws_prob <- function(gp, X_test, ndraws = 5000)
2 {
3   out <- gplite::gp_draw(gp, xnew = X_test, draws = ndraws,
4     transform = TRUE, target = FALSE, jitter = 1e-6)
5   return(t(out))
6 }
7 # Main function
8 run_trim <- function(t_trim = 0.10, N_post = 5000, h_r = 1/
9   2, cor_size = 1) {
10   # Trim by estimated PS
11   keep <- which(ps_hat >= t_trim & ps_hat <= (1 - t_trim))
12   n_eff <- length(keep)
13   if (n_eff < 50) stop("Too few observations after trimming.
14     ")
15   Yk <- Y[keep]
16   Dk <- D[keep]
17   Zk <- Z[keep, , drop = FALSE]
18   psk <- ps_hat[keep]
19   # Riesz representer and sigma_n choice
20   gamma_k <- riesz_fun(Dk, psk)
21   sigma_n <- cor_size * (log(n_eff) / ((n_eff)^(h_r) * mean
22     (abs(gamma_k))))
23   gamma_scaled <- sigma_n * gamma_k
24   # Training inputs for GP: [Z, D] and, if corrected, an
25   # extra column 'gamma'
26   X_train_nc <- data.frame(Zk)
27   X_train_nc$D <- Dk
28   colnames(X_train_nc) <- c(covars, "D")
29   X_train_c <- X_train_nc
30   X_train_c$gamma <- gamma_scaled
31   # Test inputs: for each i, (Z_i, d=0) and (Z_i, d=1)
32   X_t0 <- X_train_nc; X_t0$D <- 0
33   X_t1 <- X_train_nc; X_t1$D <- 1
34   X_t0c <- X_t0; X_t1c <- X_t1
35   X_t0c$gamma <- sigma_n * riesz_fun(0, psk)
36   X_t1c$gamma <- sigma_n * riesz_fun(1, psk)
37   # GP WITHOUT prior correction #
38   gp_nc <- make_gp(X_train = X_train_nc, y_bin = Yk, use_
39     correction = FALSE)
40   # joint draws for [m(Z,0); m(Z,1)]
41   M_nc_0 <- posterior_draws_prob(gp_nc, X_t0, ndraws = N_
42     post) # N_post x n_eff
43   M_nc_1 <- posterior_draws_prob(gp_nc, X_t1, ndraws = N_
44     post)
45   # observed arm probabilities
46   M_nc_obs <- ifelse(matrix(Dk, nrow = N_post, ncol = n_eff,
47     byrow = TRUE) == 1, M_nc_1, M_nc_0)
48   # GP WITH prior correction #
49   gp_c <- make_gp(X_train = X_train_c, y_bin = Yk, use_
50     correction = TRUE, gamma_scaled = gamma_scaled)
51   M_c_0 <- posterior_draws_prob(gp_c, X_t0c, ndraws = N_post
52     )
53   M_c_1 <- posterior_draws_prob(gp_c, X_t1c, ndraws = N_post
54     )
55   M_c_obs <- ifelse(matrix(Dk, nrow = N_post, ncol = n_eff,
56     byrow = TRUE) == 1, M_c_1, M_c_0)

```

*R code. Doubly robust Bayesian inference:
Simulation exercise*

```

1  # Bayesian bootstrap weights #
2  W <- matrix(rexp(N_post * n_eff, rate = 1), nrow = N_post)
3  W <- W / rowSums(W)
4  # ATEs
5  # (1) Unadjusted Bayes
6  Ate_nc_draws <- rowSums((M_nc_1 - M_nc_0) * W)
7  # (2) Prior-adjusted Bayes
8  Ate_c_draws <- rowSums((M_c_1 - M_c_0) * W)
9  # (3) DR Bayes (posterior recentering)
10 # Pre-centering using UNcorrected GP posterior means
11 mu_nc_diff <- colMeans(M_nc_1 - M_nc_0)
12 mu_nc_obs <- colMeans(M_nc_obs)
13 ATE_dr_pre <- mean(mu_nc_diff + gamma_k * (Yk - mu_nc_obs)
14 )
15 # Recenter draws using corrected GP draws
16 DR_rec_1 <- (matrix(gamma_k, nrow = N_post, ncol = n_eff,
17 byrow = TRUE)) * (matrix(Yk, nrow = N_post, ncol = n_eff
18 , byrow = TRUE) - M_c_obs)
19 # The first component is  $\tau_{\eta}$ 's in Algorithm 1
20 # The second component is  $\hat{m}$  a scalar. This has
21 # positive sign because
22 # minus x minus, the bias is with minus, and then, this
23 # component enters with minus in the bias term
24 # The third component is  $\gamma_k \times (y - m(d, x))$ ,  $m(d, x)$  is
25 # with prior correction
26 # The fourth component is  $m(1, x) - m(0, x)$  also with
27 # correction in the prior
28 Ate_drb_draws <- rowSums((M_c_1 - M_c_0) * W) + ATE_dr_pre
29 - rowSums(DR_rec_1) / n_eff - rowSums(M_c_1 - M_c_0) /
30 n_eff
31
32 qfun <- function(x) {
33   qs <- quantile(x, c(0.025, 0.975))
34   c(mean = mean(x), median = median(x), lo = qs[1], hi
35     = qs[2], len = diff(qs))
36 }
37
38 vals <- list(
39   Bayes = qfun(Ate_nc_draws),
40   'PA Bayes' = qfun(Ate_c_draws),
41   'DR Bayes' = qfun(Ate_drb_draws)
42 )
43
44 out_df <- as.data.frame(do.call(rbind, vals), check.names
45 = FALSE)
46 list(t_trim = t_trim, results = out_df)
47 }
48
49 N_post <- 5000
50 h_r <- 1/2
51 cor_size <- 1
52 trim <- 0.05
53 res_list <- run_trim(t_trim = trim, N_post = N_post, h_r = h
54 _r, cor_size = cor_size)
55 print(res_list$results)

```



14

Approximate Bayesian methods

Solutions of Exercises

1. g-and-k distribution for financial returns continues I

Simulate a dataset following the specification given in the g-and-k distribution for financial returns example. Set $\theta_1 = 0.8$, $a = 1$, $b = 0.5$, $g = -1$, and $k = 1$, with a sample size of 500, and use the same priors as in that example. Implement the ABC accept/reject algorithm from scratch using one million prior draws, selecting the 1,000 draws with the smallest distance.¹

- Perform a linear regression adjustment using the posterior draws of our ABC-AR algorithm (ABC-AR-Adj).
- Compare the results with those obtained using the ABC-AR implementation in the *EasyABC* package, ensuring that the computational time is relatively similar between both implementations.
- Compare the posterior results of ABC-AR, ABC-AR-Adj and EasyABC with the population values.

Answer:

The following code implements our ABC-AR algorithm, following the steps outlined in the ABC-AR Algorithm of the book. Users can observe that a linear regression adjustment is applied; however, due to multicollinearity issues, one summary statistic is omitted in the prediction step (see the *abc* package for alternative regression adjustments).

Figure 14.1 presents the posterior distributions of θ_1 , g , and k using both our ABC implementations and the *EasyABC* package. We observe that our algorithms provide less information about θ_1 compared to the *EasyABC* implementation. Conversely, our algorithms offer more information about g and k than the *EasyABC* function. Additionally, regression adjustment helps center the posterior distributions of g and k around the population parameters. But, this is not the case regarding θ_1 . Moreover, applying regression adjustment introduces a degree of smoothness to the distributions.

¹Note that this setting does not satisfy the asymptotic requirements for Bayesian consistency. However, it serves as a pedagogical exercise.

Overall, in all implementations, the 95% credible intervals encompass the population values.

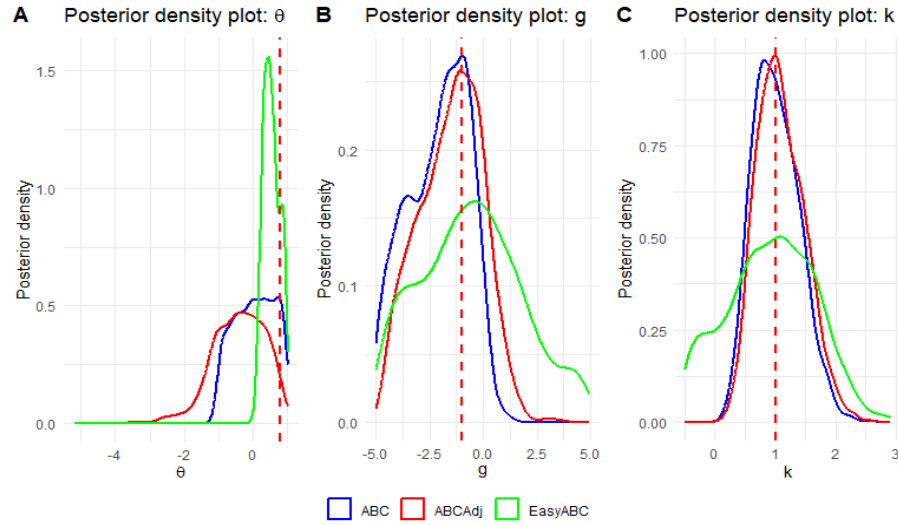


FIGURE 14.1

Posterior distributions using approximate Bayesian computation accept/reject: g-and-k distribution.

*R code. Approximate Bayesian computation
accept/reject algorithm: g-and-k distribution
simulation*

```

1 rm(list = ls()); set.seed(010101)
2 # Simulate g-and-k data
3 RGKnew <- function(par) {
4   z <- NULL
5   theta <- par[1]; a <- par[2]; b <- par[3]; g <- par[4]; k
     <- par[5]
6   e <- rnorm(n + 1)
7   for(t in 2:(n + 1)){
8     zt <- e[t] + theta * e[t-1]
9     z <- c(z, zt)
10  }
11  zs <- z / (1 + theta^2)^0.5
12  x <- a + b * (1 + 0.8 * (1 - exp(-g * zs)) / (1 + exp(-g *
     zs))) * (1 + zs^2)^k * zs
13  return(x)
14 }
15 # Summary statistics
16 SumSt <- function(y) {
17   Oct <- quantile(y, c(0.125, 0.25, 0.375, 0.5, 0.625, 0.75,
     0.875))
18   eta1 <- Oct[6] - Oct[2]
19   eta2 <- (Oct[6] + Oct[2] - 2 * Oct[4]) / eta1
20   eta3 <- (Oct[7] - Oct[5] + Oct[3] - Oct[1]) / eta1
21   autocor <- acf(y, lag = 2, plot = FALSE)
22   autocor[["acf"]][2:3]
23   Etag <- c(Oct, eta1, eta2, eta3, autocor[["acf"]][2:3])
24   return(Etag)
25 }
26 # Population parameters
27 theta1 <- 0.8; a <- 1; b <- 0.5; g <- -1; k <- 1
28 parpop <- c(theta1, a, b, g, k)
29 n <- 500
30 y <- RGKnew(par = parpop)
31 ##### ABC Function#####
32 ABC <- function(S, a, y) {
33   prior <- cbind(runif(S,-1,1), runif(S,0,5), runif(S,0,5),
     runif(S,-5,5), runif(S,-0.5,5))
34   Z <- apply(prior, 1, RGKnew)
35   EtasZ <- apply(Z, 2, SumSt)
36   Etag <- SumSt(y)
37   Dist <- sapply(1:S, function(l) {
38     dist(rbind(Etag, EtasZ[, l]))
39   })
40   OrdPrior <- prior[order(Dist), ]
41   SelPrior <- OrdPrior[1:round(S * a), ]
42   SelSumSt <- t(EtasZ)[1:round(S * a), ]
43   return(list(SelPrior = SelPrior, SelSumSt = SelSumSt))
44 }
45 S <- 1000000
46 a <- 0.001
47 tick <- Sys.time()
48 ResABC <- ABC(S = S, a = 0.001, y = y)
49 tock <- Sys.time()
50 tock - tick
51 PostABC_Arown <- ResABC[["SelPrior"]]

```

*R code. Approximate Bayesian computation
accept/reject algorithm: g-and-k distribution
simulation*

```

1 # Regression adjusted ABC
2 X <- ResABC[["SelSumSt"]]-matrix(SumSt(y), S*a, 12, byrow =
  TRUE)
3 PostABC_ARownRegAd <- PostABC_ARown
4 for(j in 1:5){
5   Reg <- lm(PostABC_ARown[,j] ~ X)
6   # Coefficient of regressor 9 is na.
7   PostABC_ARownRegAd[,j] <- PostABC_ARown[,j] - X[,-9]%*%Reg
  $coefficients[-c(1,9)]
8 }
9 RGKnewSum <- function(par) {
10   z <- NULL
11   theta <- par[1]; a <- par[2]; b <- par[3]; g <- par[4]; k
    <- par[5]
12   e <- rnorm(n + 1)
13   for(t in 2:(n + 1)){
14     zt <- e[t] + theta * e[t-1]
15     z <- c(z, zt)
16   }
17   zs <- z / (1 + theta^2)^0.5
18   x <- a + b * (1 + 0.8 * (1 - exp(-g * zs)) / (1 + exp(-g *
    zs))) * (1 + zs^2)^k * zs
19   Etaz <- SumSt(x)
20   return(Etaz)
21 }
22 sum_stat_obs <- SumSt(y)
23 toy_prior <- list(c("unif",-1,1), c("unif",0,5), c("unif",
  0,5), c("unif", -5,5), c("unif", -0.5,5))
24 library(EasyABC)
25 tick <- Sys.time()
26 ABC_AR <- ABC_rejection(model=RGKnewSum, prior=toy_prior,
27 summary_stat_target = sum_stat_obs, nb_simul=260000, tol =
  0.00385,
28 progress_bar = TRUE)
29 tock <- Sys.time()
30 tock - tick
31 PostABC_AR <- coda::mcmc(ABC_AR$param)
32 # Summary
33 summary(coda::mcmc(PostABC_ARown))
34 summary(coda::mcmc(PostABC_ARownRegAd))
35 summary(coda::mcmc(PostABC_AR))

```

*R code. Approximate Bayesian computation
accept/reject algorithm: g-and-k distribution
simulation*

```

1 library(ggplot2); library(latex2exp)
2 Sp <- 1000
3
4 df1 <- data.frame(Value = c(PostABC_AR[1:Sp,1], PostABC_
  ARown[1:Sp,1], PostABC_ARownRegAd[1:Sp,1]), Distribution
  = factor(c(rep("EasyABC", Sp), rep("ABC", Sp), rep("
    ABCAdj", Sp))))
5 dentheta <- ggplot(df1, aes(x = Value, color = Distribution)
  ) + geom_density(linewidth = 1) + geom_vline(xintercept
  = theta1, linetype = "dashed", color = "red", linewidth
  = 1) +
6 labs(title = TeX("Posterior density plot:  $\theta$ "), x = TeX
  (" $\theta$ "), y = "Posterior density") + scale_color_
  manual(values = c("blue", "red", "green")) + theme_
  minimal() +
7 theme(legend.title = element_blank())
8
9 df2 <- data.frame(Value = c(PostABC_AR[1:Sp,4], PostABC_
  ARown[1:Sp,4], PostABC_ARownRegAd[1:Sp,4]), Distribution
  = factor(c(rep("EasyABC", Sp), rep("ABC", Sp), rep("
    ABCAdj", Sp))))
10 deng <- ggplot(df2, aes(x = Value, color = Distribution)) +
  geom_density(linewidth = 1) + geom_vline(xintercept =
  g, linetype = "dashed", color = "red", linewidth = 1) +
  labs(title = TeX("Posterior density plot: g"), x = TeX("
    g"), y = "Posterior density") +
11 scale_color_manual(values = c("blue", "red", "green")) +
  theme_minimal() + theme(legend.title = element_blank())
12
13 df3 <- data.frame(Value = c(PostABC_AR[1:Sp,5], PostABC_
  ARown[1:Sp,5], PostABC_ARownRegAd[1:Sp,5]), Distribution
  = factor(c(rep("EasyABC", Sp), rep("ABC", Sp), rep("
    ABCAdj", Sp))))
14 denk <- ggplot(df3, aes(x = Value, color = Distribution)) +
  geom_density(linewidth = 1) + geom_vline(xintercept =
  k, linetype = "dashed", color = "red", linewidth = 1) +
15 labs(title = TeX("Posterior density plot: k"), x = TeX(" $k$ "),
  y = "Posterior density") +
16 scale_color_manual(values = c("blue", "red", "green")) +
  theme_minimal() + theme(legend.title = element_blank())
17
18 library(ggpubr)
19 ggarrange(dentheta, deng, denk, labels = c("A", "B", "C"),
  ncol = 3, nrow = 1,
20 legend = "bottom", common.legend = TRUE)

```

2. Simulation: g-and-k distribution continues

Perform the simulation example of the Bayesian synthetic likelihood presented in the book, using the same population parameters and setting $M = 500$, $S = 6,000$, with burn-in and thinning parameters set to 1,000 and 5, respectively. Use the *BSL* package in **R** to perform inference using the vanilla, unbiased, semi-parametric, and misspecified (mean and variance) versions of BSL. Compare the posterior distributions of the methods with the true population parameters.

Answer:

The following code shows how to perform inference in this exercise. The results using the semi-parametric and misspecified settings in mean are very different compared to the other methods. In general, the performance of these methods are not good in this particular exercise and setting.

Figure 14.2 shows the posterior plots of the vanilla, unbiased and misspecified in variance settings. All the 95% credible intervals encompass the population variables; the posterior distributions of a , b and k are really good.

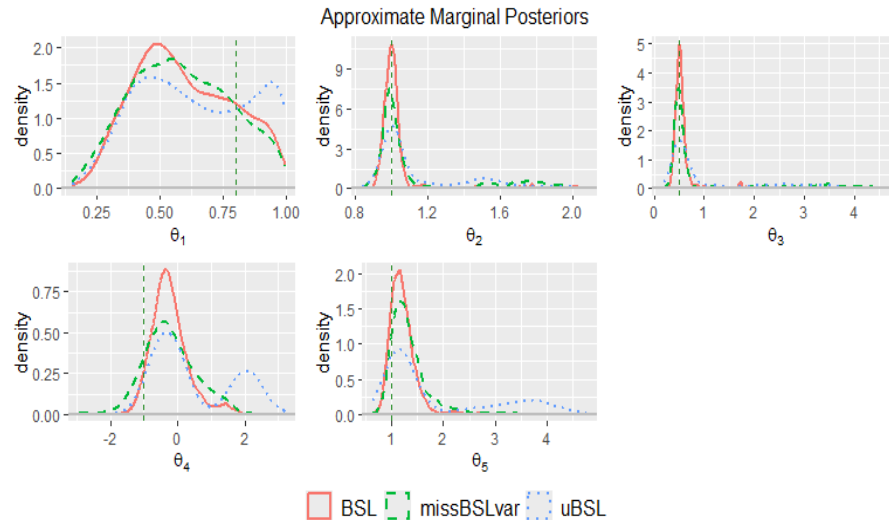


FIGURE 14.2

Posterior distributions using Bayesian synthetic likelihood: Simulation exercise g-and-k distribution.

R code. Bayesian synthetic likelihood: Simulation exercise g-and-k distribution.

```

1 rm(list = ls()); set.seed(010101)
2 library(BSL)
3 # Simulate g-and-k data
4 RGKnew <- function(par) {
5   z <- NULL
6   theta <- par[1]; a <- par[2]; b <- par[3]; g <- par[4]; k
   <- par[5]
7   e <- rnorm(n + 1)
8   for(t in 2:(n + 1)){
9     zt <- e[t] + theta * e[t-1]
10    z <- c(z, zt)
11  }
12  zs <- z / (1 + theta^2)^0.5
13  x <- a + b * (1 + 0.8 * (1 - exp(-g * zs)) / (1 + exp(-g *
    zs))) * (1 + zs^2)^k * zs
14  return(x)
15 }
16 # Summary statistics
17 SumSt <- function(y) {
18   Oct <- quantile(y, c(0.125, 0.25, 0.375, 0.5, 0.625, 0.75,
    0.875))
19   eta1 <- Oct[6] - Oct[2]
20   eta2 <- (Oct[6] + Oct[2] - 2 * Oct[4]) / eta1
21   eta3 <- (Oct[7] - Oct[5] + Oct[3] - Oct[1]) / eta1
22   autocor <- acf(y, lag = 2, plot = FALSE)
23   autocor[["acf"]][2:3]
24   Etay <- c(Oct[4], eta1, eta2, eta3, autocor[["acf"]][2:3])
25   return(Etay)
26 }
27 # Prior function
28 LogPrior <- function(par){
29   LogPi <- log(par[1] > -1 & par[1] < 1 & par[2] > 0 & par
    [2] < 5 & par[3] > 0 & par[3] < 5 & par[4] > -5 & par[4]
    < 5 & par[5] > -0.5 & par[5] < 5)
30   return(LogPi)
31 }
32 # Population parameters
33 theta1 <- 0.8; a <- 1; b <- 0.5; g <- -1; k <- 1
34 parpop <- c(theta1, a, b, g, k)
35 K <- 5
36 n <- 500
37 y <- RGKnew(par = parpop)
38 # Algorithm parameters
39 M <- 200 # Number of iterations to calculate mu and sigma
40 S <- 1100 # Number of MCMC iterations
41 burnin <- 100 # Burn in iterations
42 thin <- 2 # Thining parameter
43 keep <- seq(burnin + 1, S, thin)
44 par0 <- c(0.5, 2, 1, 0, 1)
45 Modelgk <- newModel(fnSim = RGKnew, fnSum = SumSt, theta0 =
    par0, fnLogPrior = LogPrior, verbose = FALSE)
46 validObject(Modelgk)
47 keep <- seq(burnin + 1, S, thin)
48 tune <- 0.1 # Tuning parameter RW MH
49 simgk <- simulation(Modelgk, n = M, theta = par0, seed = 10)
50 par(mfrow = c(2, 3))

```

R code. Bayesian synthetic likelihood: Simulation exercise g-and-k distribution.

```

1 # Check if the summary statistics are roughly normal
2 for (i in 1:6){
3   eval <- seq(min(simgk$ssx[, i]), max(simgk$ssx[, i]),
4             0.001)
5   densnorm <- dnorm(eval, mean = mean(simgk$ssx[, i]), sd(
6     simgk$ssx[, i]))
7   plot(density(simgk$ssx[, i]), main = "", xlab = "")
8   lines(eval, densnorm, col = "red")
9 }
10 Lims <- matrix(c(-1, 0, 0, -5, -0.5, 1, rep(5, 4)), 5, 2)
11 tick <- Sys.time()
12 Resultsgk <- bsl(y = y, n = M, M = S, model = Modelgk,
13   covRandWalk = tune*diag(5),
14   method = "BSL", thetaNames = expression(theta, a, b, g, k),
15   logitTransformBound = Lims, plotOnTheFly = TRUE)
16 tock <- Sys.time()
17 tock - tick
18 PostChain <- coda::mcmc(Resultsgk@theta[keep,])
19 CovarRWnew <- var(PostChain)
20 M <- 500 # Number of iterations to calculate mu and sigma
21 S <- 6000 # Number of MCMC iterations
22 burnin <- 1000 # Burn in iterations
23 thin <- 5 # Thining parameter
24 keep <- seq(burnin + 1, S, thin)
25 tune <- 1 # Tuning parameter RW MH
26 ModelgkNew <- newModel(fnSim = RGKnew, fnSum = SumSt, theta0
27   = par0, fnLogPrior = LogPrior, verbose = FALSE)
28 logitTransform <- function(par, a, b){
29   logtrans <- log((par - a)/(b - par))
30   return(logtrans)
31 }
32 ParTrans <- matrix(NA, dim(PostChain)[1], 5)
33 for(j in 1:5){
34   ParTrans[,j] <- logitTransform(par = PostChain[,j], a =
35     Lims[j,1], b = Lims[j,2])
36 }
37 CovarRW <- var(ParTrans)
38 # Vanilla BLS
39 tick <- Sys.time()
40 ResultsgkVanila <- bsl(y = y, n = M, M = S, model =
41   ModelgkNew, covRandWalk = tune*CovarRW, method = "BSL",
42   thetaNames = expression(theta, a, b, g, k),
43   logitTransformBound = Lims, plotOnTheFly = TRUE)
44 tock <- Sys.time()
45 tock - tick
46 # Unbiased BLS
47 tick <- Sys.time()
48 ResultsgkuBLS <- bsl(y = y, n = M, M = S, model = ModelgkNew
49   , covRandWalk = tune*CovarRW,
50   method = "uBSL", thetaNames = expression(theta, a, b, g, k),
51   logitTransformBound = Lims, plotOnTheFly = TRUE)
52 tock <- Sys.time()
53 tock - tick

```

R code. Bayesian synthetic likelihood: Simulation exercise g-and-k distribution.

```

1 # Semi-parametric BLS
2 tick <- Sys.time()
3 ResultsgksemiBLS <- bsl(y = y, n = M, M = S, model =
   ModelgkNew, covRandWalk = tune*CovarRW, method = "
   semiBSL", thetaNames = expression(theta, a, b, g, k),
4 logitTransformBound = Lims, plotOnTheFly = TRUE)
5 tock <- Sys.time()
6 tock - tick
7 # Misspecified BLS in mean
8 tick <- Sys.time()
9 ResultsgkmisspecBLSmean <- bsl(y = y, n = M, M = S, model =
   ModelgkNew, covRandWalk = tune*CovarRW, method = "
   BSLmisspec", thetaNames = expression(theta, a, b, g, k),
10 logitTransformBound = Lims, misspecType = "mean", tau = 0.5,
   plotOnTheFly = TRUE)
11 tock <- Sys.time()
12 tock - tick
13 # Misspecified BLS in variance
14 tick <- Sys.time()
15 ResultsgkmisspecBLSvar <- bsl(y = y, n = M, M = S, model =
   ModelgkNew, covRandWalk = tune*CovarRW, method = "
   BSLmisspec", thetaNames = expression(theta, a, b, g, k),
16 logitTransformBound = Lims, misspecType = "variance", tau =
   0.5, plotOnTheFly = TRUE)
17 tock <- Sys.time()
18 tock - tick
19 # Summary all models
20 gkResults <- list(ResultsgkVanila, ResultsgkuBLS,
   ResultsgksemiBLS, ResultsgkmisspecBLSmean,
   ResultsgkmisspecBLSvar)
21 names(gkResults) <- c("BSL", "uBSL", "semiBSL", "missBSLmean",
   "missBSLvar")
22 t(sapply(gkResults, summary))
23 combinePlotsBSL(gkResults, which = 2, thetaTrue = parpop,
   thin = thin, options.linetype = list(values = 1:8),
   options.size = list(values = rep(1, 8)), options.theme =
   list(plot.margin = grid::unit(rep(0.03, 4), "npc"),
   axis.title = ggplot2::element_text(size = 12), axis.text
   = ggplot2::element_text(size = 8),
24 legend.text = ggplot2::element_text(size = 12)))
25 # semi BSL and miss BSL mean are totally diferent to other
   methods
26 gkResultsNew <- list(gkResults[["BSL"]], gkResults[["uBSL"]],
   gkResults[["missBSLvar"]])
27 names(gkResultsNew) <- c("BSL", "uBSL", "missBSLvar")
28 combinePlotsBSL(gkResultsNew, which = 2, thetaTrue = parpop,
   thin = thin, options.linetype = list(values = 1:8),
   options.size = list(values = rep(1, 8)), options.theme =
   list(plot.margin = grid::unit(rep(0.03, 4), "npc"),
   legend.text = ggplot2::element_text(size = 12)))

```

3. Simulate a multinomial logit model (see Section 6.5 in the book) with 3 alternatives, 2 alternative-specific regressors, and 1 individual-specific regressor. The population parameters for the alternative-specific regressors are -0.3 and 1.2 , while the population values for the individual-specific regressor are 0.3 , 0 , and 0.5 . All regressors are assumed to follow a standard normal distribution, and the sample size is $1,000$.

Perform inference using the *INLA* package, and note that the Poisson trick should be used for multinomial models in this exercise (see [46] for details).

Answer:

R code. Multinomial model in INLA: Simulation exercise.

```

1 rm(list = ls()); set.seed(010101)
2 library(INLA)
3 beta1 <- -0.3; beta2 <- 1.2; gamma <- c(0.3, 0, 0.5)
4 param <- c(beta1, beta2, gamma); n <- 1000
5 # Alternative specific regressors
6 x11 <- rnorm(n); x12 <- rnorm(n); x13 <- rnorm(n)
7 x21 <- rnorm(n); x22 <- rnorm(n); x23 <- rnorm(n)
8 # Individual specific regressor
9 Z <- rnorm(n)
10 Multinom.sample = function(N){
11   Y <- matrix(NA, ncol = 3, nrow = n)
12   for(i in 1:n){
13     V1 <- beta1*x11[i] + beta2*x21[i] + gamma[1]*Z[i]
14     V2 <- beta1*x12[i] + beta2*x22[i] + gamma[2]*Z[i]
15     V3 <- beta1*x13[i] + beta2*x23[i] + gamma[3]*Z[i]
16     probs <- c(V1, V2, V3)
17     probs <- exp(probs)/sum(exp(probs))
18     samp <- rmultinom(1, N, prob = probs)
19     Y[i,] = as.vector(samp)
20   }
21   colnames(Y) = c("Y1", "Y2", "Y3")
22   return(Y)
23 }
24 Y <- Multinom.sample(1)
25 df <- data.frame(cbind(Y, x11, x12, x13, x21, x22, x23, Z))
26 Data.structure = function(df){
27   Data = matrix(NA, ncol = 6, nrow = n*3)
28   for(i in 1:n){
29     # simulated variable
30     Data[((i-1)*3+1):(i*3), 1] = c(df$Y1[i], df$Y2[i], df$Y3[i])
31     # alternative specific with generic coeff
32     Data[((i-1)*3+1):(i*3), 2] = c(df$x11[i], df$x12[i], df$x13[i])
33     # alternative specific with generic coeff
34     Data[((i-1)*3+1):(i*3), 3] = c(df$x21[i], df$x22[i], df$x23[i])
35     # individual specific with alternative coeff
36     Data[((i-1)*3+1):(i*3), 4] = rep(df$Z[i], 3)
37     # choice situation index
38     Data[((i-1)*3+1):(i*3), 5] = rep(i, 3)
39     # choice alternative index
40     Data[((i-1)*3+1):(i*3), 6] = c(1, 2, 3)
41   }
42   Data = data.frame(Data)
43   names(Data) = c('Y', "X1", "X2", "Z", 'phi', 'alt.idx')
44   return(Data)
45 }
46 formula = Y ~ -1 + X1 + X2 + f(phi, initial = -10, fixed = T)
47   + f(alt.idx, Z, fixed = T, constr = T)
48 Data = Data.structure(df)
49 model = inla(formula, data = Data, family = 'Poisson')
50 result = rbind(model$summary.fixed[1:5], model$summary.random$alt.idx[2:6])
51 result = cbind(result, true = param)

```

R code. Multinomial model in INLA: Simulation exercise.

```
1 row.names(result) = c("beta1", "beta2", "gamma.A", "gamma.B",  
  "gamma.C" )  
2 round(result,3)  
3 diff.result = cbind("0.025quant" = diff(model$summary.random$  
  alt.idx$'0.025quant'), "0.5quant" = diff(model$summary.  
  random$alt.idx$'0.5quant'), "0.975quant" = diff(model$  
  summary.random$alt.idx$'0.975quant'), "true" = diff(  
  gamma))  
4 row.names(diff.result) = c("gamma.B - gamma.A", "gamma.C -  
  gamma.B")  
5 round(diff.result,3)
```

4. Get the expression for the ELBO in the linear regression model with conjugate family.

Answer:

$$\begin{aligned}
\text{ELBO}(q(\boldsymbol{\beta}, \sigma^2)) &= \mathbb{E}_{\boldsymbol{\beta}, \sigma^2} [\log p(\mathbf{y}, \boldsymbol{\beta}, \sigma^2 \mid \mathbf{X})] - \mathbb{E}_{\boldsymbol{\beta}, \sigma^2} [\log q(\boldsymbol{\beta}, \sigma^2)] \\
&= \mathbb{E}_{\boldsymbol{\beta}, \sigma^2} [\log p(\mathbf{y} \mid \boldsymbol{\beta}, \sigma^2, \mathbf{X}) + \log \pi(\boldsymbol{\beta} \mid \sigma^2) + \log(\sigma^2)] \\
&\quad - \mathbb{E}_{\boldsymbol{\beta}, \sigma^2} [\log q(\boldsymbol{\beta}) + \log q(\sigma^2)] \\
&= \mathbb{E}_{\boldsymbol{\beta}, \sigma^2} \left[-\frac{N}{2} \log(2\pi) - \frac{N}{2} \log \sigma^2 - \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \right. \\
&\quad - \frac{K}{2} \log(2\pi) - \frac{K}{2} \log(\sigma^2) - \frac{1}{2} \log |\mathbf{B}_0| - \frac{1}{2\sigma^2} (\boldsymbol{\beta} - \boldsymbol{\beta}_0)^\top \mathbf{B}_0^{-1} (\boldsymbol{\beta} - \boldsymbol{\beta}_0) \\
&\quad \left. - (\alpha_0/2 + 1) \log \sigma^2 - \frac{\delta_0}{2\sigma^2} + \alpha_0/2 \log(\delta_0/2) - \log \Gamma(\alpha_0/2) \right] \\
&\quad - \left\{ \mathbb{E}_{\boldsymbol{\beta}, \sigma^2} \left[\frac{K}{2} \log \mathbb{E}_{\sigma^2}(1/\sigma^2) - \frac{1}{2} \log |\mathbf{B}_n| - \frac{1}{2} \mathbb{E}_{\sigma^2}(1/\sigma^2) (\boldsymbol{\beta} - \boldsymbol{\beta}_n)^\top \mathbf{B}_n^{-1} (\boldsymbol{\beta} - \boldsymbol{\beta}_n) \right. \right. \\
&\quad \left. \left. - \frac{K}{2} \log(2\pi) - (\alpha_n/2 + 1) \log \sigma^2 - \frac{\delta_n}{2\sigma^2} + \alpha_n/2 \log(\delta_n/2) - \log \Gamma(\alpha_n/2) \right] \right\} \\
&= -\frac{N}{2} \log(2\pi) - \frac{1}{2} \log |\mathbf{B}_0| + \frac{1}{2} \log |\mathbf{B}_n| + \alpha_0/2 \log(\delta_0/2) \\
&\quad - \alpha_n/2 \log(\delta_n/2) - \log \Gamma(\alpha_0/2) + \log \Gamma(\alpha_n/2) - \frac{K}{2} \log \mathbb{E}_{\sigma^2}(1/\sigma^2) \\
&\quad + \frac{1}{2} \mathbb{E}_{\sigma^2}(1/\sigma^2) \mathbb{E}_{\boldsymbol{\beta}} [(\boldsymbol{\beta} - \boldsymbol{\beta}_n)^\top \mathbf{B}_n^{-1} (\boldsymbol{\beta} - \boldsymbol{\beta}_n)] \\
&= -\frac{N}{2} \log(2\pi) - \frac{1}{2} \log |\mathbf{B}_0| + \frac{1}{2} \log |\mathbf{B}_n| + \alpha_0/2 \log(\delta_0/2) \\
&\quad - \alpha_n/2 \log(\delta_n/2) - \log \Gamma(\alpha_0/2) + \log \Gamma(\alpha_n/2) - \frac{K}{2} \log(\alpha_n/\delta_n) \\
&\quad + 0.5 \text{tr} \{ \text{Var}(\boldsymbol{\beta}) \mathbf{B}_n^{-1} \}
\end{aligned}$$

We take into account in the fourth equality that the terms collecting $\log \sigma^2$ and $\frac{1}{2\sigma^2}$ cancel out by the definitions of α_n and δ_n , respectively.

5. Linear regression continues

Perform inference in the linear regression example using stochastic variational inference via the automatic differentiation variational inference (ADVI) approach [25], implemented in **rstan** package. In particular, consider the model $y_i = \mathbf{x}_i^\top \boldsymbol{\beta} + \mu_i$, assuming non-informative independent priors: $\boldsymbol{\beta} \sim N(\boldsymbol{\beta}_0, \mathbf{B}_0)$ and $\sigma^2 \sim \text{IG}(\alpha_0/2, \delta_0/2)$, where $\boldsymbol{\beta}_0 = \mathbf{0}_3$, $\mathbf{B}_0 = 1000\mathbf{I}_3$, and $\alpha_0 = \delta_0 = 0.01$. The sample size is one hundred thousand.

Answer:

The posterior mean values are very close to the population parameters, but the 95% credible intervals are too narrow and do not contain the true values.

*R code. Stochastic variational inference:
Simulation exercise linear regression.*

```

1 library(rstan)
2 # --- Simulated data ---
3 N <- 100000; K <- 2; B <- rep(1, K + 1); sig2 <- 1
4 X <- cbind(1, matrix(rnorm(N * K), N, K))
5 y <- X %*% B + rnorm(N, 0, sig2^0.5)
6 # --- Priors ---
7 b0 <- 0; B0 <- 1000; a0 <- 0.01; d0 <- 0.01
8 # --- Prepare data for Stan ---
9 data_list <- list(
10 N = N,
11 K = K + 1, # K+1 because of the intercept
12 X = X,
13 y = as.vector(y),
14 b0 = b0,
15 B0 = B0,
16 a0 = a0,
17 d0 = d0
18 )
19 # --- Stan model compilation ---
20 stan_model_code <- "
21 data {
22   int<lower=0> N;           // number of data points
23   int<lower=0> K;           // number of predictors
24   matrix[N, K] X;          // predictor matrix
25   vector[N] y;             // response vector
26   real b0;                 // location parameter prior on beta
27   real<lower=0> B0;         // variance parameter prior on beta
28   real<lower=0> a0;         // shape parameter prior on sigma^2
29   real<lower=0> d0;         // rate parameter prior on sigma^2
30 }
31 parameters {
32   vector[K] beta;          // coefficients of the linear model
33   real<lower=0> sigma2;     // variance parameter (sigma^2)
34 }
35 model {
36   // Priors
37   beta ~ normal(b0, B0);    // Normal prior on beta
38   sigma2 ~ inv_gamma(a0/2, d0/2); // Inverse Gamma prior
39
40   // Likelihood
41   y ~ normal(X * beta, sqrt(sigma2)); // Normal likelihood
42   // with linear predictor
43 }
44 generated quantities {
45   vector[N] y_pred;
46   for (n in 1:N) {
47     y_pred[n] = normal_rng(dot_product(X[n], beta), sqrt(
48       sigma2));
49   }
50 }
51 "

```

*R code. Stochastic variational inference:
Simulation exercise linear regression.*

```

1 # Compile Stan model
2 stan_model <- stan_model(model_code = stan_model_code)
3 # --- Perform Stochastic Variational Inference (SVI) ---
4 fit <- vb(stan_model, data = data_list, algorithm = "
    meanfield", iter = 10000)
5 # --- Extract the results ---
6 mu_estimate <- extract(fit)$beta
7 summary(coda::mcmc(mu_estimate))
8 sigma2_estimate <- extract(fit)$sigma2
9 summary(coda::mcmc(as.vector(sigma2_estimate)))
10

```

6. Let's retake the mixture regression model of Chapter 11, that is, the simple regression mixture with two components such that $z_{i1} \sim \text{Ber}(0.5)$, consequently, $z_{i2} = 1 - z_{i1}$, and assume one regressor, $x_i \sim N(0, 1)$, $i = 1, 2, \dots, 1,000$. Then,

$$p(y_i | \mathbf{x}_i) = 0.5\phi(y_i | 2 + 1.5x_i, 1^2) + 0.5\phi(y_i | -1 + 0.5x_i, 0.8^2).$$

Let's set $\alpha_{h0} = \delta_{h0} = 0.01$, $\beta_{h0} = \mathbf{0}_2$, $\mathbf{B}_{h0} = \mathbf{I}_2$, and $\mathbf{a}_0 = [1/2 \ 1/2]^\top$.

Perform VB inference in this model using the CAVI algorithm.

Answer:

The likelihood function in this model is $p(\mathbf{y} | \mathbf{z}, \beta, \sigma^2) = \prod_{i=1}^N \prod_{h=1}^H \phi(y_i | \mathbf{x}_i^\top \beta_h, \sigma_h^2)^{z_{ih}}$, and $\pi(\mathbf{z} | \lambda) = \prod_{i=1}^N \prod_{h=1}^H \lambda_h^{z_{ih}}$, $\lambda \sim \text{Dir}(\mathbf{a}_0)$, $\beta_h \sim N(\beta_{h0}, \mathbf{B}_{h0})$ and $\sigma_h^2 \sim \text{IG}(\alpha_{h0}/2, \delta_{h0}/2)$. In this notation, $\mathbf{z}, \beta, \sigma^2$ and λ stacks all the latent indicators, all locations and scale parameters of each cluster, and all the probabilities, respectively.

Let's work with the mean-field variational family $q(\mathbf{z}, \lambda, \beta, \sigma^2) = q(\mathbf{z})q(\lambda) \prod_{h=1}^H q(\beta_h) \prod_{h=1}^H q(\sigma_h^2)$.

Then,

$$\begin{aligned}
\log q(\mathbf{z}) &= \mathbb{E}_{\boldsymbol{\lambda}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2} [\log p(\mathbf{y}, \mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2)] + c_1 \\
&= \mathbb{E}_{\boldsymbol{\lambda}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2} [\log p(\mathbf{y} \mid \mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2) + \log \pi(\mathbf{z} \mid \boldsymbol{\lambda})] + c_2 \\
&= \mathbb{E}_{\boldsymbol{\lambda}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2} \left[\sum_{i=1}^N \sum_{h=1}^H z_{ih} \left\{ -0.5 \log 2\pi - 0.5 \log \sigma_h^2 - \right. \right. \\
&\quad \left. \left. - \frac{1}{2\sigma_h^2} (y_i - \mathbf{x}_i \boldsymbol{\beta}_h)^2 + \log \lambda_h \right\} \right] + c_2 \\
&= \sum_{i=1}^N \sum_{h=1}^H \psi_{ih} \log \rho_{ih} + c_2.
\end{aligned}$$

Thus, $q(\mathbf{z}) = \prod_{i=1}^N \prod_{h=1}^H r_{ih}^{z_{ih}}$, such that $\mathbb{E}_{z_{ih}}[z_{ih}] = r_{ih}$, where $r_{ih} = \frac{\rho_{ih}}{\sum_{j=1}^H \rho_{ij}}$, and

$$\log \rho_{ih} = -0.5 \log 2\pi - 0.5 \mathbb{E}_{\sigma_h^2} \log \sigma_h^2 - 0.5 \mathbb{E}_{\sigma_h^2} \left(\frac{1}{\sigma_h^2} \right) \mathbb{E}_{\boldsymbol{\beta}_h} (y_i - \mathbf{x}_i^\top \boldsymbol{\beta}_h)^2 + \mathbb{E}_{\lambda_h} \log \lambda_h.$$

$$\begin{aligned}
\log q(\boldsymbol{\lambda}) &= \mathbb{E}_{\mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2} [\log p(\mathbf{y}, \mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2)] + c_1 \\
&= \mathbb{E}_{\mathbf{z}} [\log \pi(\mathbf{z} \mid \boldsymbol{\lambda}) + \log \pi(\boldsymbol{\lambda})] + c_2 \\
&= \mathbb{E}_{\mathbf{z}} \left[\sum_{i=1}^N \sum_{h=1}^H z_{ih} \log \lambda_h + \sum_{h=1}^H (a_{h0} - 1) \log \lambda_h \right] + c_3 \\
&= \sum_{i=1}^N \sum_{h=1}^H \mathbb{E}_{z_{ih}}[z_{ih}] \log \lambda_h + \sum_{h=1}^H (a_{h0} - 1) \log \lambda_h + c_3 \\
&= \sum_{i=1}^N \sum_{h=1}^H r_{ih} \log \lambda_h + \sum_{h=1}^H (a_{h0} - 1) \log \lambda_h + c_3 \\
&= \sum_{h=1}^H \left(\sum_{i=1}^N r_{ih} + a_{h0} - 1 \right) \log \lambda_h + c_3.
\end{aligned}$$

This implies that $q(\boldsymbol{\lambda})$ is Dirichlet with parameters $a_{hn} = a_{h0} + \sum_{i=1}^N r_{ih}$.

$$\begin{aligned}
\log q(\boldsymbol{\beta}_h) &= \mathbb{E}_{\boldsymbol{\lambda}, \mathbf{z}, \boldsymbol{\sigma}^2} [\log p(\mathbf{y}, \mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2)] + c_1 \\
&= \mathbb{E}_{\mathbf{z}, \boldsymbol{\sigma}^2} [\log p(\mathbf{y} \mid \mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2) + \log \pi(\boldsymbol{\beta}_h)] + c_2 \\
&= \mathbb{E}_{\mathbf{z}, \boldsymbol{\sigma}^2} \left[\sum_{i=1}^N \sum_{h=1}^H z_{ih} \left\{ -\frac{1}{2\sigma_h^2} (y_i - \mathbf{x}_i^\top \boldsymbol{\beta}_h)^2 \right\} \right. \\
&\quad \left. - 0.5(\boldsymbol{\beta}_h - \boldsymbol{\beta}_{h0})^\top \mathbf{B}_{h0}^{-1} (\boldsymbol{\beta}_h - \boldsymbol{\beta}_{h0}) \right] + c_3.
\end{aligned}$$

Following same steps as those to obtain the posterior distribution,

and taking into account the expected values, we have that $q(\beta_h)$ is $N(\beta_{hn}, \mathbf{B}_{hn})$, where $\mathbf{B}_{hn} = \left(\mathbf{B}_{h0}^{-1} + \mathbb{E}_{\sigma_h^2} \left(\frac{1}{\sigma_h^2} \right) \sum_{i=1}^N r_{ih} \mathbf{x}_i \mathbf{x}_i^\top \right)^{-1}$ and $\beta_{hn} = \mathbf{B}_{hn} \left(\mathbf{B}_{h0}^{-1} \beta_{h0} + \mathbb{E}_{\sigma_h^2} \left(\frac{1}{\sigma_h^2} \right) \sum_{i=1}^N r_{ih} \mathbf{x}_i y_i \right)$.

$$\begin{aligned} \log q(\sigma_h^2) &= \mathbb{E}_{\lambda, \mathbf{z}, \beta} [\log p(\mathbf{y}, \mathbf{z}, \beta, \sigma^2)] + c_1 \\ &= \mathbb{E}_{\mathbf{z}, \beta} [\log p(\mathbf{y} | \mathbf{z}, \beta, \sigma^2) + \log \pi(\sigma_h^2)] + c_2 \\ &= \mathbb{E}_{\mathbf{z}, \beta} \left[\sum_{i=1}^N \sum_{h=1}^H z_{ih} \left\{ -0.5 \log \sigma_h^2 - \frac{1}{2\sigma_h^2} (y_i - \mathbf{x}_i^\top \beta_h)^2 \right\} \right. \\ &\quad \left. - (\alpha_{h0}/2 + 1) \log \sigma_h^2 - \frac{\delta_{h0}}{2\sigma_h^2} \right] + c_1. \end{aligned}$$

Collecting terms we see that $q(\sigma_h^2)$ is $IG(\alpha_{hn}/2, \delta_{hn}/2)$, where $\alpha_{hn} = \sum_{i=1}^N r_{ih} + \alpha_{h0}$ and $\delta_{hn} = \sum_{i=1}^N r_{ih} \mathbb{E}_{\beta_h} (y_i - \mathbf{x}_i^\top \beta_h)^2 + \delta_{h0}$.

Note that the previous results imply that $\mathbb{E}_{\sigma_h^2} \left(\frac{1}{\sigma_h^2} \right) = \alpha_{hn}/\delta_{hn}$, $\mathbb{E}_{\beta_h} (y_i - \mathbf{x}_i^\top \beta_h)^2 = (y_i - \mathbf{x}_i^\top \beta_{hn})^2 + \mathbf{x}_i^\top \mathbf{B}_{hn} \mathbf{x}_i$, and

$$\begin{aligned} \log \rho_{ih} &= -0.5 \log 2\pi - 0.5 \mathbb{E}_{\sigma_h^2} \log \sigma_h^2 - 0.5 \mathbb{E}_{\sigma_h^2} \left(\frac{1}{\sigma_h^2} \right) \mathbb{E}_{\beta_h} (y_i - \mathbf{x}_i^\top \beta_h)^2 + \mathbb{E}_{\lambda_h} \log \lambda_h \\ &= -0.5 \log 2\pi - 0.5 (\log(\delta_{hn}/2) - \psi(\alpha_{hn}/2)) - 0.5 (\alpha_{hn}/\delta_{hn}) (y_i - \mathbf{x}_i^\top \beta_{hn})^2 \\ &\quad + \mathbf{x}_i^\top \mathbf{B}_{hn} \mathbf{x}_i + \psi(a_{hn}) - \psi \left(\sum_{h=1}^H a_{hn} \right), \end{aligned}$$

where ψ is the digamma function.

In addition, the evidence lower bound (ELBO) is given by

$$\begin{aligned}
\text{ELBO}(q(\boldsymbol{\theta})) &= \mathbb{E}_q [\log p(\mathbf{y}, \mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2)] - \mathbb{E}_q [\log q(\mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2)] \\
&= \mathbb{E}_q \left[\log p(\mathbf{y} \mid \mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2) + \log \pi(\mathbf{z} \mid \boldsymbol{\lambda}) + \log \pi(\boldsymbol{\lambda}) + \sum_{h=1}^H (\log \pi(\boldsymbol{\beta}_h) + \log \pi(\sigma_h^2)) \right] \\
&\quad - \mathbb{E}_q \left[\log q(\mathbf{z}) + \log \pi(\boldsymbol{\lambda}) + \sum_{h=1}^H (\log q(\boldsymbol{\beta}_h) + \log \pi(\sigma_h^2)) \right] \\
&= \mathbb{E}_q \left[\sum_{i=1}^N \sum_{h=1}^H z_{ih} \left\{ -0.5 \log(2\pi) - 0.5 \log \sigma_h^2 - 0.5 \frac{1}{\sigma_h^2} (y_i - \mathbf{x}_i^\top \boldsymbol{\beta}_h)^2 \right\} \right. \\
&\quad + \sum_{i=1}^N \sum_{h=1}^H z_{ih} \log \lambda_h + \sum_{h=1}^H \left\{ -\frac{K}{2} \log(2\pi) - 0.5 \log |\mathbf{B}_{h0}| - 0.5 (\boldsymbol{\beta}_h - \boldsymbol{\beta}_{h0})^\top \mathbf{B}_{h0}^{-1} (\boldsymbol{\beta}_h - \boldsymbol{\beta}_{h0}) \right\} \\
&\quad + \sum_{h=1}^H \left\{ -(\alpha_{h0}/2 + 1) \log \sigma_h^2 - \frac{\delta_{h0}}{2\sigma_h^2} + (\alpha_{h0}/2) \log(\delta_{h0}/2) - \log \Gamma(\alpha_{h0}/2) \right\} \Big] \\
&\quad - \mathbb{E}_q \left[\sum_{i=1}^N \sum_{h=1}^H z_{ih} \log r_{ih} - \log B(\mathbf{a}_n) + \sum_{h=1}^H (a_{hn} - 1) \log \lambda_h \right. \\
&\quad + \sum_{h=1}^H \left\{ -\frac{K}{2} \log(2\pi) - 0.5 \log |\mathbf{B}_{hn}| - 0.5 (\boldsymbol{\beta}_h - \boldsymbol{\beta}_{hn})^\top \mathbf{B}_{hn}^{-1} (\boldsymbol{\beta}_h - \boldsymbol{\beta}_{hn}) \right\} \\
&\quad \left. + \sum_{h=1}^H \left\{ -(\alpha_{hn}/2 + 1) \log \sigma_h^2 - \frac{\delta_{hn}}{2\sigma_h^2} + (\alpha_{hn}/2) \log(\delta_{hn}/2) - \log \Gamma(\alpha_{hn}/2) \right\} \right].
\end{aligned}$$

The first part of this expression, that is, $\mathbb{E}_q [\log p(\mathbf{y}, \mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2)]$, is given by

$$\begin{aligned}
&\sum_{i=1}^N \sum_{h=1}^H r_{ih} \left\{ -0.5 \log(2\pi) - 0.5 (\log(\delta_{hn}/2) - \psi(\alpha_{hn}/2)) - \frac{\alpha_{hn}}{2\delta_{hn}} [(y_i - \mathbf{x}_i^\top \boldsymbol{\beta}_{hn})^2 + \mathbf{x}_i^\top \mathbf{B}_{hn} \mathbf{x}_i] \right\} + \\
&\sum_{i=1}^N \sum_{h=1}^H r_{ih} \log r_{ih} + \sum_{h=1}^H \left\{ -\frac{K}{2} \log(2\pi) - 0.5 \log |\mathbf{B}_{h0}| - \text{tr}(\mathbf{B}_{hn} \mathbf{B}_{h0}^{-1}) - (\boldsymbol{\beta}_{hn} - \boldsymbol{\beta}_{h0})^\top (\boldsymbol{\beta}_{hn} - \boldsymbol{\beta}_{h0}) \right\} + \\
&\sum_{h=1}^H \left\{ -(\alpha_{h0}/2 + 1) (\log(\delta_{hn}/2) - \psi(\alpha_{hn}/2)) - \frac{\delta_{h0}}{2} \frac{\alpha_{hn}}{\delta_{hn}} + (\alpha_{h0}/2) \log(\delta_{h0}/2) - \log \Gamma(\alpha_{h0}/2) \right\}.
\end{aligned}$$

The second part of the ELBO, that is, $\mathbb{E}_q [\log q(\mathbf{z}, \boldsymbol{\beta}, \boldsymbol{\sigma}^2)]$ is given by

$$\begin{aligned}
&\sum_{i=1}^N \sum_{h=1}^H r_{ih} \log r_{ih} - \log B(\mathbf{a}_n) + \sum_{h=1}^H (a_{hn} - 1) \left(\psi(a_{hn}) - \psi \left(\sum_{h=1}^H a_{hn} \right) \right) + \\
&\sum_{h=1}^H \left\{ -\frac{K}{2} (\log(2\pi) + 1) - 0.5 \log |\mathbf{B}_{hn}| + (\alpha_{hn}/2 + 1) \psi(\alpha_{hn}/2) - 0.5 \alpha_{hn} - \log(\delta_{hn}/2) - \log \Gamma(\alpha_{hn}/2) \right\}.
\end{aligned}$$

We use

$$\begin{aligned}
\mathbb{E}_{\boldsymbol{\beta}_h} [(\boldsymbol{\beta}_h - \boldsymbol{\beta}_{hn})^\top \mathbf{B}_{hn}^{-1} (\boldsymbol{\beta}_h - \boldsymbol{\beta}_{hn})] &= \text{tr} \{ \mathbb{E}_{\boldsymbol{\beta}_h} (\boldsymbol{\beta}_h - \boldsymbol{\beta}_{hn})^\top \mathbf{B}_{hn}^{-1} (\boldsymbol{\beta}_h - \boldsymbol{\beta}_{hn}) \} \\
&= \mathbb{E}_{\boldsymbol{\beta}_h} [\text{tr} \{ (\boldsymbol{\beta}_h - \boldsymbol{\beta}_{hn})^\top \mathbf{B}_{hn}^{-1} (\boldsymbol{\beta}_h - \boldsymbol{\beta}_{hn}) \}] \\
&= \text{tr} \{ \mathbb{E}_{\boldsymbol{\beta}_h} [(\boldsymbol{\beta}_h - \boldsymbol{\beta}_{hn})(\boldsymbol{\beta}_h - \boldsymbol{\beta}_{hn})^\top \mathbf{B}_{hn}^{-1}] \} \\
&= \text{tr} \{ \text{Var}(\boldsymbol{\beta}_h) \mathbf{B}_{hn}^{-1} \} \\
&= \text{tr} \{ \mathbf{I}_K \} \\
&= K
\end{aligned}$$

R code. Variational Bayes: Mixture Gaussian regression.

```

1 rm(list = ls())
2 set.seed(010101)
3 # Simulate data from a 2-component mixture model
4 n <- 1000
5 K <- 2; H <- 2
6 x <- rnorm(n)
7 z <- rbinom(n, 1, 0.5) # Latent class indicator
8 y <- ifelse(z == 0, rnorm(n, 2 + 1.5*x, 1), rnorm(n, -1 +
9   0.5*x, 0.8))
9 X <- cbind(1,x)
10 # Iterations
11 S <- 10000
12 ##### From scratch #####
13 # Hyperparameters
14 b0h <- rep(0, K); B0h <- diag(1000,K); B0hi <- solve(B0h)
15 a0h <- 0.01; d0h <- 0.01; a0 <- rep(1/H, H)
16 # Innitial values
17 dhn <- c(50, 50); ahn <- c(500, 500)
18 bhn <- matrix(c(2, 1.5, -1, 0.5), K, H)
19 Bhn <- array(c(1,0,0,1,1,0,0,1), c(K, K, H))
20 an <- c(500, 500)
21 VarParPsi <- function(dhn, ahn, bhn, Bhn, an){
22   logrhoih <- matrix(NA, n, H)
23   for(i in 1:n){
24     for(h in 1:H){
25       logrhoih[i,h] <- -0.5*log(2*pi) - 0.5*(log(dhn[h]/2) -
26         digamma(ahn[h]/2)) -
27       0.5*(ahn[h]/dhn[h])*((y[i] - t(X[i,])%*%bhn[,h])^2 + t
28       (X[i,])%*%Bhn[,h]%*%X[i,]) +
29       digamma(an[h]) - digamma(sum(an))
30     }
31   }
32   rhoih <- exp(logrhoih - apply(logrhoih, 1, max))
33   rih <- rhoih / rowSums(rhoih)
34   return(rih)
35 }
36 Rih <- VarParPsi(dhn = dhn, ahn = ahn, bhn = bhn, Bhn = Bhn,
37   an = an)
38 VarParLambda <- function(rih){
39   an <- a0 + colSums(rih)
40   return(an)
41 }
42 VarParLambda(rih = Rih)
43 VarParBetah <- function(ahn, dhn, rh){
44   rXX <- matrix(0, K, K)
45   rXy <- matrix(0, K, 1)
46   for(i in 1:n){
47     rXXi <- rh[i]*X[i,]%*%t(X[i,])
48     rXX <- rXX + rXXi
49     rXyi <- rh[i]*X[i,]*y[i]
50     rXy <- rXy + rXyi
51   }
52   Bhn <- solve(B0hi + ahn/dhn * rXX)
53   bhn <- Bhn%*%(B0hi%*%b0h + ahn/dhn * rXy)
54   return(list(bhn = bhn, Bhn = Bhn))
55 }

```

R code. Variational Bayes: Mixture Gaussian regression.

```

1 VarParSigma2h <- function(bhn, Bhn, rh){
2   ahn <- a0h + sum(rh); dhnterm <- 0
3   for(i in 1:n){
4     dhntermi <- rh[i]*((y[i] - t(X[i,])%*%bhn)^2 + t(X[i,])%
5       *%Bhn%*%X[i,])
6     dhnterm <- dhnterm + dhntermi
7   }
8   dhn <- dhnterm + d0h; return(list(ahn = ahn, dhn = dhn))
9 }
10 multivariate_beta <- function(alpha) {
11   prod_gammas <- sum(sapply(alpha, lgamma))
12   sum_alpha <- sum(alpha)
13   gamma_sum_alpha <- lgamma(sum_alpha)
14   return(prod_gammas - gamma_sum_alpha)
15 }
16 ELBO <- function(rih, dhn, ahn, bhn, Bhn, an){
17   LogLikh <- 0; LogPrioPsih <- 0
18   LogPriorBeta <- 0; LogPriorSig2 <- 0
19   for(h in 1:H){
20     for(i in 1:n){
21       LogLikih <- rih[i, h]*(-0.5*log(2*pi) - 0.5*(log(dhn[h]
22         ]/2) - digamma(ahn[h]/2)) - 0.5*(ahn[h]/dhn[h])*((y[i] -
23         t(X[i,])%*%bhn[,h])^2 + t(X[i,])%*%Bhn[,h]%*%X[i,]))
24       LogLikh <- LogLikh + LogLikih
25       LogPrioPsiih <- rih[i, h]*(digamma(an[h]) - digamma(
26         sum(an)))
27       LogPrioPsih <- LogPrioPsih + LogPrioPsiih
28     }
29     LogPriorBetah <- -0.5*K*log(2*pi) - 0.5*log(det(B0h)) -
30       sum(diag(Bhn[,h]%*%B0hi)) - t(bhn[,h] - b0h)%*%B0hi%*(
31         bhn[,h] - b0h)
32     LogPriorBeta <- LogPriorBeta + LogPriorBetah
33     LogPriorSig2h <- -(a0h/2+1)*(log(dhn[h]/2) - digamma(ahn
34       [h]/2)) - 0.5*d0h*(ahn[h]/dhn[h]) + (a0h/2)*log(d0h/2) -
35       lgamma(a0h/2)
36     LogPriorSig2 <- LogPriorSig2 + LogPriorSig2h
37   }
38   LogJoint <- LogLikh + LogPrioPsih + LogPriorBeta +
39     LogPriorSig2
40   logqPsi <- 0; logqLambda <- 0; logqBeta <- 0; logqSig2 <- 0
41   for(h in 1:H){
42     for(i in 1:n){
43       logqPsiih <- rih[i, h] * log(rih[i, h])
44       logqPsi <- logqPsi + logqPsiih
45     }
46     logqLambdah <- (an[h] - 1)*(digamma(an[h]) - digamma(sum
47       (an)))
48     logqLambda <- logqLambda + logqLambdah
49     logqBetah <- -0.5*K*log(2*pi + 1) - 0.5*log(det(Bhn[,h]
50       ))
51     logqBeta <- logqBeta + logqBetah
52     logqSig2h <- -log(dhn[h]/2) + (ahn[h]/2+1)*digamma(ahn[h]
53       /2) - ahn[h]/2 - lgamma(ahn[h]/2)
54     logqSig2 <- logqSig2 + logqSig2h
55   }
56   logqLambda <- logqLambda - multivariate_beta(alpha = an)
57   logq <- logqPsi + logqLambda + logqBeta + logqSig2
58   ELBO <- LogJoint - logq
59   return(ELBO)
60 }

```


R code. Variational Bayes: Mixture Gaussian regression.

```

1 # Initial values
2 fit_lm1 <- lm(y ~ x)
3 bhn[,1] <- coef(fit_lm1)
4 bhn[,2] <- coef(fit_lm1) + rnorm(K, 0, 0.5)
5 Bhn[,1] <- diag(0.1, K); Bhn[,2] <- diag(0.1, K)
6 ahn <- rep(10, H); dhn <- rep(10, H)
7 an <- c(300, 700)
8 ELBOs <- rep(NA, S)
9 ELBOs[1] <- ELBO(rih = Rih, dhn = dhn, ahn = ahn, bhn = bhn,
10   Bhn = Bhn, an = an)
11 epsilon <- 1e-5
12 for(s in 2:S){
13   Rih <- VarParPsi(dhn = dhn, ahn = ahn, bhn = bhn, Bhn =
14     Bhn, an = an)
15   an <- VarParLambda(rih = Rih)
16   bhn <- matrix(NA, K, H)
17   Bhn <- array(NA, c(K, K, H))
18   for(h in 1:H){
19     EnsVarParBetah <- VarParBetah(ahn = ahn[h], dhn = dhn[h],
20       rh = Rih[,h])
21     bhn[,h] <- EnsVarParBetah[["bhn"]]
22     Bhn[,h] <- EnsVarParBetah[["Bhn"]]
23   }
24   ahn <- rep(NA, H)
25   dhn <- rep(NA, H)
26   for(h in 1:H){
27     EnsVarParSig2h <- VarParSigma2h(bhn = bhn[,h], Bhn = Bhn[,h],
28       rh = Rih[,h])
29     ahn[h] <- EnsVarParSig2h[["ahn"]]
30     dhn[h] <- EnsVarParSig2h[["dhn"]]
31   }
32   ELBOs[s] <- ELBO(rih = Rih, dhn = dhn, ahn = ahn, bhn = bhn,
33     Bhn = Bhn, an = an)
34   # Check if lower bound decreases
35   if (ELBOs[s] < ELBOs[s - 1]) { message("Lower bound
36     decreases!\n")}
37   # Check for convergence
38   if (ELBOs[s] - ELBOs[s - 1] < epsilon) { break }
39   # Check if VB converged in the given maximum iterations
40   if (s == S) {warning("VB did not converge!\n")}
41 }
42 plot(ELBOs[1:S], type = "l", main = "ELBO over iterations",
43   ylab = "ELBO", xlab = "Iteration")
44 Sig21 <- MCMCpack::rinvgamma(S, ahn[1]/2, dhn[1]/2)
45 summary(coda::mcmc(Sig21))
46 Sig22 <- MCMCpack::rinvgamma(S, ahn[2]/2, dhn[2]/2)
47 summary(coda::mcmc(Sig22))
48 Beta1 <- MASS::mvrnorm(S, mu = bhn[,1], Sigma = Bhn[,1])
49 summary(coda::mcmc(Beta1))
50 Beta2 <- MASS::mvrnorm(S, mu = bhn[,2], Sigma = Bhn[,2])
51 summary(coda::mcmc(Beta2))
52 Lambda <- MCMCpack::rdirichlet(S, alpha = an)
53 summary(coda::mcmc(Lambda))

```



Bibliography

- [1] Jim Albert. *Bayesian Computation with R*. Use R! Springer, New York, NY, 2nd edition, 2009.
- [2] Izhar Asael Alonzo and Cristian Cruz. varstan: An R package for Bayesian time series models with Stan. *ARXIV Preprint*, 2020.
- [3] R. Baath. *Package bayesboot*, 2018.
- [4] Thomas Bayes. LII. an essay towards solving a problem in the doctrine of chances. by the late rev. mr. bayes, FRS communicated by Mr. Price, in a letter to John Canton, AMFR S. *Philosophical transactions of the Royal Society of London*, 53:370–418, 1763.
- [5] Bob Carpenter, Andrew Gelman, Matthew D Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of statistical software*, 76:1–32, 2017.
- [6] Joshua Chan, Gary Koop, Dale J Poirier, and Justin L Tobias. *Bayesian econometric methods*, volume 7. Cambridge University Press, 2019.
- [7] W. Chang. *Web Application Framework for R: Package shiny*. R Studio, 2018.
- [8] T. Conley, C. Hansen, and P. Rossi. Plausibly exogenous. *The Review of Economics and Statistics*, 94(1):260–272, 2012.
- [9] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, Hoboken, NJ, 2nd edition, 2006.
- [10] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.
- [11] Andrew Gelman, John B Carlin, Hal S Stern, David Dunson, Aki Vehtari, and Donald B Rubin. *Bayesian data analysis*. Chapman and Hall/CRC, 2021.
- [12] J. Geweke. *Bayesian Statistics*, chapter Evaluating the accuracy of sampling-based approaches to calculating posterior moments. Clarendon Press, Oxford, UK., 1992.

- [13] John Geweke. Getting it right: Joint distribution tests of posterior simulators. *Journal of the American Statistical Association*, 99(467):799–804, 2004.
- [14] John Geweke. *Contemporary Bayesian econometrics and statistics*, volume 537. John Wiley & Sons, 2005.
- [15] N. J. Gordon, D. J. Salmond, and A. F. Smith. Novel approach to nonlinear/non-gaussian bayesian state estimation. *IEE Proceedings F (Radar and Signal Processing)*, 140(2):107–113, 1993.
- [16] Edward Greenberg. *Introduction to Bayesian econometrics*. Cambridge University Press, 2012.
- [17] P. Heidelberger and P. D. Welch. Simulation run length control in the presence of an initial transient. *Operations Research*, 31(6):1109–1144, 1983.
- [18] Daniel Hosszejni and Gregor Kastner. Modeling univariate and multivariate stochastic volatility in r with `stochvol` and `factorstochvol`. *Journal of Statistical Software*, 100(12):1–34, 2021.
- [19] H. Jeffreys. *Theory of Probability*. Oxford University Press, London, 1961.
- [20] M. Jetter and A. Ramírez Hassan. Want export diversification? Educate the kids first. *Economic Inquiry*, 53(4):1765–1782, 2015.
- [21] Michael Jetter, Rafat Mahmood, Christopher F. Parmeter, and Andrés Ramírez-Hassan. Post-cold war civil conflict and the role of history and religion: A stochastic search variable selection approach. *Economic Modelling*, 114:105907, 2022.
- [22] Valen E Johnson and David Rossell. On the use of non-local prior densities in bayesian hypothesis tests. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 72(2):143–170, 2010.
- [23] G. Karabatsos. A menu-driven software package of Bayesian nonparametric (and parametric) mixed models for regression analysis and density estimation. *Behavior Research Methods*, 49:335–362, 2016.
- [24] Gary M Koop. *Bayesian econometrics*. John Wiley & Sons Inc., 2003.
- [25] Alp Kucukelbir, Dustin Tran, Rajesh Ranganath, Andrew Gelman, and David M Blei. Automatic differentiation variational inference. *Journal of machine learning research*, 18(14):1–45, 2017.
- [26] Tony Lancaster. *An introduction to modern Bayesian econometrics*. Blackwell Oxford, 2004.

- [27] Alex Lenkoski, Anna Karl, and Andreas Neudecker. *Package ivbma*, 2013.
- [28] Dennis V Lindley. A statistical paradox. *Biometrika*, 44(1/2):187–192, 1957.
- [29] Andrew D. Martin, Kevin M. Quinn, and Jong Hee Park. *Package MCM-Cpack*, 2018.
- [30] Tyler H. McCormick, Adrian Raftery, David Madigan, Sevvandi Kandanaarachchi, and Hana Sevcikova. *dma: Dynamic Model Averaging*, 2018. R package version 1.4-0.
- [31] Franz X. Mohr. *bvartools: Bayesian Inference of Vector Autoregressive and Error Correction Models*, 2024. R package version 0.2.4.
- [32] P. Müller, F. Quintana, A. Jara, and T. Hanson. *Bayesian Nonparametric Data Analysis*. Springer, New York, 2015.
- [33] Giovanni Petris. An R package for dynamic linear models. *Journal of Statistical Software*, 36(12):1–16, 2010.
- [34] Martyn Plummer, Nicky Best, Kate Cowles, Karen Vines, Deepayan Sarkar, Douglas Bates, Russell Almond, and Arni Magnusson. *Output Analysis and Diagnostics for MCMC*, 2016.
- [35] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2021.
- [36] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2023.
- [37] Adrian Raftery, Jennifer Hoeting, Chris Volinsky, Ian Painter, and Ka Yee Yeung. *Package BMA*, 2012.
- [38] A.E. Raftery and S.M. Lewis. One long run with diagnostics: Implementation strategies for Markov chain Monte Carlo. *Statistical Science*, 7:493–497, 1992.
- [39] Andrés Ramírez-Hassan. Dynamic variable selection in dynamic logistic regression: an application to internet subscription. *Empirical Economics*, 59(2):909–932, 2020.
- [40] Andrés Ramírez-Hassan and Daniela A Carvajal-Rendón. Specification uncertainty in modeling internet adoption: A developing city case analysis. *Utilities Policy*, 70:101218, 2021.
- [41] Andrés Ramírez-Hassan and David T Frazier. Testing model specification in approximate bayesian computation using asymptotic properties. *Journal of Computational and Graphical Statistics*, 33(3):1–14, 2024.

- [42] A. Ramírez-Hassan and M. Graciano-Londoño. A guided tour of Bayesian regression. Technical report, Universidad EAFIT, 2020.
- [43] P. Rossi. *Package bayesm*, 2017.
- [44] Peter E Rossi, Greg M Allenby, and Rob McCulloch. *Bayesian statistics and marketing*. John Wiley & Sons, 2012.
- [45] Peter E. Rossi, Greg M. Allenby, and Robert McCulloch. *Bayesian Statistics and Marketing*. John Wiley and Sons, Hoboken, NJ, 2005.
- [46] Francesco Serafini. Multinomial logit models with inla. *R-INLA Tutorial*. <https://inla.r-inla-download.org/r-inla.org/doc/vignettes/multinomial.pdf>, 2019.
- [47] M. Serna Rodríguez, A. Ramírez Hassan, and A. Coad. Uncovering value drivers of high performance soccer players. *Journal of Sport Economics*, 20(6):819–849, 2019.
- [48] A. F. M. Smith. A General Bayesian Linear Model. *Journal of the Royal Statistical Society. Series B (Methodological)*., 35(1):67–75, 1973.
- [49] Stan Development Team. shinystan: Interactive visual and numerical diagnostics and posterior analysis for Bayesian models., 2017. R package version 2.3.0.
- [50] Samuel Thomas and Wanzhu Tu. Learning hamiltonian monte carlo in r. *The American Statistician*, 75(4):403–413, 2021.
- [51] Regina Tüchler. Bayesian variable selection for logistic models using auxiliary mixture sampling. *Journal of Computational and Graphical Statistics*, 17(1):76–94, 2008.
- [52] P. Woodward. BugsXLA: Bayes for the common man. *Journal of Statistical Software*, 14(5):1–18, 2005.
- [53] Jeffrey M. Wooldridge. *Introductory Econometrics: A Modern Approach*. Cengage Learning, Mason, Ohio: South-Western, fifth edition, 2012.
- [54] A. Zellner. *Introduction to Bayesian inference in econometrics*. John Wiley & Sons Inc., 1996.

A

Appendix

TABLE A.1

Libraries and commands in BEsmarter GUI.

Univariate models			
Model	Library	Command	Reference
Normal	MCMCpack	MCMCregress	[29]
Logit	MCMCpack	MCMClogit	[29]
Probit	bayesm	rbprobitGibbs	[43]
Multinomial(Mixed) Probit	bayesm	rmnpGibbs	[43]
Multinomial(Mixed) Logit	bayesm	rmnlIndepMetrop	[43]
Ordered Probit	bayesm	rordprobitGibbs	[43]
Negative Binomial(Poisson)	bayesm	rnegbinRw	[43]
Tobit	MCMCpack	MCMCtobit	[29]
Quantile	MCMCpack	MCMCquantreg	[29]
Bayesian bootstrap	bayesboot	bayesboot	[3]
Multivariate models			
Model	Library	Command	Reference
Multivariate	bayesm	rmultireg	[43]
Seemingly Unrelated Regression	bayesm	rsurGibbs	[43]
Instrumental Variable	bayesm	rivGibbs	[43]
Bivariate Probit	bayesm	rmvpGibbs	[43]
Time series models			
Model	Library	Command	Reference
Normal dynamic linear model	dln	dlnGibbsDIGs	[33]
ARMA	bayesforecast	stan_sarima	[2]
Stochastic volatility models	stochvol	svsample	[18]
VAR	bvartools	draw_posterior	[31]
Hierarchical longitudinal models			
Model	Library	Command	Reference
Normal	MCMCpack	MCMChregress	[29]
Logit	MCMCpack	MCMChlogit	[29]
Poisson	MCMCpack	MCMChpoisson	[29]
Bayesian model averaging			
Model	Library	Command	Reference
Normal (BIC)	BMA	bicreg	[37]
Normal (MC ³)	BMA	MC3.REG	[37]
Normal (instrumental variables)	ivbma	ivbma	[27]
Normal (Dynamic BMA)	dma	dma	[30]
Logit (BIC)	BMA	bic.glm	[37]
Gamma (BIC)	BMA	bic.glm	[37]
Poisson (BIC)	BMA	bic.glm	[37]
Diagnostics			
Diagnostic	Library	Command	Reference
Trace plot	coda	traceplot	[34]
Autocorrelation plot	coda	autocorr.plot	[34]
Geweke test	coda	geweke.diag	[34]
Raftery & Lewis test	coda	raftery.diag	[34]
Heidelberger & Welch test	coda	heidel.diag	[34]

TABLE A.2Datasets templates in folder *DataSim*.

Univariate models		
Model	Data set file	Data set simulation
Normal	11SimNormalmodel.csv	11SimNormal.R
Logit	12SimLogitmodel.csv	12SimLogit
Probit	13SimProbitmodel.csv	13SimProbit.R
Multinomial(Mixed) Probit	14SimMultProbmodel.csv	14SimMultinomialProbit.R
Multinomial(Mixed) Logit	15SimMultLogitmodel.csv	15SimMultinomialLogit.R
Ordered Probit	16SimOrderedProbitmodel.csv	16SimOrderedProbit.R
Negative Binomial(Poisson)	17SimNegBinmodel.csv	17SimNegBin.R
Tobit	18SimTobitmodel.csv	18SimTobit.R
Quantile	19SimQuantilemodel.csv	19SimQuantile.R
Bayesian bootstrap	110SimBootstrapmodel.csv	110SimBootstrapmodel.R
Multivariate models		
Model	Data set file	Data set simulation
Multivariate	21SimMultivariate.csv	21SimMultReg.R
Seemingly Unrelated Regression	22SimSUR.csv	22SimSUR.R
Instrumental Variable	23SimIV.csv	23SimIV.R
Bivariate Probit	24SimMultProbit.csv	24SimMultProbit.R
Time series models		
Model	Data set file	Data set simulation
Dynamic Linear Models	31SimDynamicLinearModel.csv	31SimDynamicLinearModel.R
ARMA	32SimARMAmodels.csv	32SimARMAmodels.R
Stochastic volatility Models	33SimStochasticVolatility.csv	33SimStochasticVolatility.R
VAR	34SimVARmodels.csv	34SimVARmodels.R
Hierarchical longitudinal models		
Model	Data set file	Data set simulation
Normal	41SimLogitudinalNormal.csv	41SimLogitudinalNormal.R
Logit	42SimLogitudinalLogit.csv	42SimLogitudinalLogit.R
Poisson	43SimLogitudinalPoisson.csv	43SimLogitudinalPoisson.R
Bayesian model averaging		
Model	Data set file	Data set simulation
Normal (BIC)	511SimNormalBMA.csv	511SimNormalBMA.R
Normal (MC ³)	512SimNormalBMA.csv	512SimNormalBMA.R
Normal (instrumental variables)	513SimNormalBMAivYXW.csv	513SimNormalBMAiv.R
	513SimNormalBMAivZ.csv	
Normal (Dynamic BMA)	514SimDynamicBMA.csv	514SimDynamicBMA.R
	514SimModels.csv	
Logit (BIC)	52SimLogitBMA.csv	52SimLogitBMA.R
Gamma (BIC)	53SimGammaBMA.csv	53SimGammaBMA.R
Poisson (BIC)	53SimPoissonBMA.csv	53SimPoissonBMA.R

TABLE A.3Real datasets in folder *DataApp*.

Univariate models		
Model	Data set file	Dependent variable
Normal	1ValueFootballPlayers.csv	log(Value)
Logit	2HealthMed.csv	Hosp
Probit	2HealthMed.csv	Hosp
Multinomial(Mixed) Probit	Fishing.csv	mode
Multinomial(Mixed) Logit	Fishing.csv	mode
Ordered Probit	2HealthMed.csv	MedVisPrevOr
Negative Binomial(Poisson)	2HealthMed.csv	MedVisPrev
Tobit	1ValueFootballPlayers.csv	log(ValueCens)
Quantile	1ValueFootballPlayers.csv	log(Value)
Bayesian bootstrap	1ValueFootballPlayers.csv	log(Value)
Multivariate models		
Model	Data set file	Dependent variable
Multivariate	4Institutions.csv	logpcGDP95 and PAER
Seemingly Unrelated Regression	5Institutions.csv	logpcGDP95 and PAER
Instrumental Variable	6Institutions.csv	logpcGDP95 and PAER
Bivariate Probit	7HealthMed.csv	$\mathbf{y} = [\text{Hosp SHI}]^\top$
Time series models		
Model	Data set file	Dependent variable
Dynamic Linear Models	16INTDEF.csv	$\Delta(i3)$
ARMA	16INTDEF.csv	$\Delta(i3)$
Stochastic volatility models	17ExcRate.csv	USDEUR
VAR	18USAfiscal	$\mathbf{y}_t = [\Delta(ttr_t) \Delta(gst) \Delta(gdp_t)]^\top$
Hierarchical longitudinal models		
Model	Data set file	Dependent variable
Normal	8PublicCap.csv	log(gsp)
Logit	9VisitDoc.csv	DocVis
Poisson	9VisitDoc.csv	DocNum
Bayesian model averaging		
Model	Data set file	Dependent variable
Normal (BIC)	10ExportDiversificationHHI.csv	avghhi
Normal (MC ³)	10ExportDiversificationHHI.csv	avghhi
Normal (instrumental variables)	11ExportDiversificationHHI.csv	avghhi and avlgdpcap
Logit (BIC)	12ExportDiversificationHHIInstr.csv	internet
Gamma (BIC)	13InternetMed.csv	log market value
Poisson (BIC)	14ValueFootballPlayers.csv	ceb
	15Fertile2.csv	